Deep Learning

Lecture 3: Deep Architectures

Adapted from UMich EECS 498 (Justin Johnson), Stanford CS231n and Stanford CS336

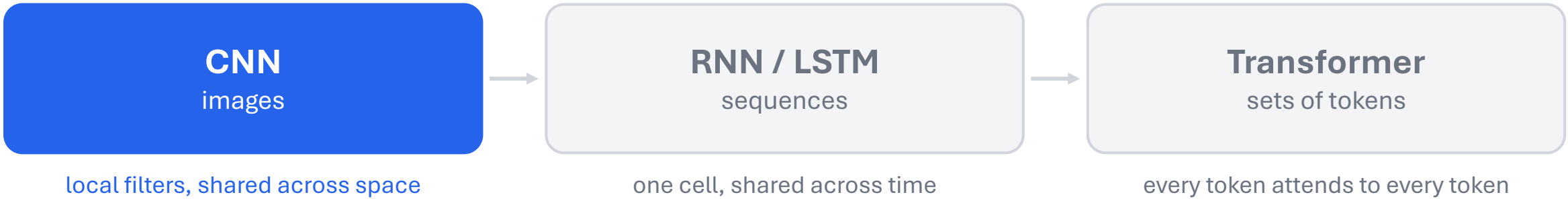
Today: three architectures



Question for the end: what makes all three trainable when they get deep?

1. **CNNs** AlexNet, VGG, GoogLeNet, ResNet, MobileNet
2. **RNNs** vanilla RNN, language models, captioning, LSTM
3. **Attention** seq2seq attention, self-attention, the Transformer, BERT, GPT

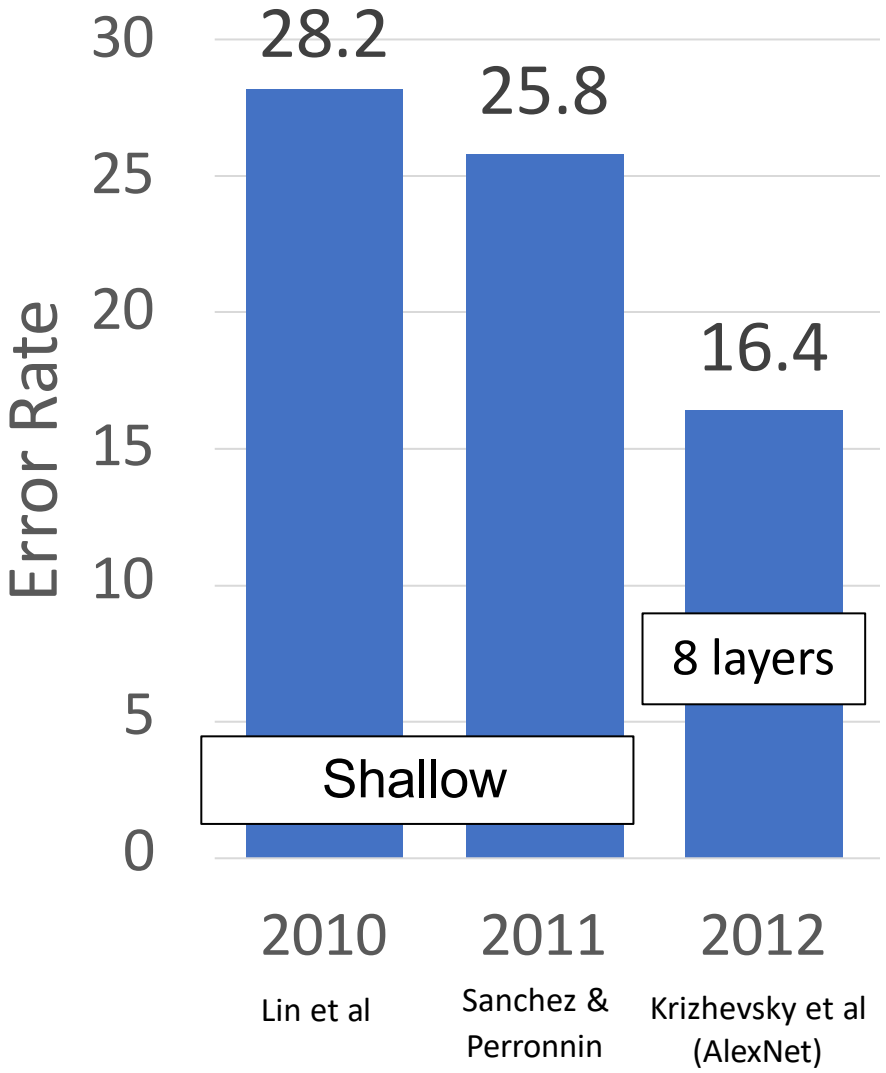
Part 1 · CNN architectures: going deeper



In this part

- AlexNet → VGG → GoogLeNet: deeper and cheaper
- Normalization: BatchNorm and LayerNorm
- ResNet: the residual connection
- Efficient CNNs: grouped and depthwise convolutions

ImageNet Classification Challenge



AlexNet

- 227 x 227 inputs
- 5 Convolutional layers Max pooling
- 3 Fully-Connected layers
- ReLU nonlinearities
- Used “Local response normalization”
  - Not used anymore
- Trained on two GTX 580 GPUs
  - Only 3GB of memory each
  - Model split over two GPUs

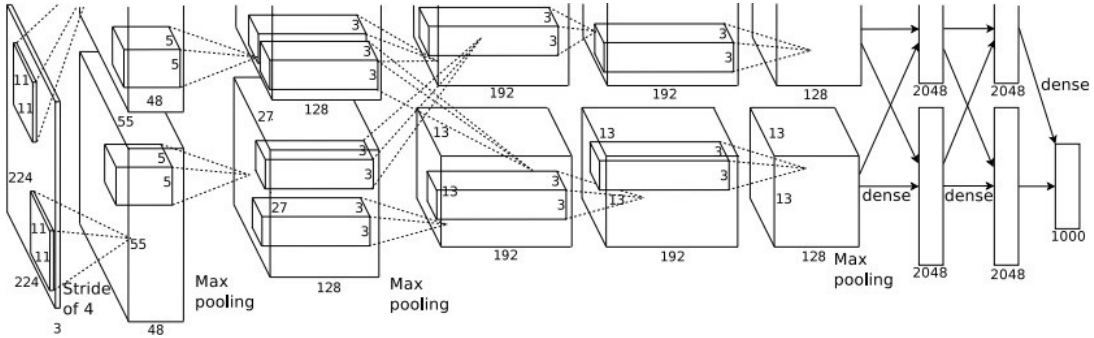
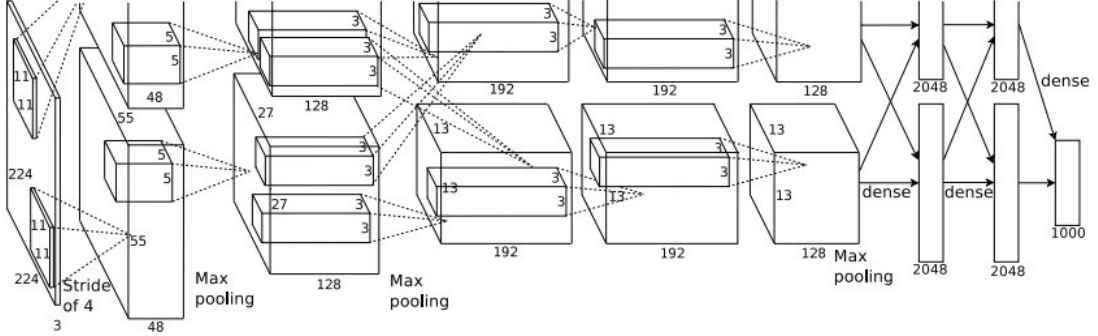
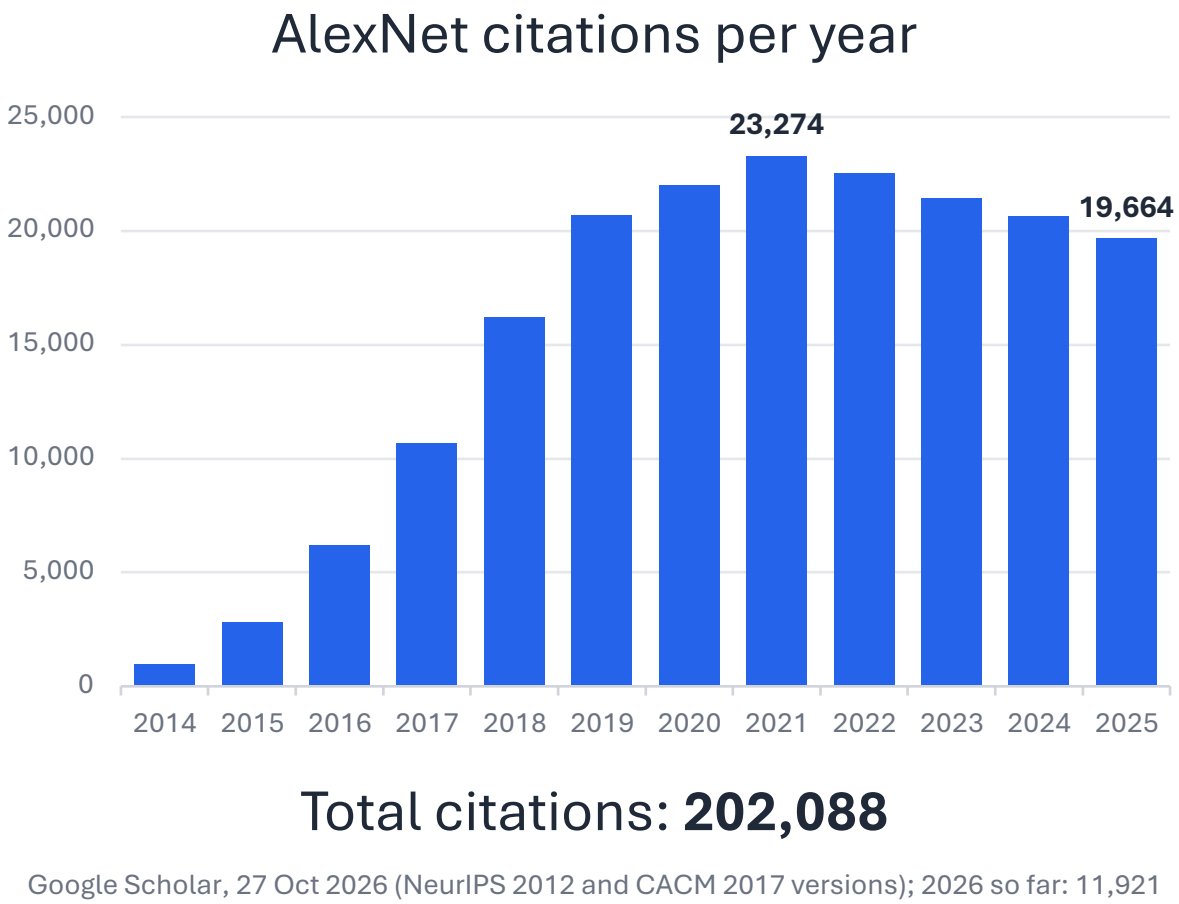


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet





Total citations: **202,088**

Google Scholar, 27 Oct 2026 (NeurIPS 2012 and CACM 2017 versions); 2026 so far: 11,921

Citation counts Google Scholar, Oct 2026

Darwin, “On the origin of species”, 1859: **44,967**

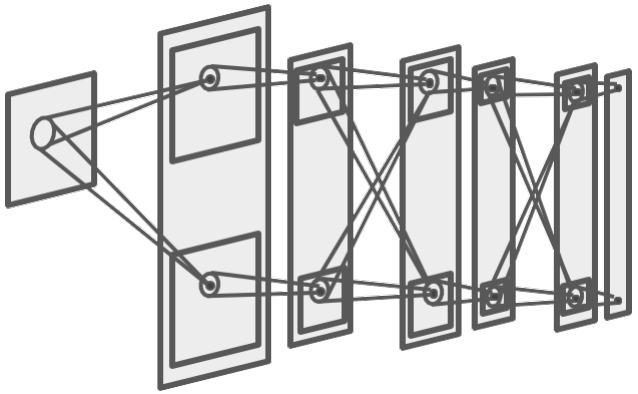
Shannon, “A mathematical theory of communication”, 1948: **185,603**

Watson and Crick, “Molecular Structure of Nucleic Acids”, 1953: **22,088**

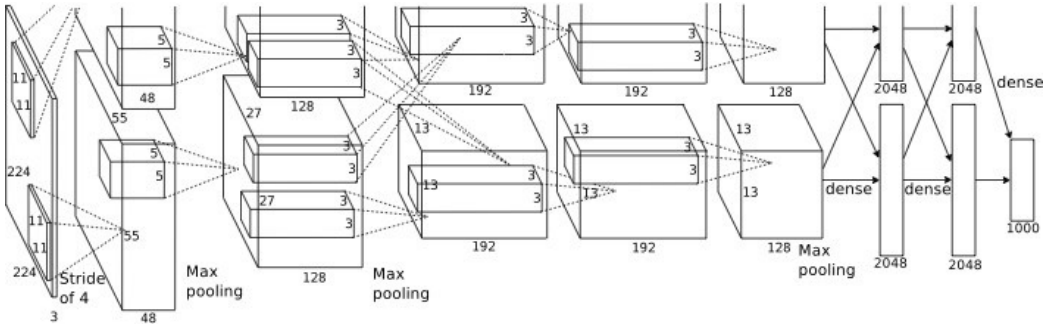
ATLAS Collaboration, “Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC”, 2012: **26,883**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet vs. Neocognitron: 32 years apart

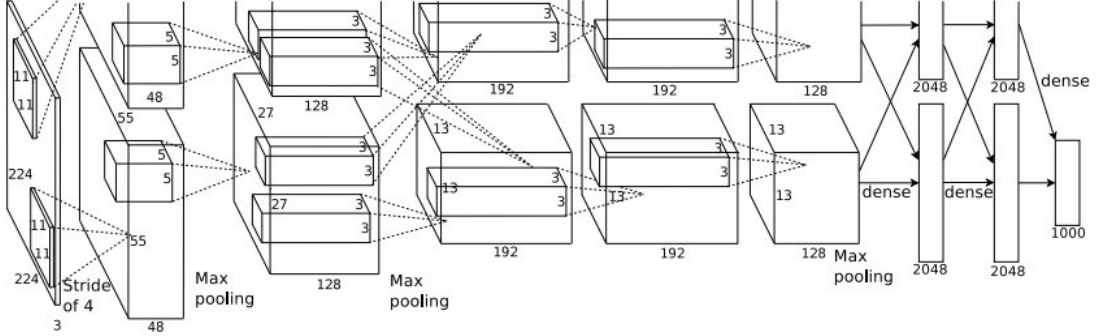


Fukushima, 1980



AlexNet, 2012

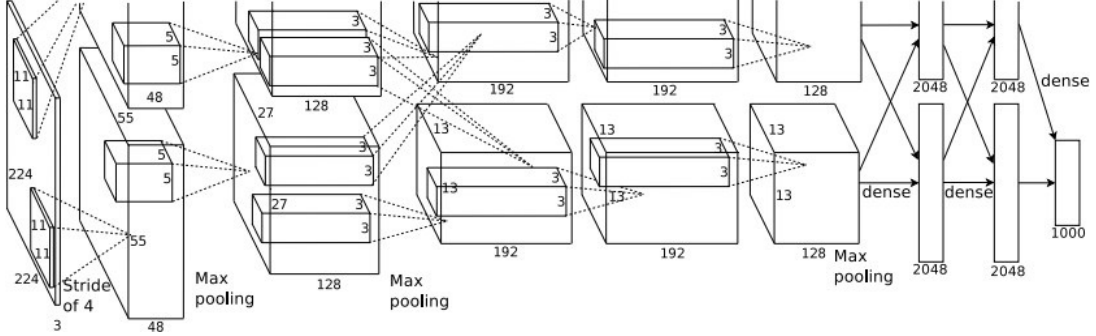
AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

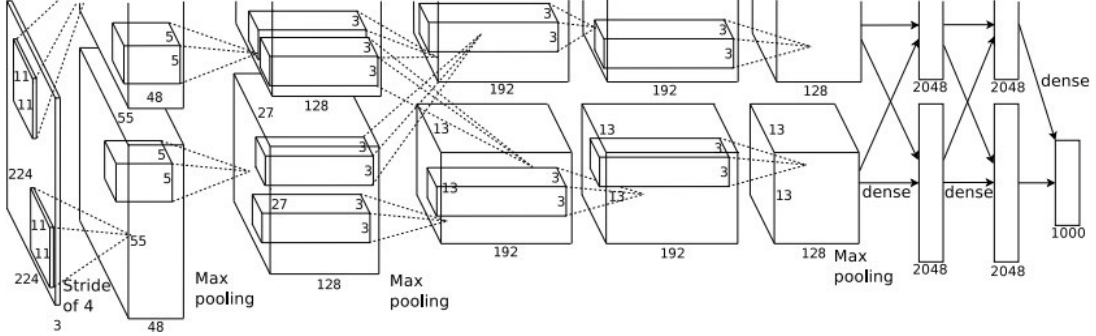


	Input size		Layer				Output size	
Layer	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	?

Recall: Output channels = number of filters

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size			Layer				Output size	
Layer	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	56

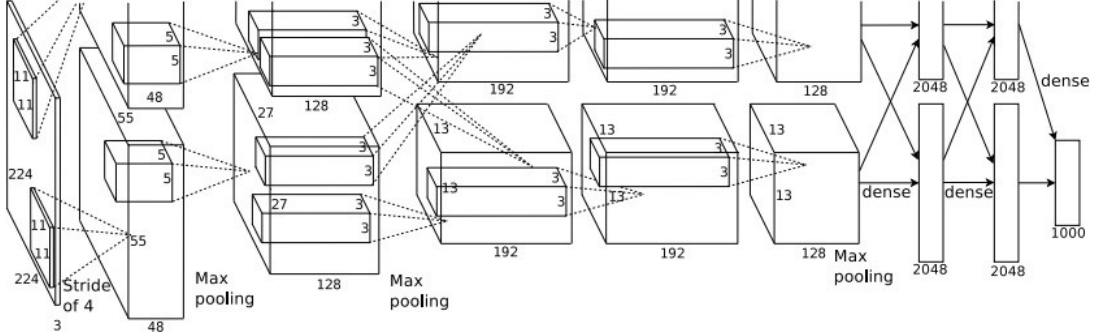
Recall: $W' = (W - K + 2P) / S + 1$

$= 227 - 11 + 2*2) / 4 + 1$

$= 220/4 + 1 = 56$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

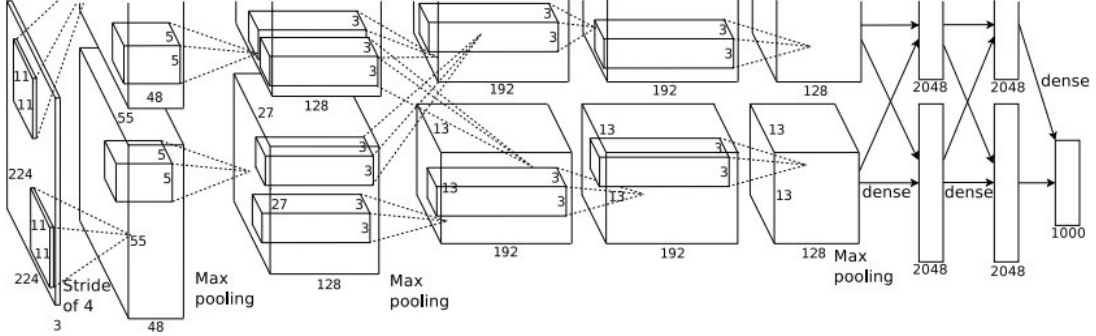
AlexNet



	Input size		Layer				Output size		
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size			Layer				Output size		
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	784

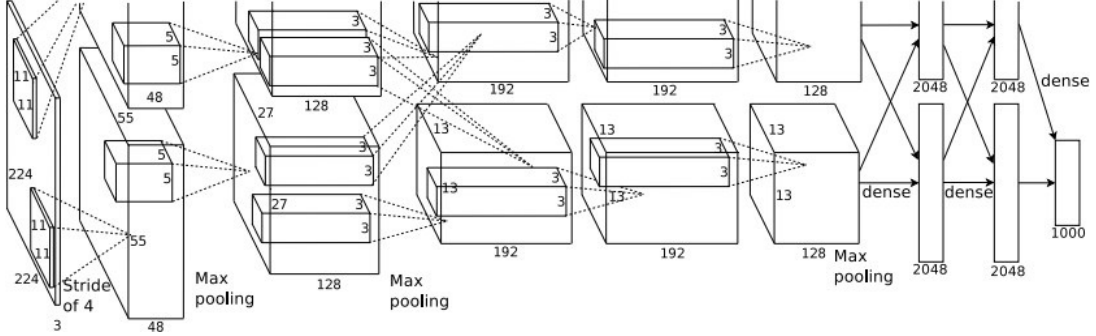
Number of output elements = C * H' * W'
= 64*56*56 = 200,704

Bytes per element = 4 (for 32-bit floating point)

KB = (number of elements) * (bytes per elem) / 1024
= 200704 * 4 / 1024
= **784**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

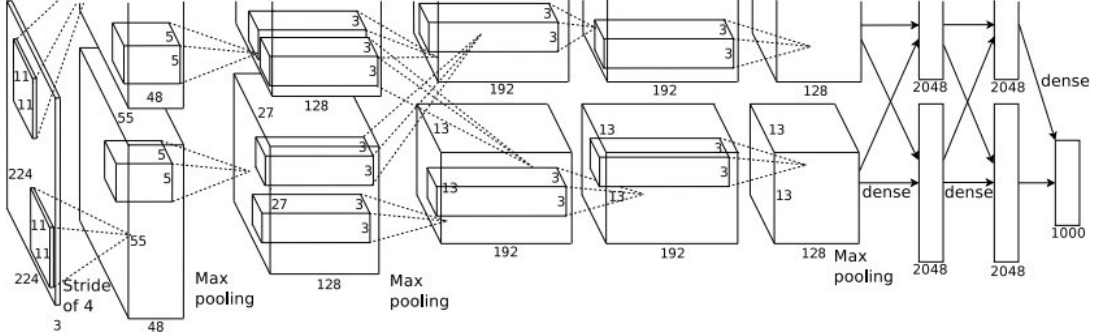
AlexNet



	Input size		Layer				Output size			
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1	3	227	64	11	4	2	64	56	784	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size			Layer				Output size			
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1	3	227	64	11	4	2	64	56	784	23

Weight shape = C_{out} x C_{in} x K x K

= 64 x 3 x 11 x 11

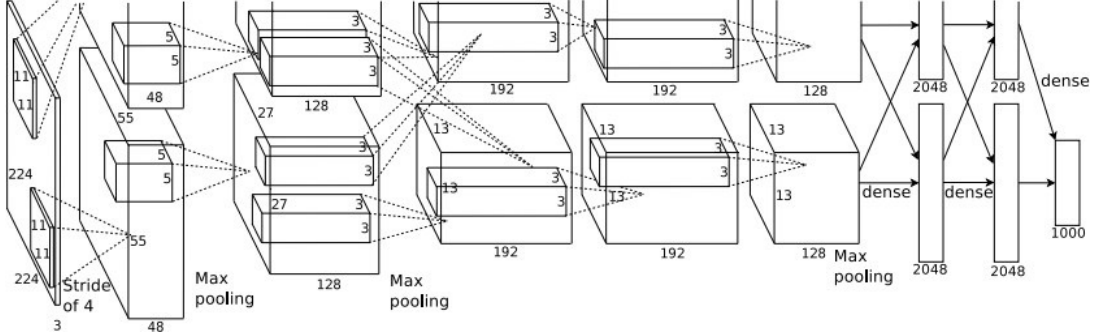
Bias shape = C_{out} = 64

Number of weights = 64*3*11*11 + 64

= **23,296**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

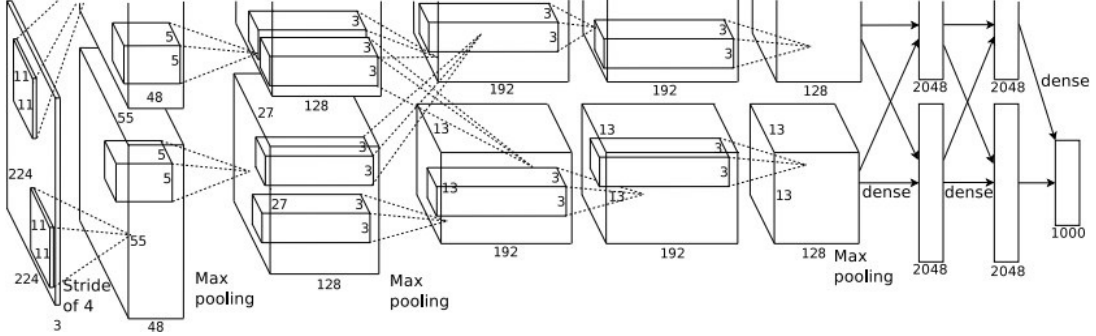
AlexNet



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



	Input size			Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73

Number of floating point operations (multiply+add)

= (number of output elements) * (ops per output elem)

= (C_{out} x H' x W') * (C_{in} x K x K)

= (64 * 56 * 56) * (3 * 11 * 11)

= 200,704 * 363

= **72,855,552**

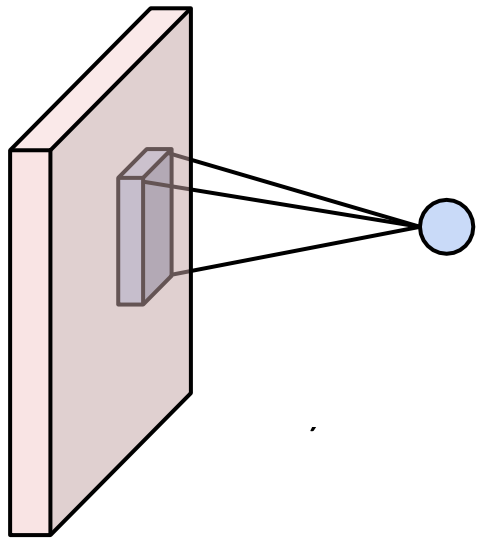
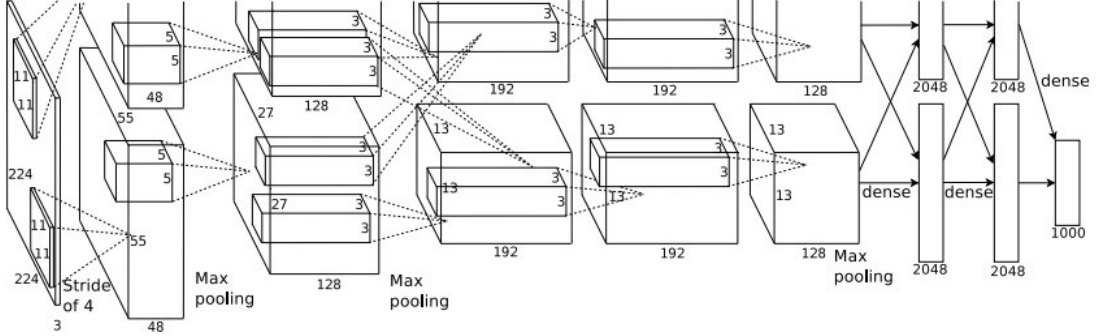


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

How to choose this?
Trial and error =(

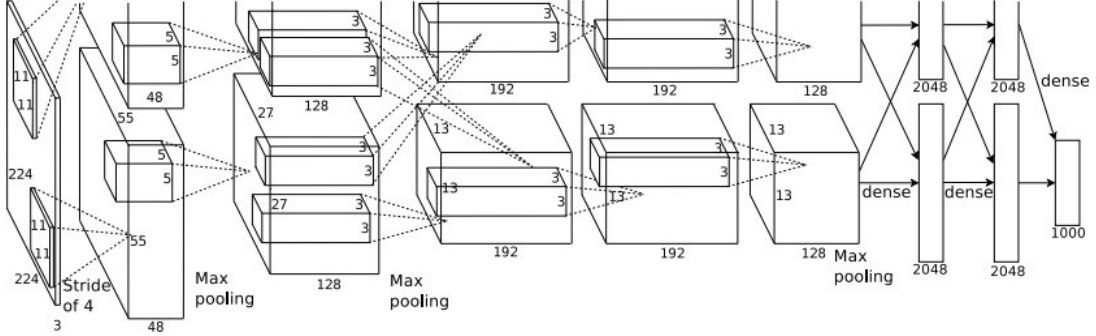


	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

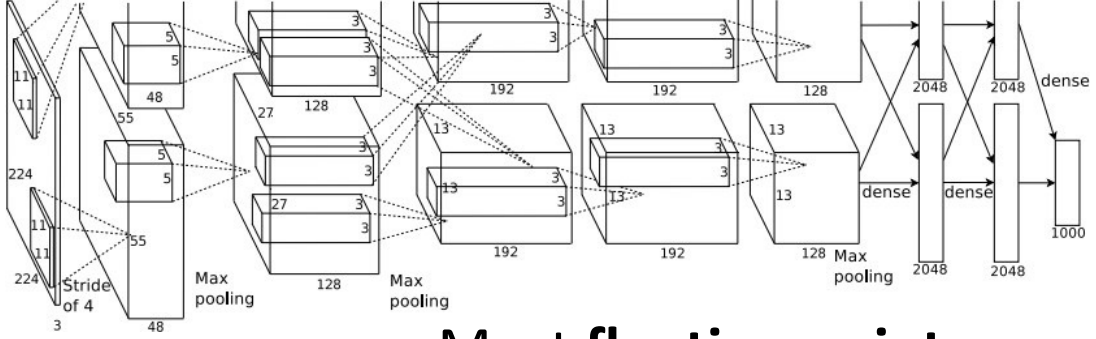
Interesting trends here!



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

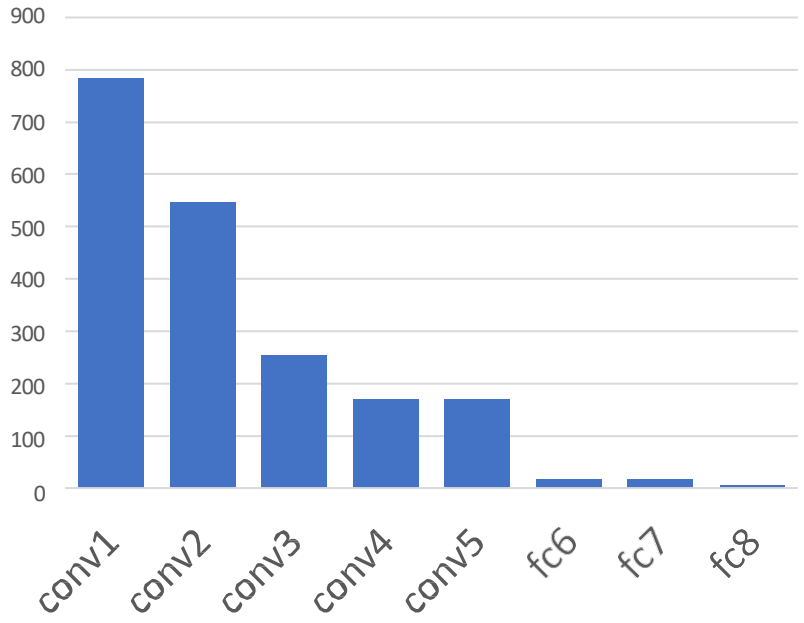


Most of the **memory usage** is in the early convolution layers

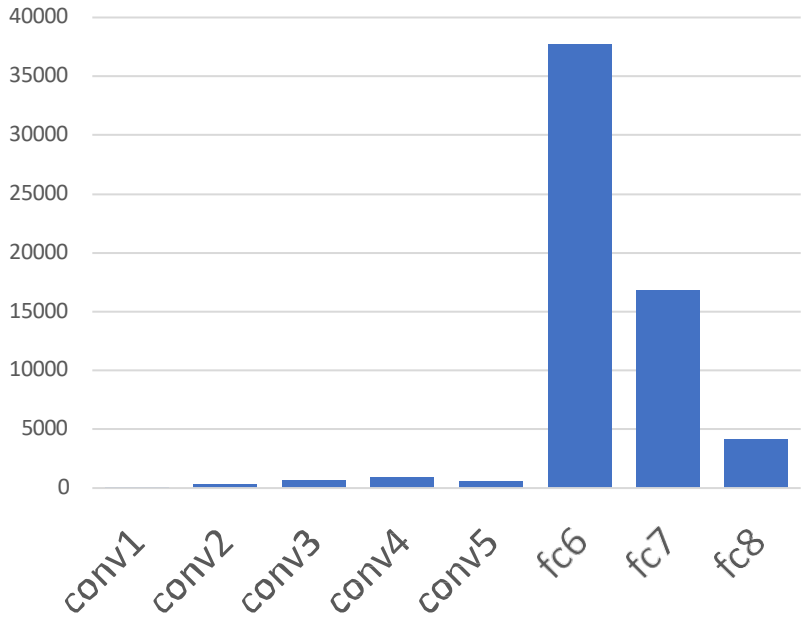
Nearly all **parameters** are in the fully-connected layers

Most **floating-point ops** occur in the convolution layers

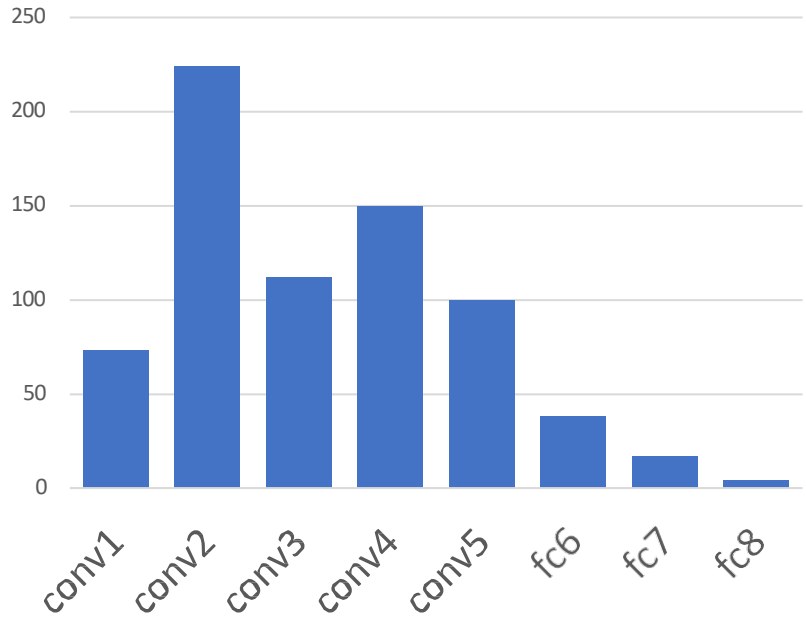
Memory (KB)



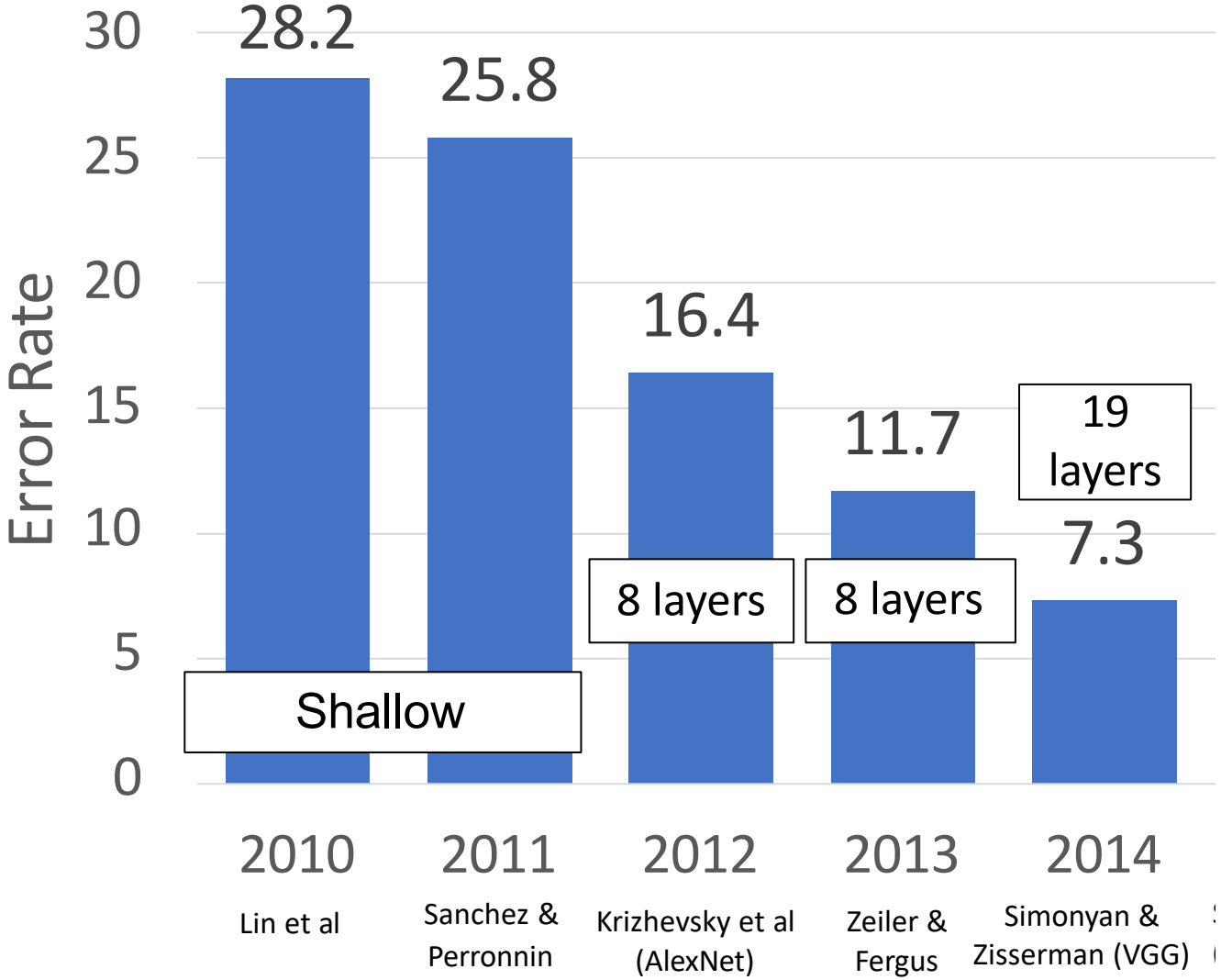
Params (K)



MFLOP



ImageNet Classification Challenge



VGG: Deeper Networks, Regular Design

Two 3x3 conv has same receptive field as a single 5x5 conv, but has fewer parameters and takes less computation!

VGG Design rules:

- **All conv are 3x3 stride 1 pad 1**
- All max pool are 2x2 stride 2
- After pool, double #channels

Option 1:

Conv(5x5, C -> C)

Params: $25C^2$

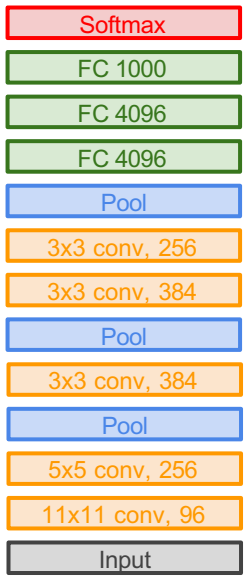
FLOPs: $25C^2HW$

Option 2:

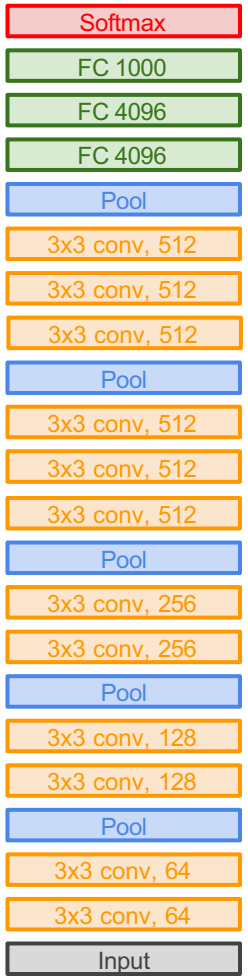
Conv(3x3, C -> C)
Conv(3x3, C -> C)

Params: $18C^2$

FLOPs: $18C^2HW$



AlexNet



VGG16



VGG19

Simonyan and Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

VGG: Deeper Networks, Regular Design

Conv layers at each spatial resolution take the same amount of computation!

<u>VGG Design rules:</u>

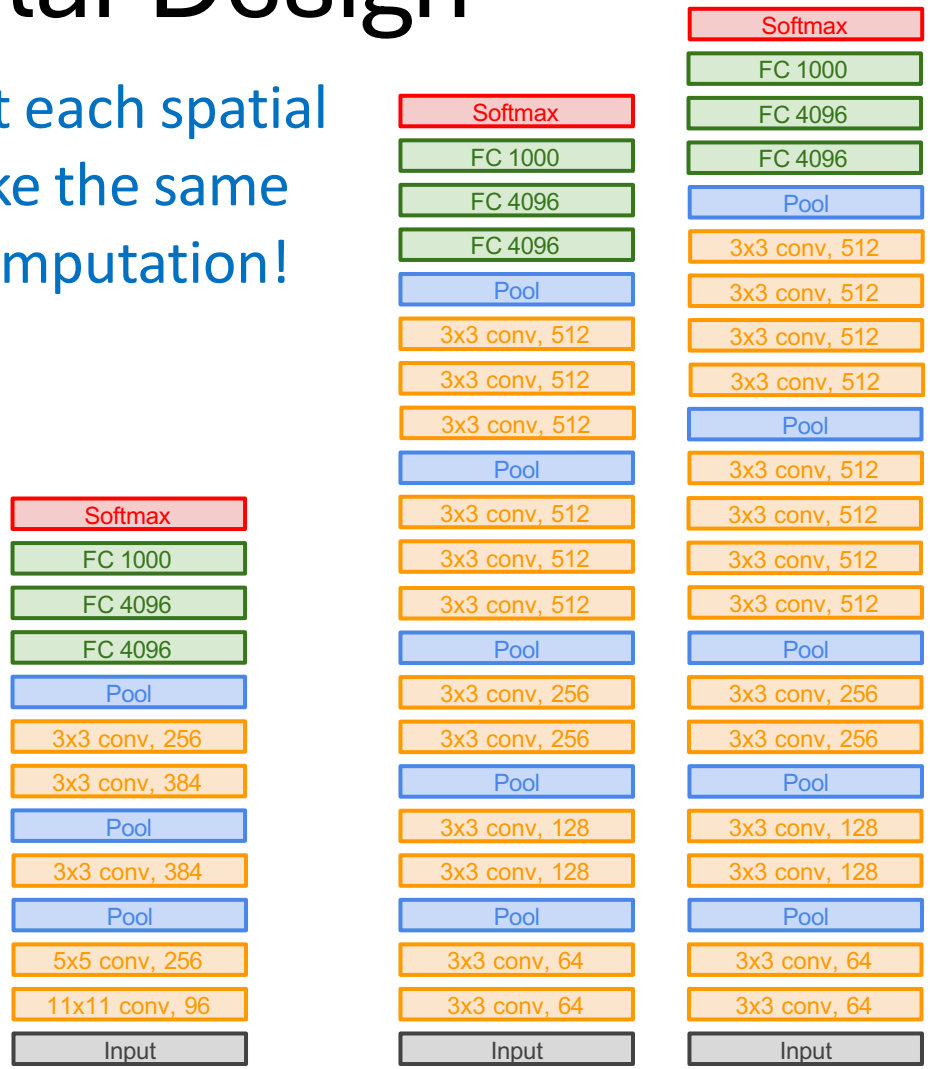
- All conv are 3x3 stride 1 pad 1
- All max pool are 2x2 stride 2
- After pool, double #channels

Input: C x 2H x 2W
Layer: Conv(3x3, C->C)

Memory: 4HWC
Params: $9C^2$
FLOPs: $36HWC^2$

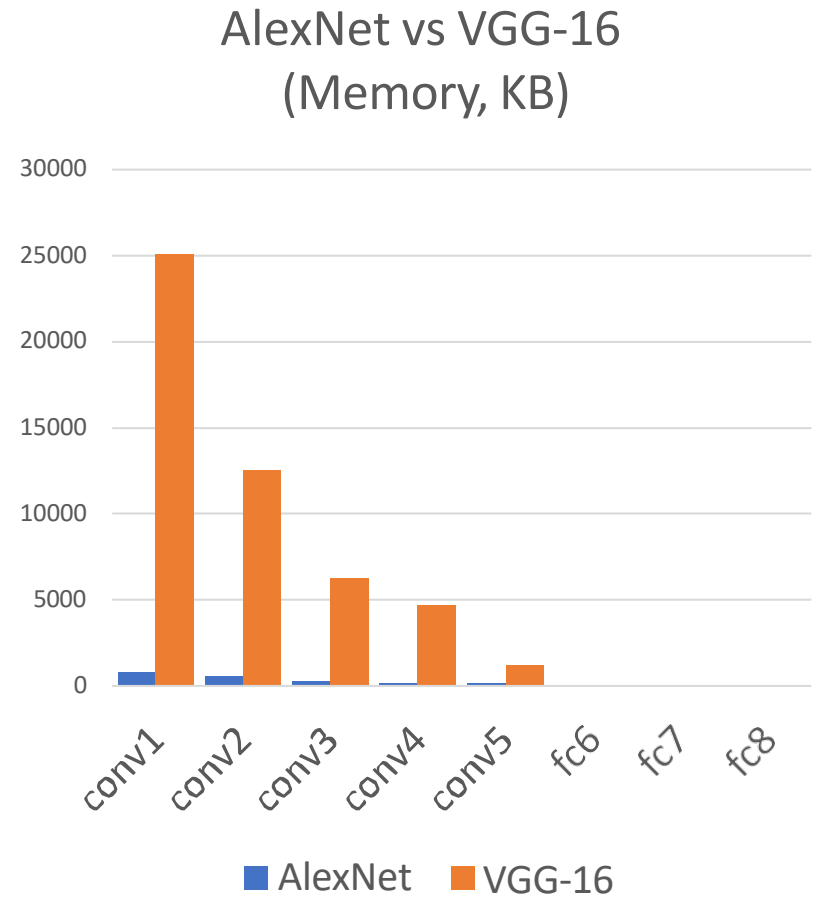
Input: 2C x H x W
Conv(3x3, 2C -> 2C)

Memory: 2HWC
Params: $36C^2$
FLOPs: $36HWC^2$



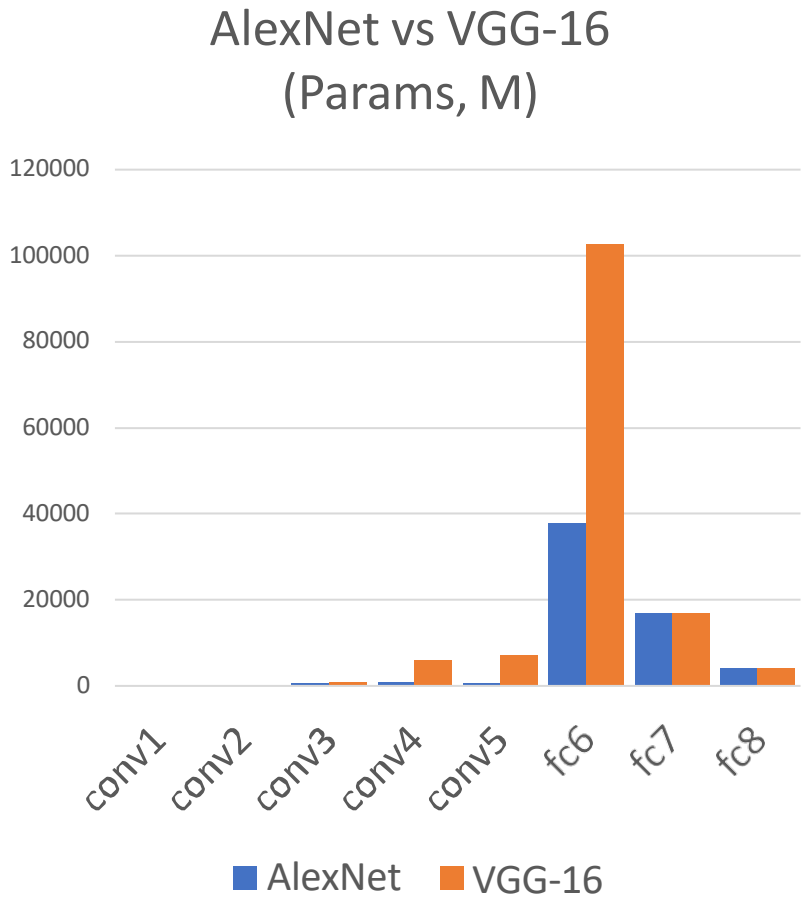
Simonyan and Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

AlexNet vs VGG-16: Much bigger network!



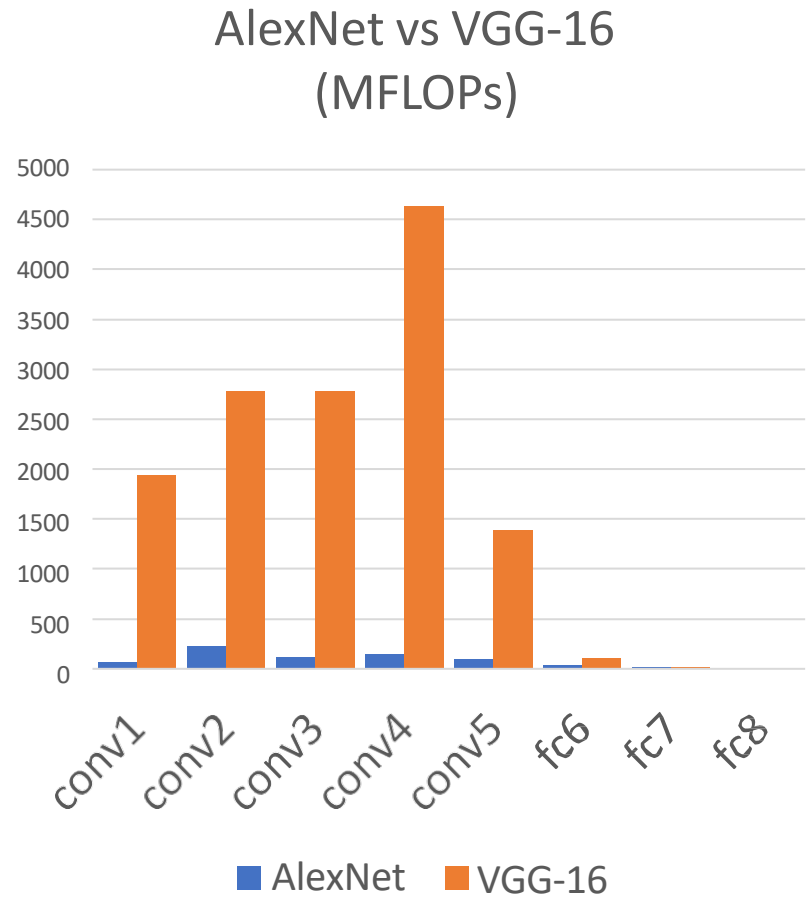
AlexNet total: 1.9 MB

VGG-16 total: 48.6 MB (25x)



AlexNet total: 61M

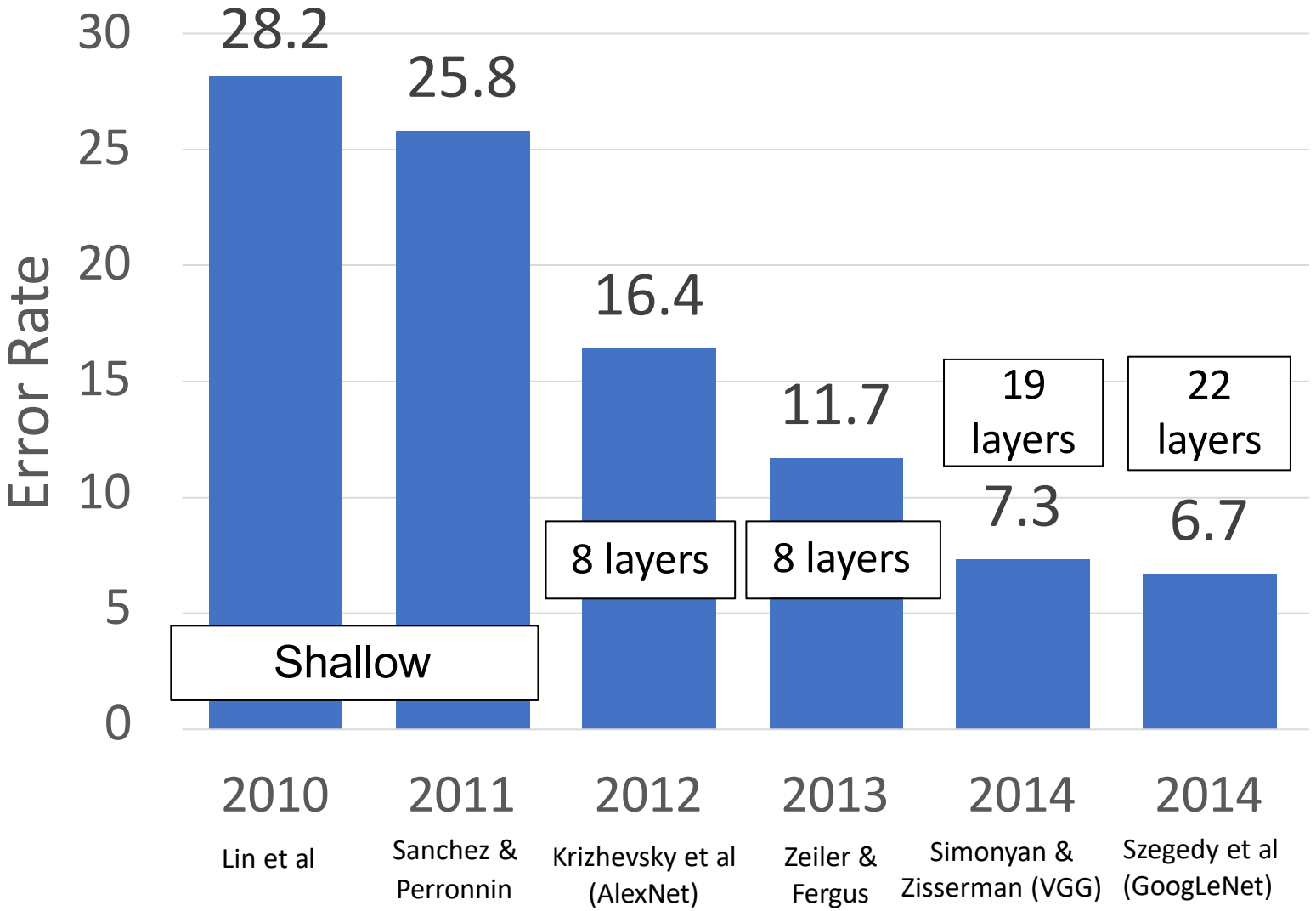
VGG-16 total: 138M (2.3x)



AlexNet total: 0.7 GFLOP

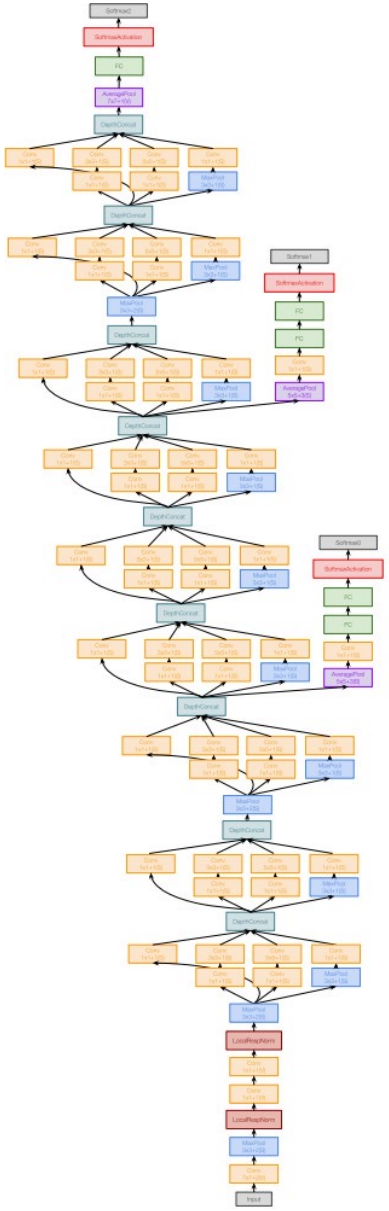
VGG-16 total: 13.6 GFLOP (19.4x)

ImageNet Classification Challenge



GoogLeNet: Focus on Efficiency

- Many innovations for efficiency: reduce parameter count, memory usage, and computation

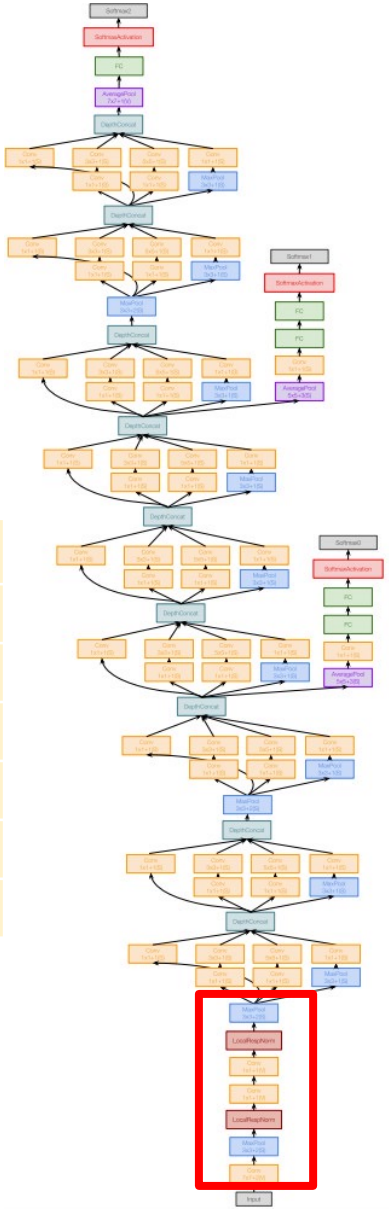


Szegedy et al, “Going deeper with convolutions”, CVPR 2015

Aggressive Stem

- **Stem network** at the start aggressively downsamples input (Recall in VGG-16: Most of the compute was at the start)

	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H/W	memory (KB)	params (K)	flop (M)
conv	3	224	64	7	2	3	64	112	3136	9	118
max-pool	64	112		3	2	1	64	56	784	0	2
conv	64	56	64	1	1	0	64	56	784	4	13
conv	64	56	192	3	1	1	192	56	2352	111	347
max-pool	192	56		3	2	1	192	28	588	0	1



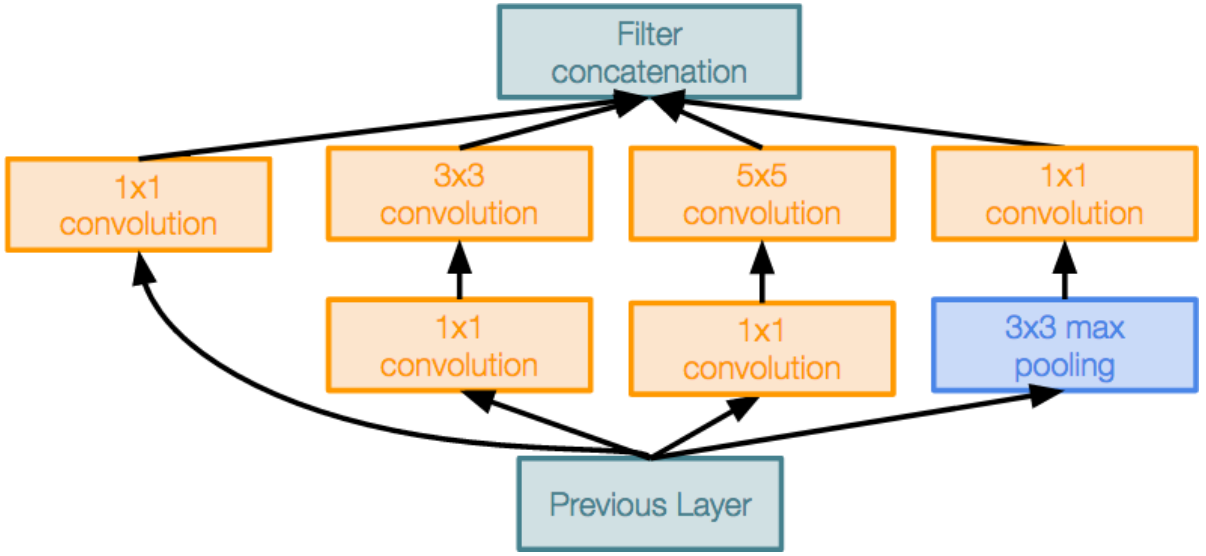
Total from 224 to 28 spatial resolution:
Memory: 7.5 MB
Params: 124K
MFLOP: 418

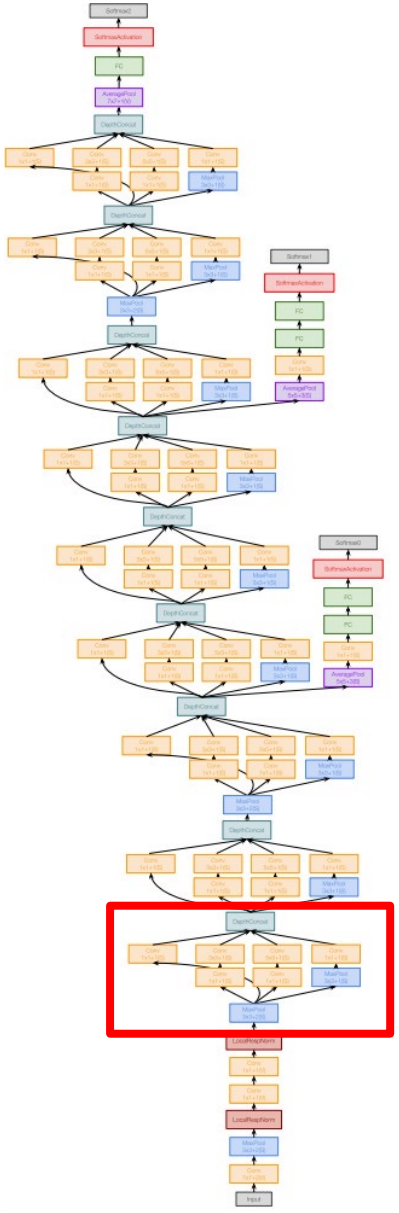
Compare VGG-16:
Memory: 42.9 MB (5.7x)
Params: 1.1M (8.9x)
MFLOP: 7485 (17.8x)

Szegedy et al, "Going deeper with convolutions", CVPR 2015

Inception module

- Parallel branches, different resolutions
- Local structure repeated many times throughout the network
- Uses 1x1 “Bottleneck” layers to reduce channel dimension before expensive conv

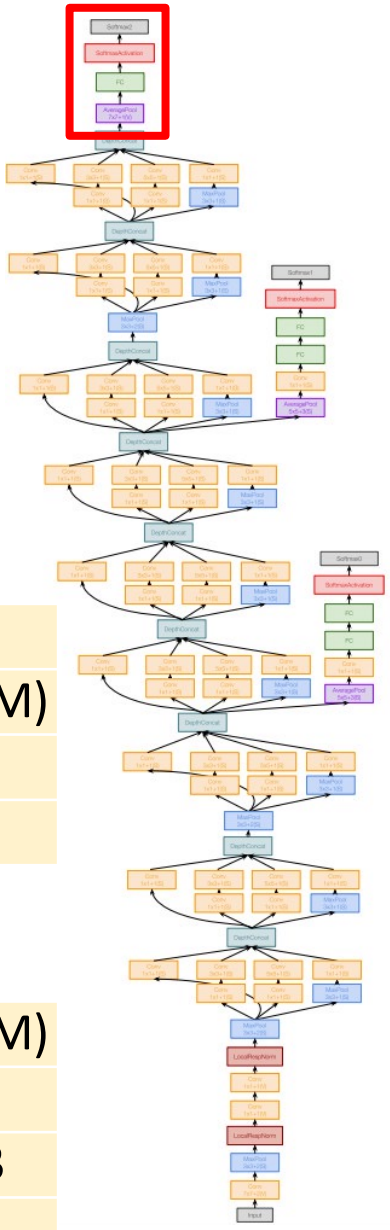




Szegedy et al, “Going deeper with convolutions”, CVPR 2015

Global Average Pooling

- No large FC layers at the end
- Instead uses “global average pooling” to collapse spatial dimensions, and one linear layer to produce class scores
- Recall VGG-16: Most parameters were in the FC layers!



	Input size		Layer				Output size				
Layer	C	H/W	filters	kernel	stride	pad	C	H/W	memory (KB)	params (k)	flop (M)
avg-pool	1024	7		7	1	0	1024	1	4	0	0
fc	1024		1000				1000		0	1025	1

Compare with VGG-16:

Layer	C	H/W	filters	kernel	stride	pad	C	H/W	memory (KB)	params (K)	flop (M)
flatten	512	7					25088		98		
fc6	25088			4096			4096		16	102760	103
fc7	4096			4096			4096		16	16777	17
fc8	4096			1000			1000		4	4096	4

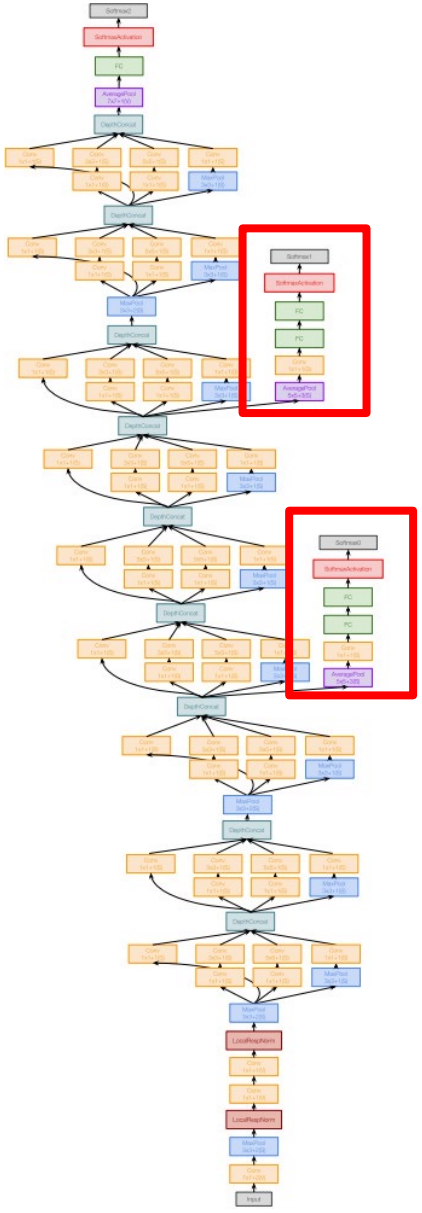
Szegedy et al, “Going deeper with convolutions”, CVPR 2015

Auxiliary Classifiers

- Training using loss at the end of the network didn't work well: Network is too deep, gradients don't propagate cleanly

- As a hack, attach "auxiliary classifiers" at several intermediate points in the network that also try to classify the image and receive loss

- GoogLeNet was before batch normalization! With BatchNorm no longer need to use this trick



Batch Normalization

Idea: “Normalize” the outputs of a layer so they have zero mean and unit variance

Why?

- Helps reduce “internal covariate shift”, improves optimization

We can normalize a batch of activations like this:

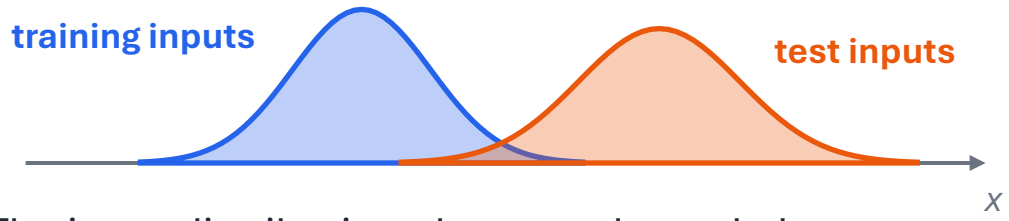
$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

This is a **differentiable function**, so we can use it as an operator in our networks and backprop through it!

Ioffe and Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, ICML 2015

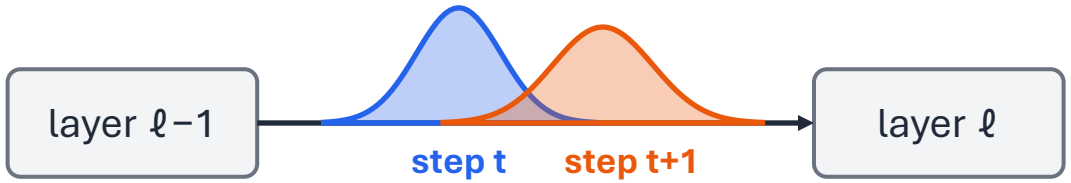
What is “internal covariate shift”?

Covariate shift Shimodaira, 2000



The input distribution changes, the task does not:
$p_{train}(x) \neq p_{test}(x)$, same $p(y \mid x)$
e.g., trained on daytime photos, used at night

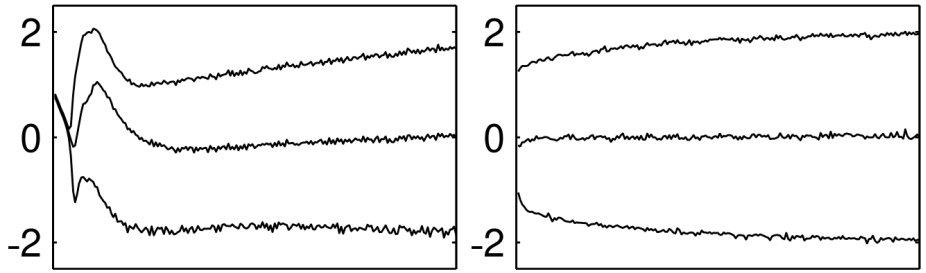
Internal covariate shift Ioffe & Szegedy, 2015



Layer ℓ's inputs are layer $\ell-1$'s outputs: every update of $W_{\ell-1}$ changes their distribution.

BN's claim

Fix each layer's inputs to mean 0, variance 1
→ stable inputs → faster training

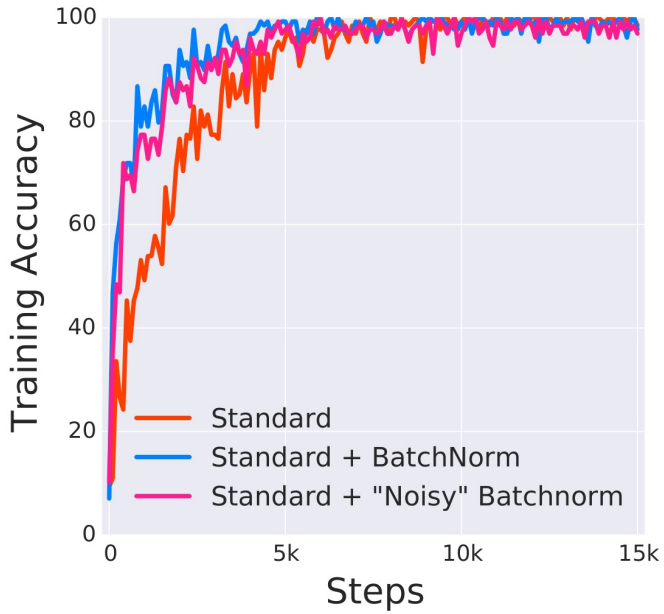


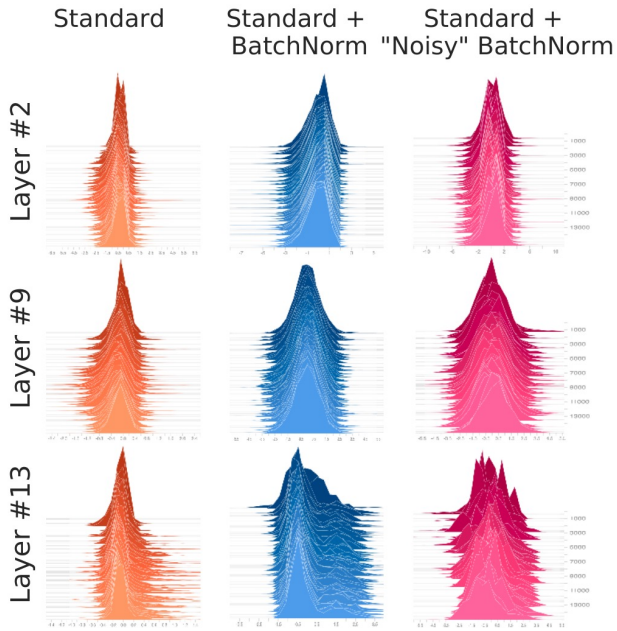
Their evidence (MNIST): one sigmoid's input over training, 15th / 50th / 85th percentiles

Shimodaira, “Improving predictive inference under covariate shift by weighting the log-likelihood function”, 2000; Ioffe and Szegedy, ICML 2015, Fig. 1(b, c)

Internal covariate shift, debunked

Santurkar et al. (NeurIPS 2018) add the shift on purpose: after every BN layer, noise whose mean and variance are re-drawn at every step. Layer inputs now shift more than without BN.



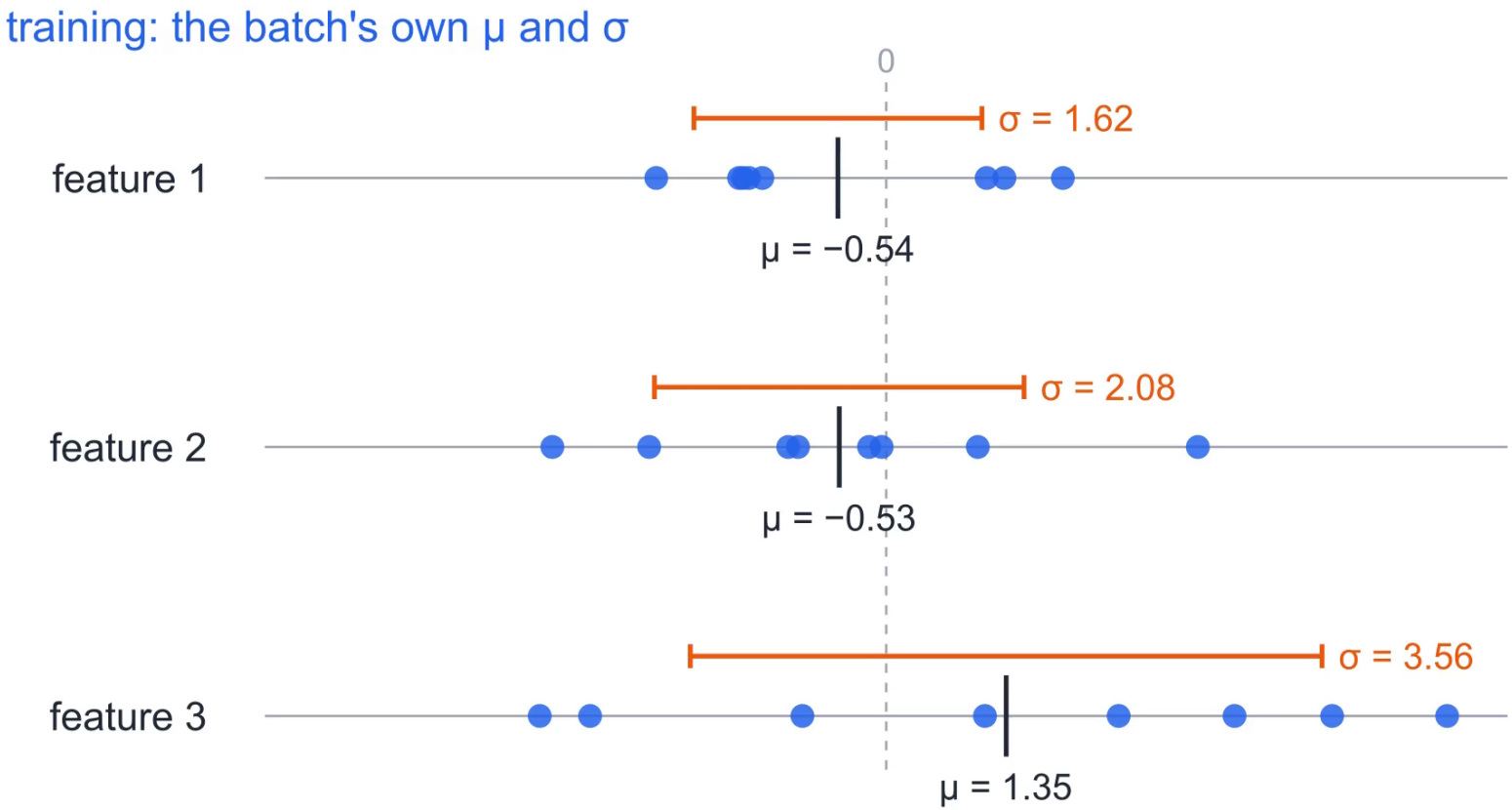


- **Training: noisy BN** is about as fast as **BN**; both beat **no BN** (standard)
- **Layer inputs:** the noisy network's move the most, and it still trains fast
- **Measured directly,** networks with BN often have more internal covariate shift, not less

What BN does instead: a smoother loss. The gradient changes slowly, so it stays accurate over a large step, and larger learning rates train stably.

Santurkar, Tsipras, Ilyas and Madry, “How Does Batch Normalization Help Optimization?”, NeurIPS 2018, Fig. 2 (VGG on CIFAR-10)

Batch normalization, step by step



code: github.com/idansc/dl-course-2026/animations/code/batchnorm_steps.py

Real numbers: the lecture-2 MLP with a BatchNorm layer, 5 epochs on CIFAR-10 (val acc 49.8%); 3 of its 100 features, a batch of 8 test images.

Batch Normalization

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
$\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$

Per-channel std, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

Test-time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
$\beta = \mu$ will recover the identity function!

$$\mu_j = \text{(Running) average of values seen during training}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \text{(Running) average of values seen during training}$$

Per-channel std, shape is D

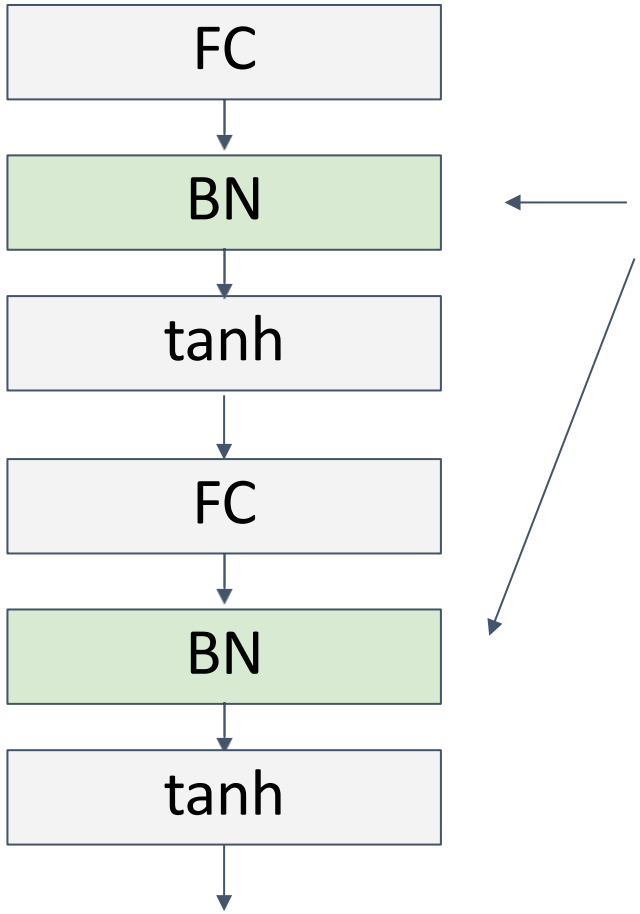
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

Batch Normalization

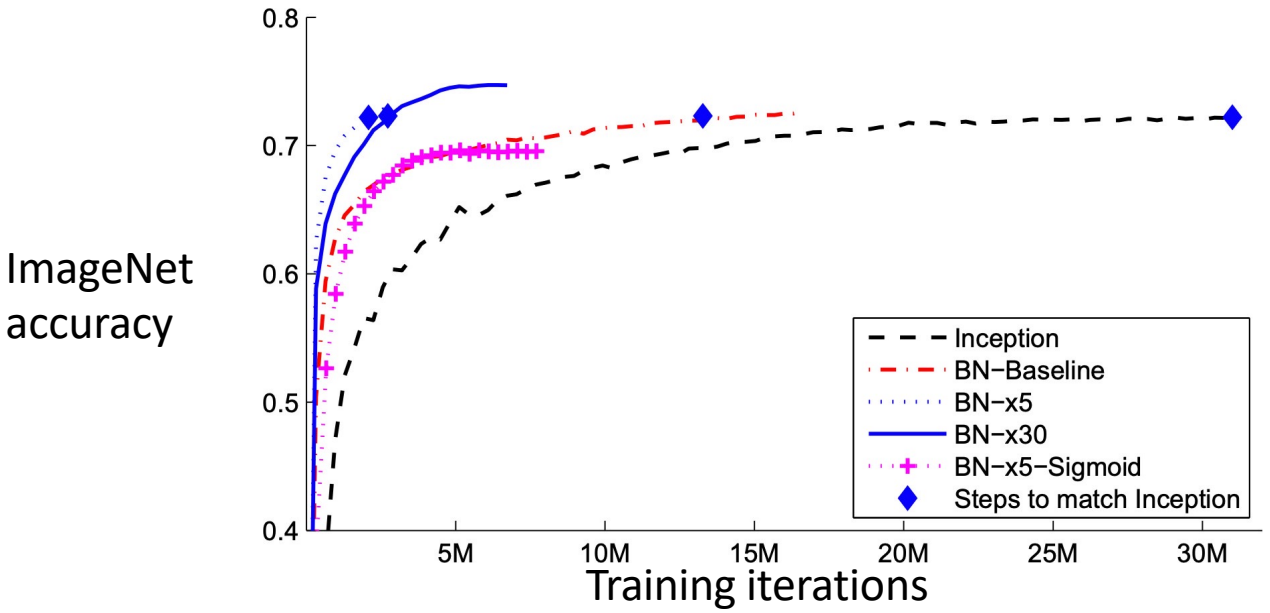


Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

Ioffe and Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, ICML 2015

Batch Normalization Advantages

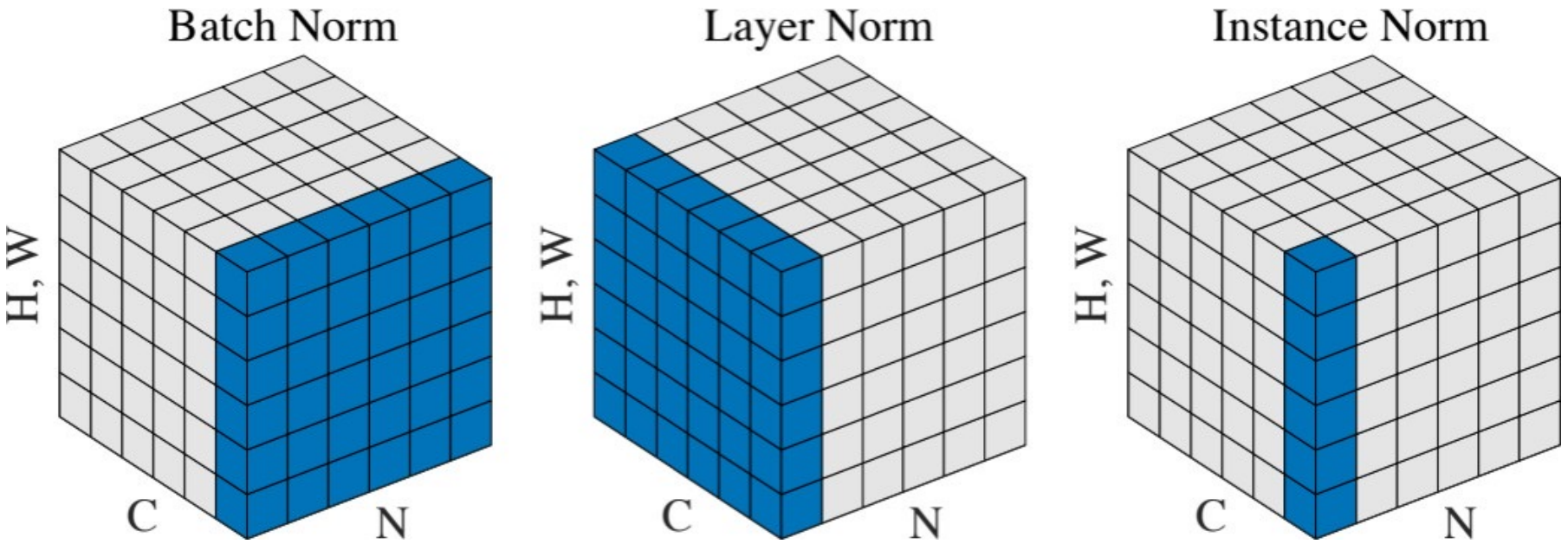
- Makes deep networks **much** easier to train!
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization



But:

- Training and testing behave differently: a common source of bugs
- Depends on the other examples in the batch: noisy with small batches

Other Alternatives of Normalization Layers



per channel, over N, H, W

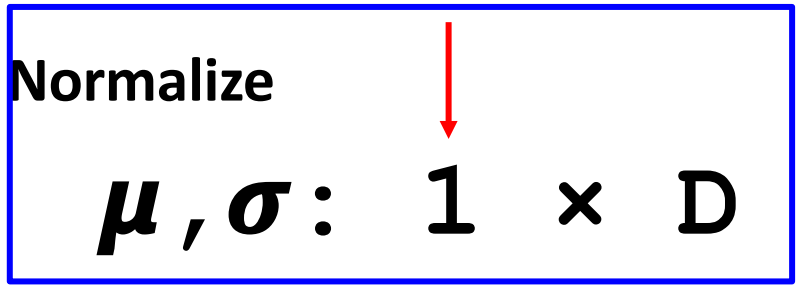
per example, over C, H, W

per example and channel, over H, W

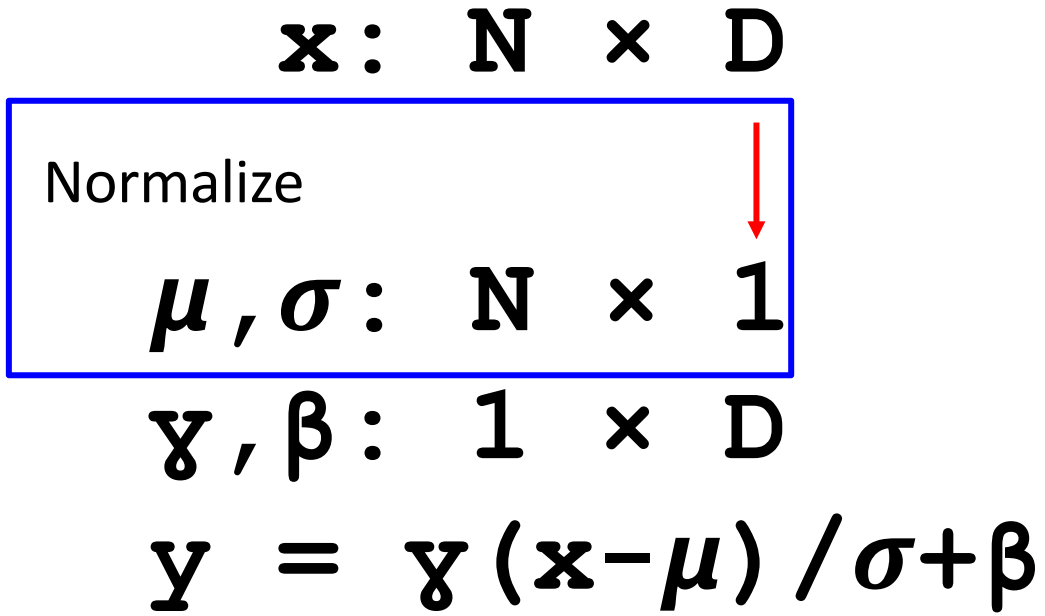
Solves the test-train different behaviour issue

Wu and He, “Group Normalization”, ECCV 2018

Batch Normalization for fully-connected networks

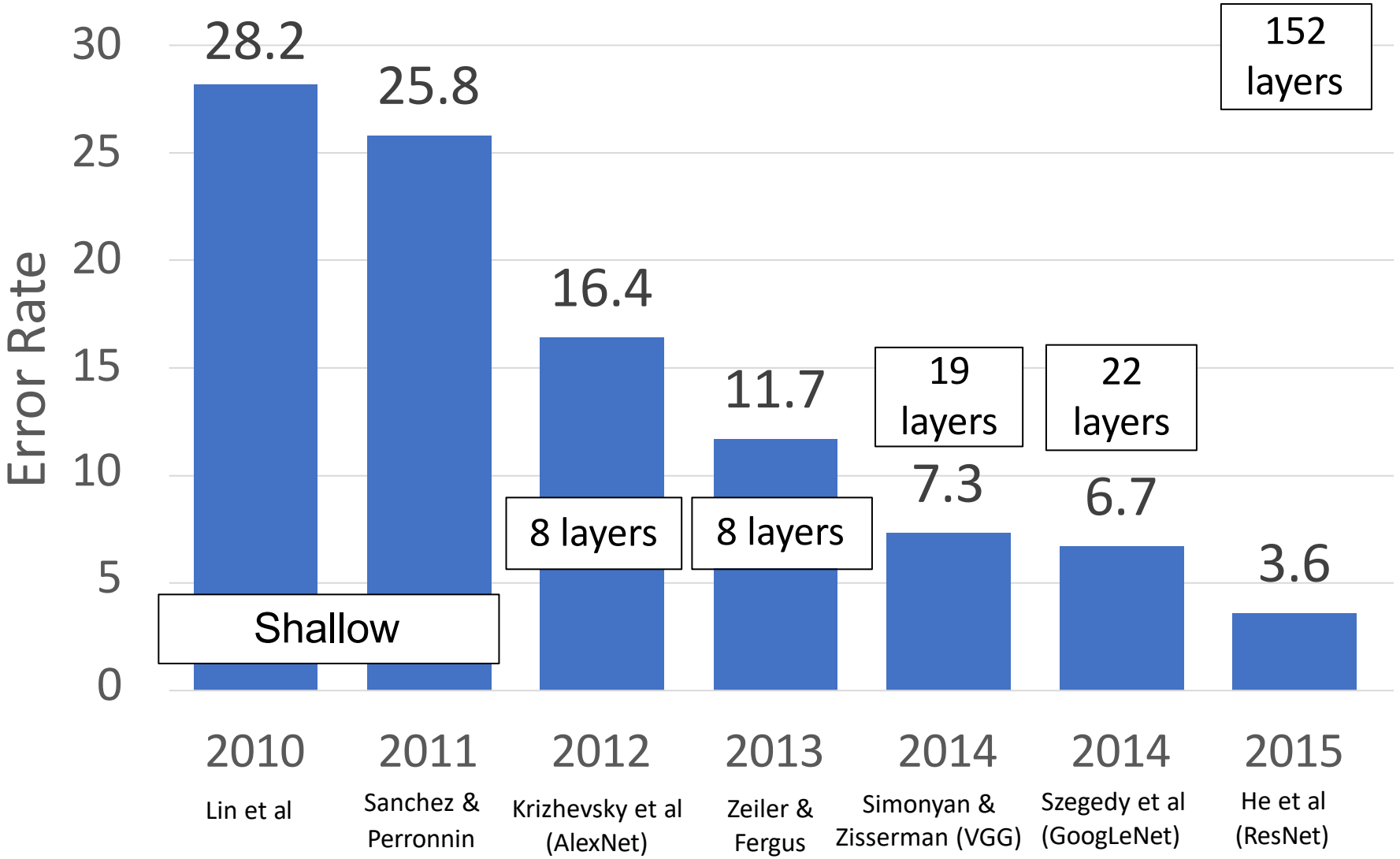


Layer Normalization for fully-connected networks
Same behavior at train and test!
Used in RNNs, Transformers



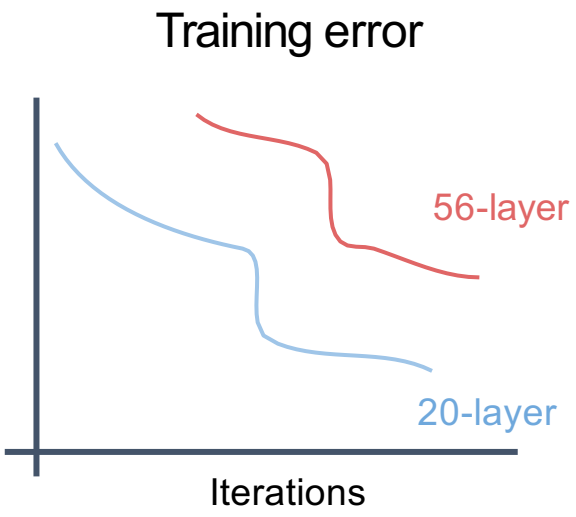
Ba, Kiros, and Hinton, "Layer Normalization", arXiv 2016

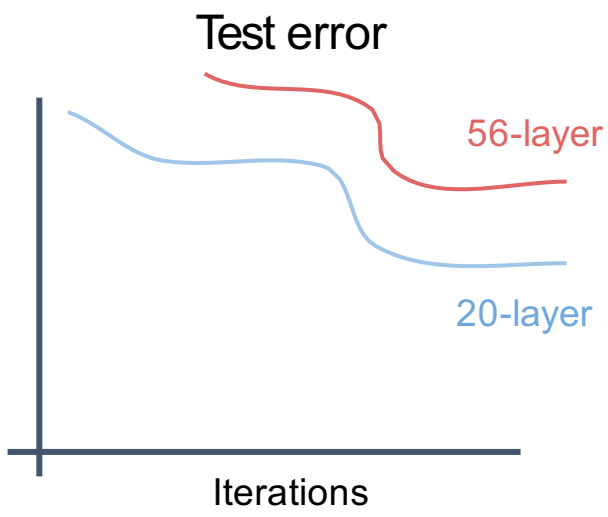
ImageNet Classification Challenge



Residual Networks

Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?





In fact the deep model seems to be **underfitting** since it also performs worse than the shallow model on the training set! It is actually **underfitting**

He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

Residual Networks

- A deeper model can emulate a shallower model: copy layers from shallower model, set extra layers to identity

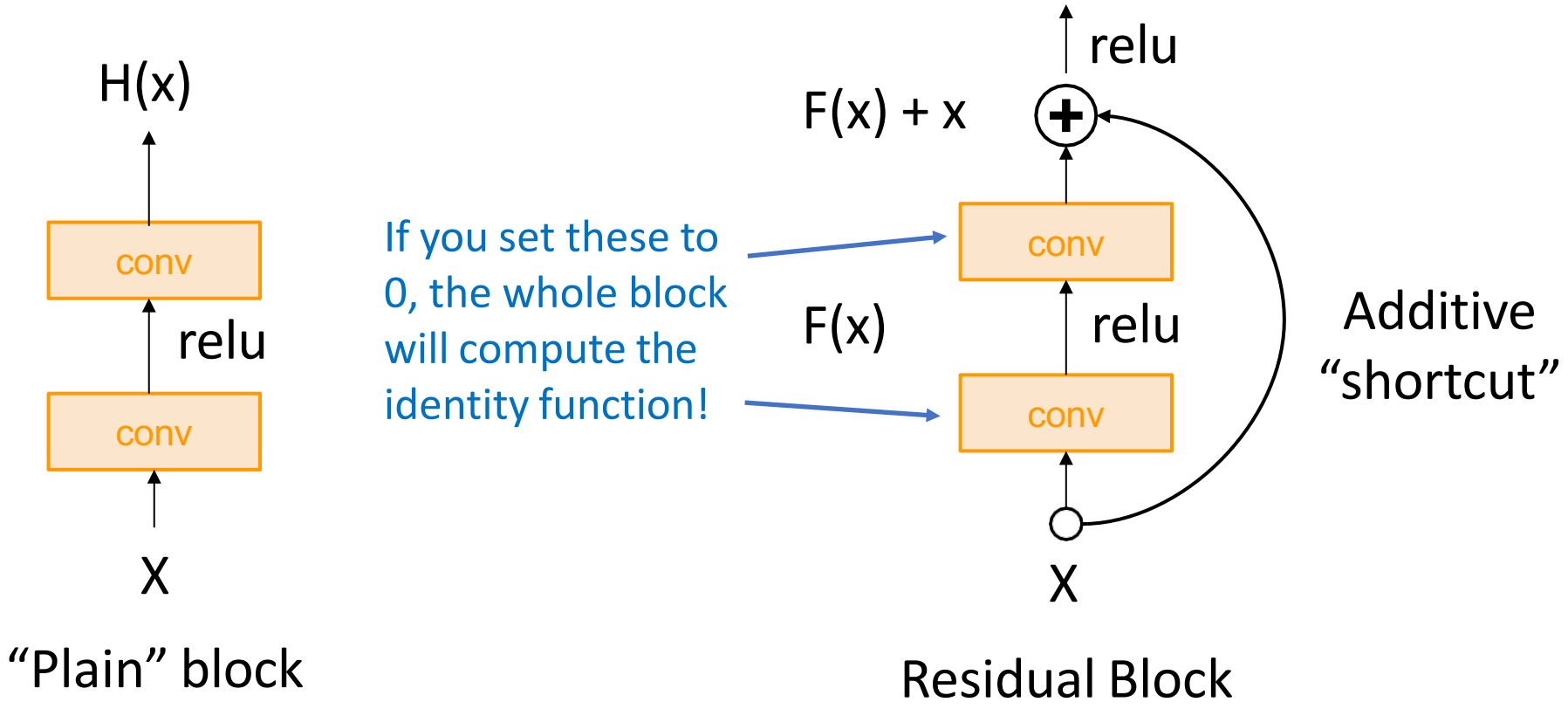
- Thus, deeper models should do at least as good as shallow models

- **Hypothesis**: This is an optimization problem: Deeper models are harder to optimize, and don't learn identity functions to emulate shallow models

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

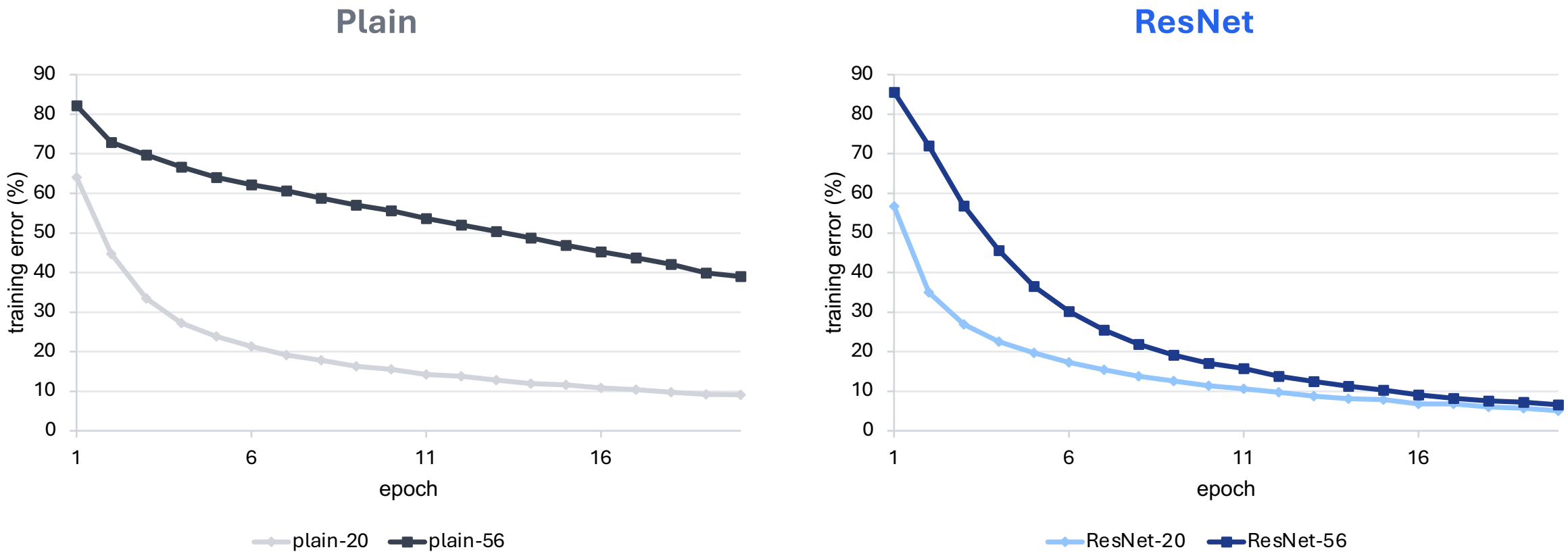
Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

Plain vs residual on CIFAR-10



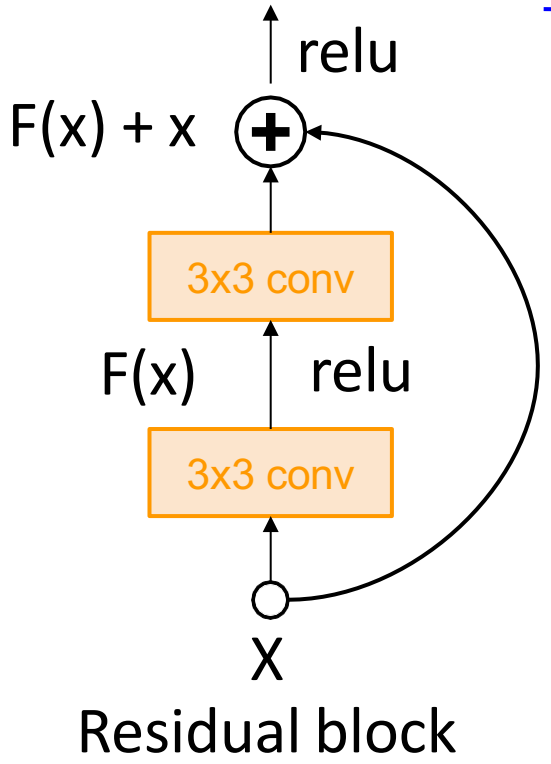
56 layers: plain trains worse than 20, with shortcuts about as well

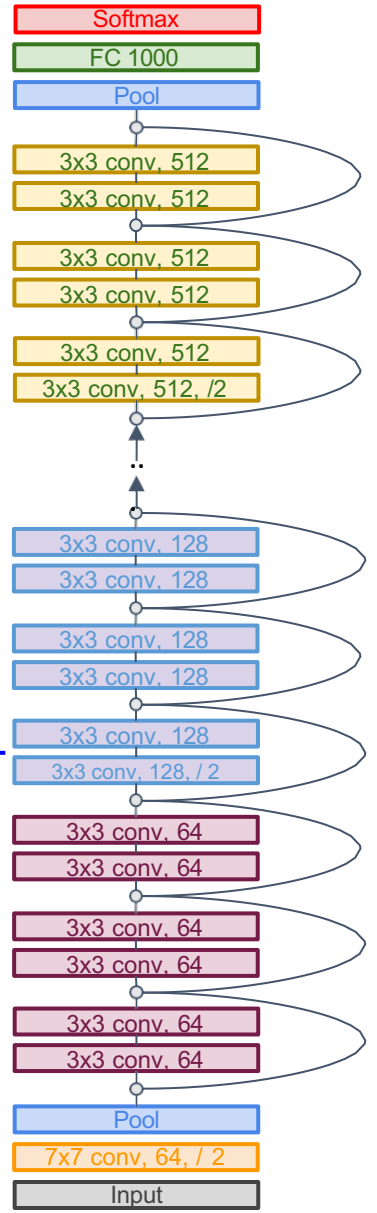
code: github.com/idansc/dl-course-2026/animations/code/resnet_degradation.py

Reproduces He et al., “Deep Residual Learning for Image Recognition”, CVPR 2016, Fig. 6, with the lecture-2 training loop (no augmentation, 20 epochs).

Residual Networks

- A residual network is a stack of many residual blocks
- Regular design, like VGG: each residual block has two 3x3 conv
- Network is divided into **stages**: the first block of each stage halves the resolution (with stride-2 conv) and doubles the number of channels





He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

Residual Networks

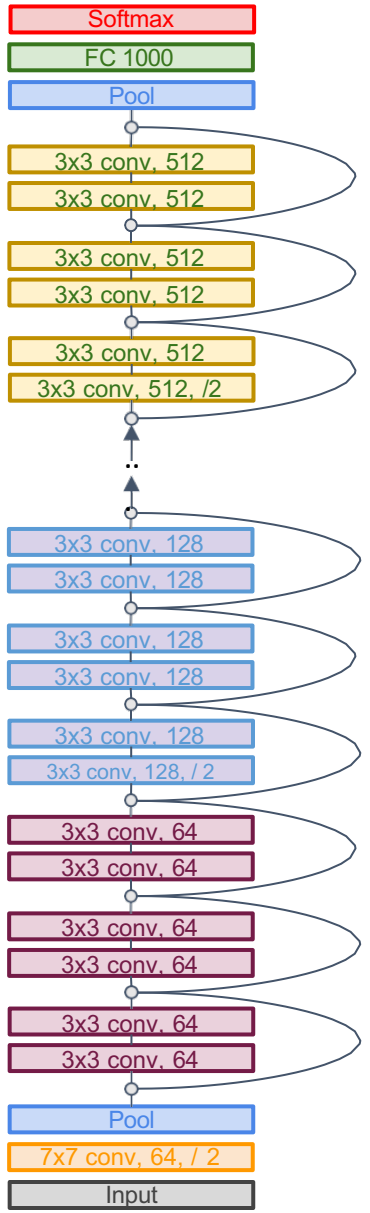
ResNet-18:
Stem: 1 conv layer
Stage 1 (C=64): 2 res. block = 4 conv
Stage 2 (C=128): 2 res. block = 4 conv
Stage 3 (C=256): 2 res. block = 4 conv
Stage 4 (C=512): 2 res. block = 4 conv
Linear

ImageNet top-5 error: 8.58
GFLOP: 3.6

ResNet-34:
Stem: 1 conv layer
Stage 1: 3 res. block = 6 conv
Stage 2: 4 res. block = 8 conv
Stage 3: 6 res. block = 12 conv
Stage 4: 3 res. block = 6 conv Linear

ImageNet top-5 error: 8.58
GFLOP: 3.6

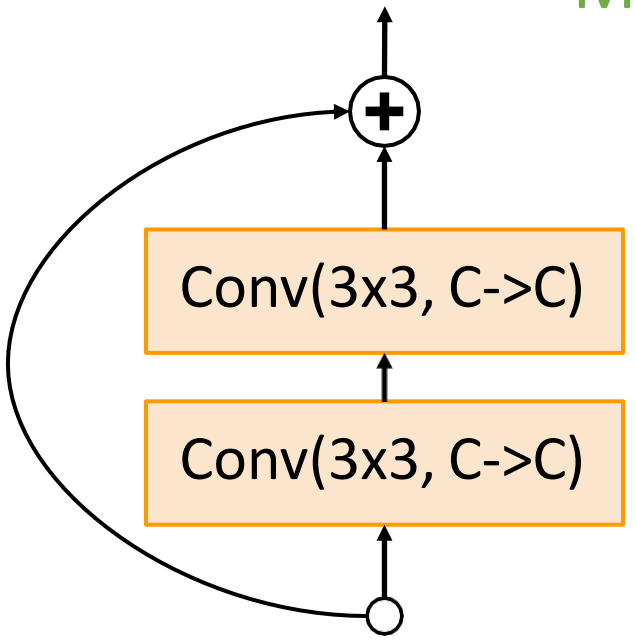
VGG-16:
ImageNet top-5 error: 9.62 GFLOP: 13.6



He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

Residual Networks: Bottleneck Block

More layers, less computational cost!



FLOPs: $9HWC^2$

FLOPs: $9HWC^2$

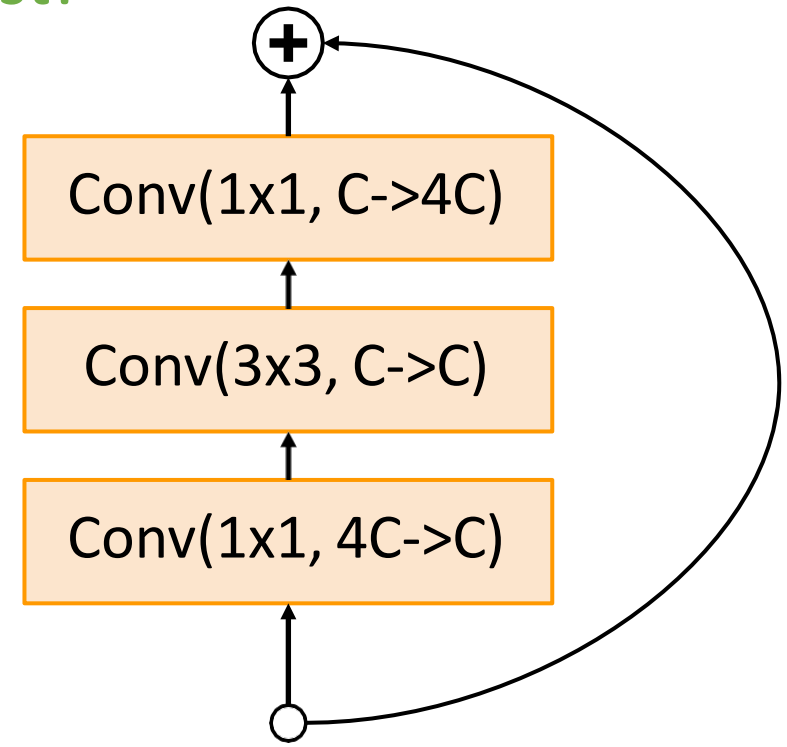
"Basic"
Residual block

Total FLOPs:
$18HWC^2$

FLOPs: $4HWC^2$

FLOPs: $9HWC^2$

FLOPs: $4HWC^2$



Total FLOPs:
$17HWC^2$

"Bottleneck"
Residual block

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

Residual Networks

- Deeper ResNet-101 and ResNet-152 models are more accurate, but also more computationally heavy

			Stage 1		Stage 2		Stage 3		Stage 4				
	Block type	Stem layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	FC layers	GFLOP	ImageNet top-5 error
ResNet-18	Basic	1	2	4	2	4	2	4	2	4	1	1.8	10.92
ResNet-34	Basic	1	3	6	4	8	6	12	3	6	1	3.6	8.58
ResNet-50	Bottle	1	3	9	4	12	6	18	3	9	1	3.8	7.13
ResNet-101	Bottle	1	3	9	4	12	23	69	3	9	1	7.6	6.44
ResNet-152	Bottle	1	3	9	8	24	36	108	3	9	1	11.3	5.94

He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

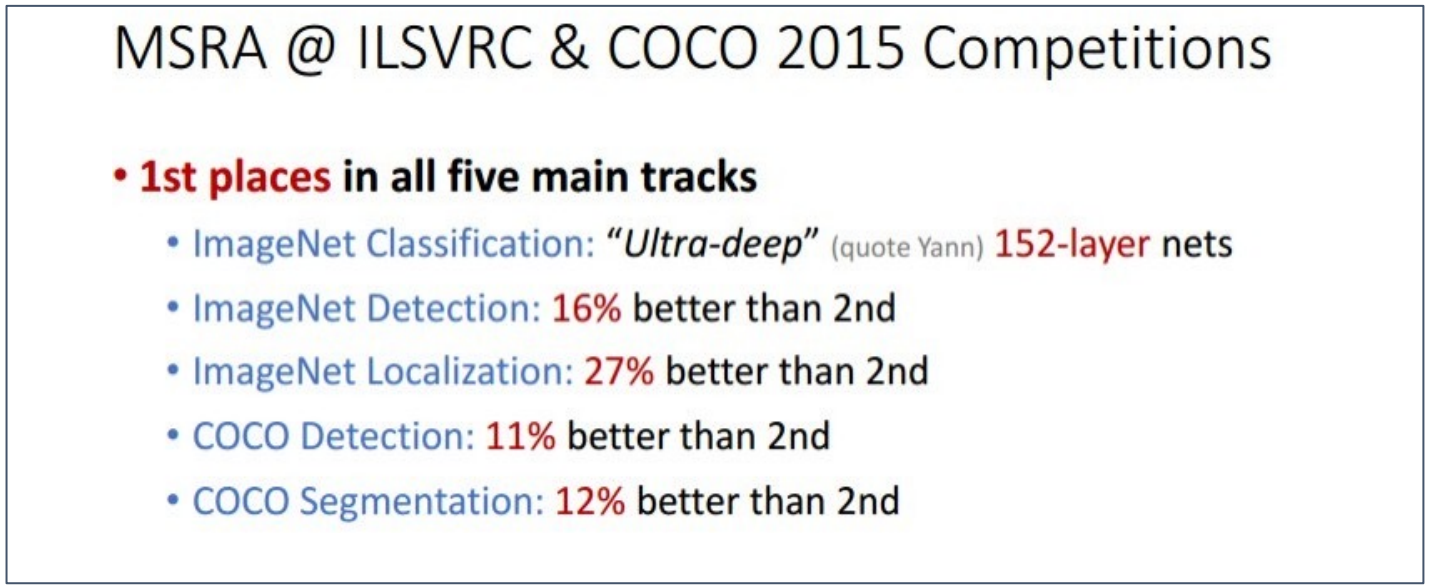
ResNet: Able to train very deep networks

- Swept 1st place in all ILSVRC and COCO 2015 competitions
- Still widely used today!

353,189 citations

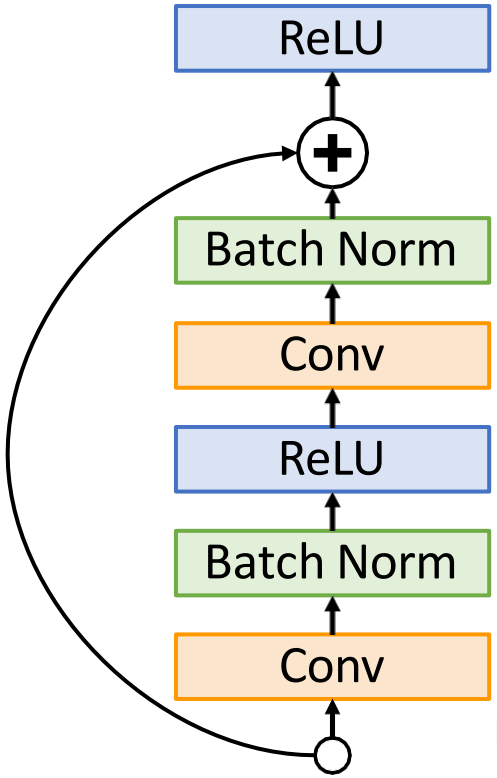
the most-cited paper of the 21st century (Nature, April 2025)

Google Scholar, 27 Oct 2026



Improving the Block Design

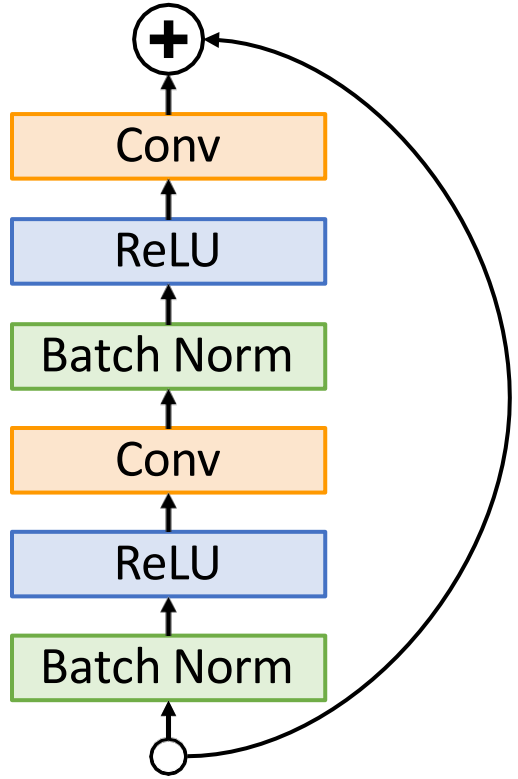
Original ResNet block



Note ReLU **after** residual:

Output is always ≥ 0: the shortcut path goes through a ReLU in every block!

"Pre-Activation" ResNet Block



Note ReLU **inside** residual:

Can learn true identity function by setting Conv weights to zero!

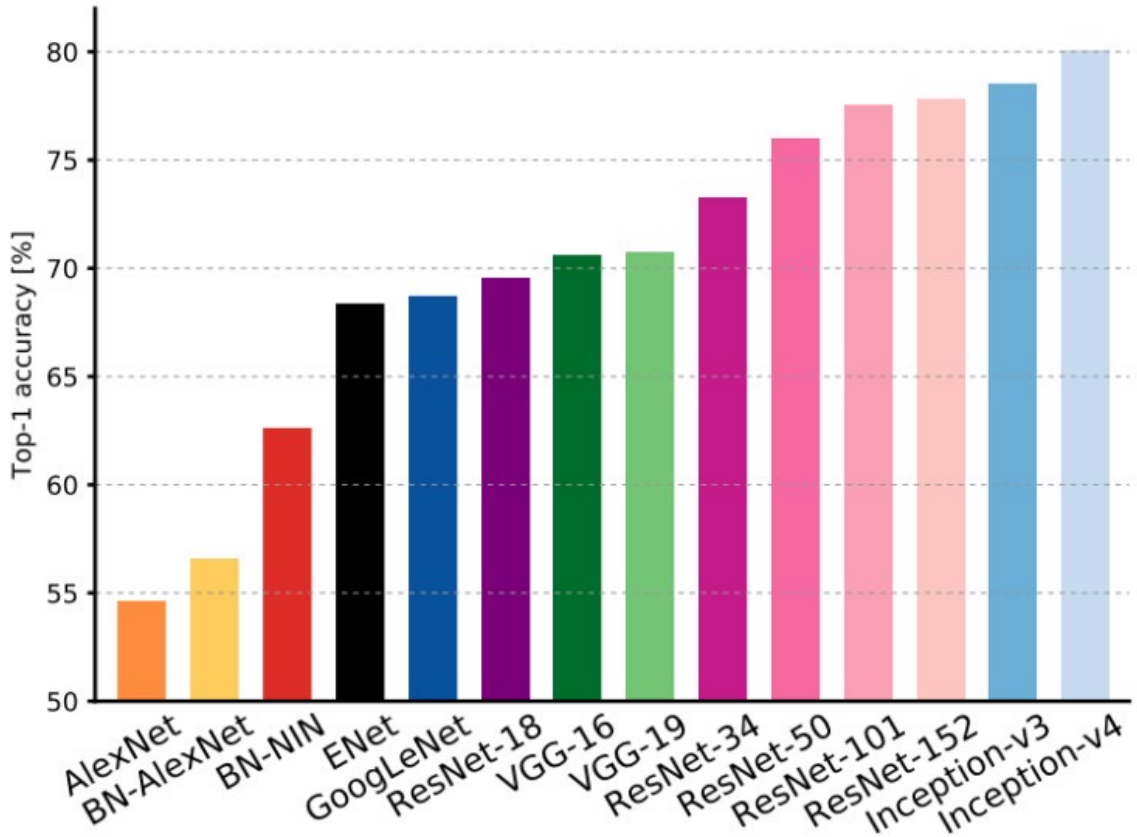
Pre-activation, unrolled: $x_L = x_l + \sum_{l \le i < L} F(x_i)$

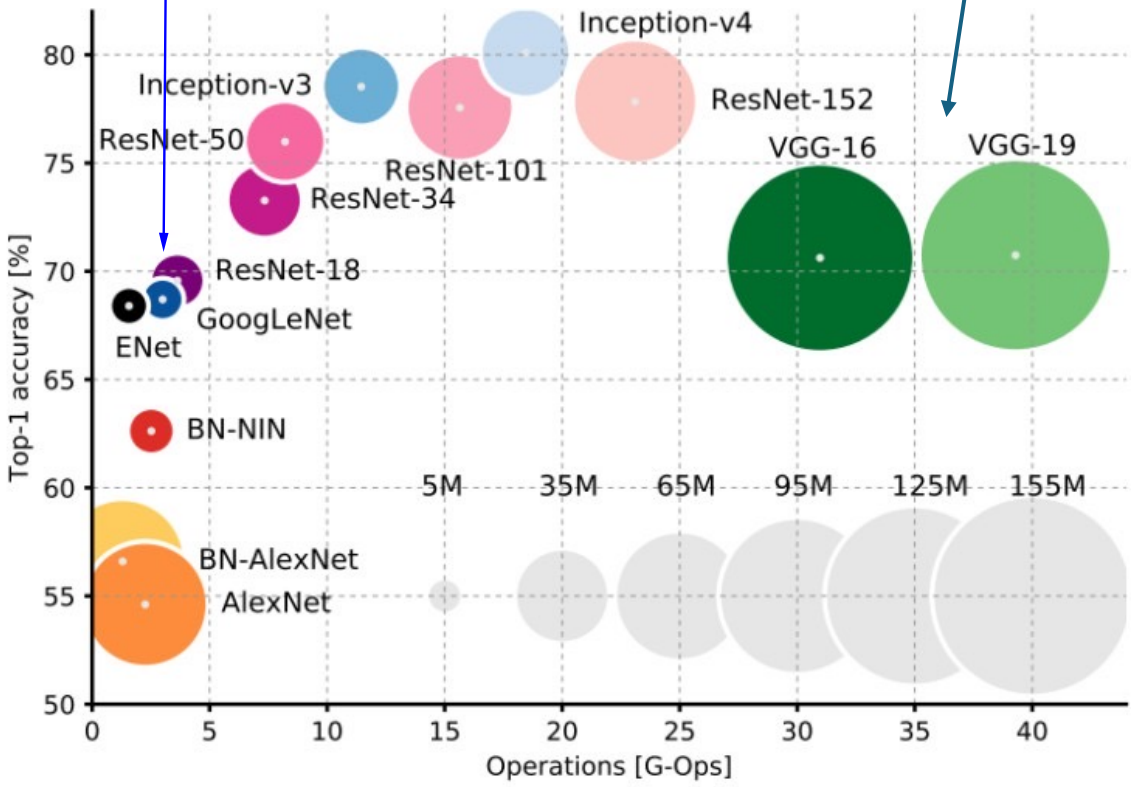
He et al, "Identity mappings in deep residual networks", ECCV 2016

Comparing Complexity

GoogLeNet: Very efficient!

VGG: Highest memory, most operations





anziani et al, “An analysis of deep neural network models for practical applications”, 2017

Grouped Convolution

Convolution with groups=1:
Normal convolution

Input: C_{in} x H x W
Weight: C_{out} x C_{in} x K x K
Output: C_{out} x H' x W'
FLOPs: $C_{out}C_{in}K^2HW$

All convolutional kernels touch all C_{in} channels of the input

Depthwise Convolution

Special case: G=C_{in}, C_{out} = nC_{in}

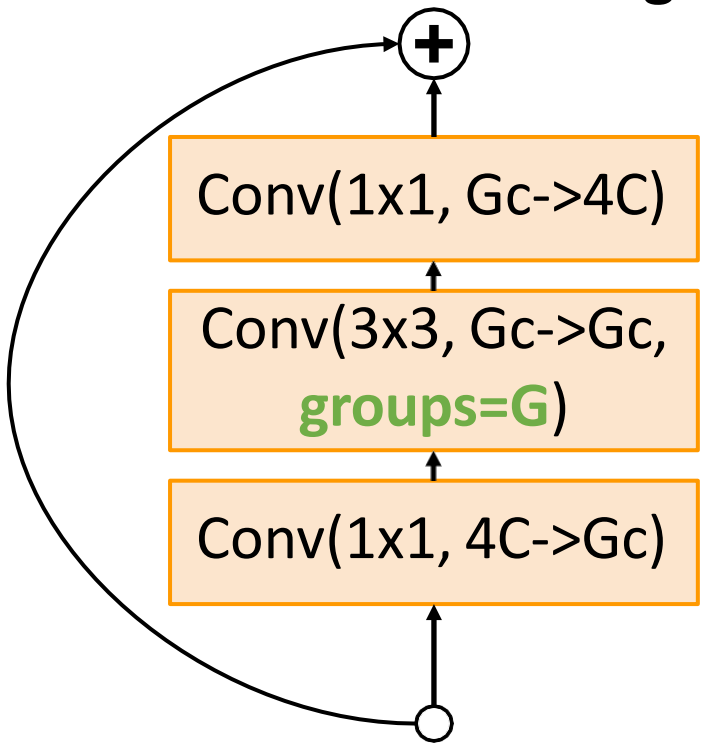
Each input channel is convolved with n different K x K filters to produce n output channels

Convolution with groups=G:
G parallel conv layers; each "sees" C_{in}/G input channels and produces C_{out}/G output channels

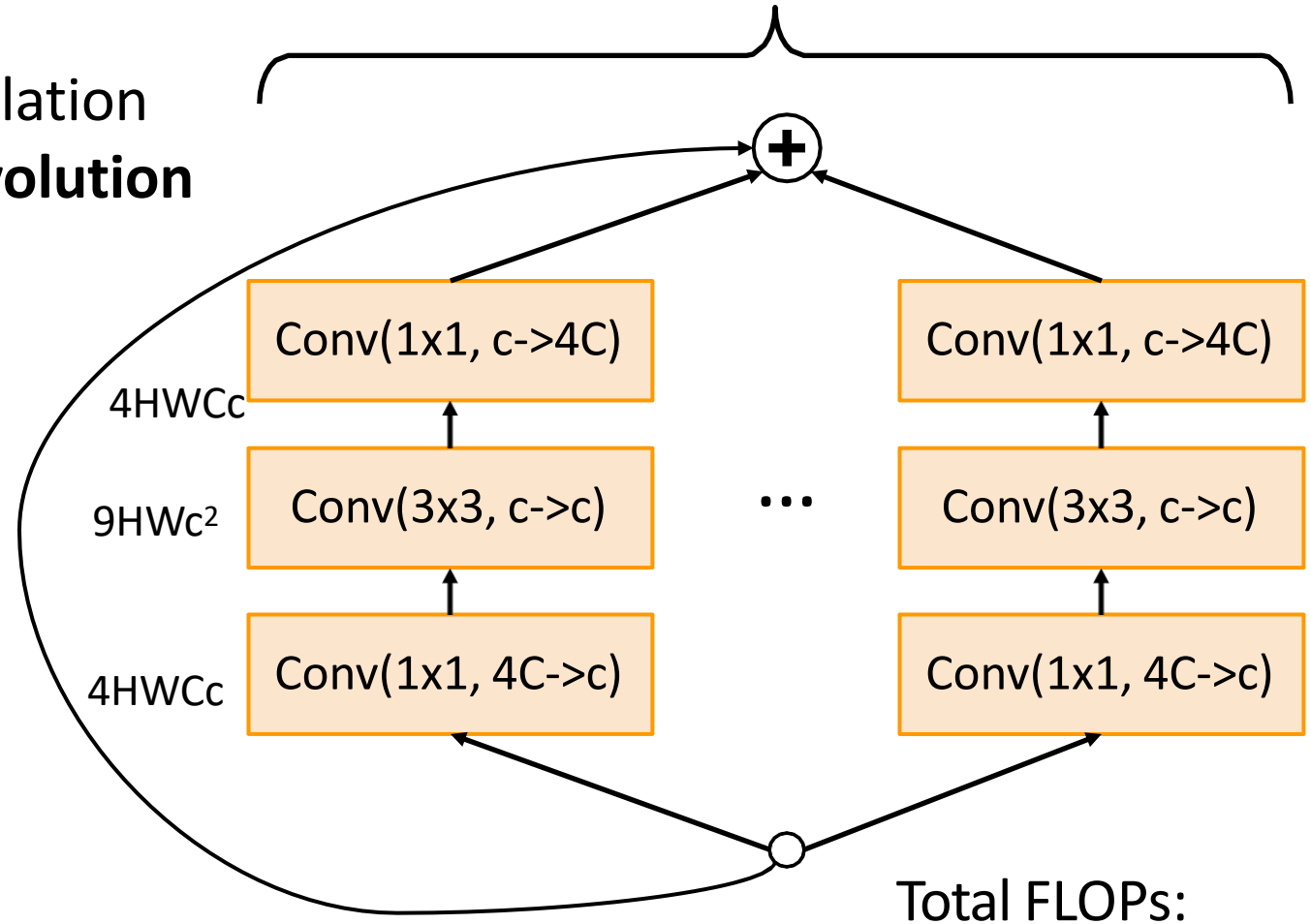
Input: C_{in} x H x W
Split to G x [(C_{in} / G) x H x W]
Weight: G x (C_{out} / G) x (C_{in} x G) x K x K
G parallel convolutions
Output: G x [(C_{out} / G) x H' x W']
Concat to C_{out} x H' x W'
FLOPs: $C_{out}C_{in}K^2HW/G$

ResNeXt

Equivalent formulation
with **grouped convolution**



ResNeXt block:
Grouped convolution



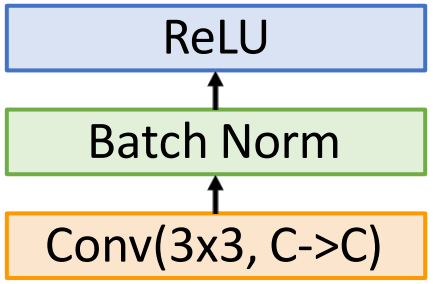
Total FLOPs:
$(8Cc + 9c^2)*HWG$

Xie et al, “Aggregated residual transformations for deep neural networks”, CVPR 2017

MobileNets: Tiny Networks (For Mobile Devices)

Standard Convolution Block

Total cost: $9C^2HW$



$9C^2HW$

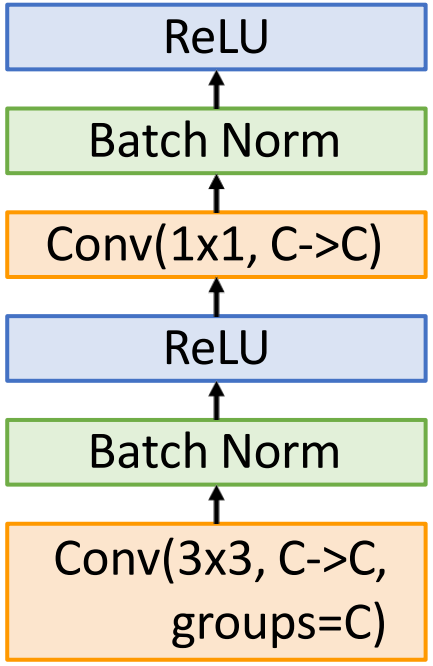
Speedup = $9C^2/(9C+C^2)$

= $9C/(9+C)$

=> 9 (as C->inf)

Depthwise Separable Convolution

Total cost: $(9C + C^2)HW$



C^2HW "Pointwise Convolution"

$9CHW$ "Depthwise Convolution"

Howard et al, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", 2017

Summary: CNN Architectures

- Early work (AlexNet -> ZFNet -> VGG) shows that **bigger networks work better**
- GoogLeNet one of the first to focus on **efficiency** (aggressive stem, 1x1 bottleneck convolutions, global avg pool instead of FC layers)
- ResNet showed us how to train extremely deep networks – limited only by GPU memory! Started to show diminishing returns as networks got bigger
- After ResNet: **Efficient networks** became central: how can we improve the accuracy without increasing the complexity?
- Lots of **tiny networks** aimed at mobile devices: MobileNet, ShuffleNet, etc
- Since 2020: **Vision Transformers** rival CNNs; ConvNeXt (2022) modernizes ResNet

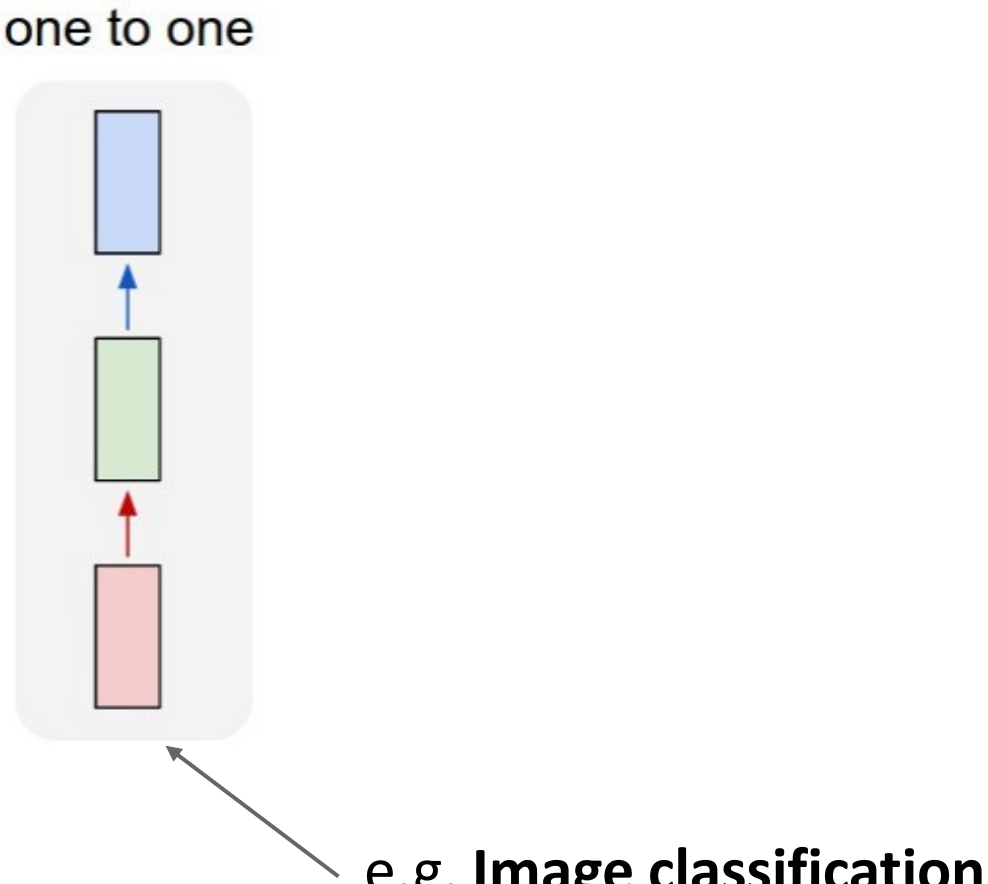
Part 2 · RNNs and LSTMs: sharing over time



In this part

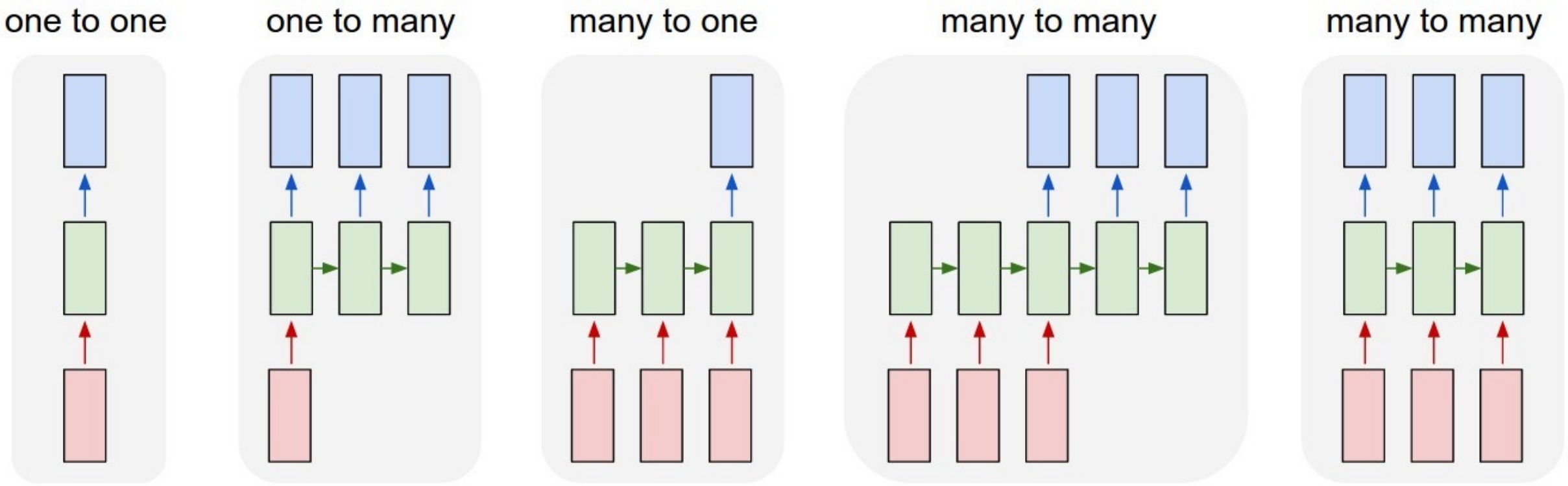
- One weight matrix, applied at every time step
- Language modeling and image captioning
- Vanishing and exploding gradients through time
- LSTM: an additive cell state

So far: “Feedforward” Neural Networks



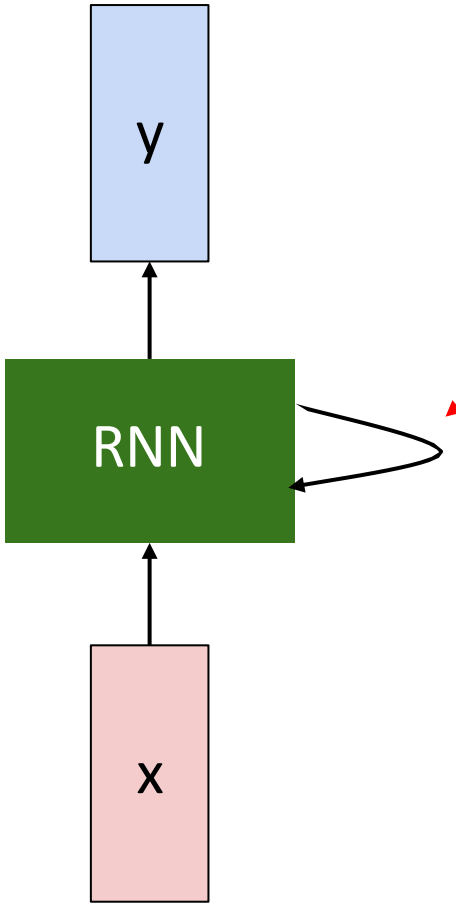
e.g. **Image classification**
Image -> Label

Recurrent Neural Networks: Process Sequences



e.g. **Per-frame video classification:**
Sequence of images -> Sequence of labels

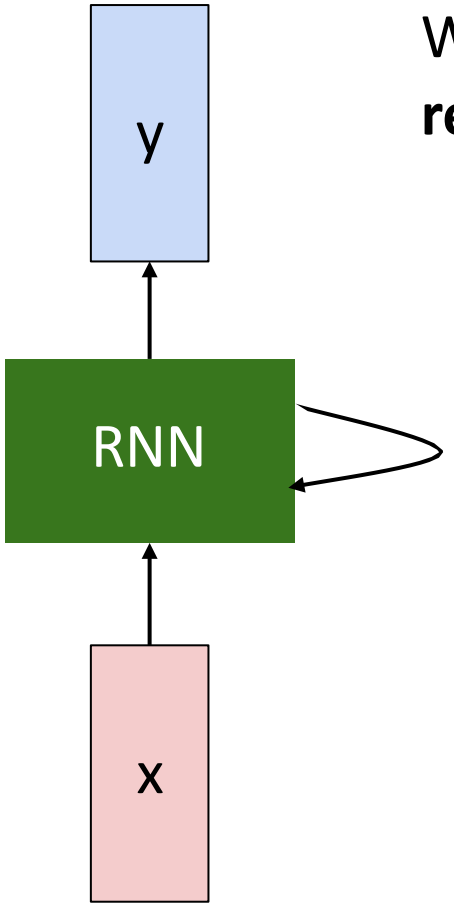
Recurrent Neural Networks



Key idea: RNNs have an "internal state" that is updated as a sequence is processed

Recurrent Neural Networks

Notice: the same function and the same set of parameters are used at every time step.



We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

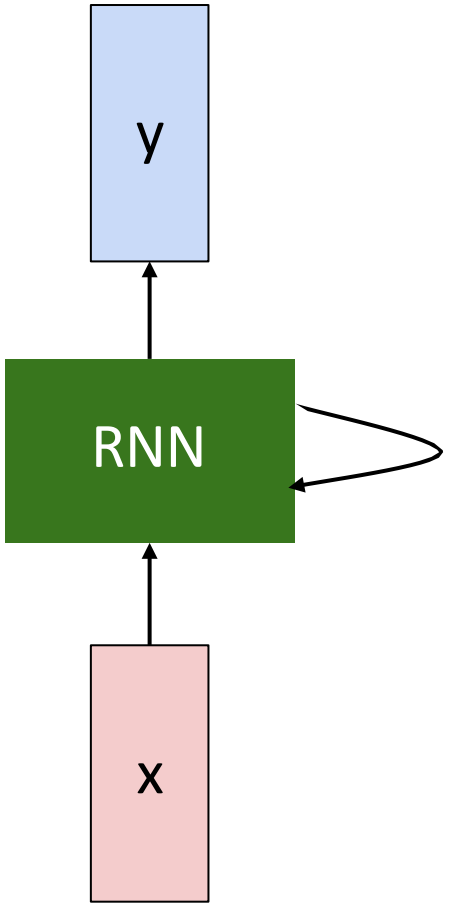
new state

some function with parameters W

old state

input vector at some time step

(Vanilla) Recurrent Neural Networks



The state consists of a single *"hidden"* vector **h**:

$$h_t = f_W(h_{t-1}, x_t)$$

(also bias term)

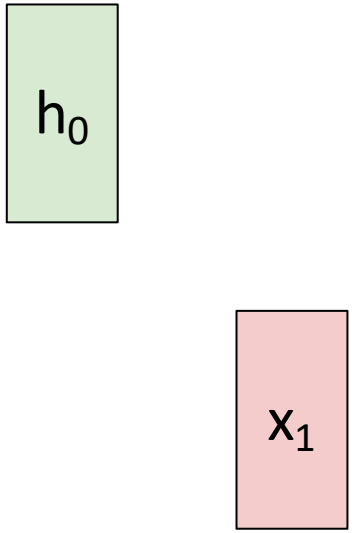
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Sometimes called a "Vanilla RNN" or an "Elman RNN" after Prof. Jeffrey Elman

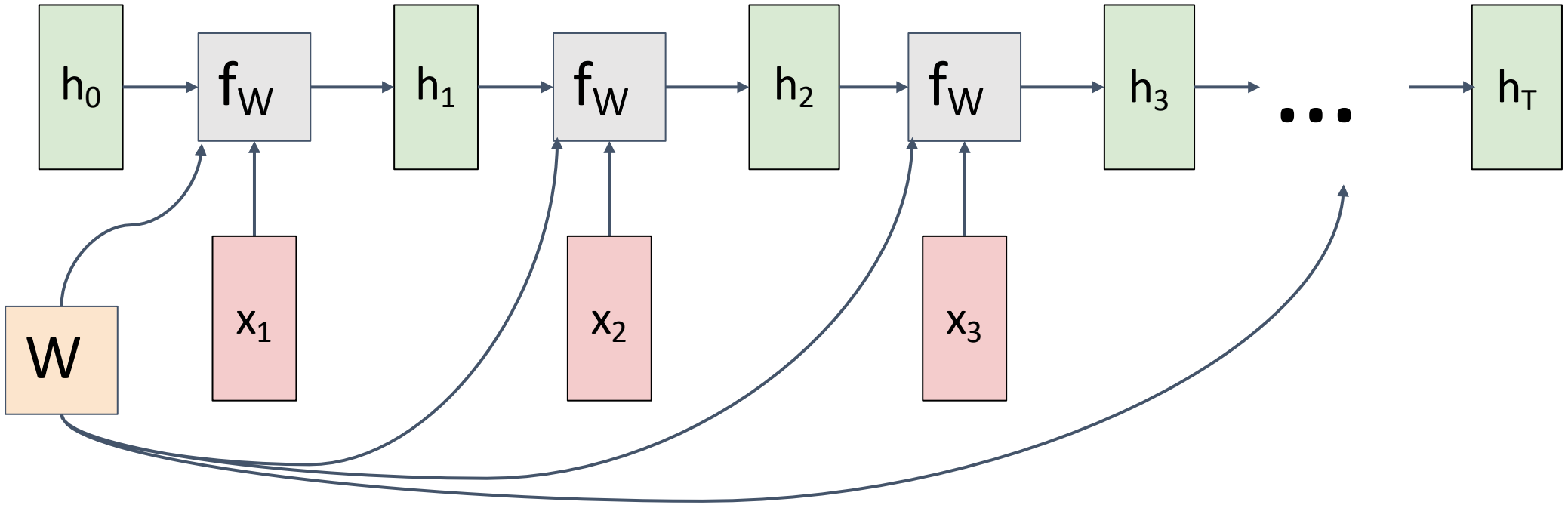
RNN Computational Graph

Initial hidden state
Either set to all 0,
Or learn it

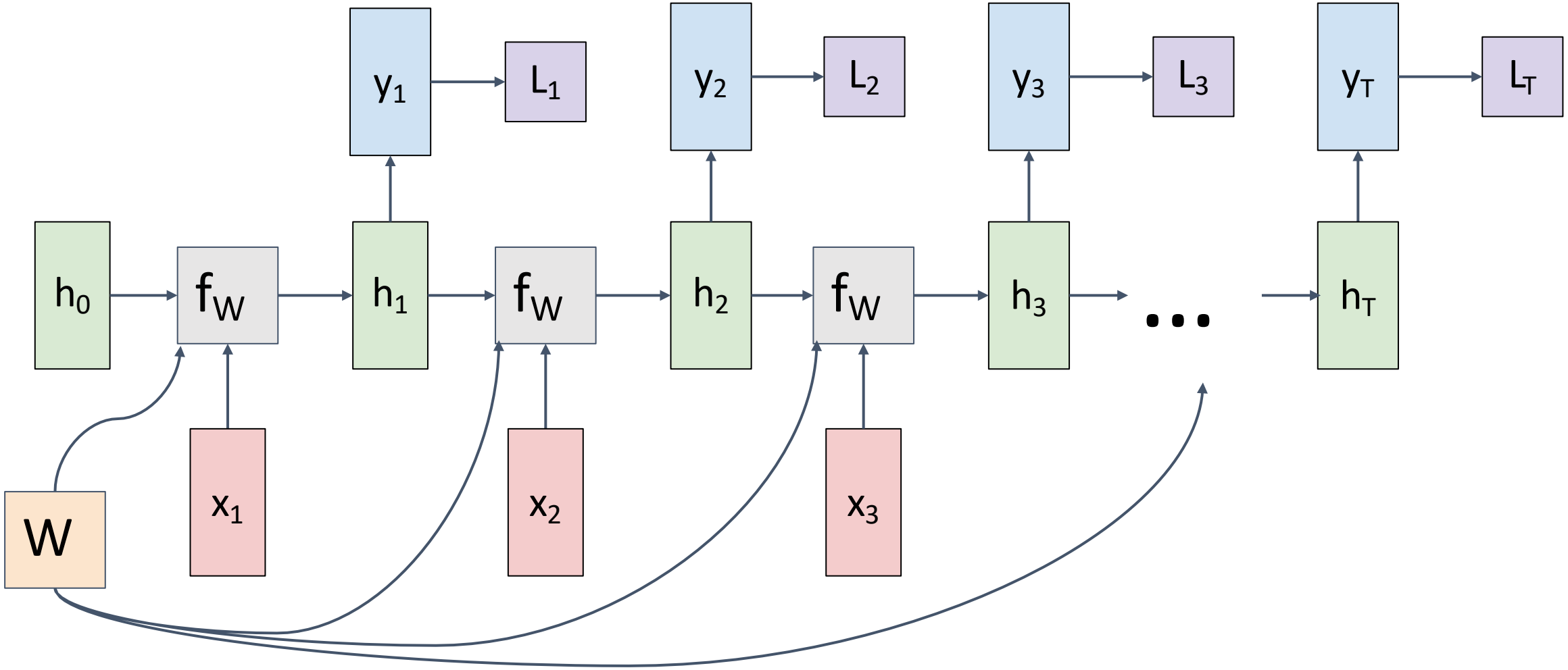


RNN Computational Graph

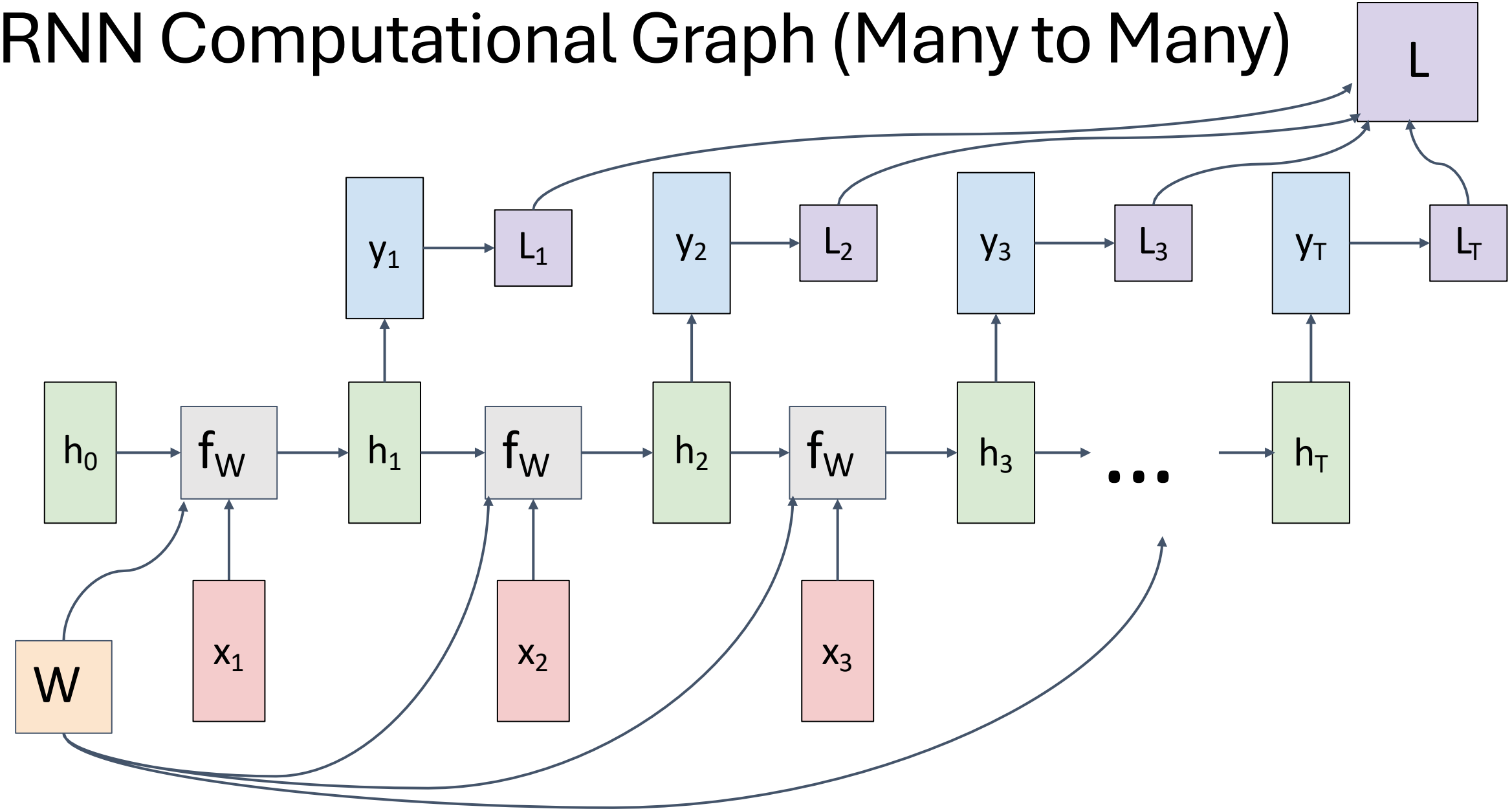
Re-use the same weight matrix at every time-step



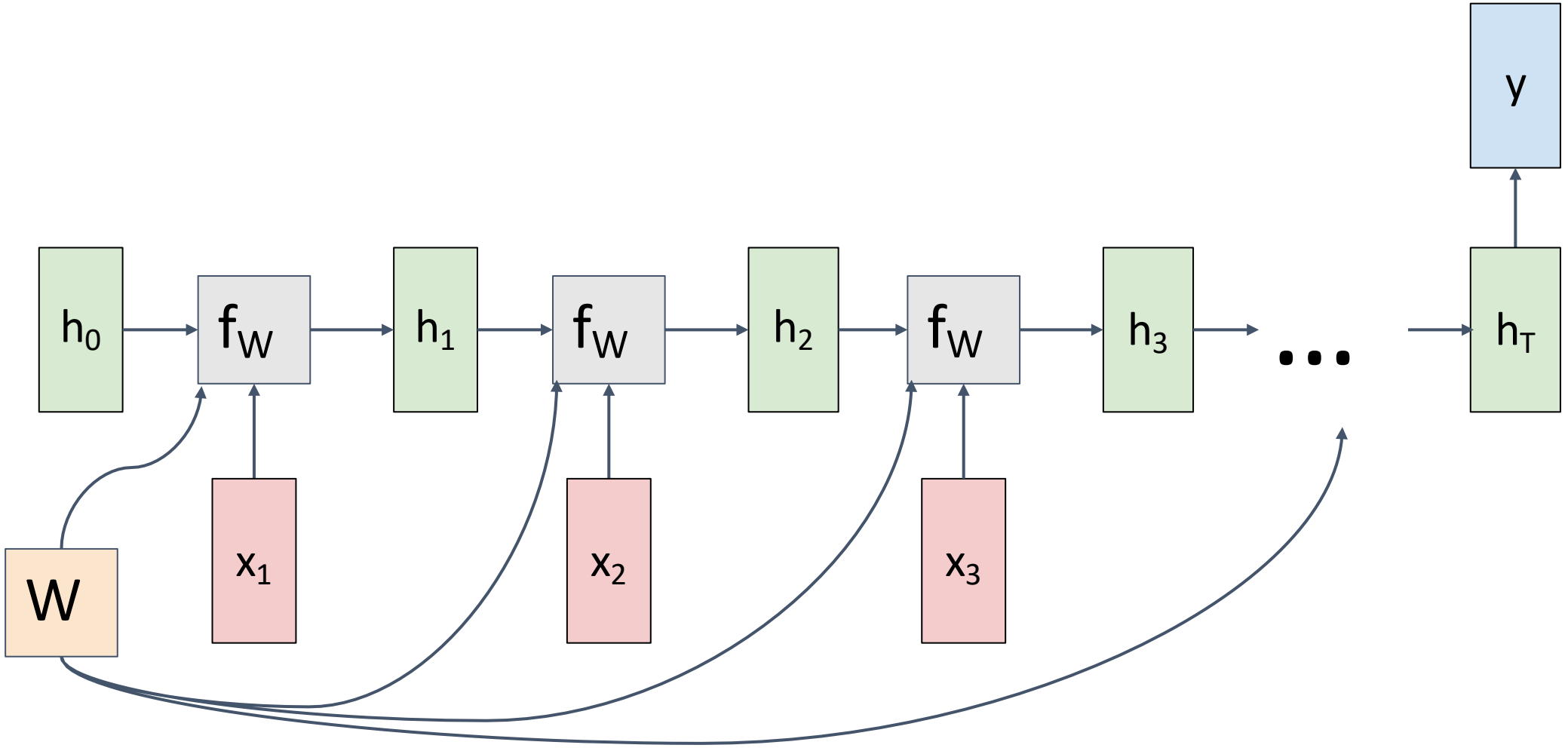
RNN Computational Graph (Many to Many)



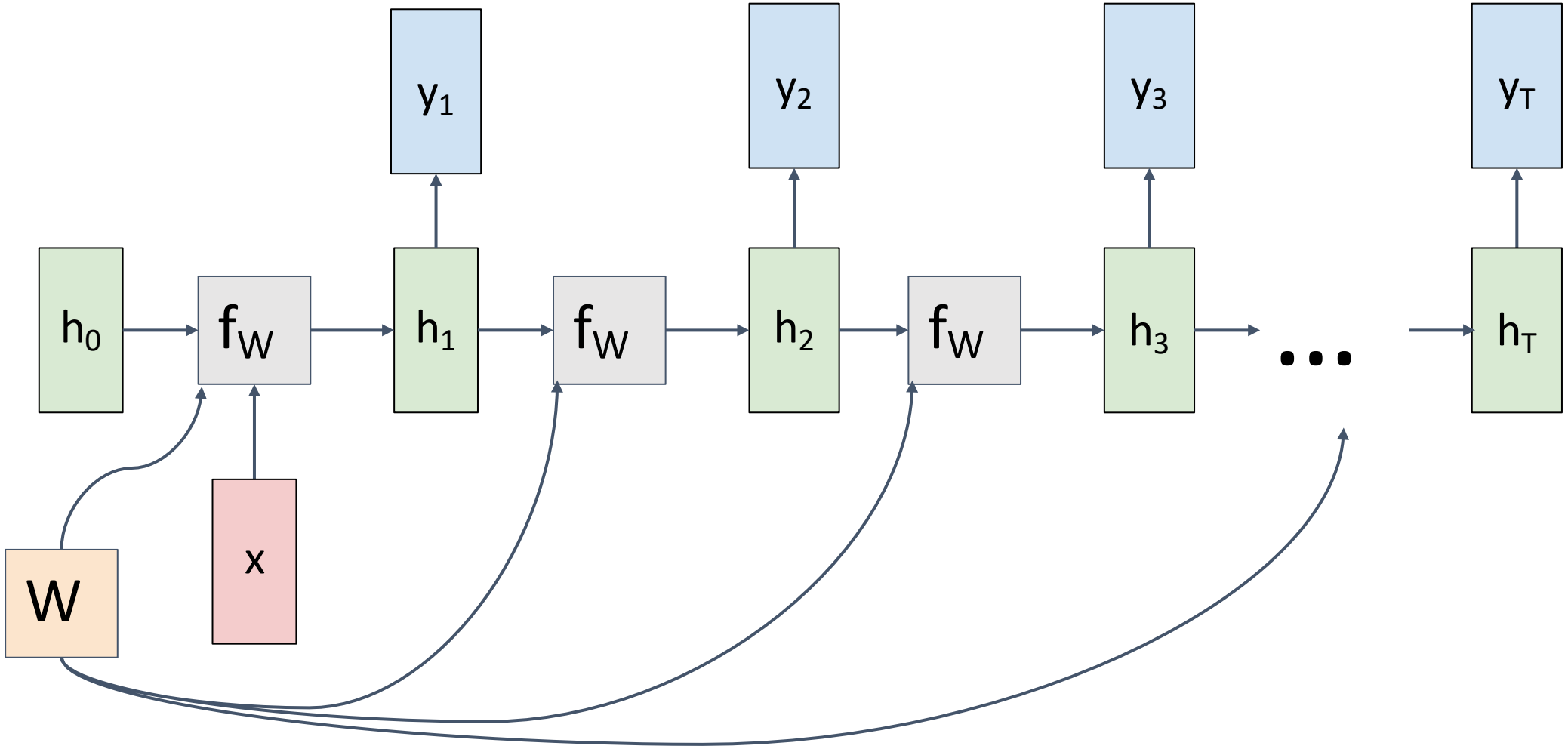
RNN Computational Graph (Many to Many)



RNN Computational Graph (Many to One)



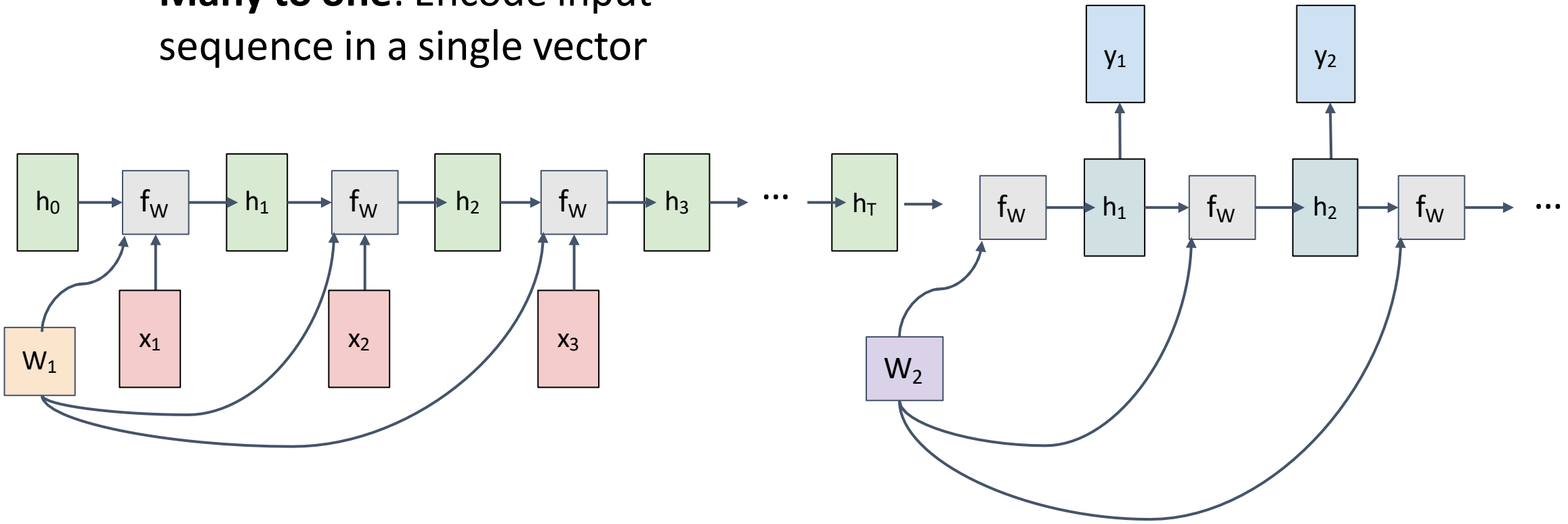
RNN Computational Graph (One to Many)



Sequence to Sequence (seq2seq)
(Many to one) + (One to many)

One to many: Produce output sequence from single input vector

Many to one: Encode input sequence in a single vector



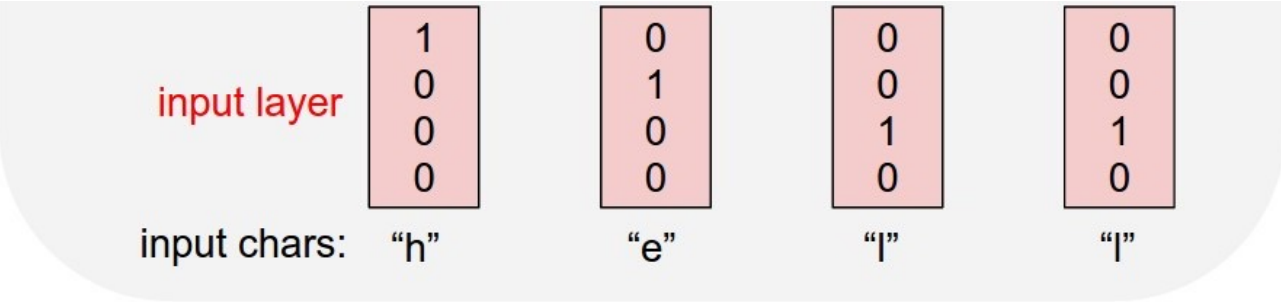
Sutskever et al, “Sequence to Sequence Learning with Neural Networks”, NIPS 2014

Example: Language Modeling

Given characters 1, 2, …, t,
model predicts character t

Training sequence: ”hello”

Vocabulary: [h, e, l, o]



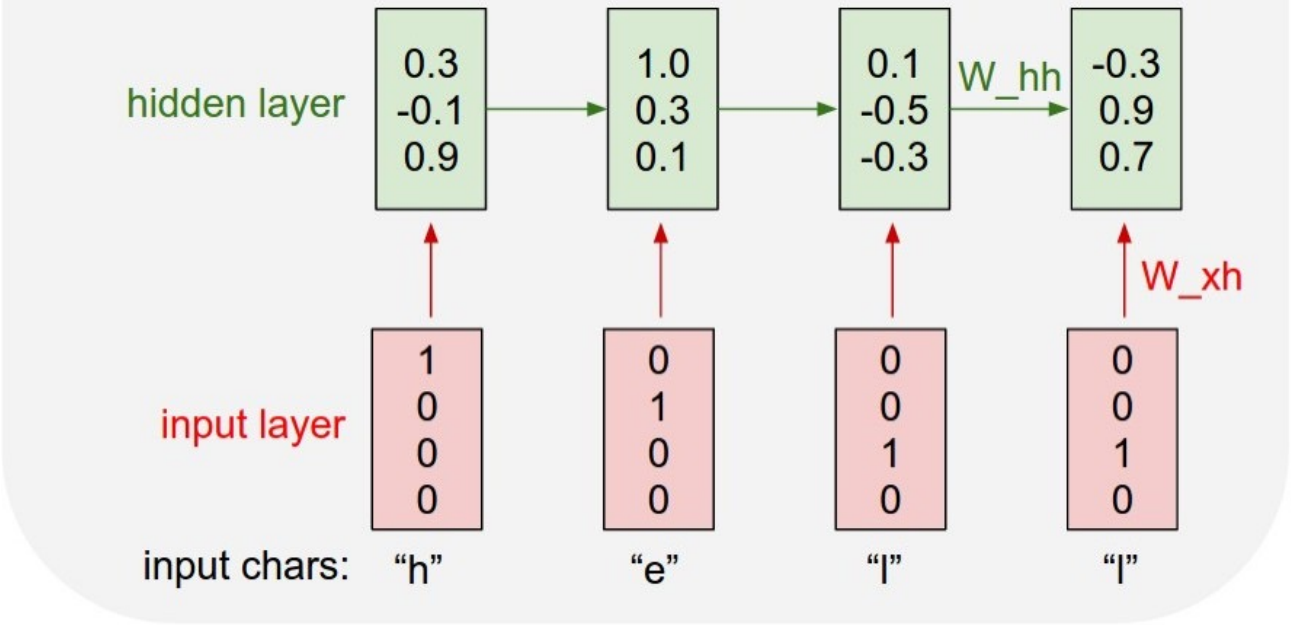
Example: Language Modeling

Given characters 1, 2, …, t,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]



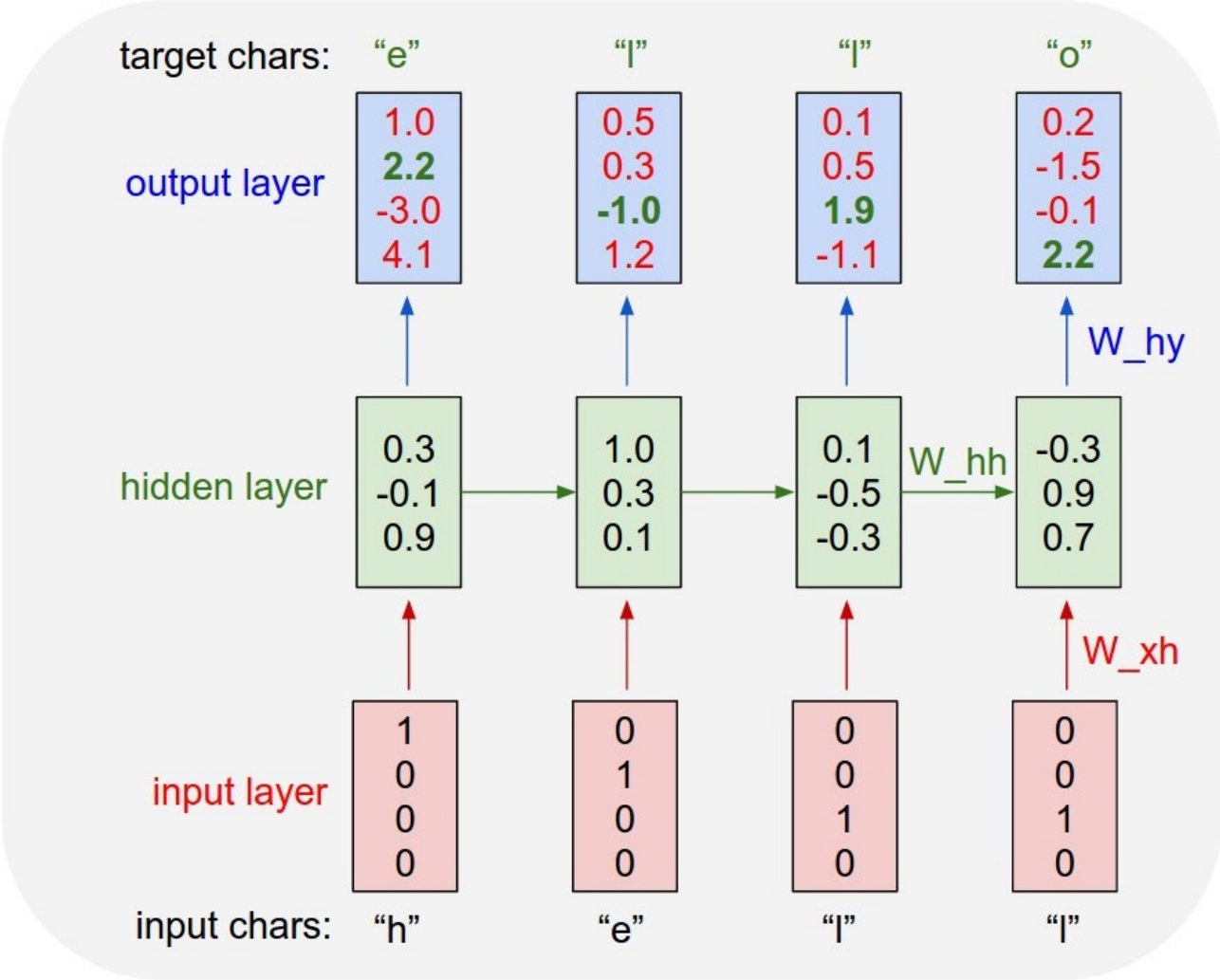
Example: Language Modeling

Given characters 1, 2, …, t,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]



Example: Language Modeling

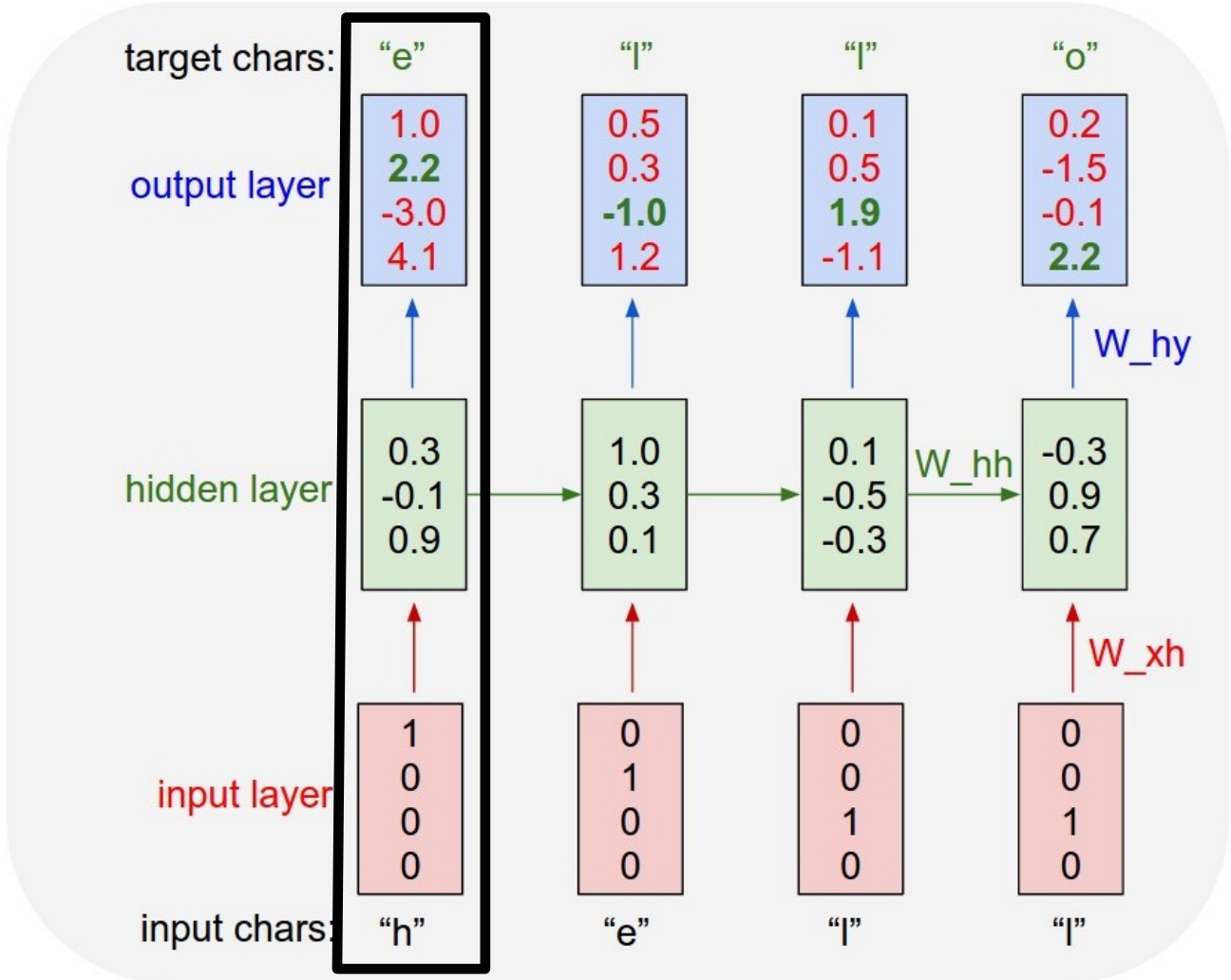
Given characters 1, 2, …, t,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

Given “h”, predict “e”



Example: Language Modeling

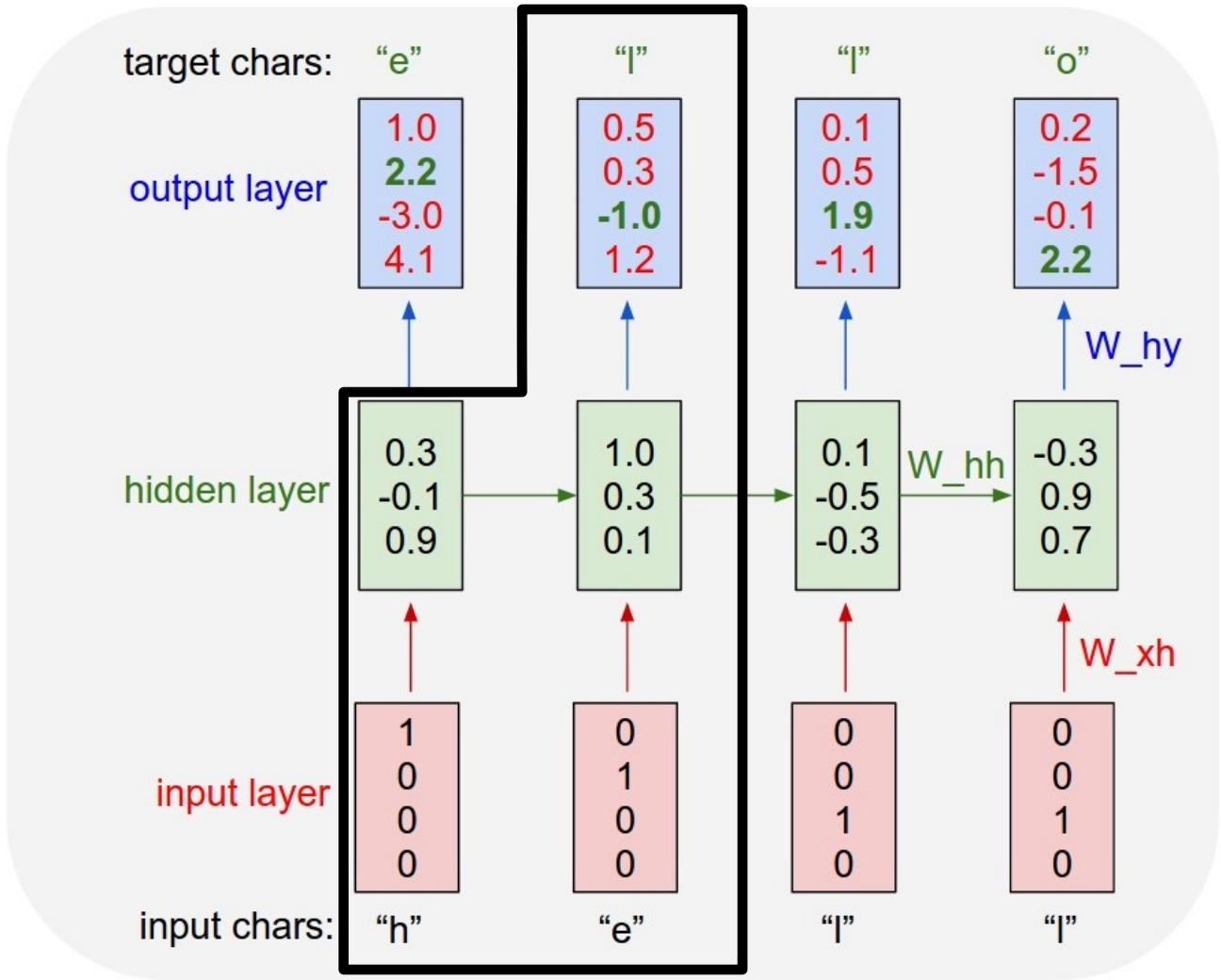
Given characters 1, 2, …, t,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

Given “he”, predict “l”



Example: Language Modeling

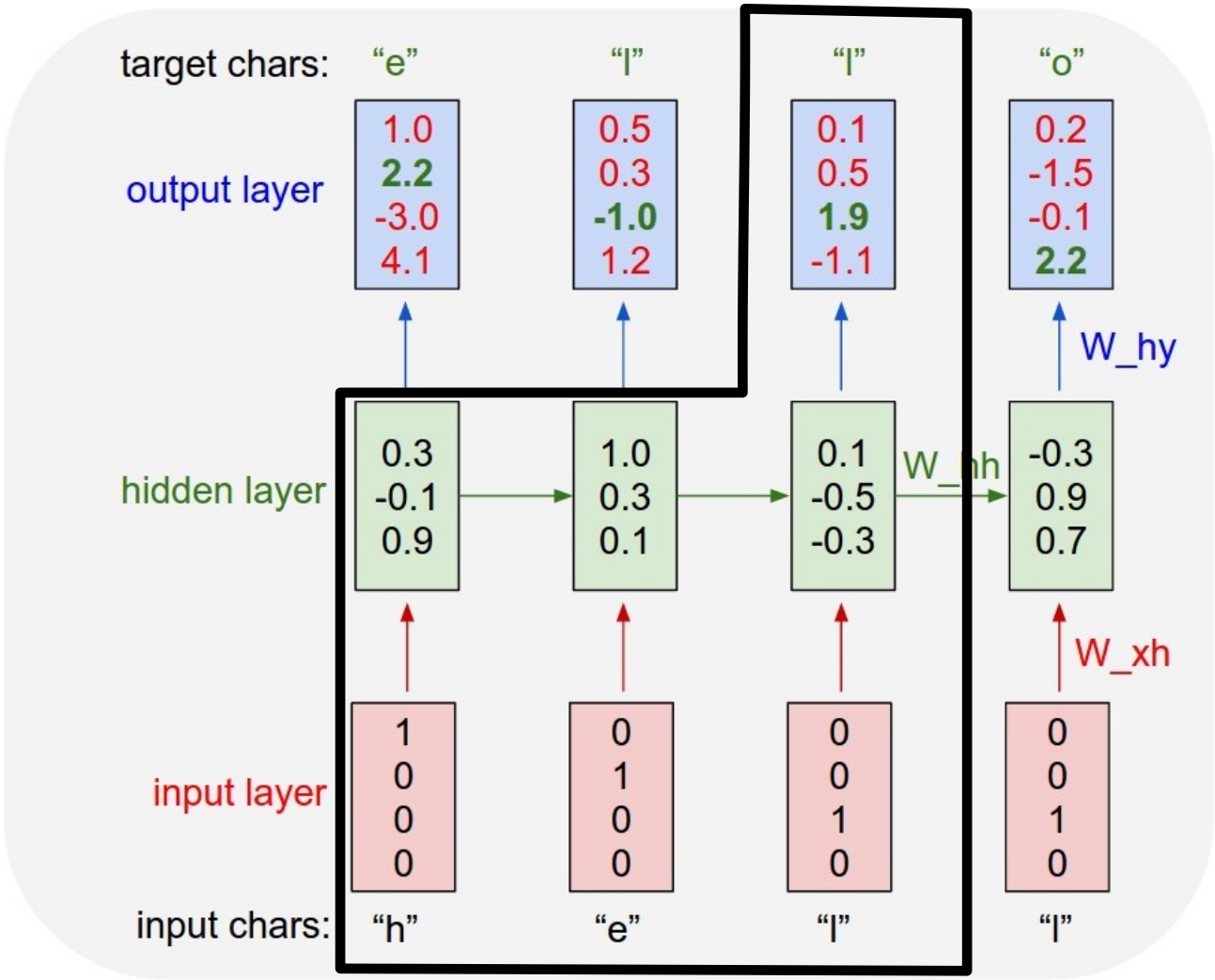
Given characters 1, 2, …, t,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

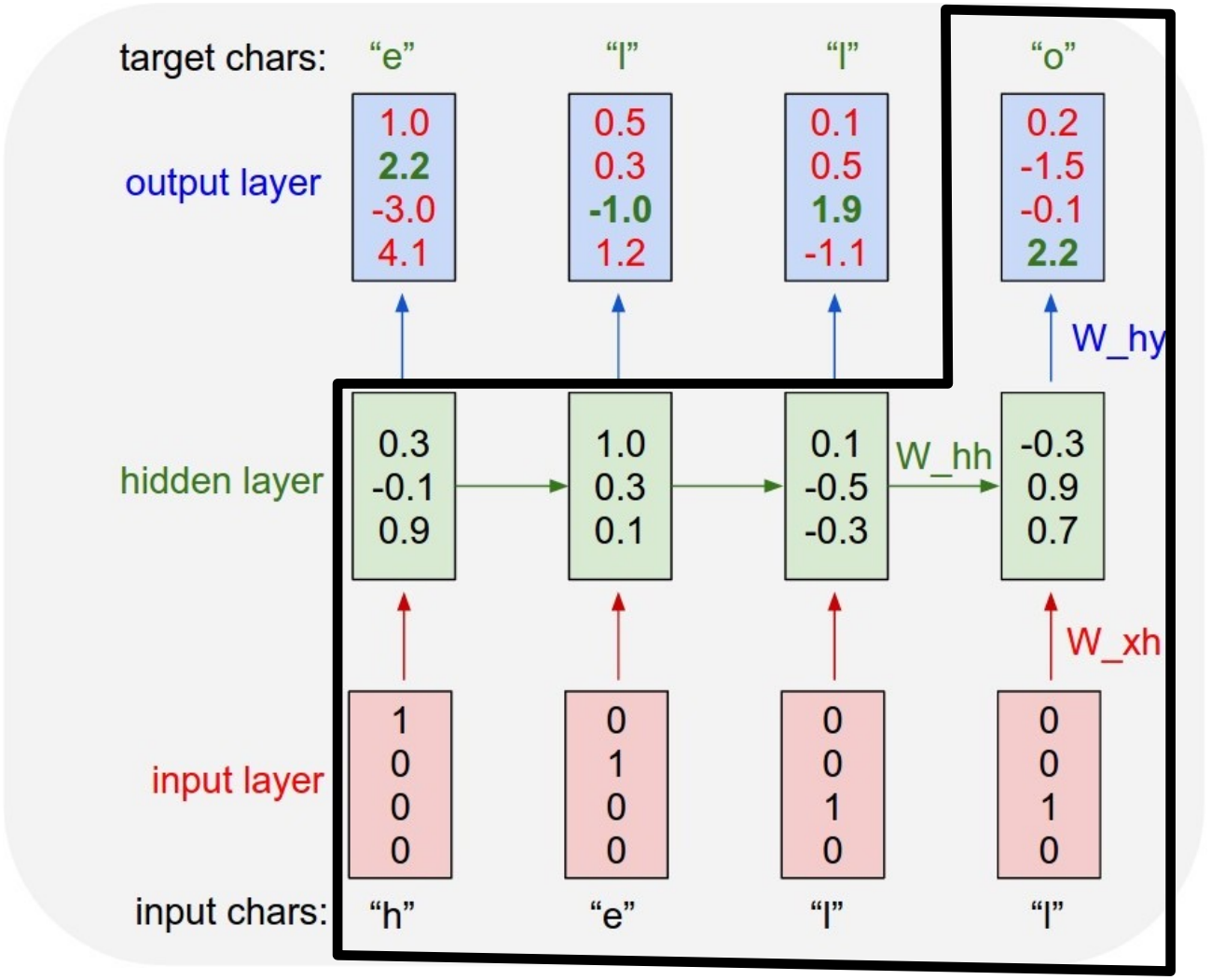
Given “hel”, predict “l”



Example: Language Modeling

Given characters 1, 2, …, t, model predicts character t

Given “hell”, predict “o”



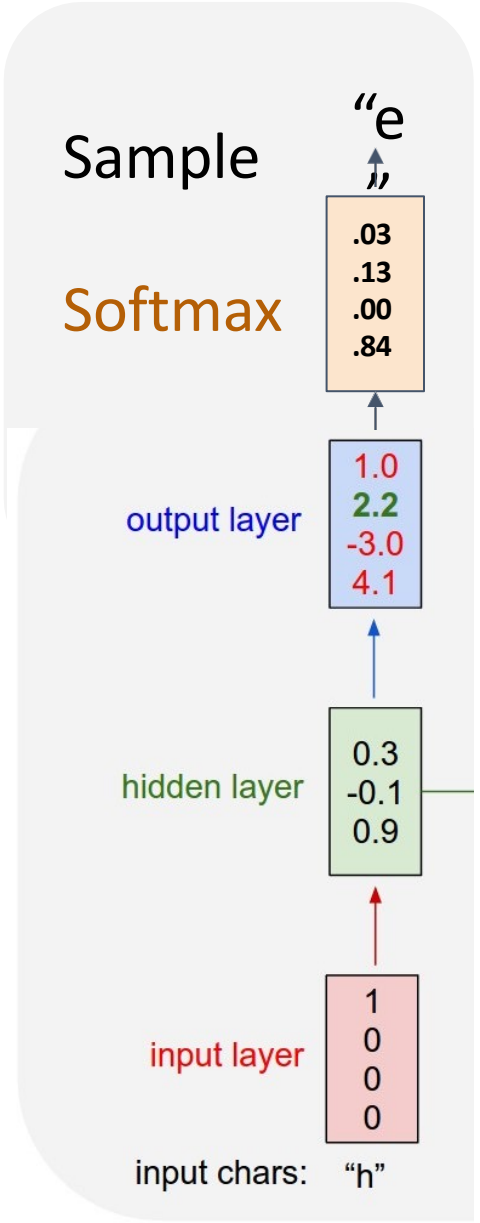
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

Example: Language Modeling

At test-time, **generate** new text: sample characters one at a time, feed back to model



Training sequence: ”hello”

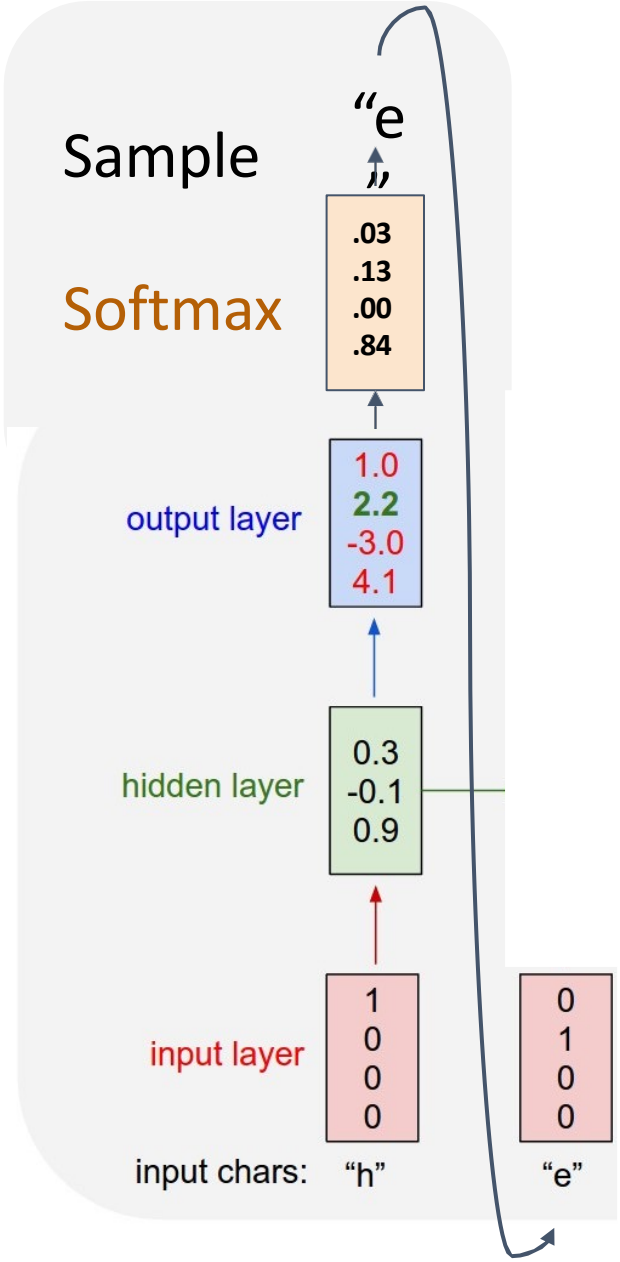
Vocabulary: [h, e, l, o]

Example: Language Modeling

At test-time, **generate** new text: sample characters one at a time, feed back to model

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

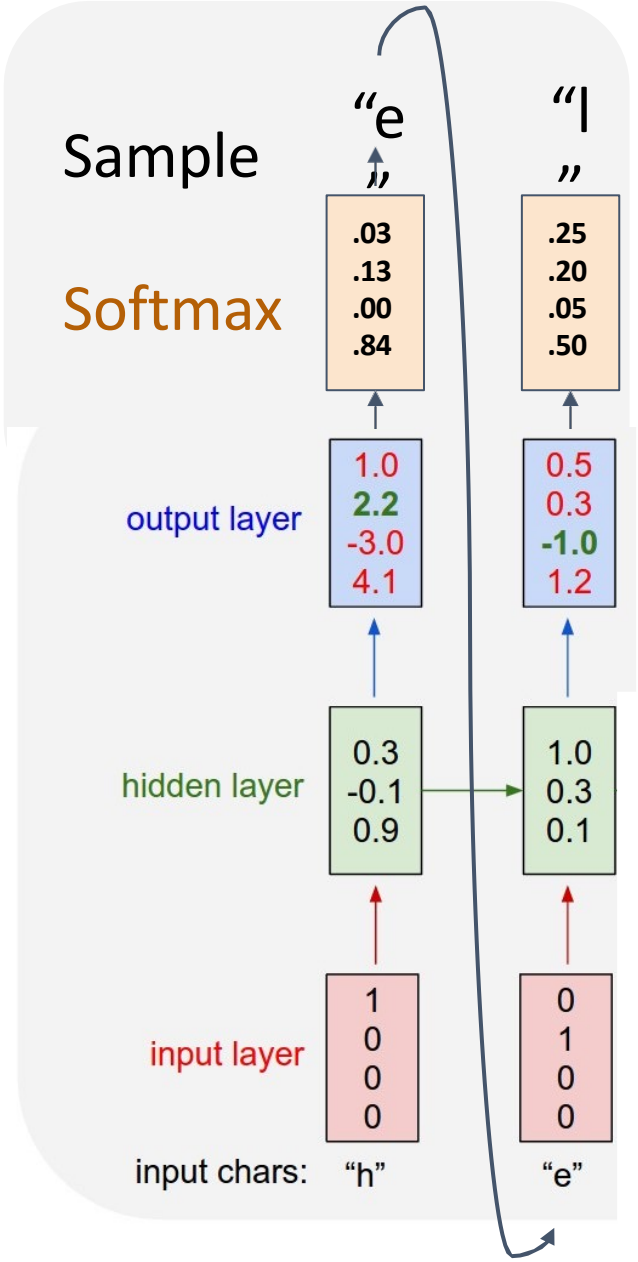


Example: Language Modeling

At test-time, **generate** new text: sample characters one at a time, feed back to model

Training sequence: ”hello”

Vocabulary: [h, e, l, o]

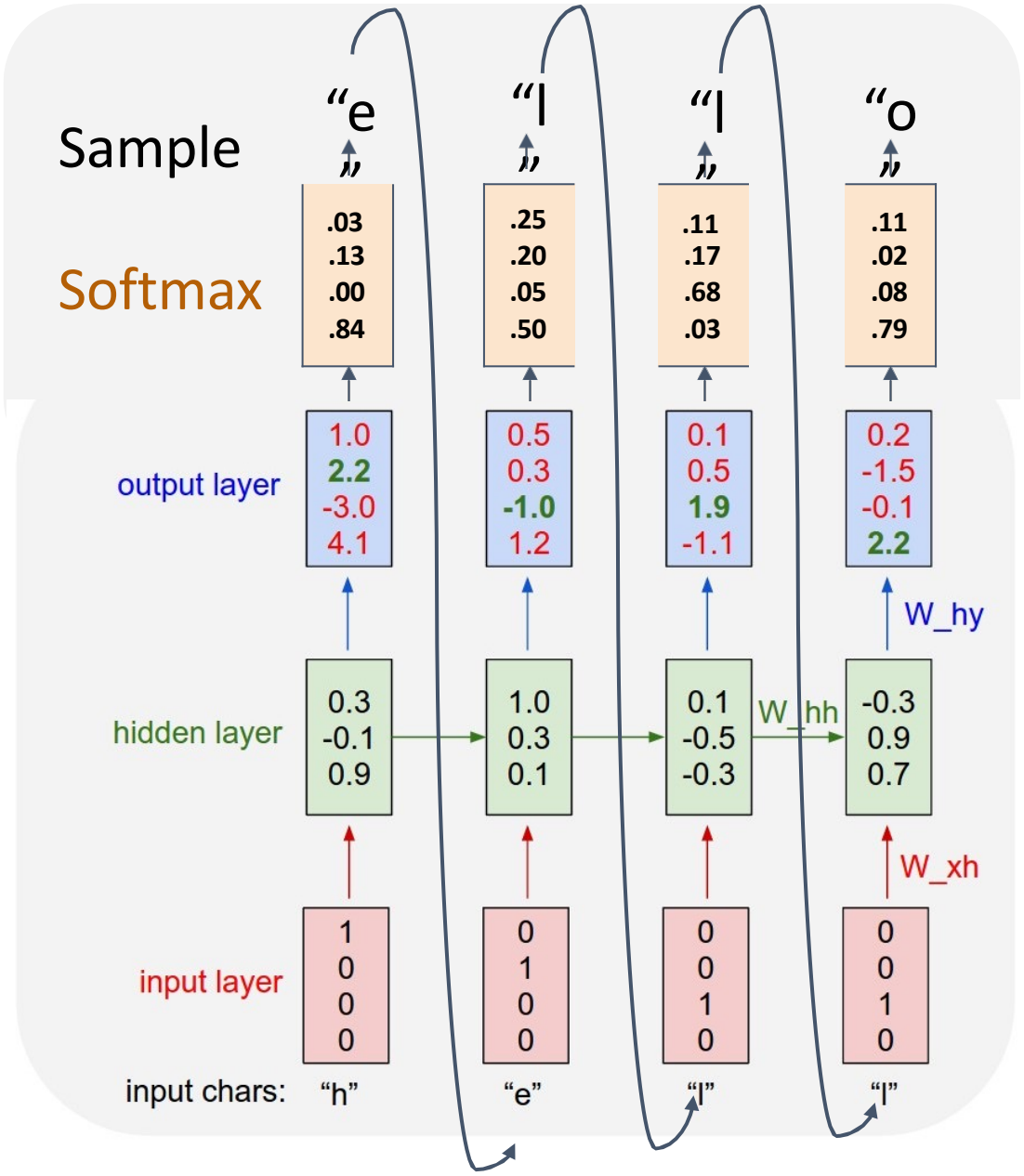


Example: Language Modeling

At test-time, **generate** new text: sample characters one at a time, feed back to model

Training sequence: ”hello”

Vocabulary: [h, e, l, o]



at first:

```
tyntd-iafhatawiaoihrdemot  lytdws  e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgd t o idoe ns,smtt   h ne etie h,hregtrs nigtike,aoaenns lng
```



train more

```
"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```



train more

```
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.
```



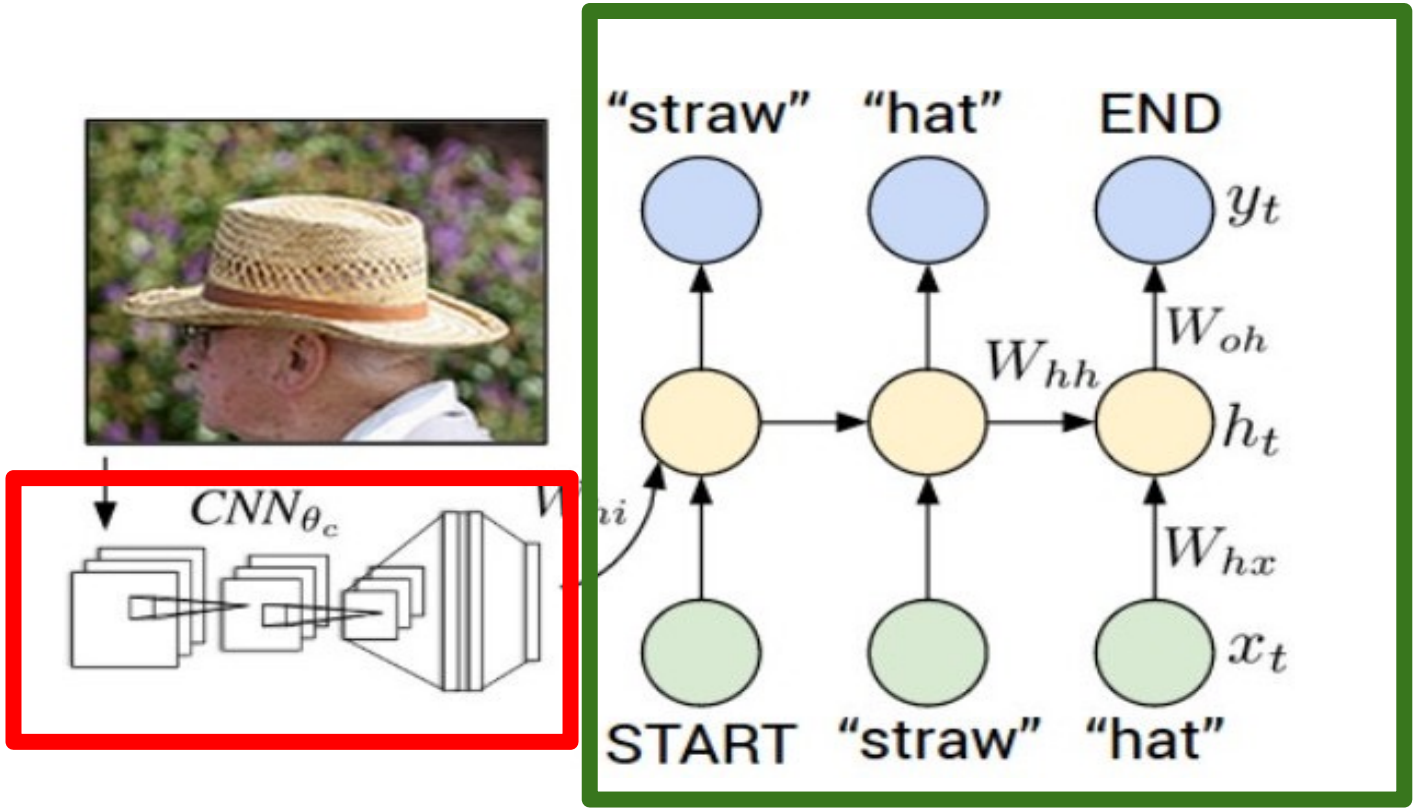
train more

```
"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.
```

Generated C code

```
static void do_command(struct seq_file *m, void *v)
{
  int column = 32 << (cmd[2] & 0x80);
  if (state)
    cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
  else
    seq = 1;
  for (i = 0; i < 16; i++) {
    if (k & (1 << 1))
      pipe = (in_use & UMXTHREAD_UNCCA) +
        ((count & 0x00000000fffffff8) & 0x000000f) << 8;
    if (count == 0)
      sub(pid, ppc_md.kexec_handle, 0x20000000);
    pipe_set_bytes(i, 0);
  }
  /* Free our user pages pointer to place camera if all dash */
  subsystem_info = &of_changes[PAGE_SIZE];
  rek_controls(offset, idx, &soffset);
  /* Now we want to deliberately put it to device */
  control_check_polarity(&context, val, 0);
  for (i = 0; i < COUNTER; i++)
    seq_puts(s, "policy ");
}
```

Example: Image Captioning



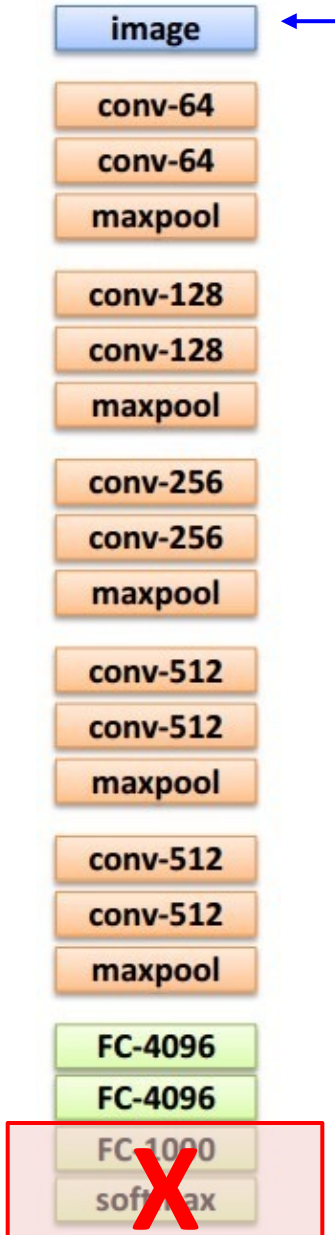
Recurrent Neural Network

Convolutional Neural Network

Figure from Karpathy et al, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

This image is CC0 public domain





Transfer learning: Take CNN trained on ImageNet, chop off last layer

This image is CC0 public domain



image

conv-64
conv-64
maxpool

conv-128
conv-128
maxpool

conv-256
conv-256
maxpool

conv-512
conv-512
maxpool

conv-512
conv-512
maxpool

FC-4096
FC-4096

x0

<START>

This image is CC0 public domain



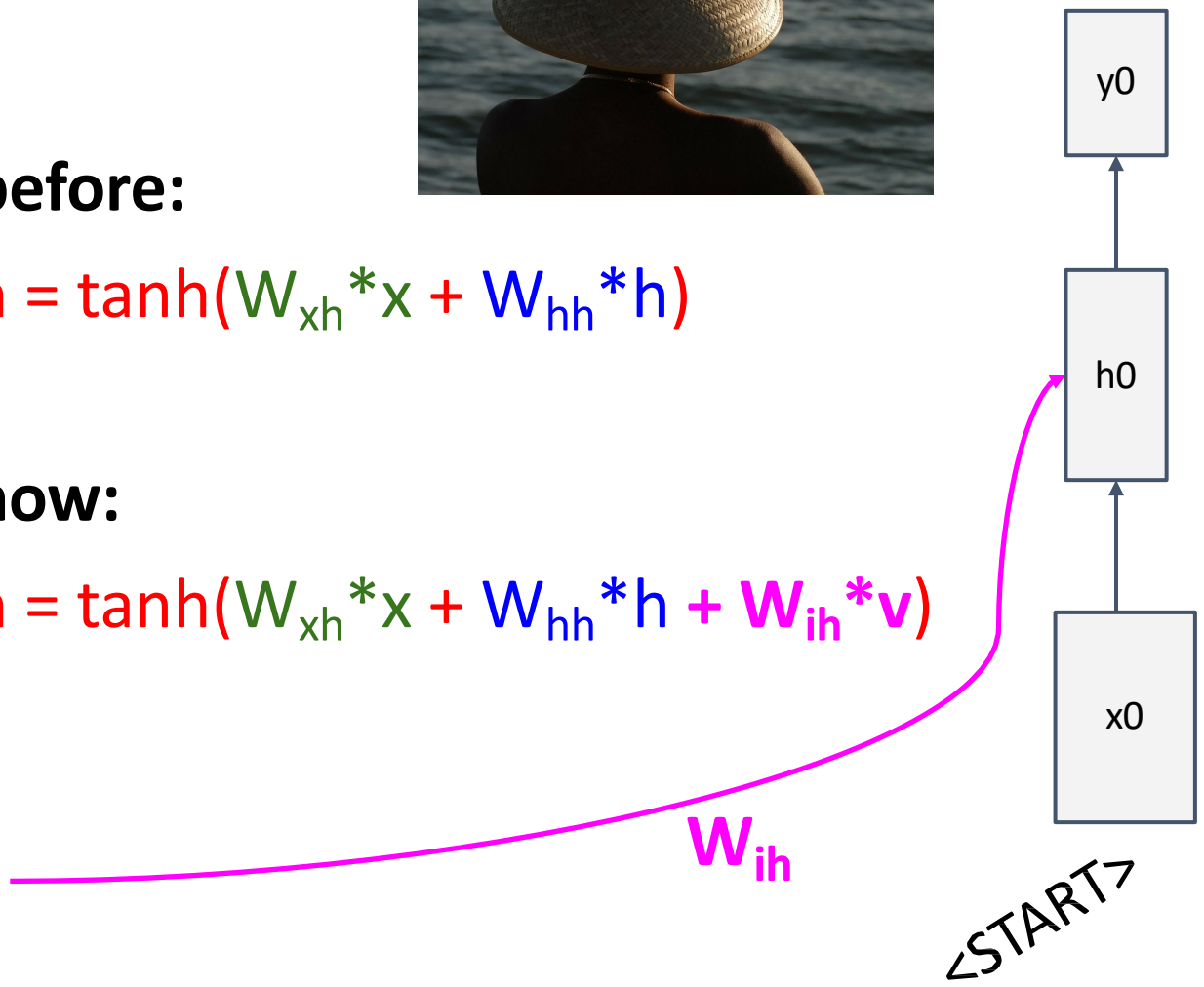
image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$



This image is CC0 public domain

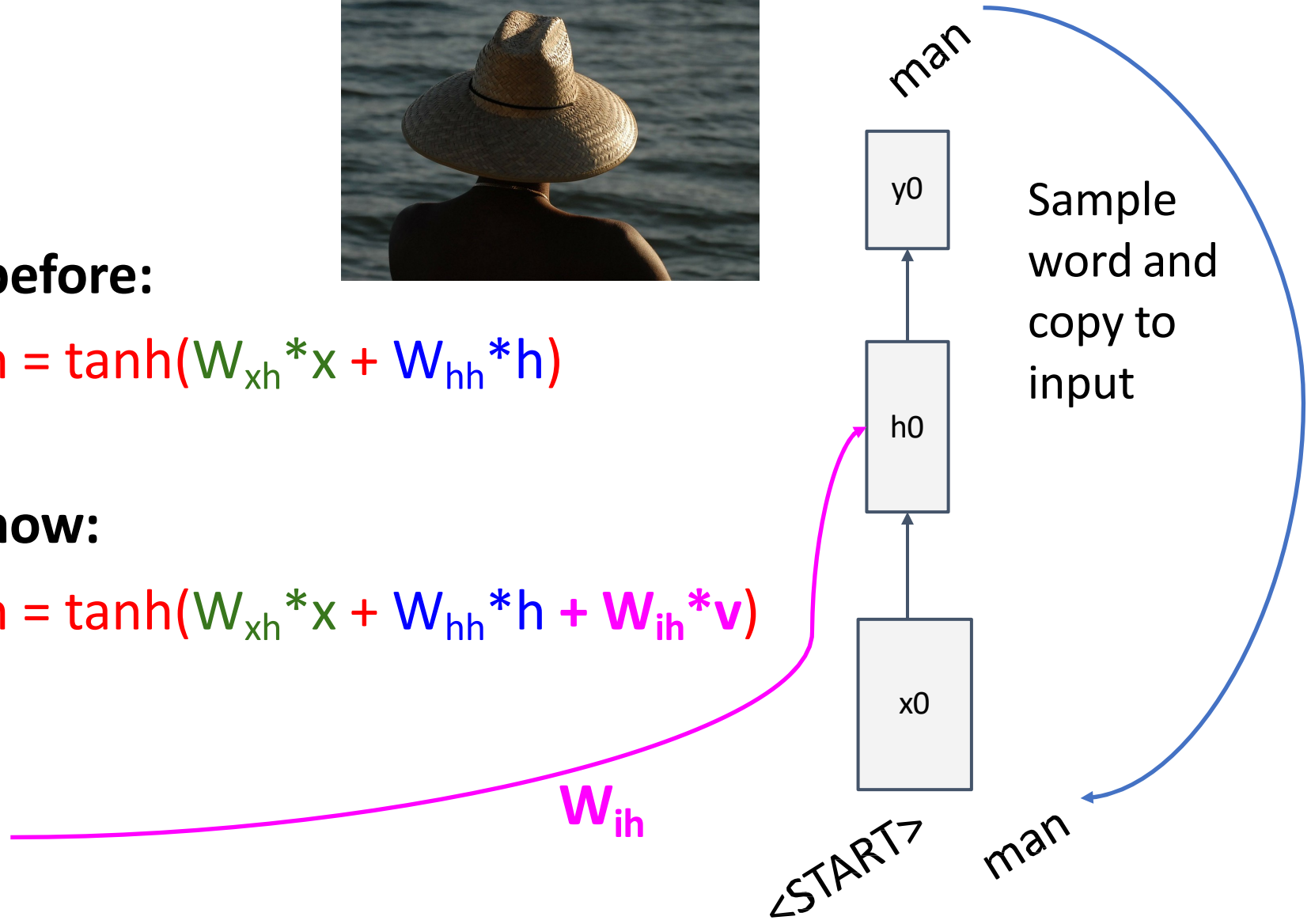
image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$



This image is CC0 public domain



image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

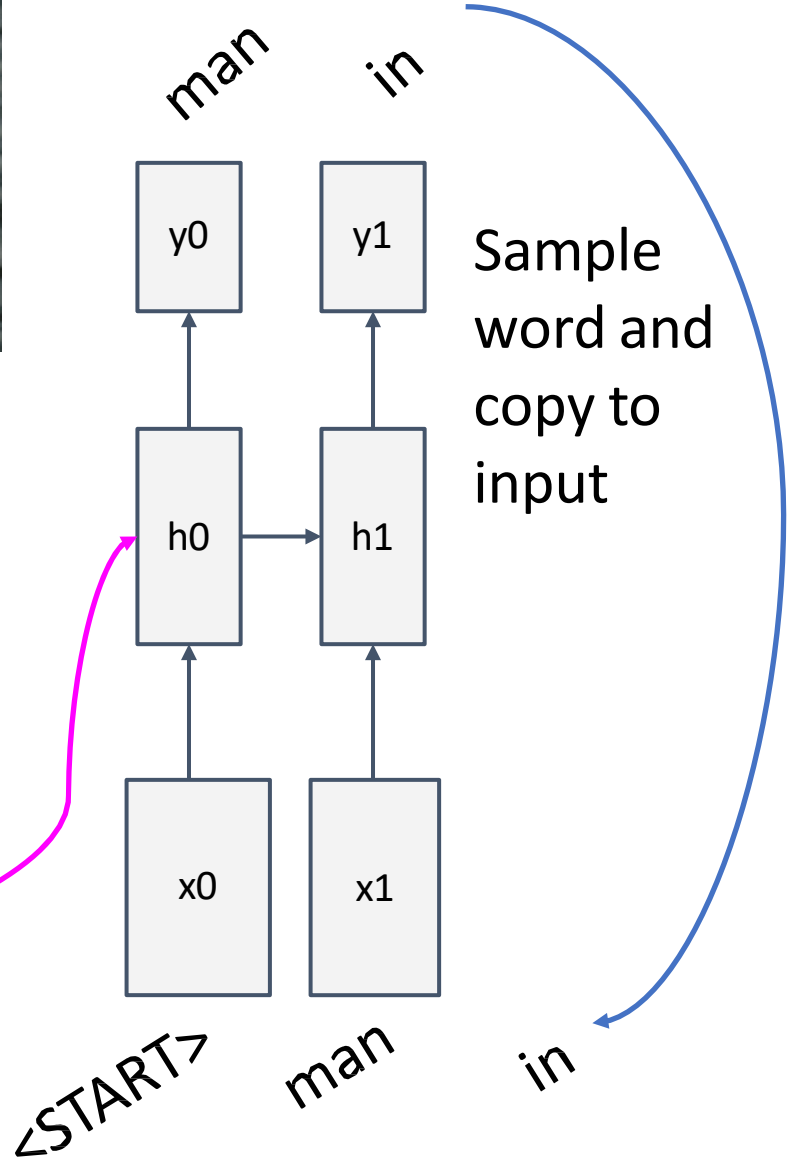
before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$

W_{ih}



This image is CC0 public domain



image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

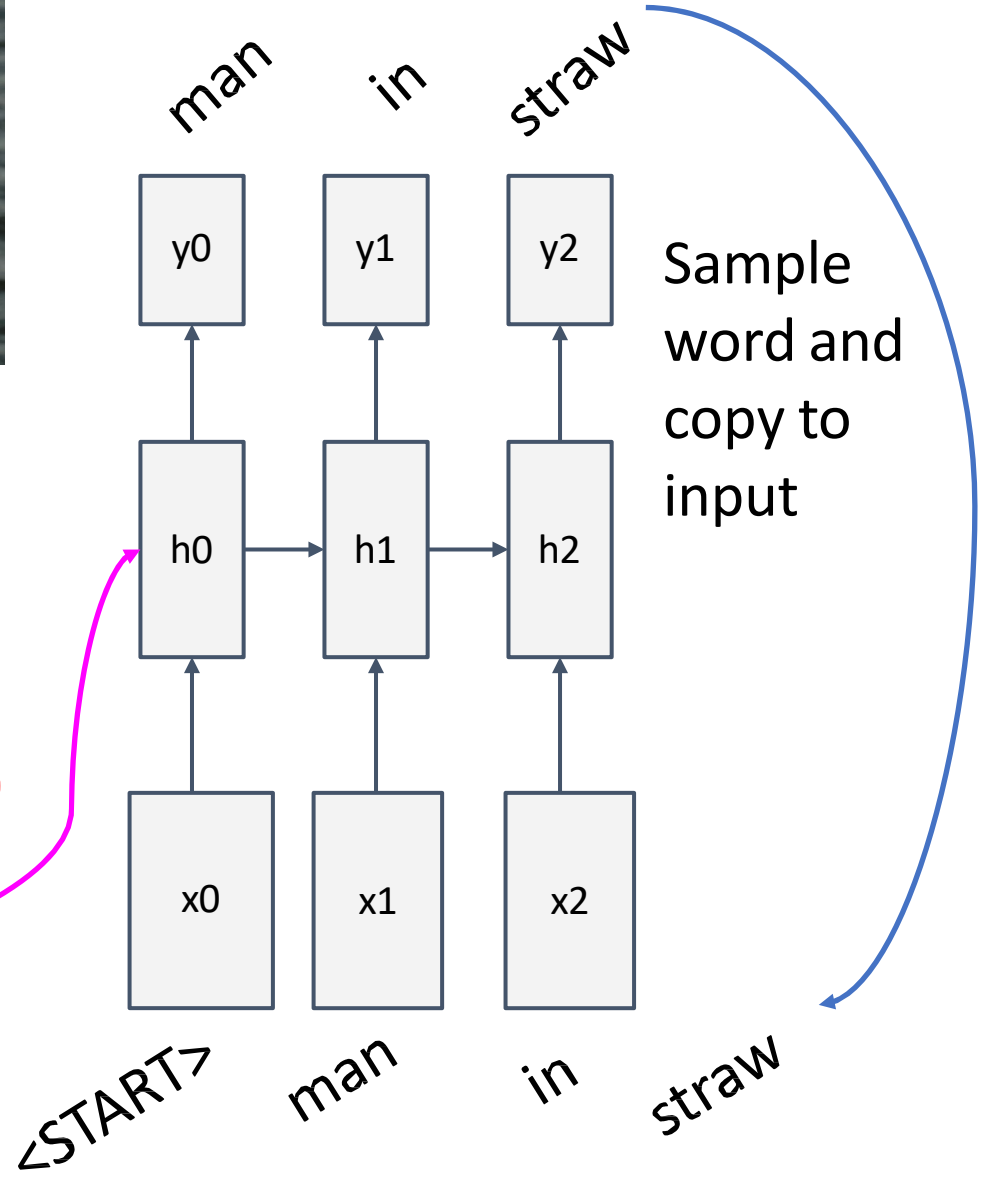
before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$

W_{ih}



This image is CC0 public domain



image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

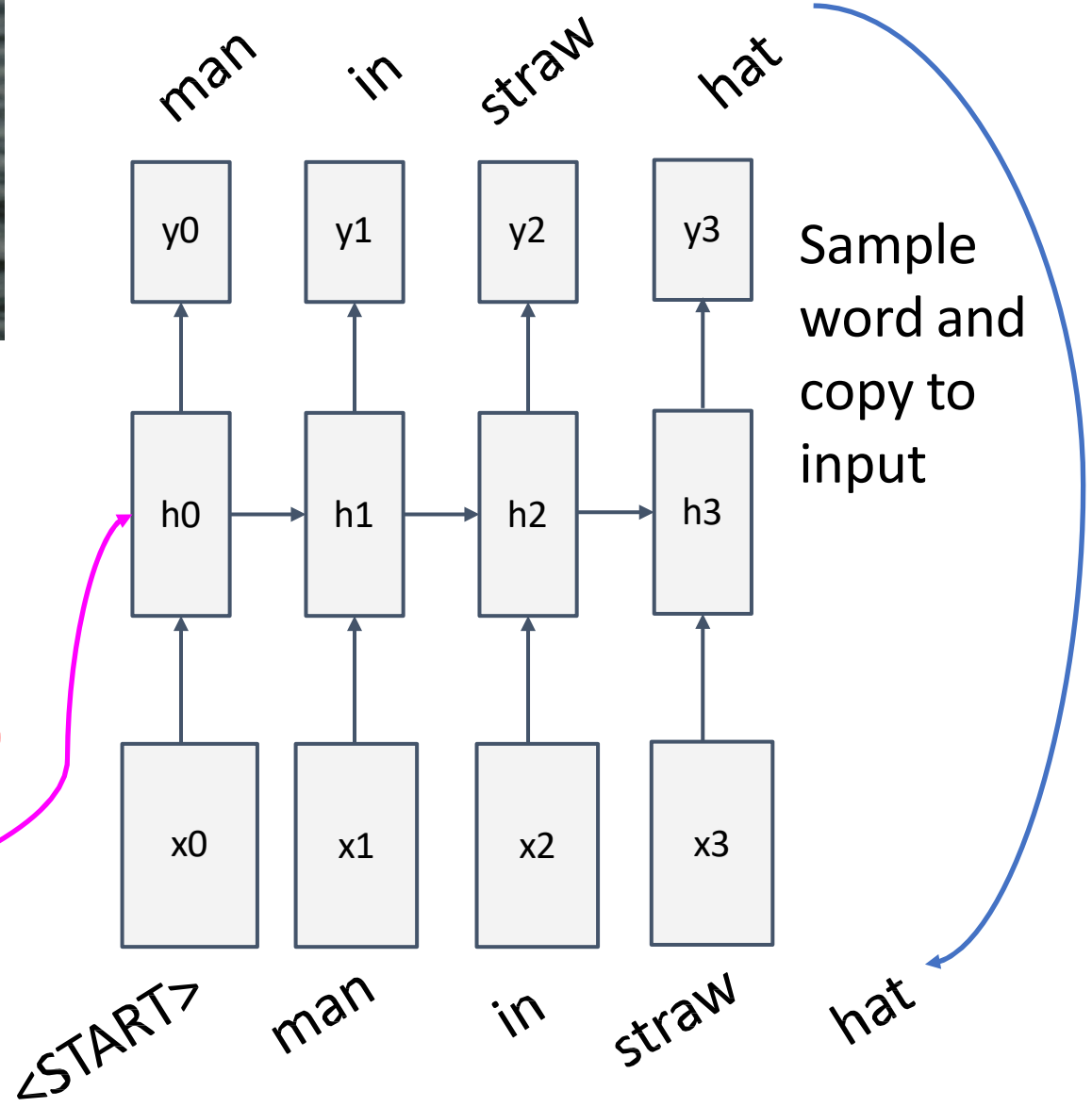
before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$

W_{ih}



This image is CC0 public domain



image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096

before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$

W_{ih}

Stop after sampling <END> token

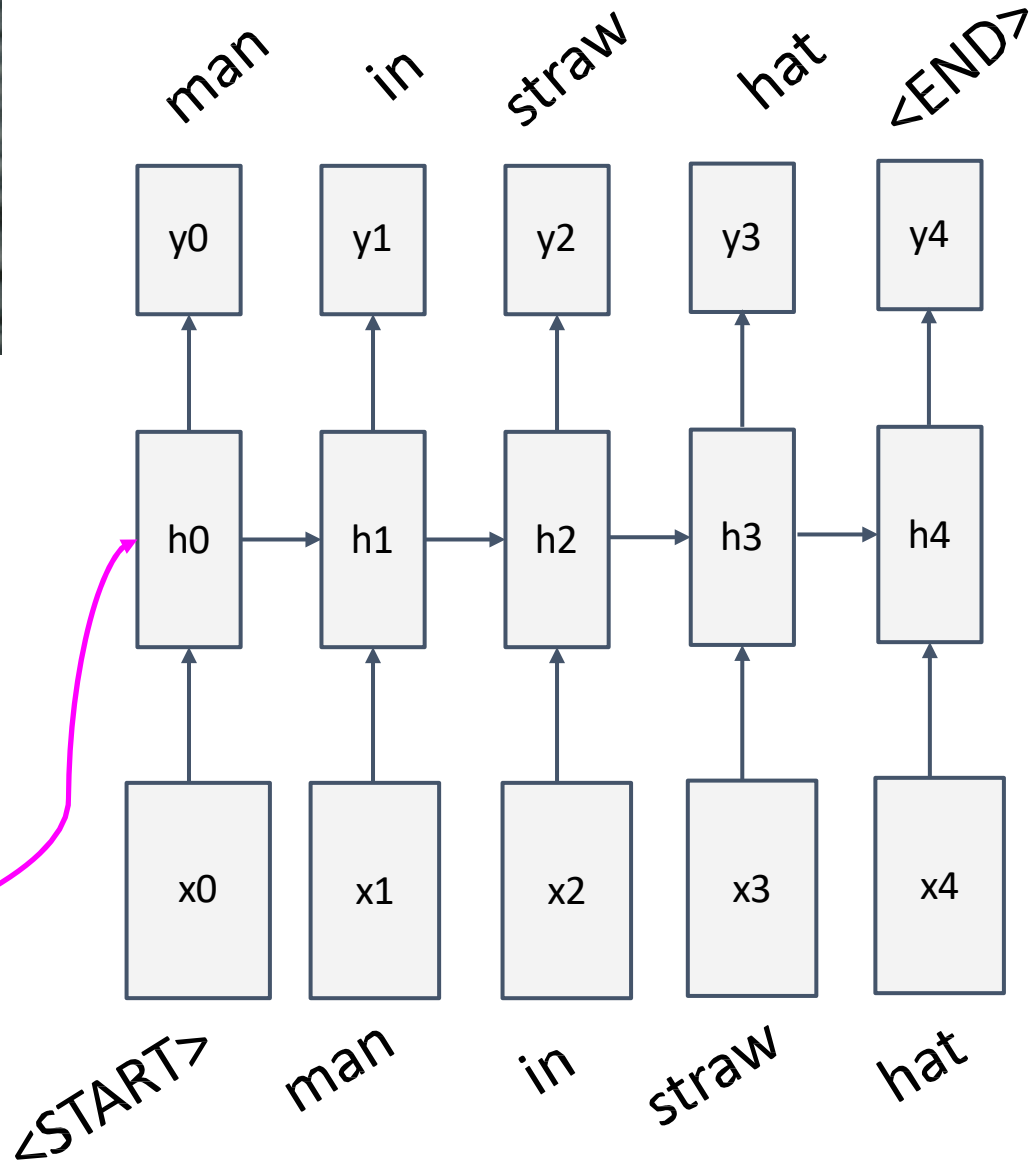


Image Captioning: Example Results

Captions generated using neuraltalk2
All images are CC0 Public domain: cat suitcase, cat tree, dog, bear, surfers, tennis, giraffe, motorcycle



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



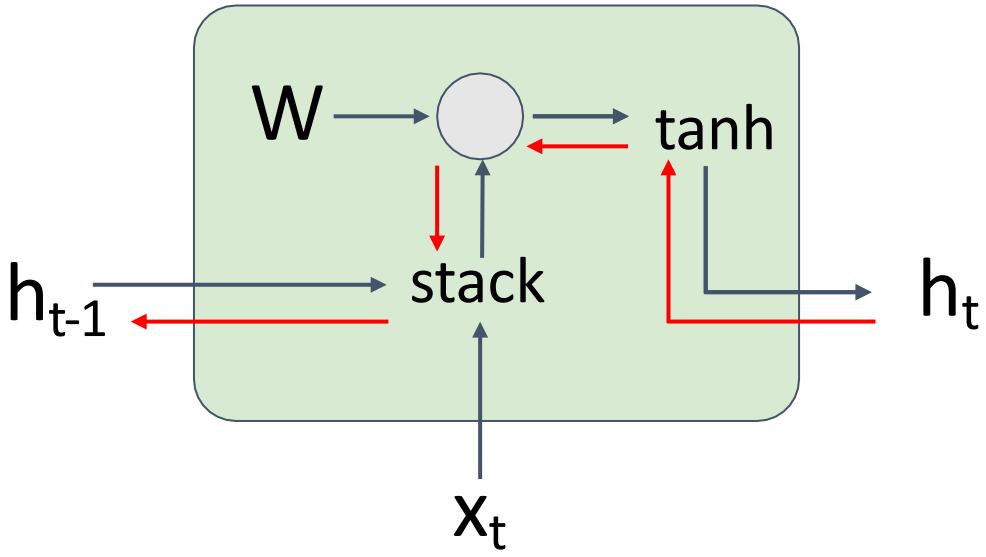
Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

Vanilla RNN Gradient Flow

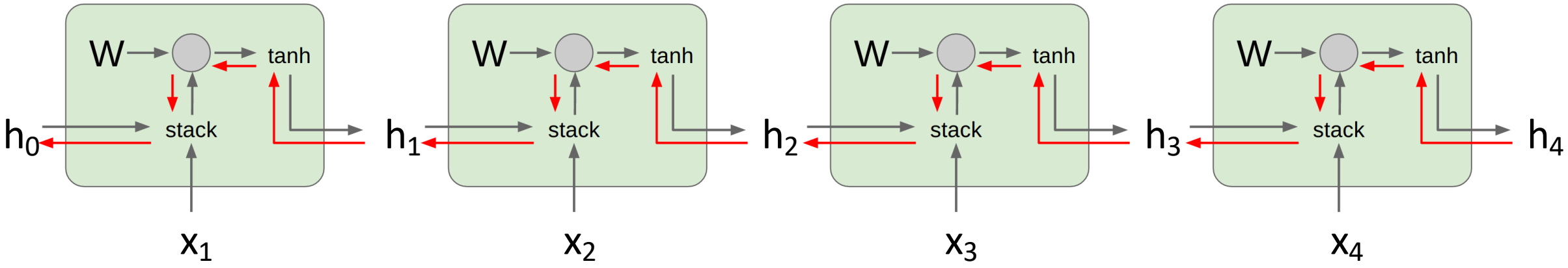
Backpropagation from
h_t to h_{t-1} multiplies by W
(actually W_{hh}^T)



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

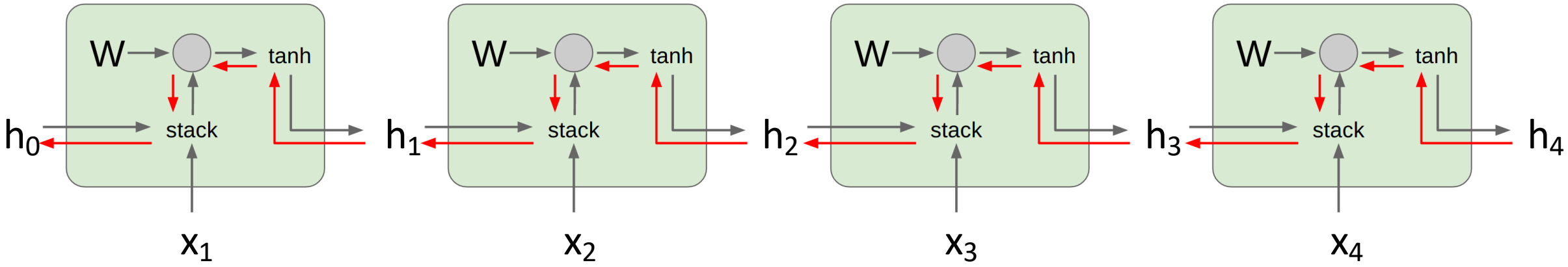
Bengio et al, “Learning long-term dependencies with gradient descent is difficult”, IEEE Transactions on Neural Networks, 1994
Pascanu et al, “On the difficulty of training recurrent neural networks”, ICML 2013

Vanilla RNN Gradient Flow



Computing gradient of
h_0 involves many
factors of W
(and repeated tanh)

Vanilla RNN Gradient Flow



Computing gradient of h_0 involves many factors of W (and repeated tanh)

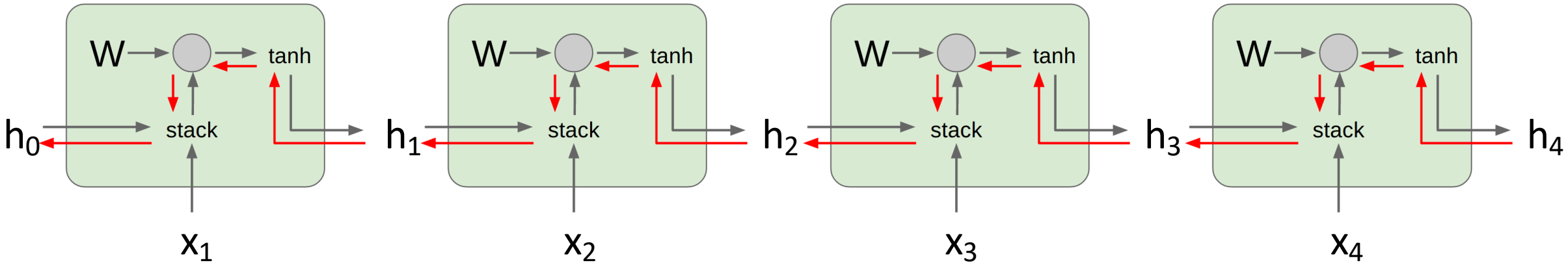
Largest singular value > 1:
Exploding gradients

→ **Gradient clipping**: Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
  grad *= (threshold / grad_norm)
```

Largest singular value < 1:
Vanishing gradients

Vanilla RNN Gradient Flow



Computing gradient of h_0 involves many factors of W (and repeated tanh)

Largest singular value > 1:
Exploding gradients

Largest singular value < 1:
Vanishing gradients → **Change RNN architecture!**

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Two vectors at each timestep:

Cell state

Hidden state

Hochreiter and Schmidhuber, “Long Short Term Memory”, Neural Computation 1997

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

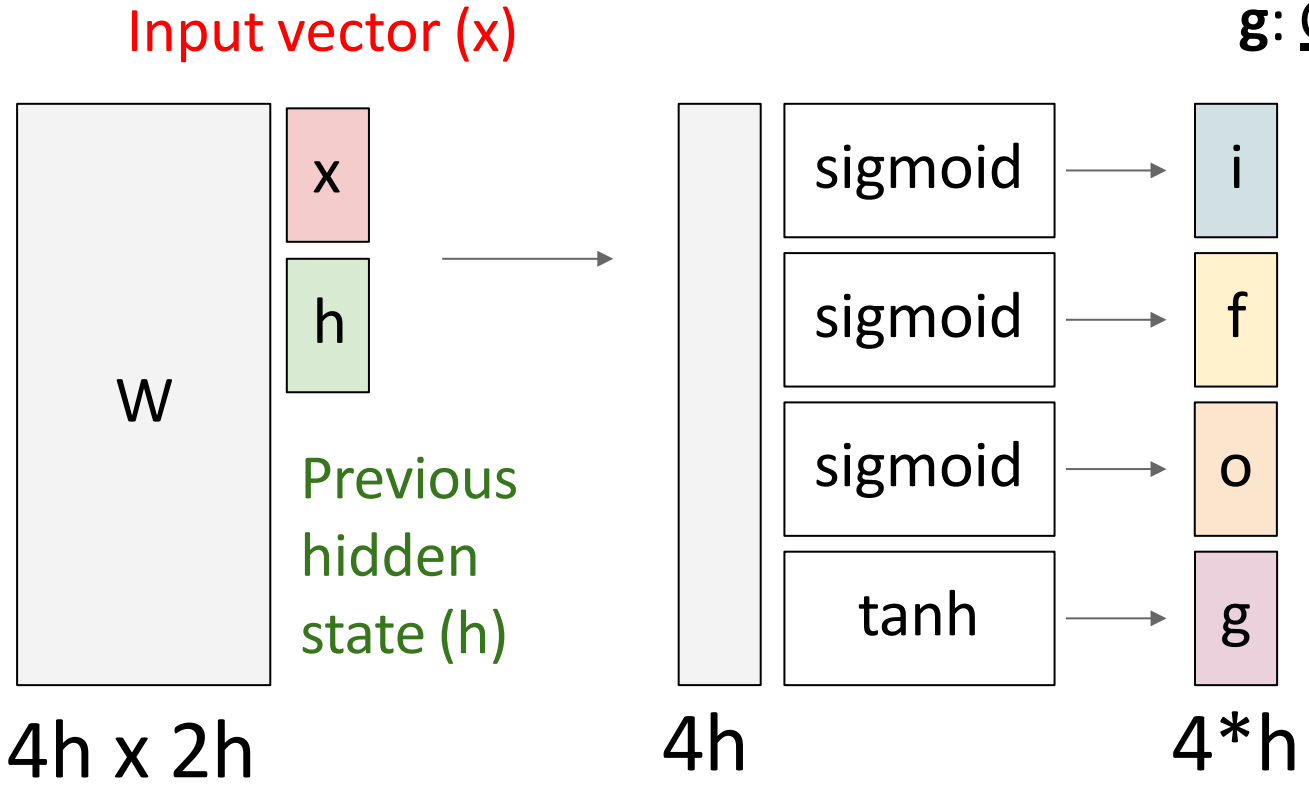
$$h_t = o \odot \tanh(c_t)$$

Compute four **gates** at each timestep

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)

i: Input gate, whether to write to cell
f: Forget gate, Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell



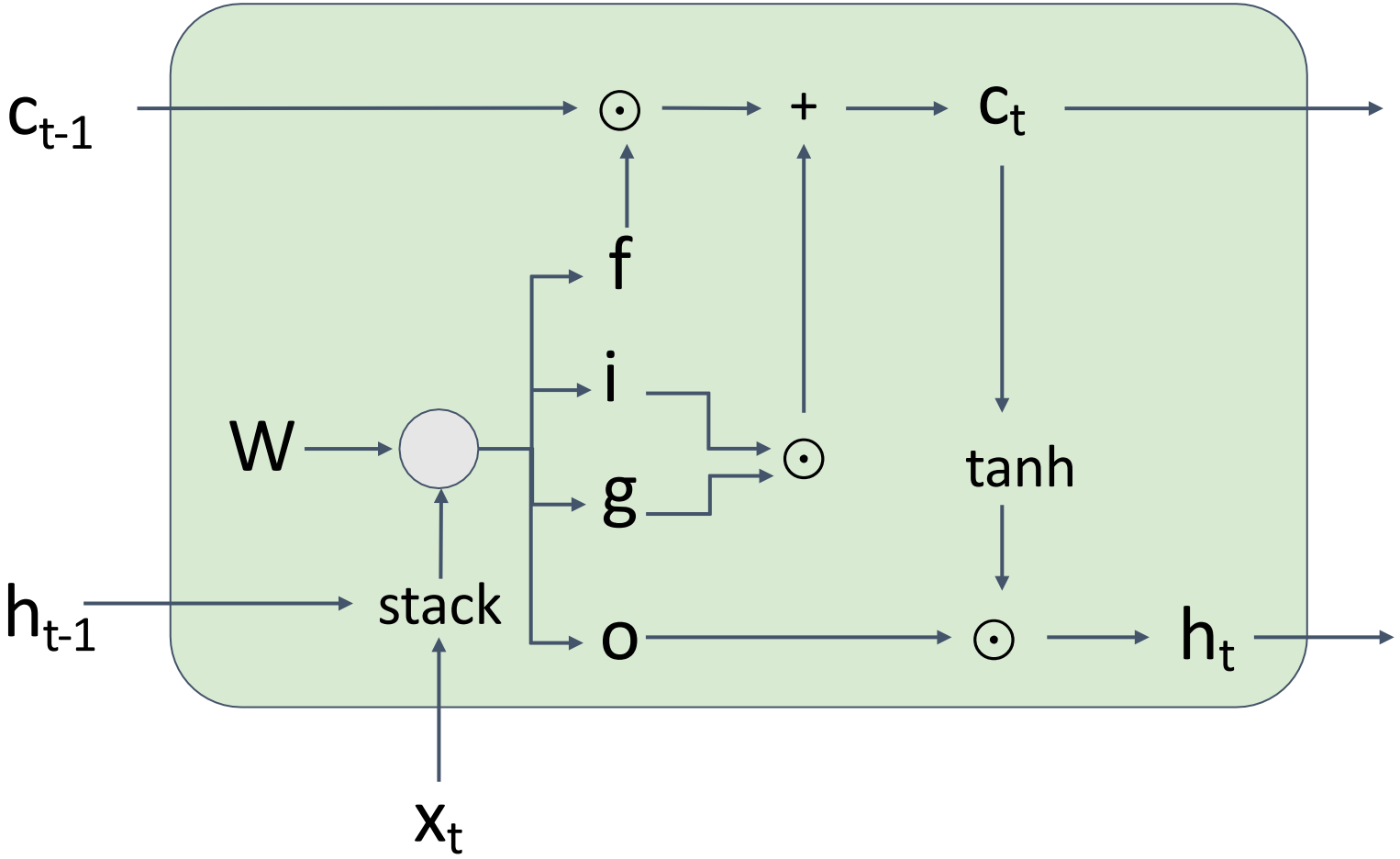
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)



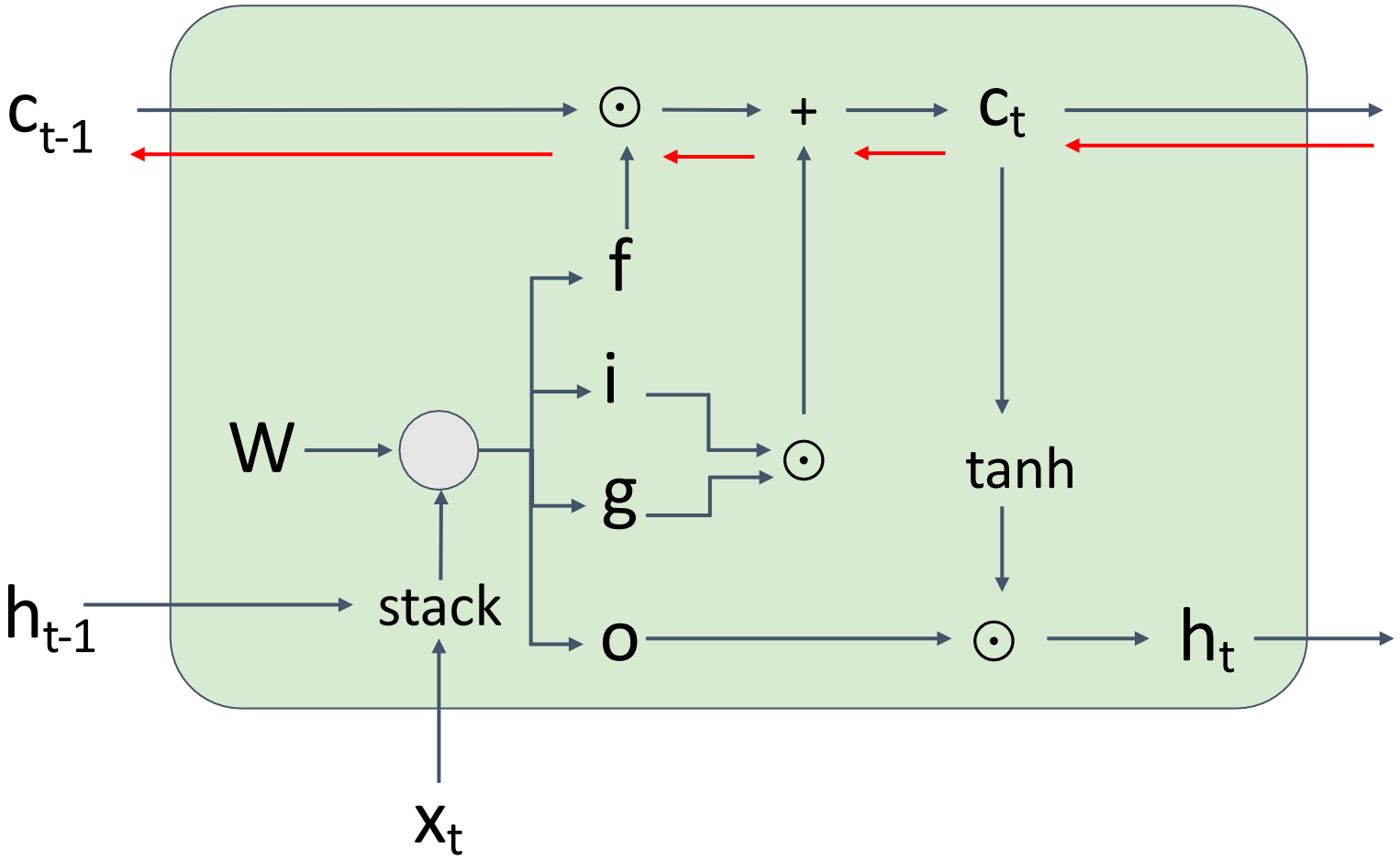
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, “Long Short Term Memory”, Neural Computation 1997

Long Short Term Memory (LSTM): Gradient Flow



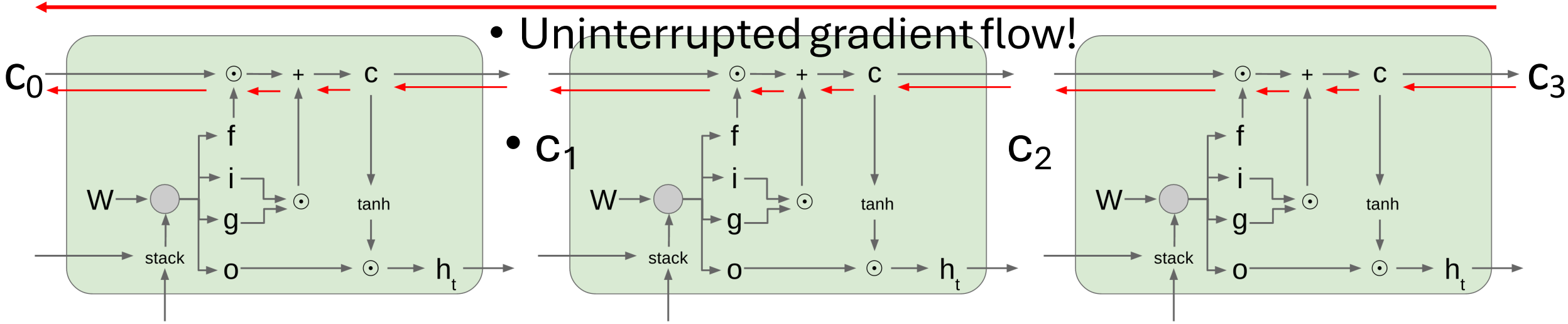
Backpropagation from c_t to c_{t-1} only elementwise multiplication by f, no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

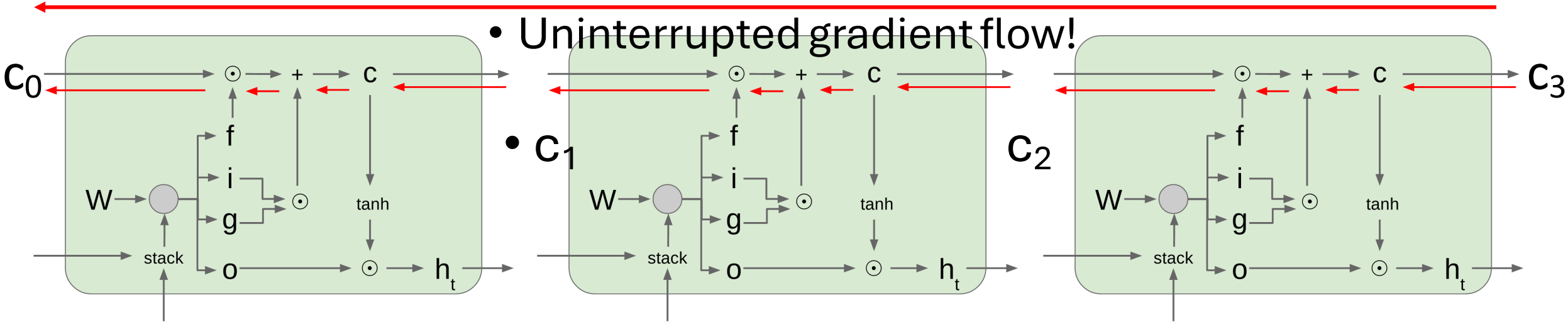
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

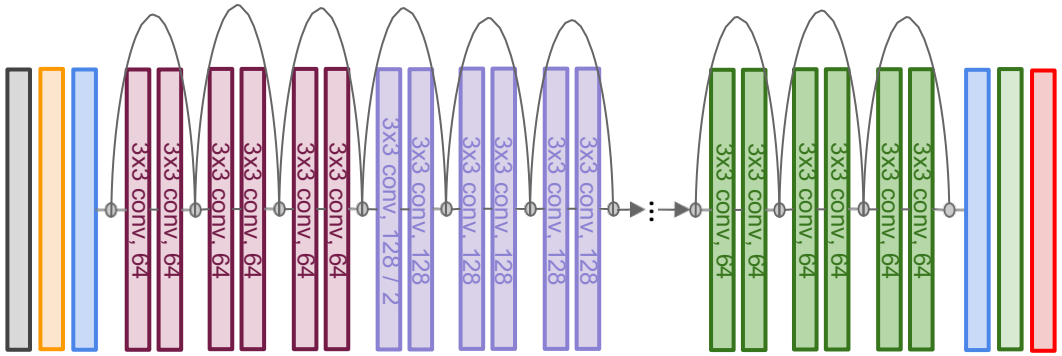


Hochreiter and Schmidhuber, “Long Short Term Memory”, Neural Computation 1997

Long Short Term Memory (LSTM): Gradient Flow



- Uninterrupted gradient flow!



Similar to ResNet!

In between: **Highway Networks**

$$g = T(x, W_T)$$

$$y = g \odot H(x, W_H) + (1 - g) \odot x$$

Srivastava et al, "Highway Networks", ICML DL Workshop 2015

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$ $\quad W^l \ [n \times 2n]$

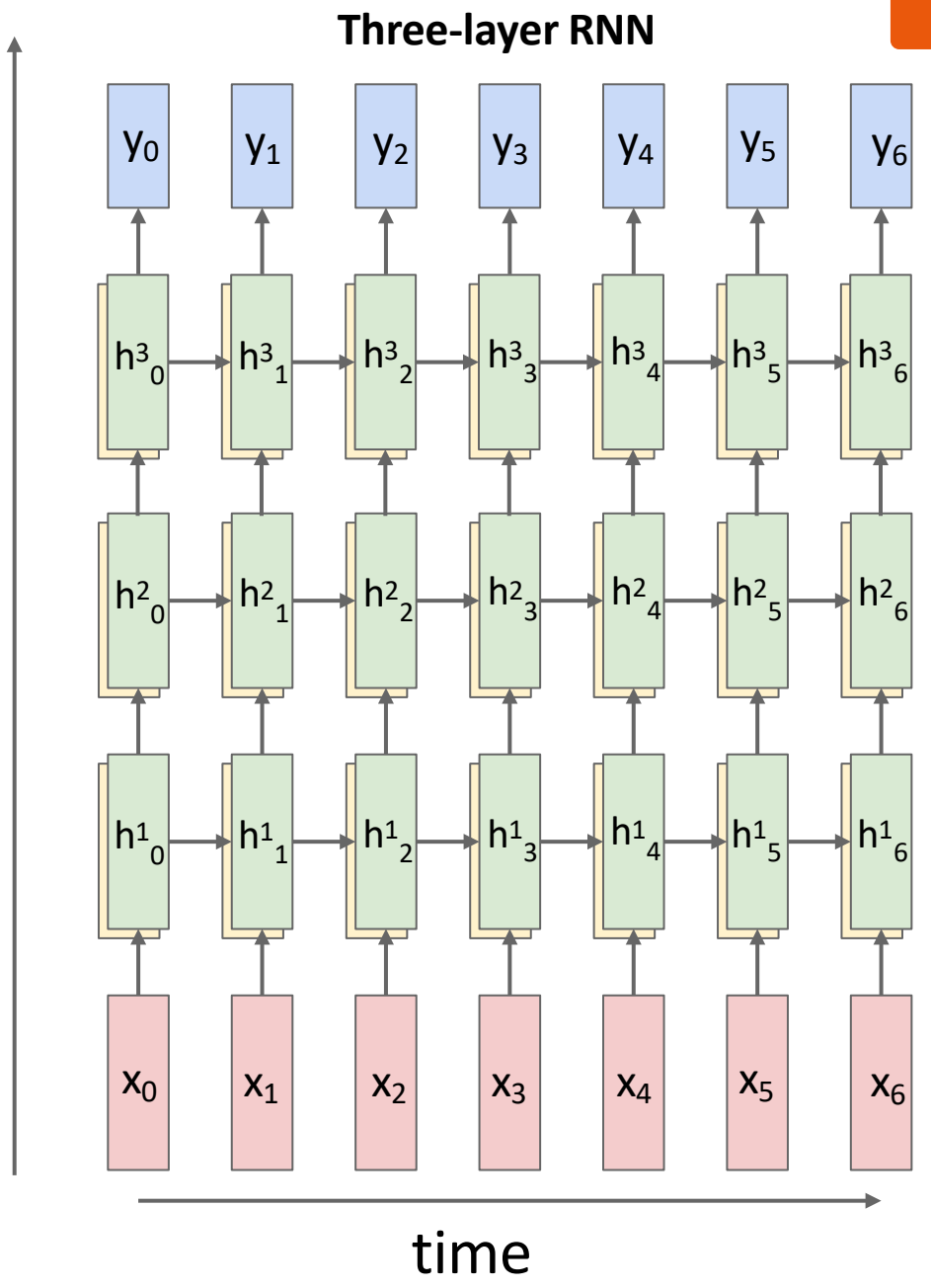
LSTM:

$W^l \ [4n \times 2n]$

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \tanh \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

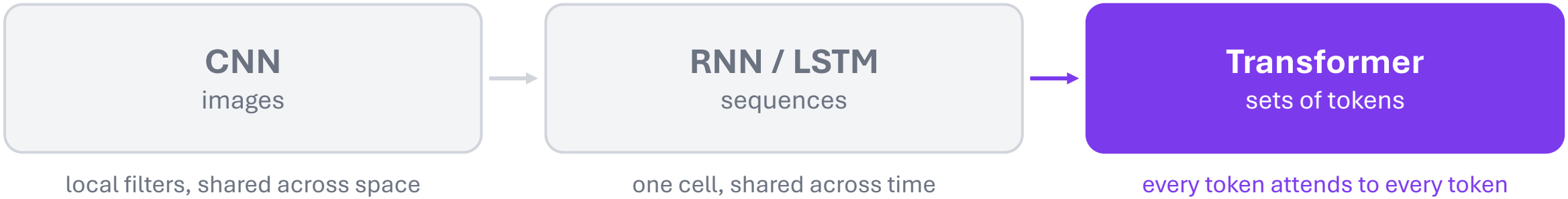


Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don’t work very well
- Common to use LSTM or GRU: additive interactions improve gradient flow
- Backward flow of gradients in RNN can explode or vanish.
  - Exploding is controlled with gradient clipping.
  - Vanishing is controlled with additive interactions (LSTM)
- 2023–26: linear RNNs are back in LLMs (Mamba, xLSTM, DeltaNet)
- Better understanding (both theoretical and empirical) is needed.

Part 3 · Attention and the Transformer



In this part

- Seq2seq: the bottleneck, then attention
- Self-attention: queries, keys and values
- Positions, masking, multiple heads
- The Transformer block; BERT, GPT and scaling

Sequence-to-Sequence with RNNs

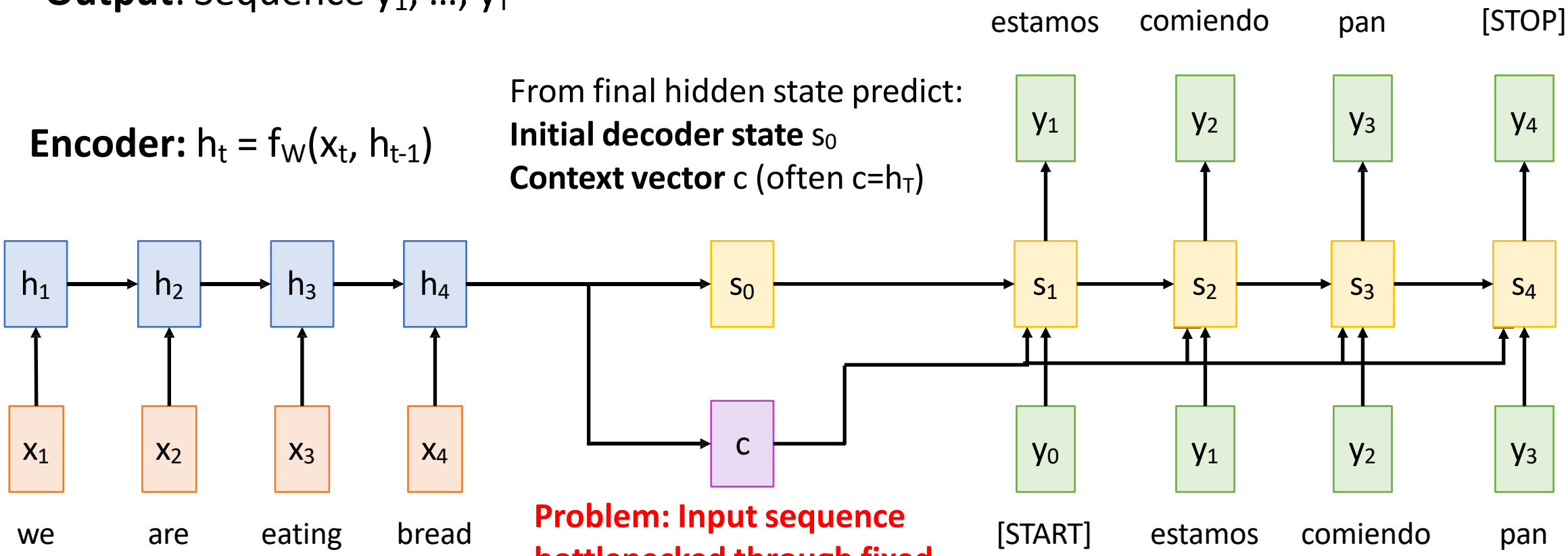
Input: Sequence $x_1, \ldots x_T$

Output: Sequence $y_1, \ldots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Problem: Input sequence bottlenecked through fixed-sized vector. What if T=1000?

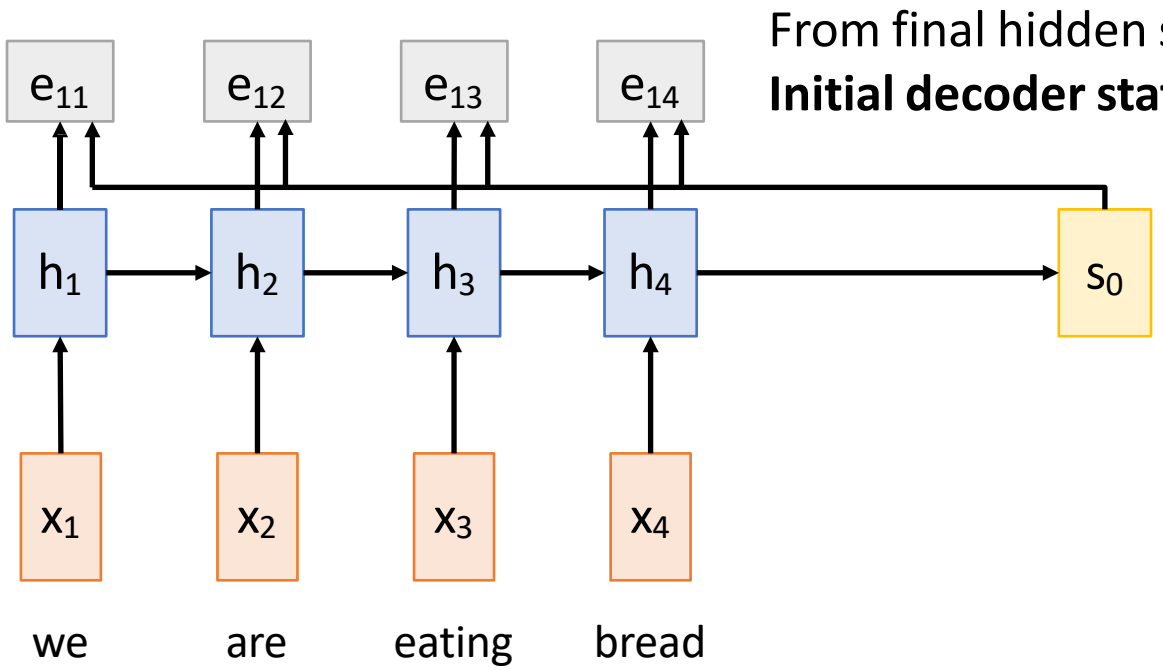
Idea: use new context vector at each step of decoder!

Sutskever et al, "Sequence to sequence learning with neural networks", NeurIPS 2014

Sequence-to-Sequence with RNNs and Attention

Compute (scalar) **alignment scores**

$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is an MLP)

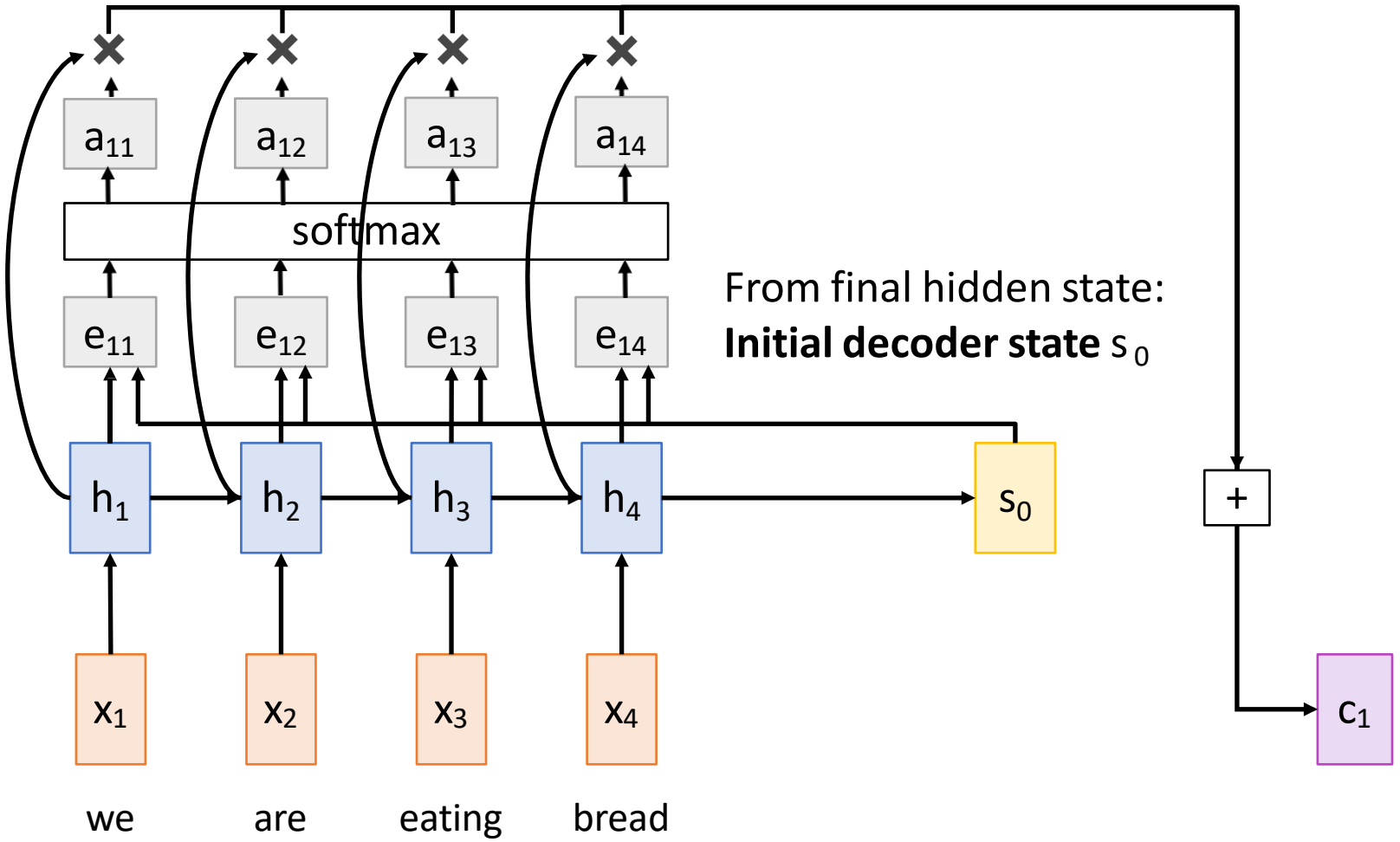


From final hidden state:

Initial decoder state s_0

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence-to-Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

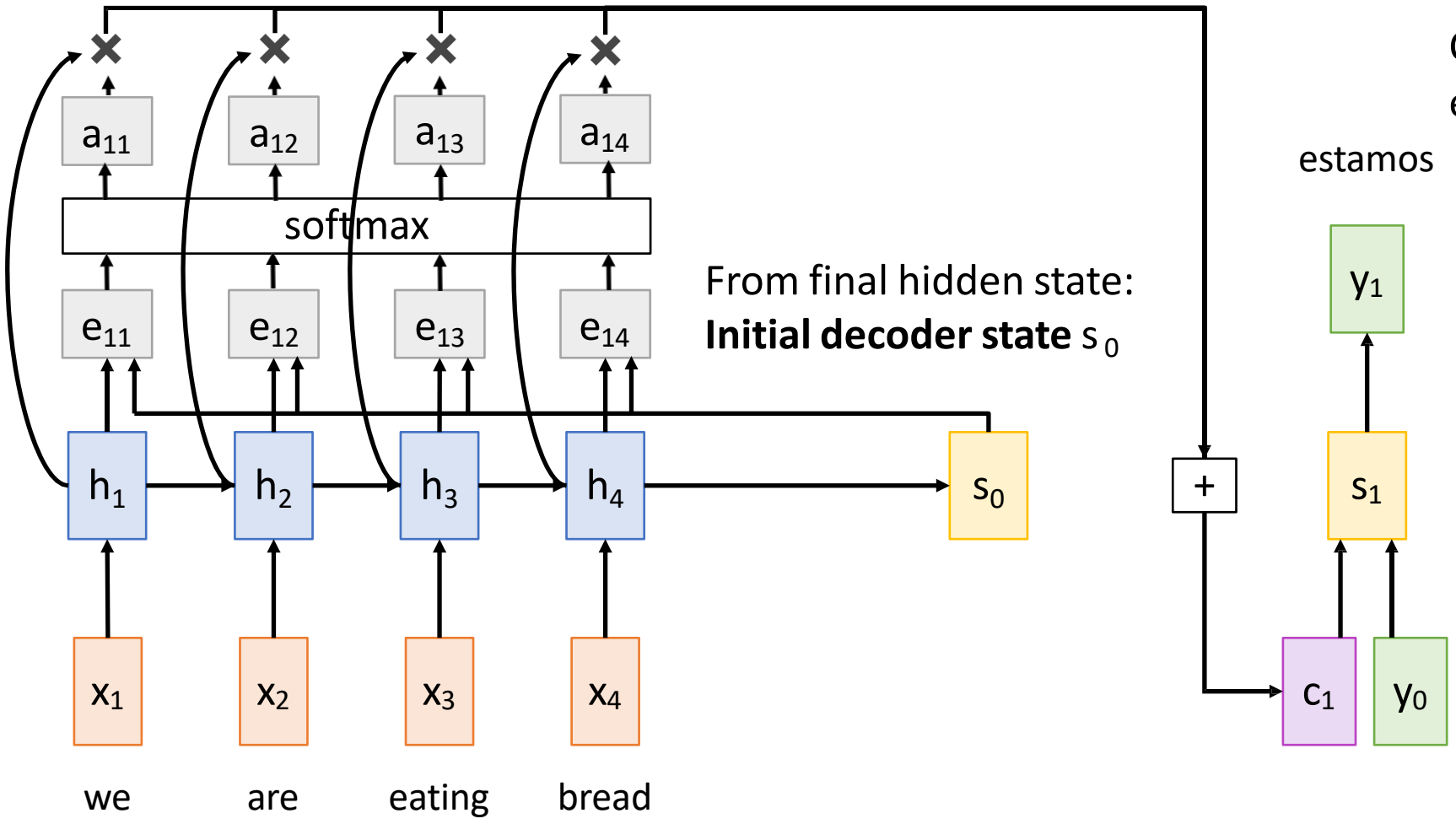
Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 0$

Compute context vector as linear combination of hidden states
$c_t = \sum_i a_{t,i} h_i$

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence-to-Sequence with RNNs and Attention



Compute (scalar) **alignment scores**

$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**

$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 0$

Compute context vector as linear combination of hidden states

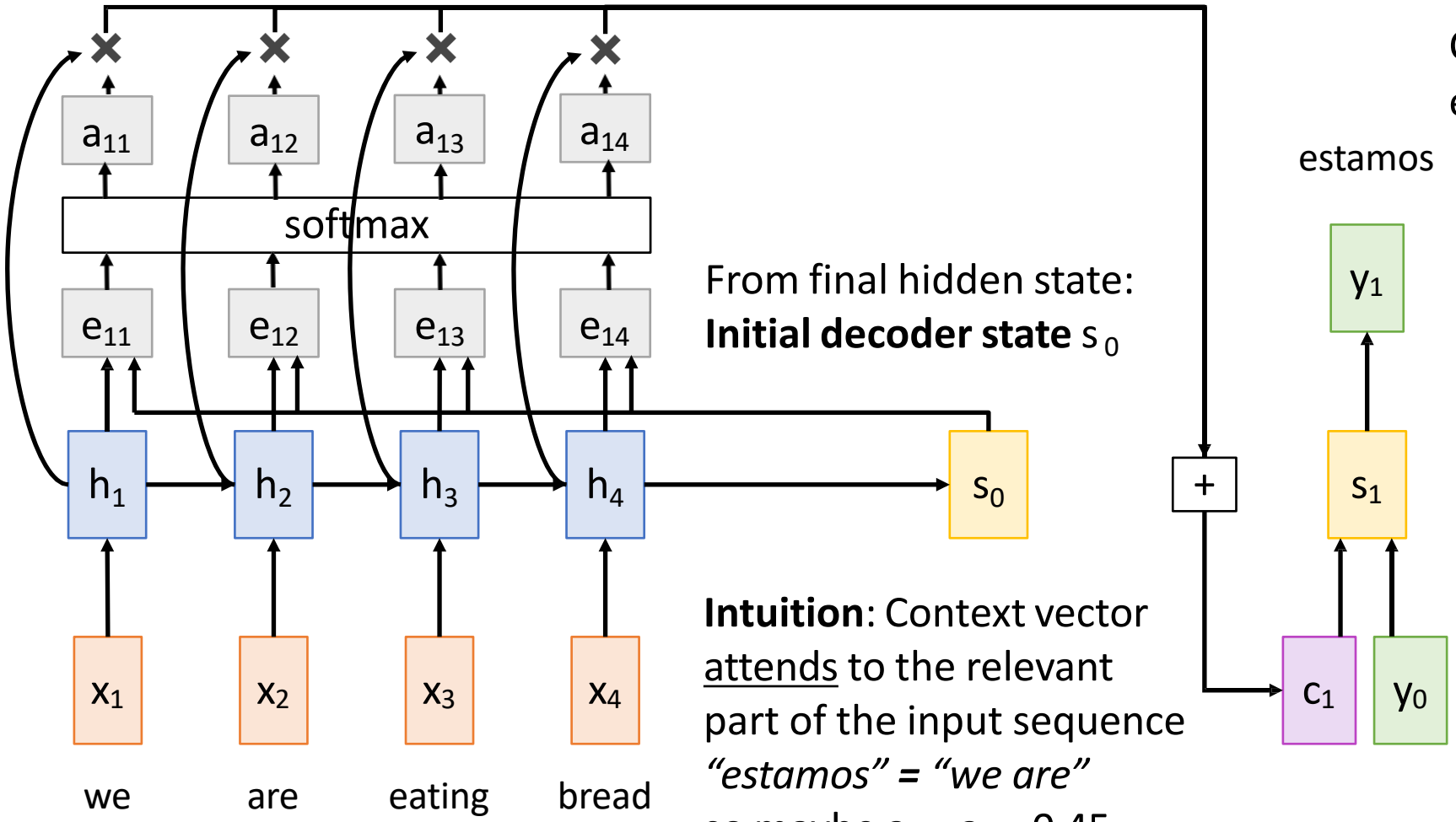
$c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

This is all differentiable! Do not supervise attention weights – backprop through everything

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence-to-Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

Intuition: Context vector attends to the relevant part of the input sequence
"estamos" = "we are"
so maybe a_{11}=a_{12}=0.45, a_{13}=a_{14}=0.05

Compute (scalar) **alignment scores**
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
$0 < a_{t,i} < 1$ $\sum_i a_{t,i} = 0$

Compute context vector as linear combination of hidden states
$c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

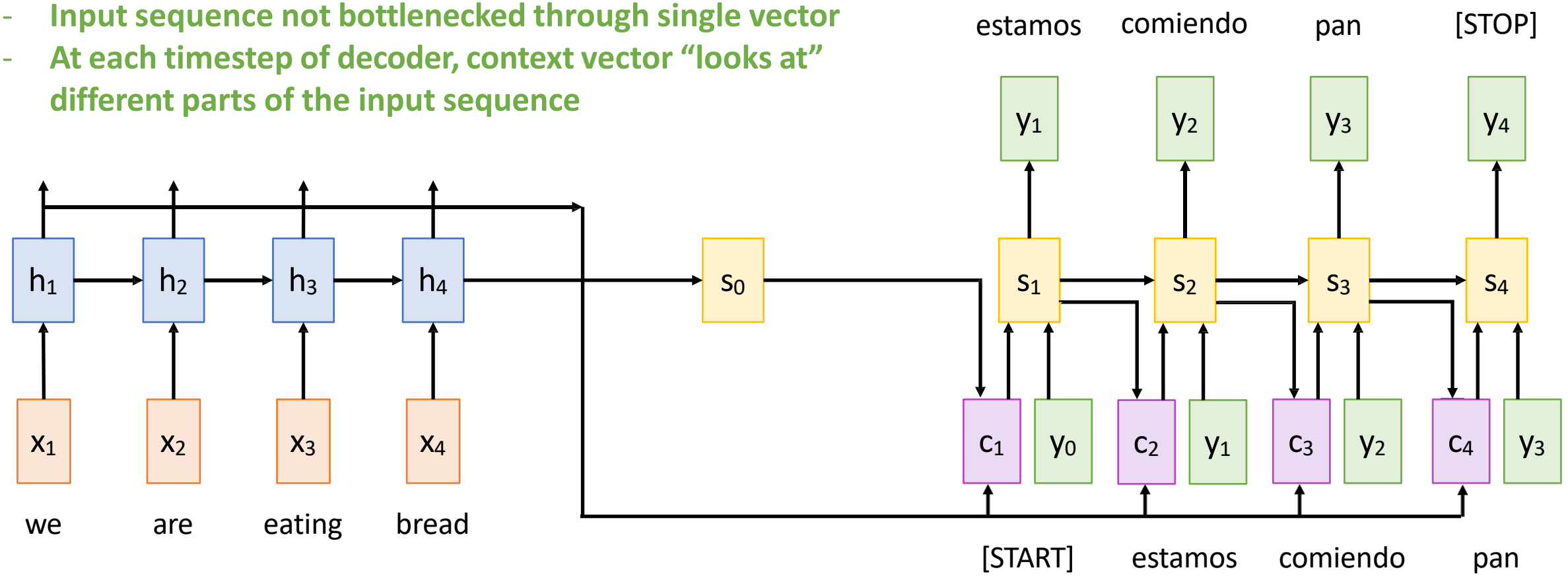
This is all differentiable! Do not supervise attention weights – backprop through everything

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence-to-Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- **Input sequence not bottlenecked through single vector**
- **At each timestep of decoder, context vector "looks at" different parts of the input sequence**



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

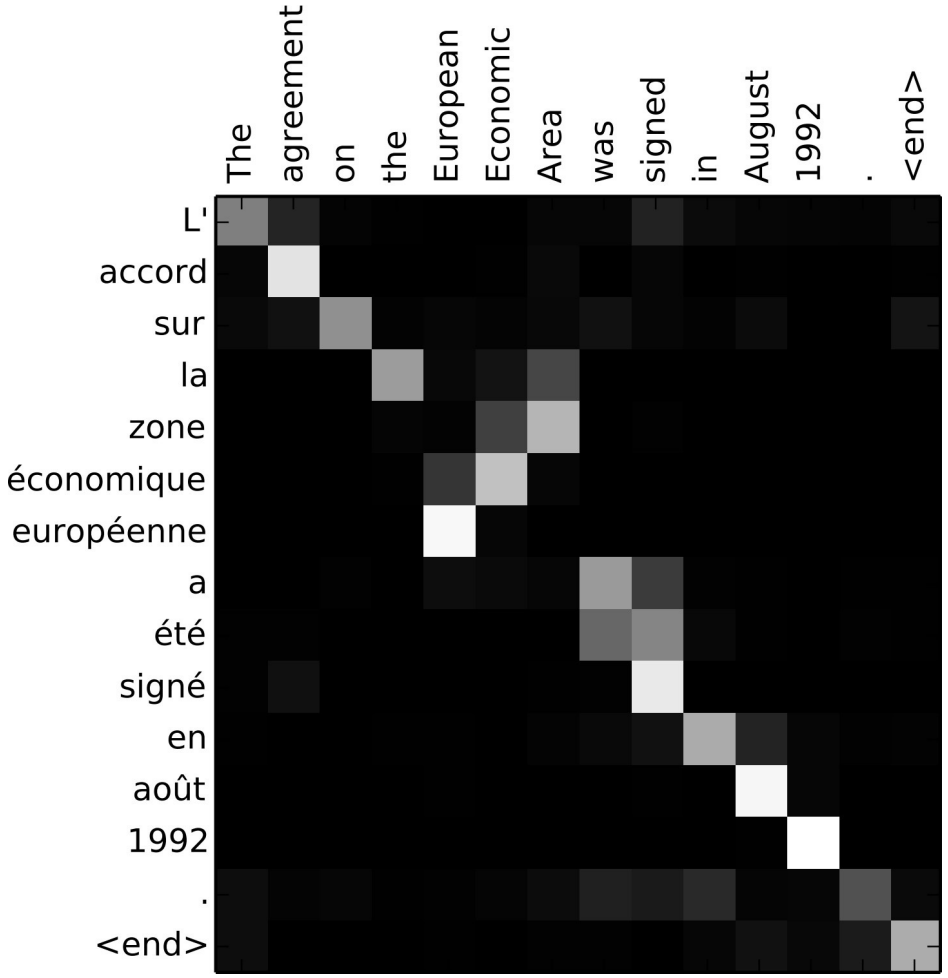
Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: "The agreement on the European Economic Area was signed in August 1992."

Output: "L'accord sur la zone économique européenne a été signé en août 1992."

Visualize attention weights $a_{t,i}$



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

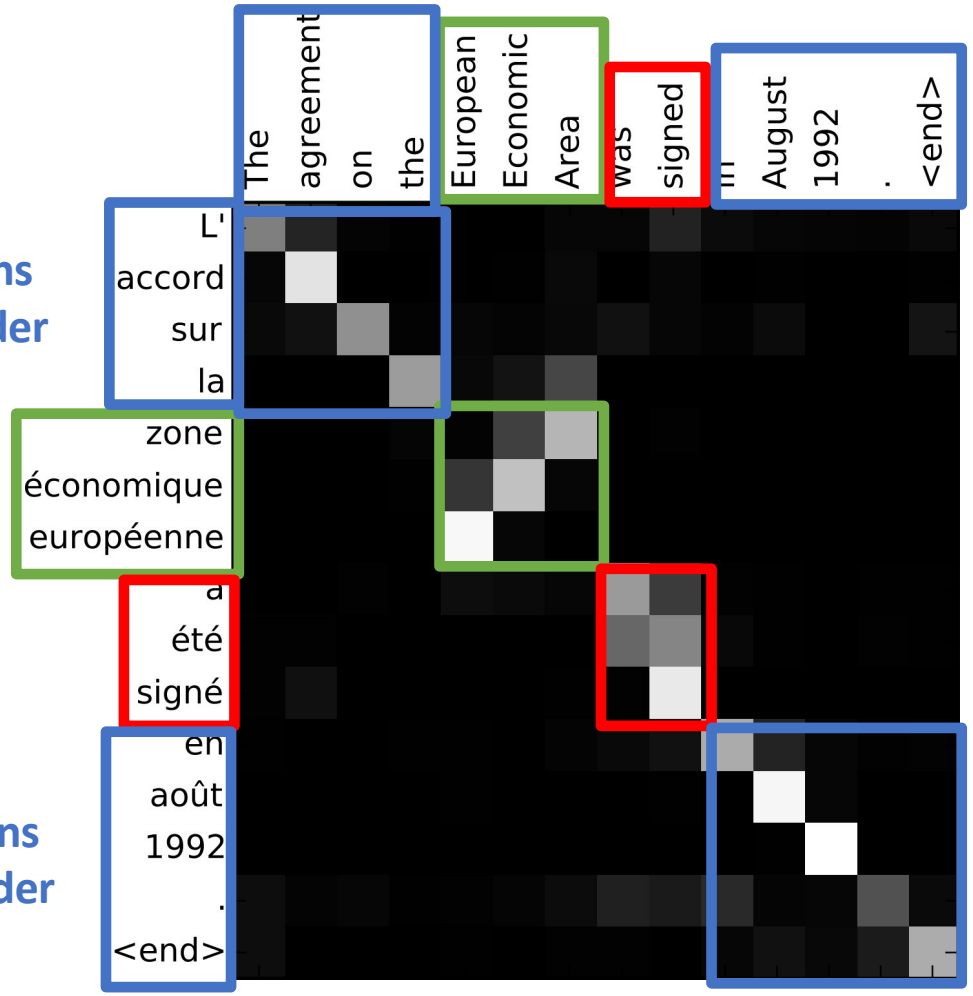
Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: "The agreement on the European Economic Area was signed in August 1992."

Output: "L'accord sur la zone économique européenne a été signé en août 1992."

Visualize attention weights $a_{t,i}$

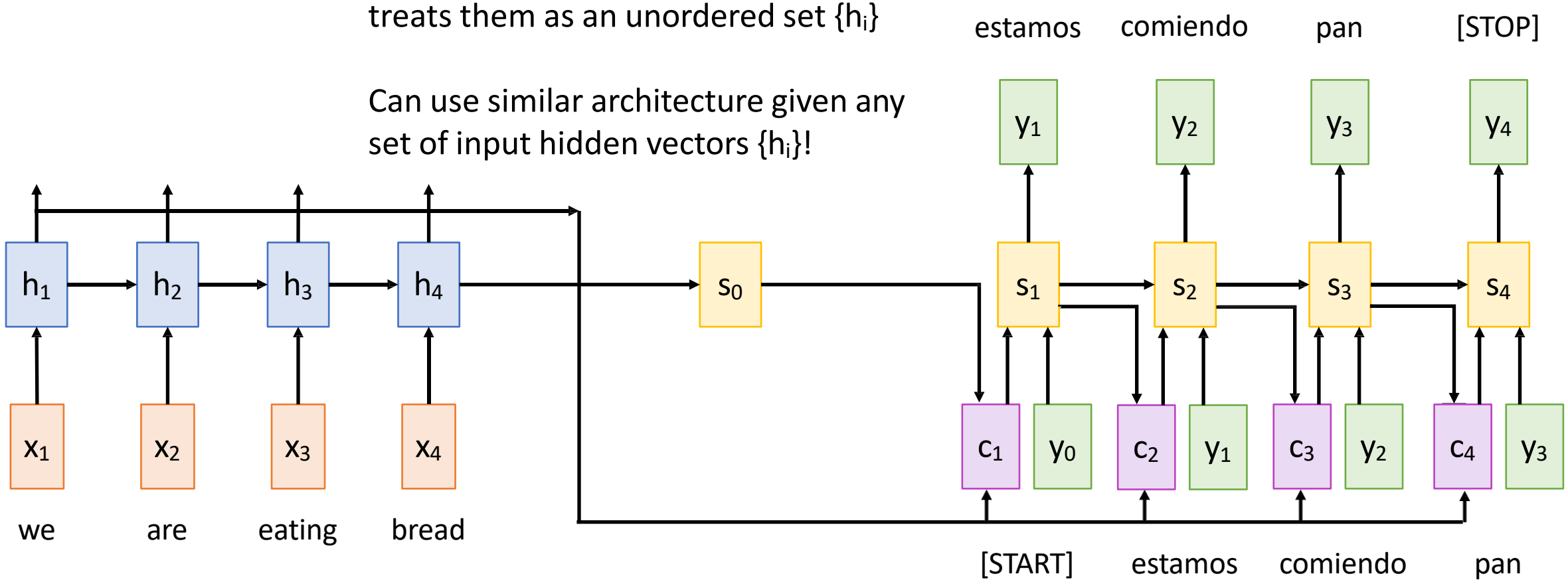


Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Sequence-to-Sequence with RNNs and Attention

The decoder doesn't use the fact that h_i form an ordered sequence – it just treats them as an unordered set $\{h_i\}$

Can use similar architecture given any set of input hidden vectors $\{h_i\}$!



Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Image Captioning with RNNs and Attention

$e_{t,i,j} = f_{att}(s_{t-1}, h_{i,j})$
$a_{t,:,:} = \text{softmax}(e_{t,:,:})$
$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$

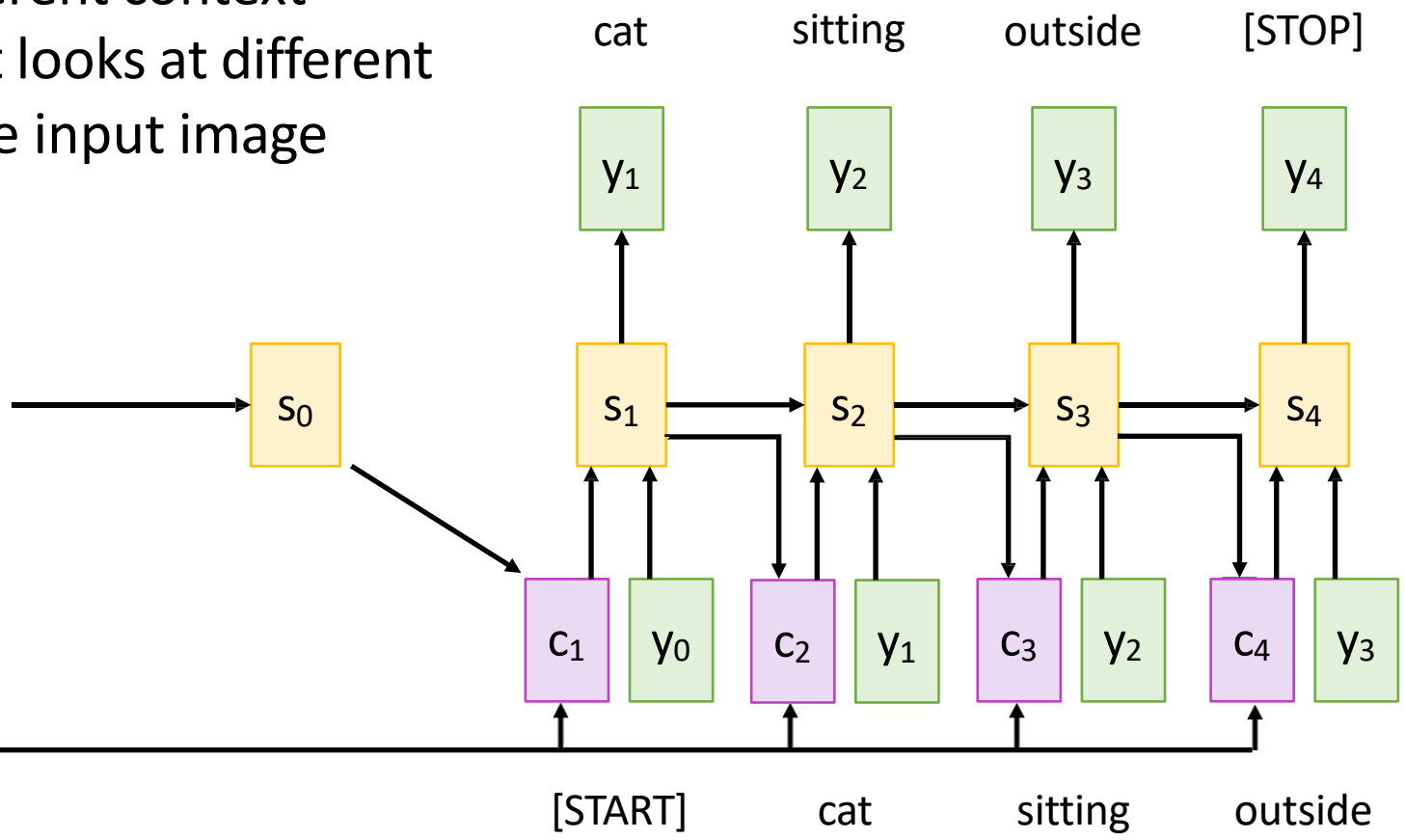
Each timestep of decoder uses a different context vector that looks at different parts of the input image



CNN

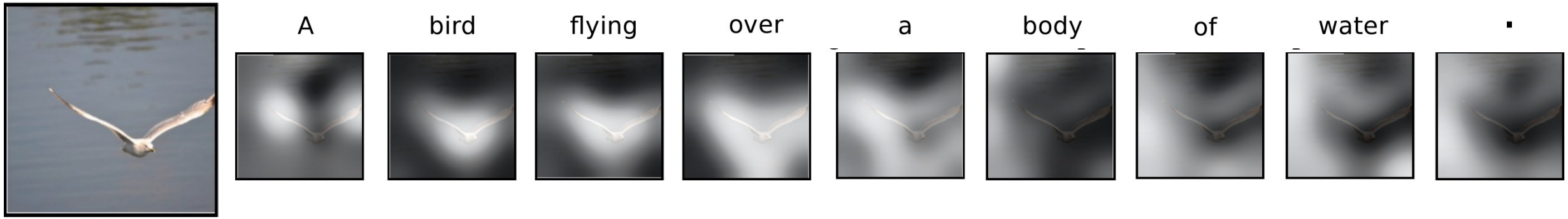
$h_{1,1}$ $h_{1,2}$ $h_{1,3}$
$h_{2,1}$ $h_{2,2}$ $h_{2,3}$
$h_{3,1}$ $h_{3,2}$ $h_{3,3}$

Use a CNN to compute a grid of features for an image



Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention



Xu et al, “Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention”, ICML 2015

X, Attend, and Y

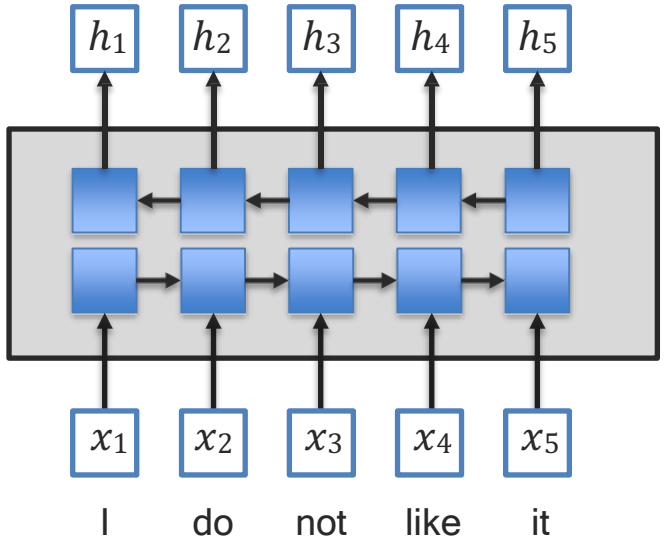
"**Show, attend, and tell**" *(Xu et al, ICML 2015)*
Look at image, attend to image regions, produce question

"**Ask, attend, and answer**" *(Xu and Saenko, ECCV 2016)*
"**Show, ask, attend, and answer"** *(Kazemi and Elqursh, 2017)*
Read text of question, attend to image regions, produce answer

"**Listen, attend, and spell**" *(Chan et al, ICASSP 2016)*
Process raw audio, attend to audio regions while producing text

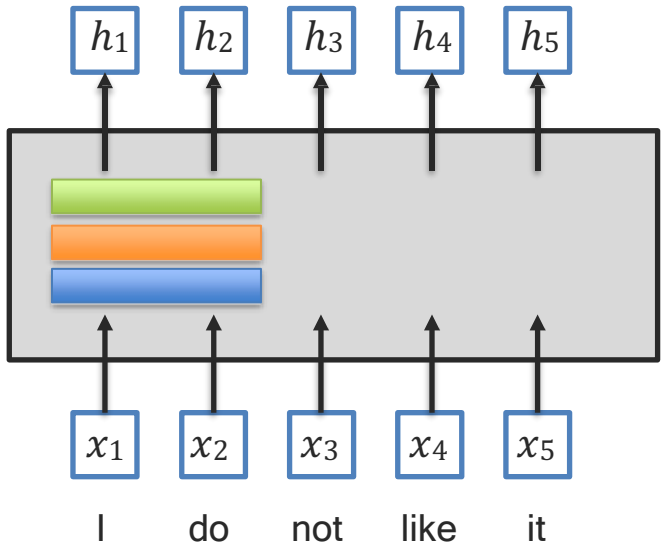
"**Listen, attend, and walk**" *(Mei et al, AAAI 2016)*
Process text, attend to text regions, output navigation commands

Attention for sequence representation?
Recall: Bi-directional LSTM



But harder to parallelize…

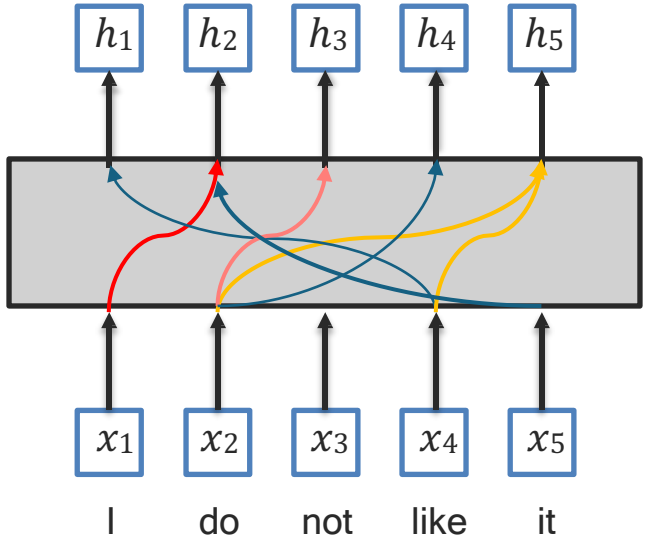
Recall : Convolutions



Can be parallelized!

But modeling long-range dependencies require multiple layers

Self-attention



Can be parallelized!

Long-range dependencies

Dynamic attention weights

Attention Layer

Inputs:

Query vectors: Q (Shape: N_Q x D_Q)

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

Value matrix: W_V (Shape: D_X x D_V)

Computation:

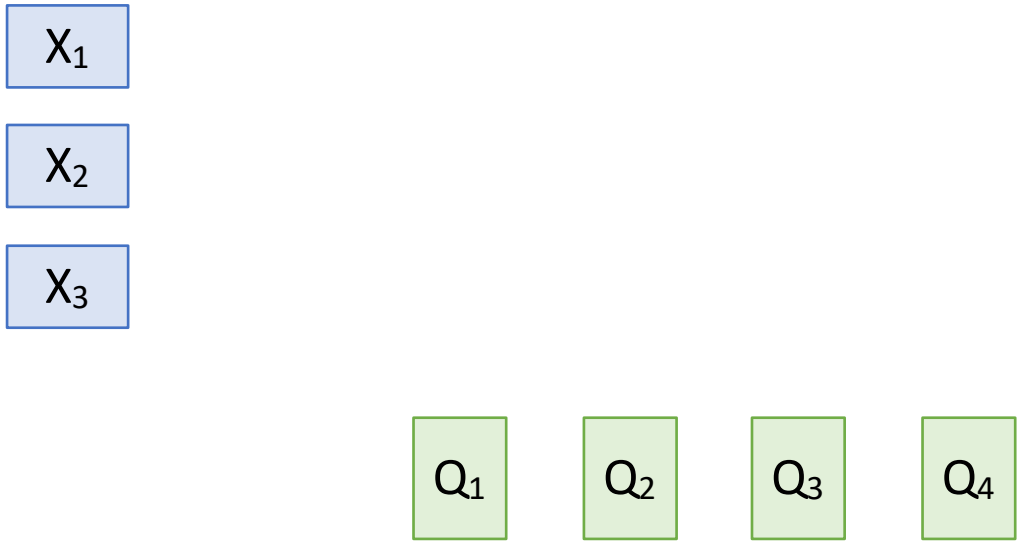
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)

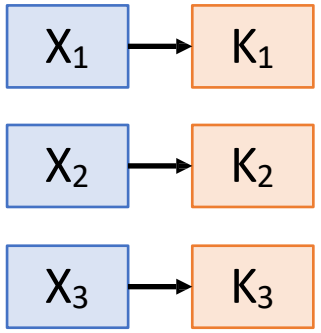
Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$

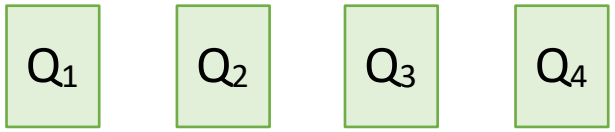


Attention Layer

Inputs:
Query vectors: Q (Shape: N_Q x D_Q)
Input vectors: X (Shape: N_X x D_X)
Key matrix: W_K (Shape: D_X x D_Q)
Value matrix: W_V (Shape: D_X x D_V)

Computation:
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
Value Vectors: $V = XW_V$ (Shape: N_X x D_V)
Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)
Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$





Attention Layer

Inputs:

Query vectors: Q (Shape: N_Q x D_Q)

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

Value matrix: W_V (Shape: D_X x D_V)

Computation:

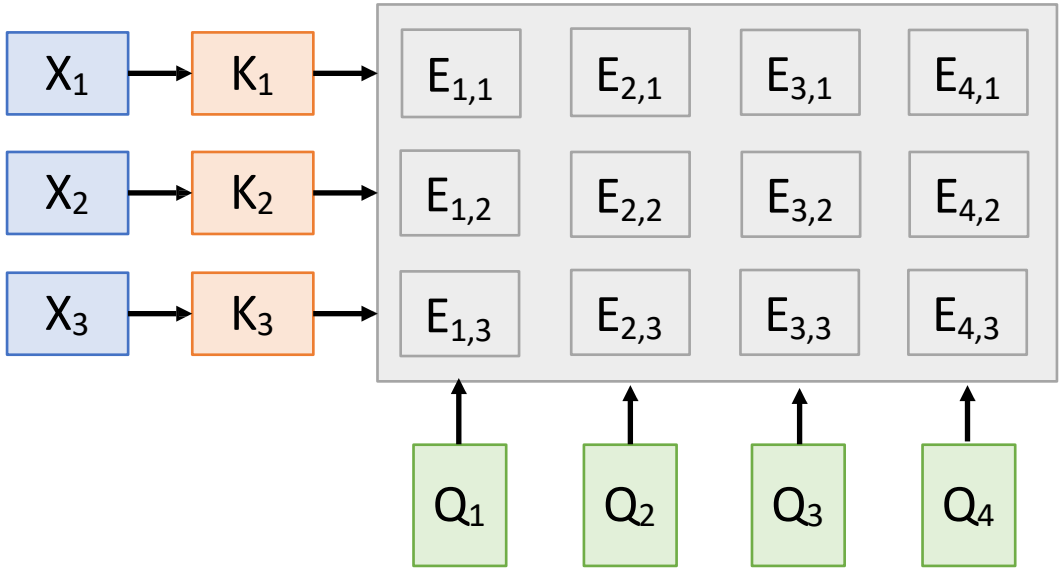
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)

Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$



Attention Layer

We used the correlation matrix before for attention pooling

			
What	1.2	2.3	1.7
animal	-0.4	6.5	4.2
is	1.3	1.2	0.6

Inputs:

Query vectors: Q (Shape: N_Q x D_Q)

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

Value matrix: W_V (Shape: D_X x D_V)

Computation:

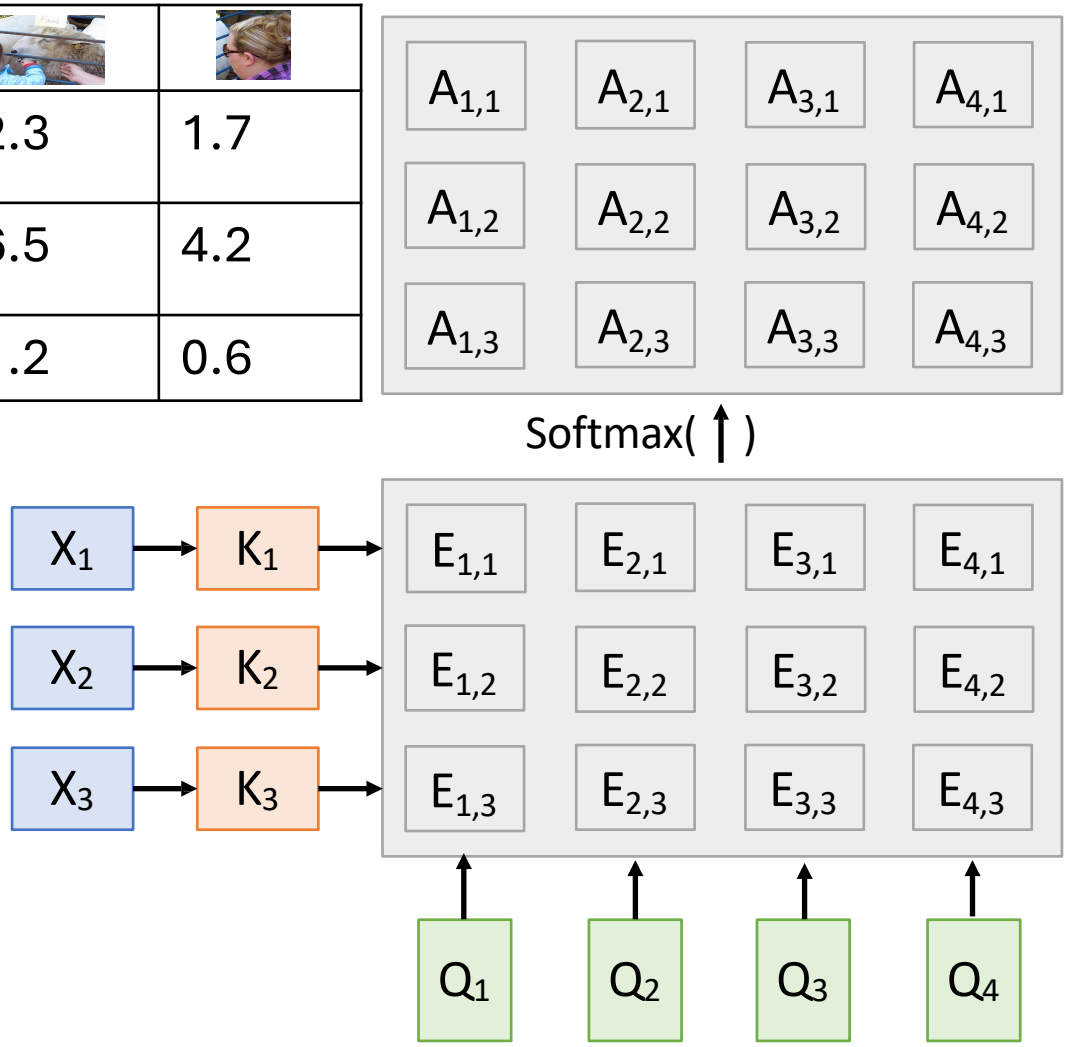
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)

Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$



Attention Layer

Inputs:

Query vectors: Q (Shape: N_Q x D_Q)

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

Value matrix: W_V (Shape: D_X x D_V)

Computation:

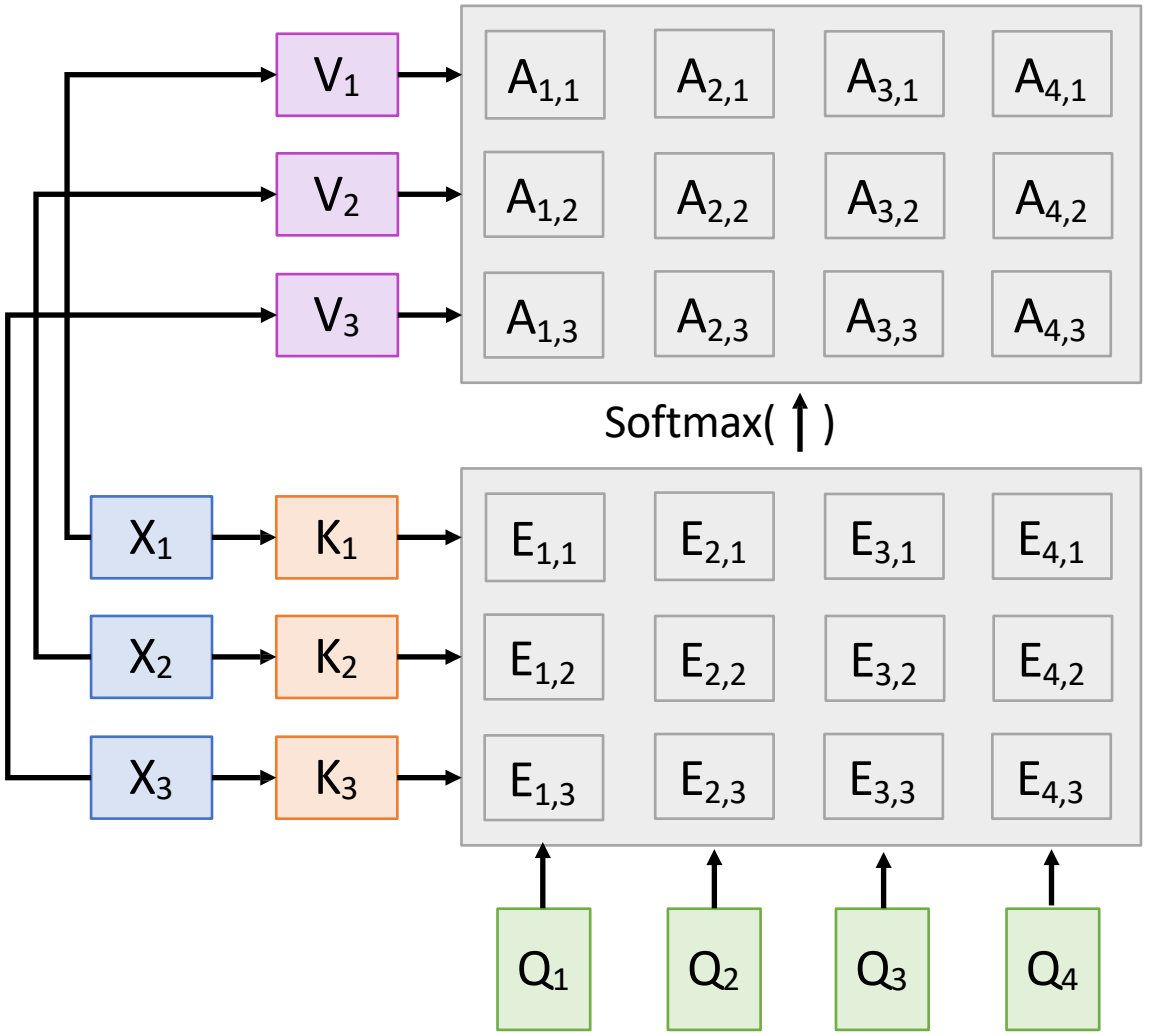
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)

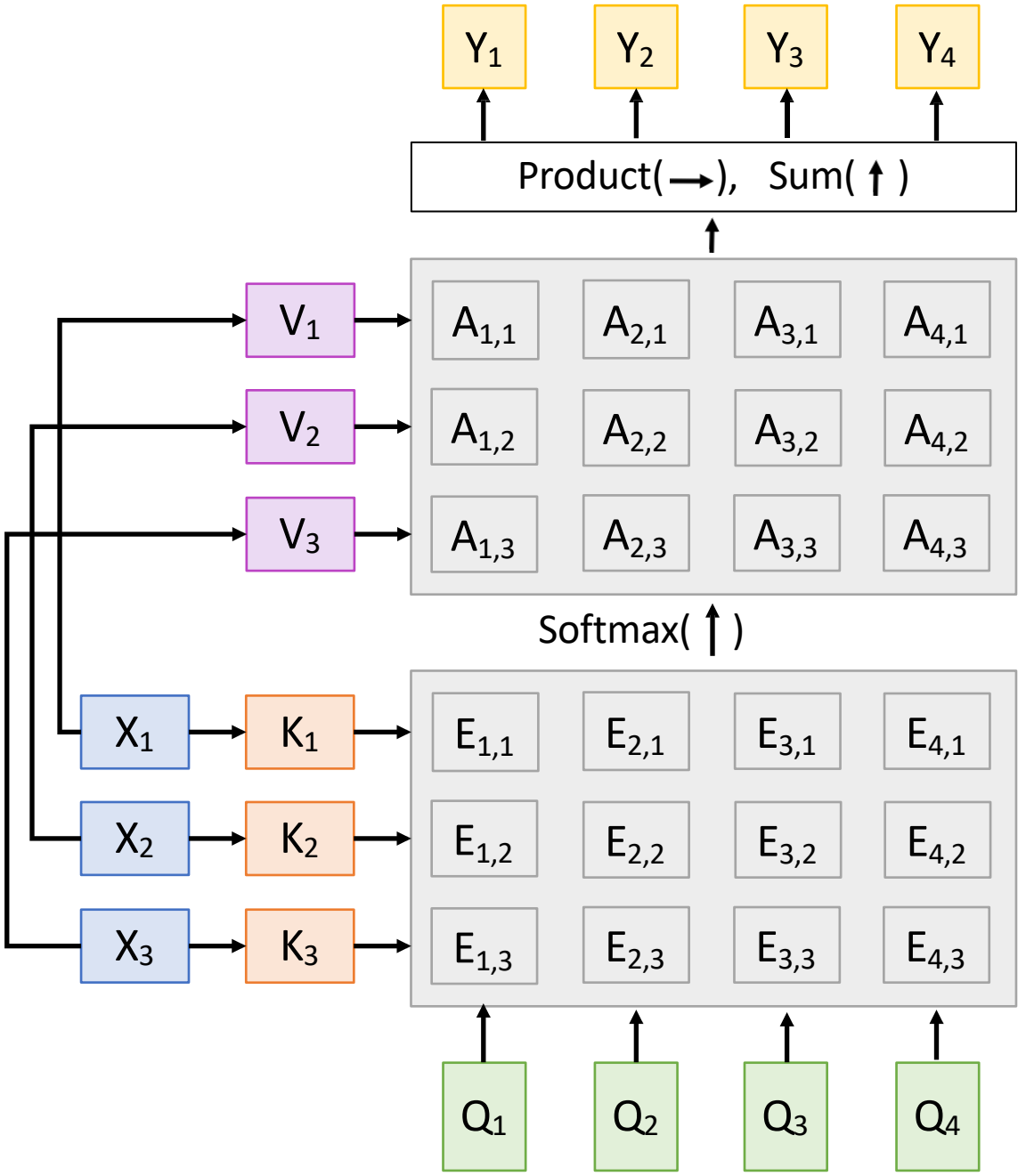
Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$



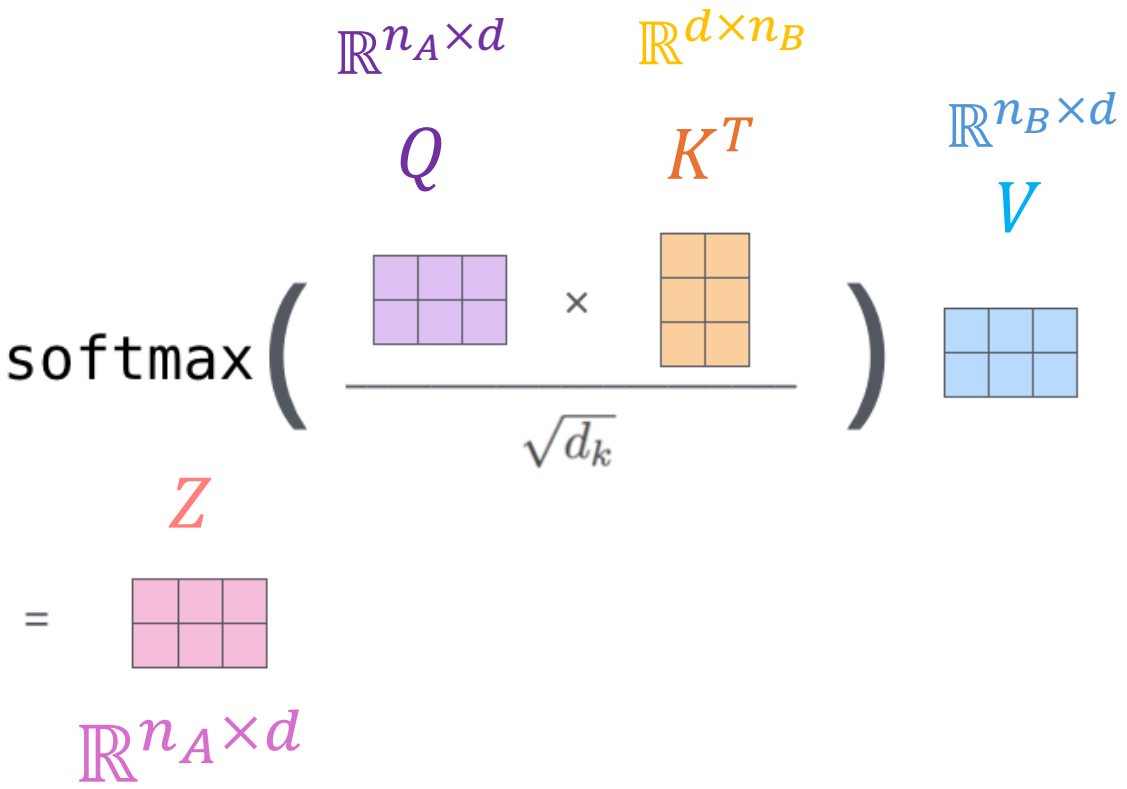
Attention Layer

Inputs:
Query vectors: Q (Shape: N_Q x D_Q)
Input vectors: X (Shape: N_X x D_X)
Key matrix: W_K (Shape: D_X x D_Q)
Value matrix: W_V (Shape: D_X x D_V)

Computation:
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
Value Vectors: $V = XW_V$ (Shape: N_X x D_V)
Similarities: $E = QK^T$ (Shape: N_Q x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: A = softmax(E, dim=1) (Shape: N_Q x N_X)
Output vectors: $Y = AV$ (Shape: N_Q x D_V) $Y_i = \sum_j A_{i,j} V_j$



Intuitively, attention layer

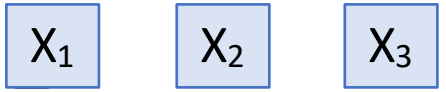


Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

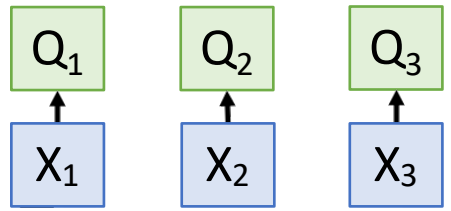


Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

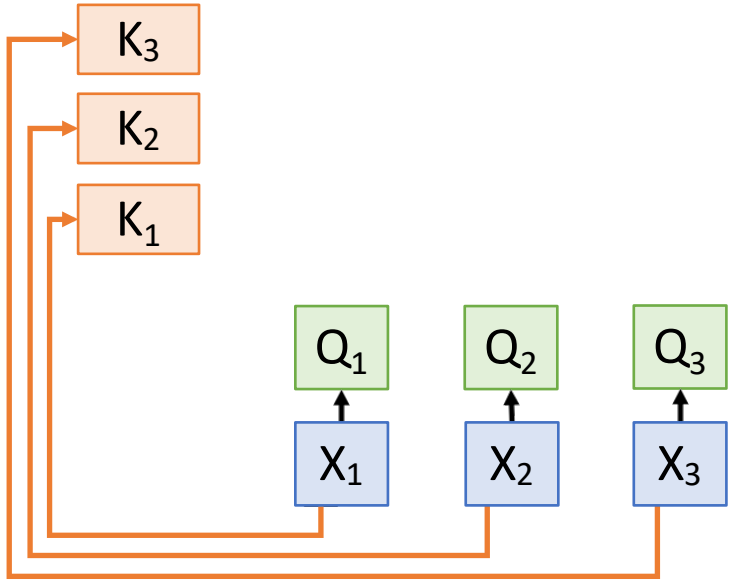
Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)



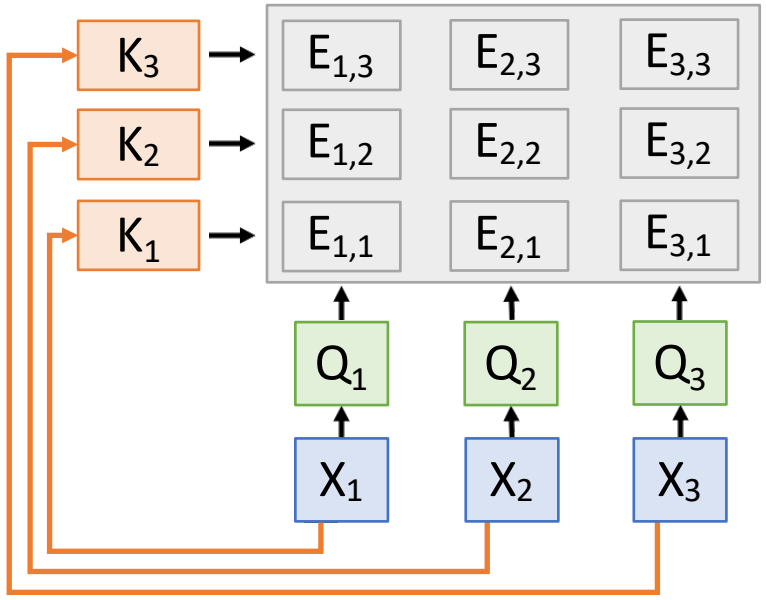
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)



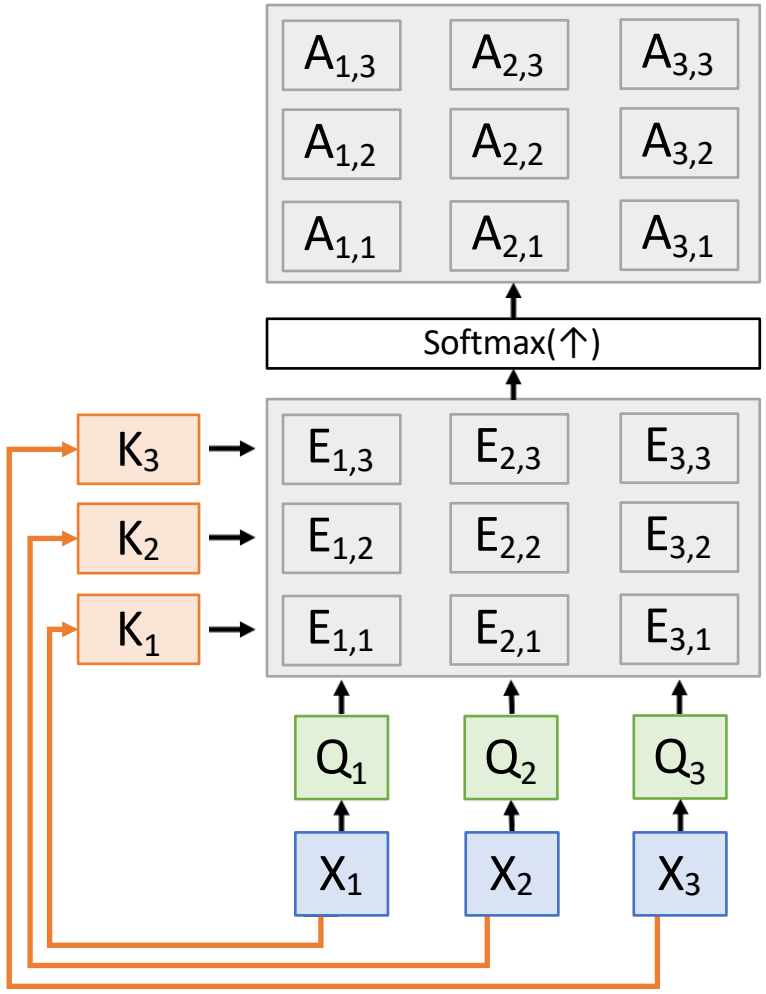
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)



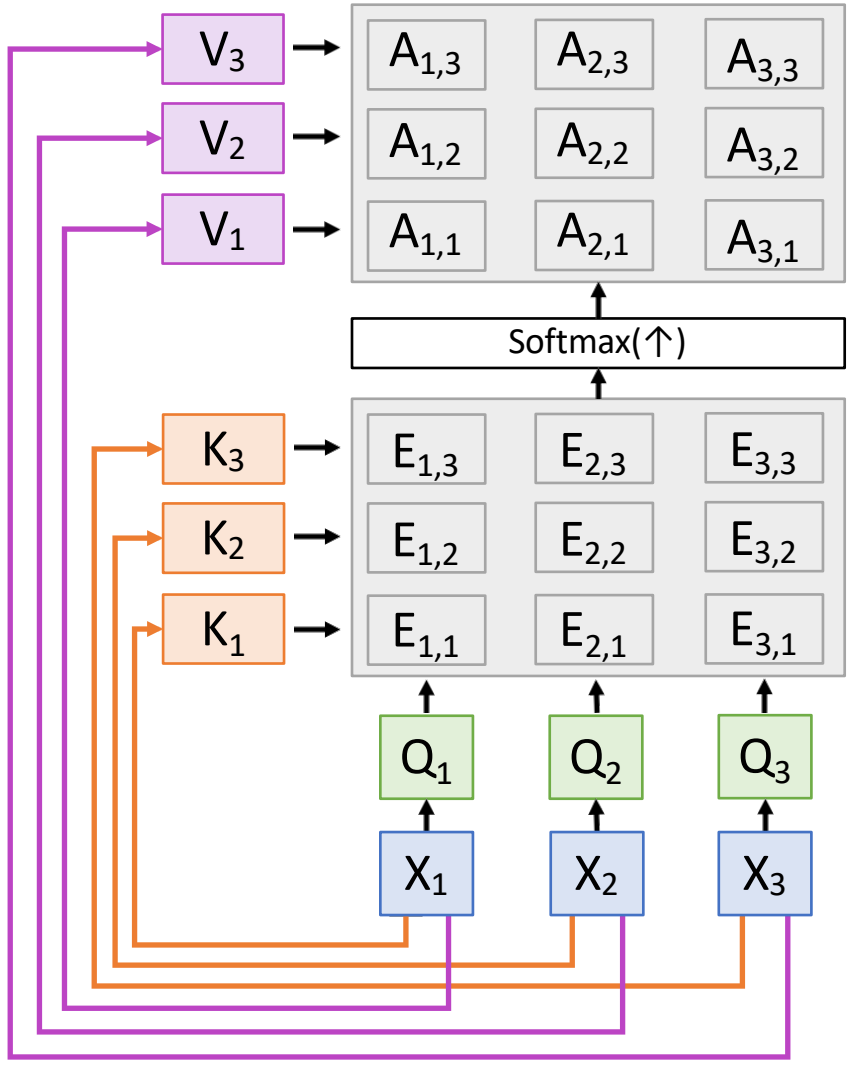
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$



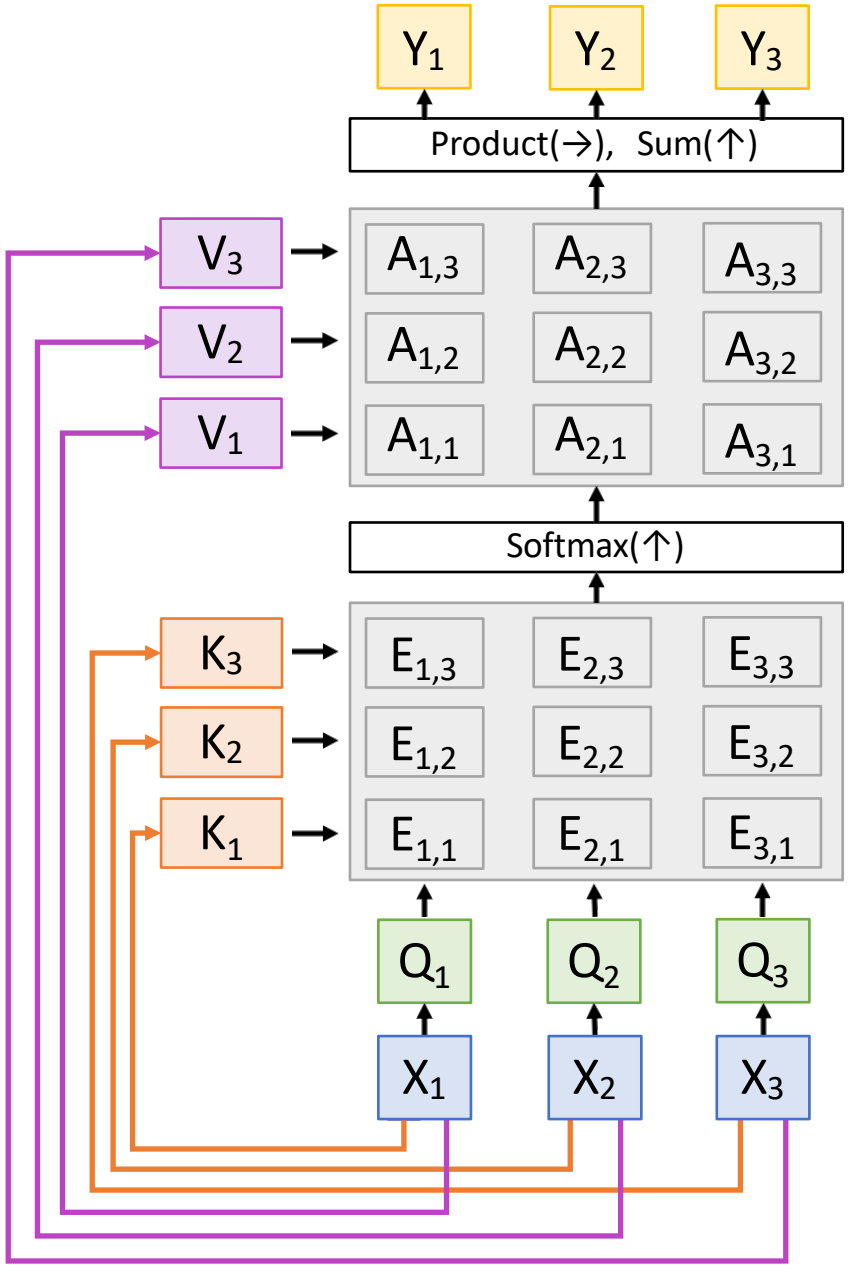
Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

Self-Attention Layer

One **query** per **input vector**

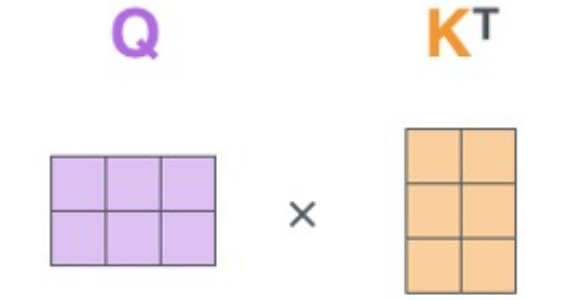
- Inputs:
- Input vectors: X (Shape: N_X x D_X) Key matrix: W_K (Shape: D_X x D_Q) Value matrix: W_V (Shape: D_X x D_V) Query matrix: W_Q (Shape: D_X x D_Q)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: N_X x D_Q)
- Value Vectors: $V = XW_v$ (Shape: N_X x D_V)
- Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \sqrt{D_Q}$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X) Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$



Self-correlation matrix

Correlation Matrix:

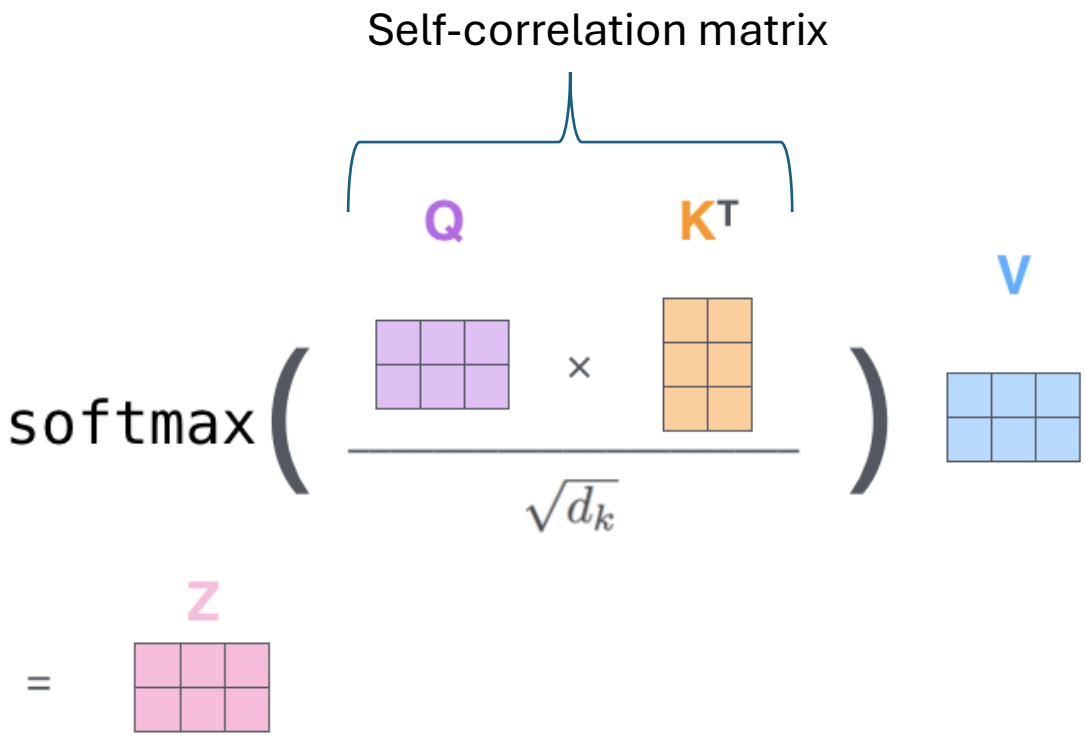


$$\in \mathbb{R}^{n \times n}$$

	What	animal	is
What	0.4	0.1	0.1
animal	0.1	0.3	0.1
is	0.1	0.3	0.3

Intuitively, Self-Attention-layer

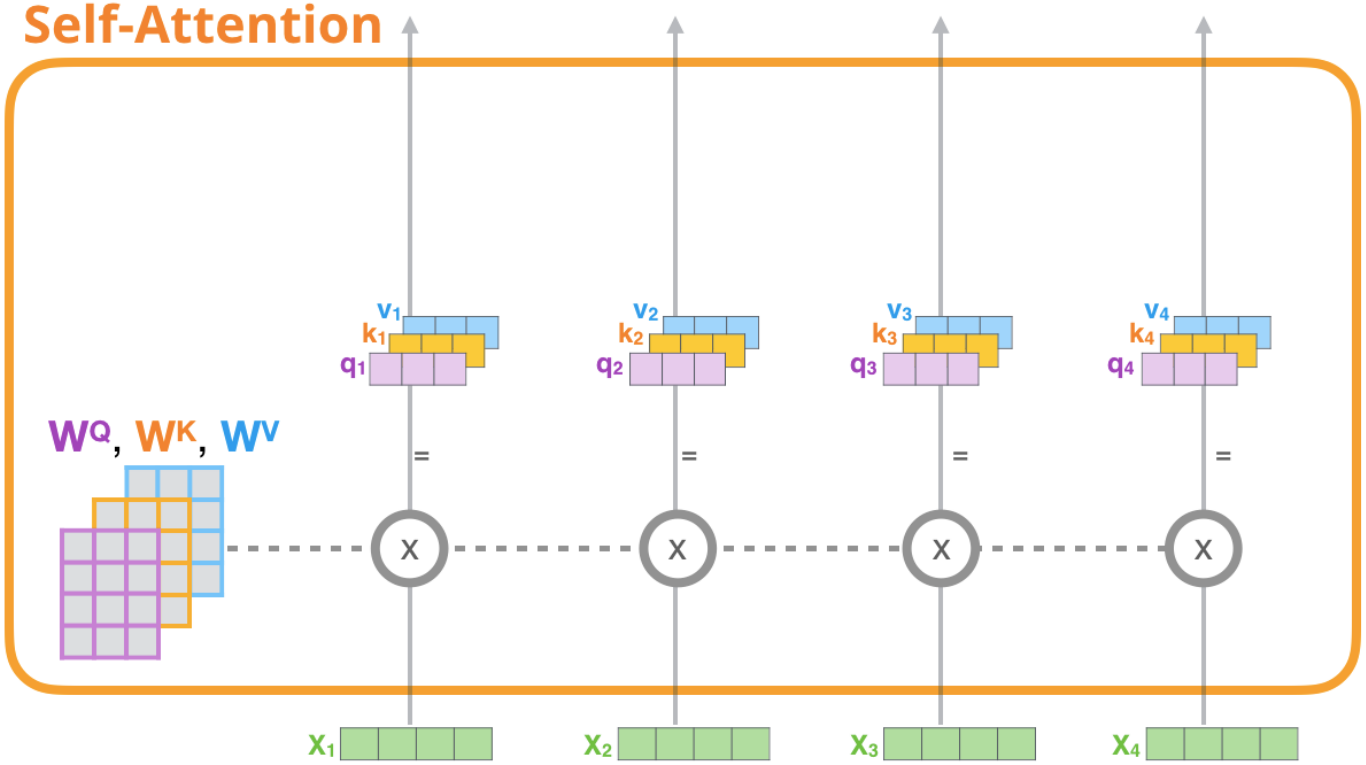
- Given a sequence $x \in \mathbb{R}^{n \times d}$ (e.g., a sentence)



$$Q = x \cdot W_Q \in \mathbb{R}^{n \times d}$$
$$K = x \cdot W_K \in \mathbb{R}^{n \times d}$$
$$V = x \cdot W_V \in \mathbb{R}^{n \times d}$$
$$W_Q, W_K, W_v \in \mathbb{R}^{d \times d}$$

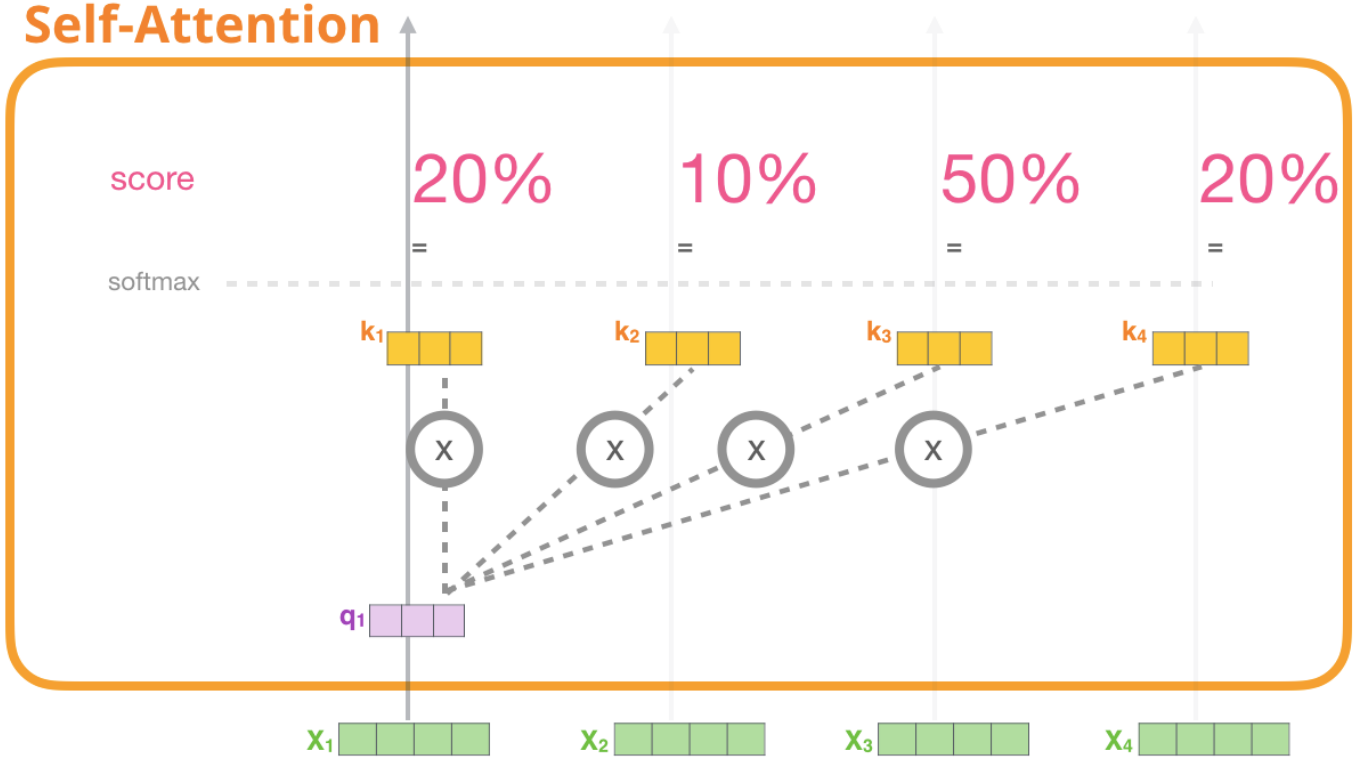
Summary: Self-Attention-layer

1) For each input token, create a query vector, a key vector, and a value vector by multiplying by weight Matrices W^Q, W^K, W^V



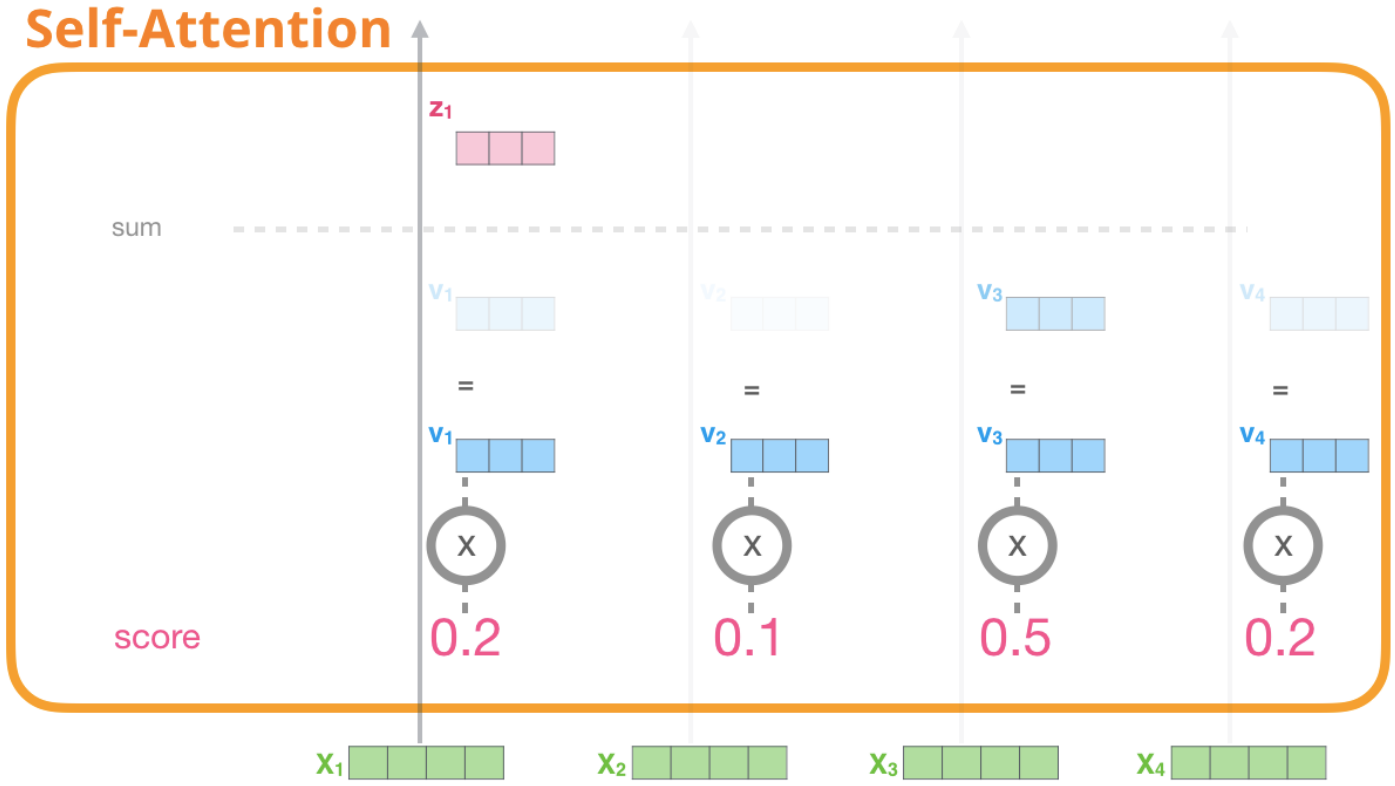
Self-Attention-layer

2) Multiply (dot product) the current query vector, by all the key vectors, to get a score of how well they match



Self-Attention-layer

3) Multiply the value vectors by the scores, then sum up



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting** the input vectors:

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$

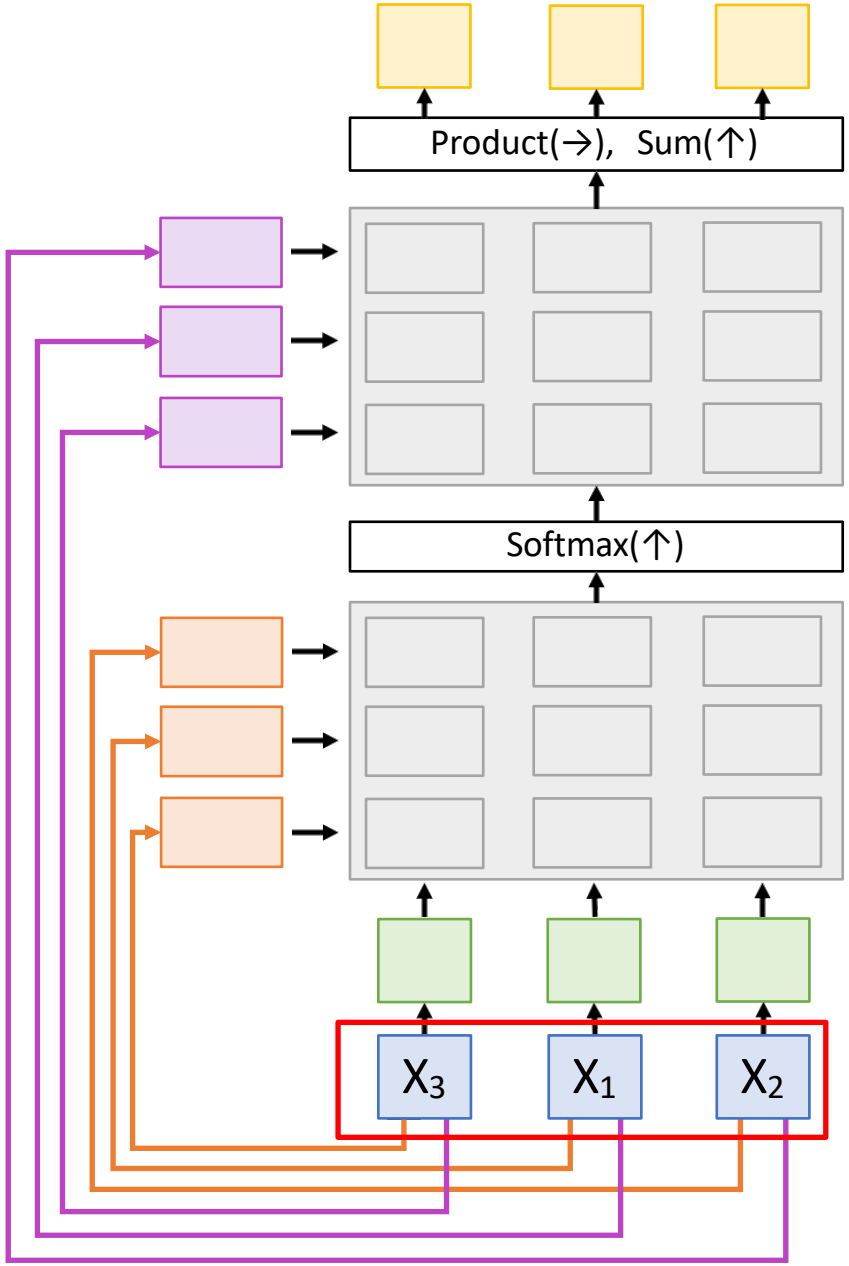
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting** the input vectors:

Queries and Keys will be the same, but permuted

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$

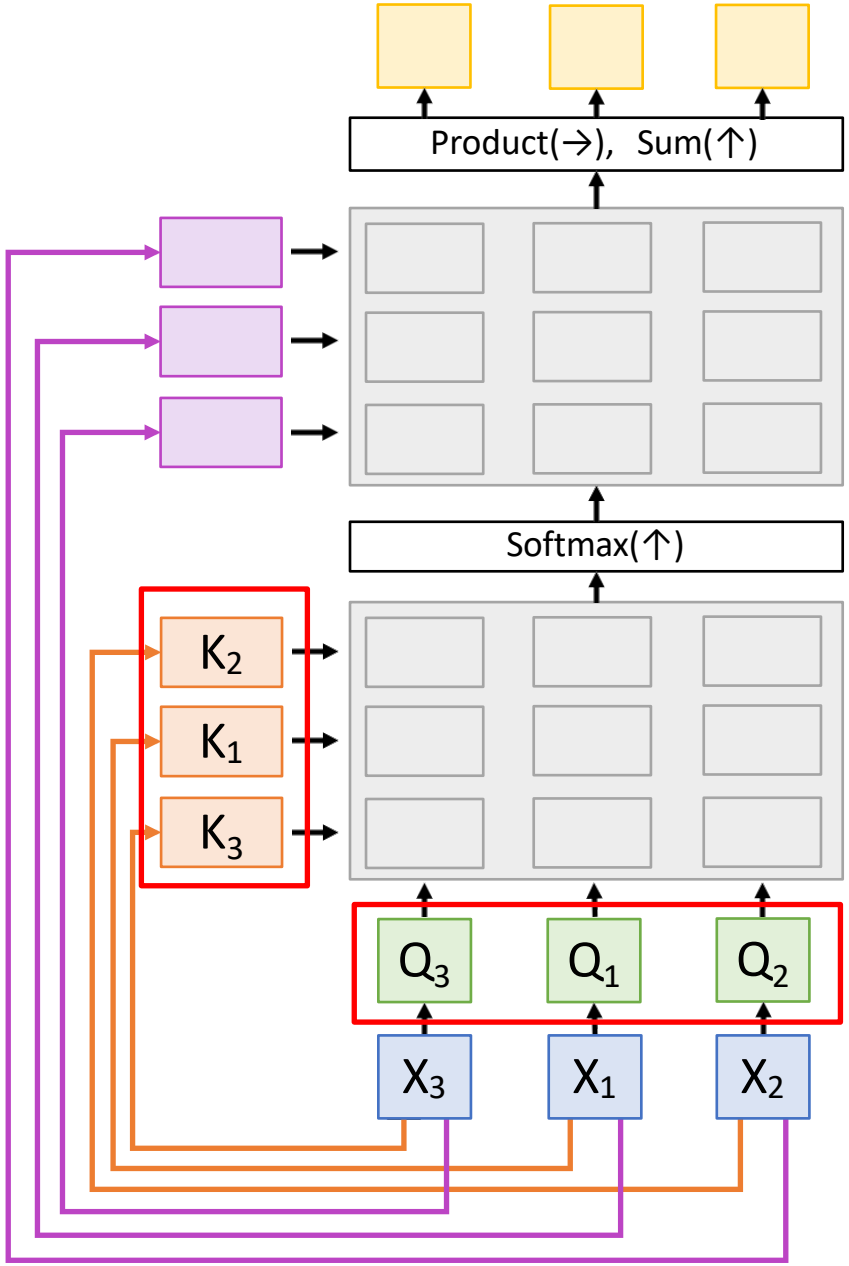
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting** the input vectors:

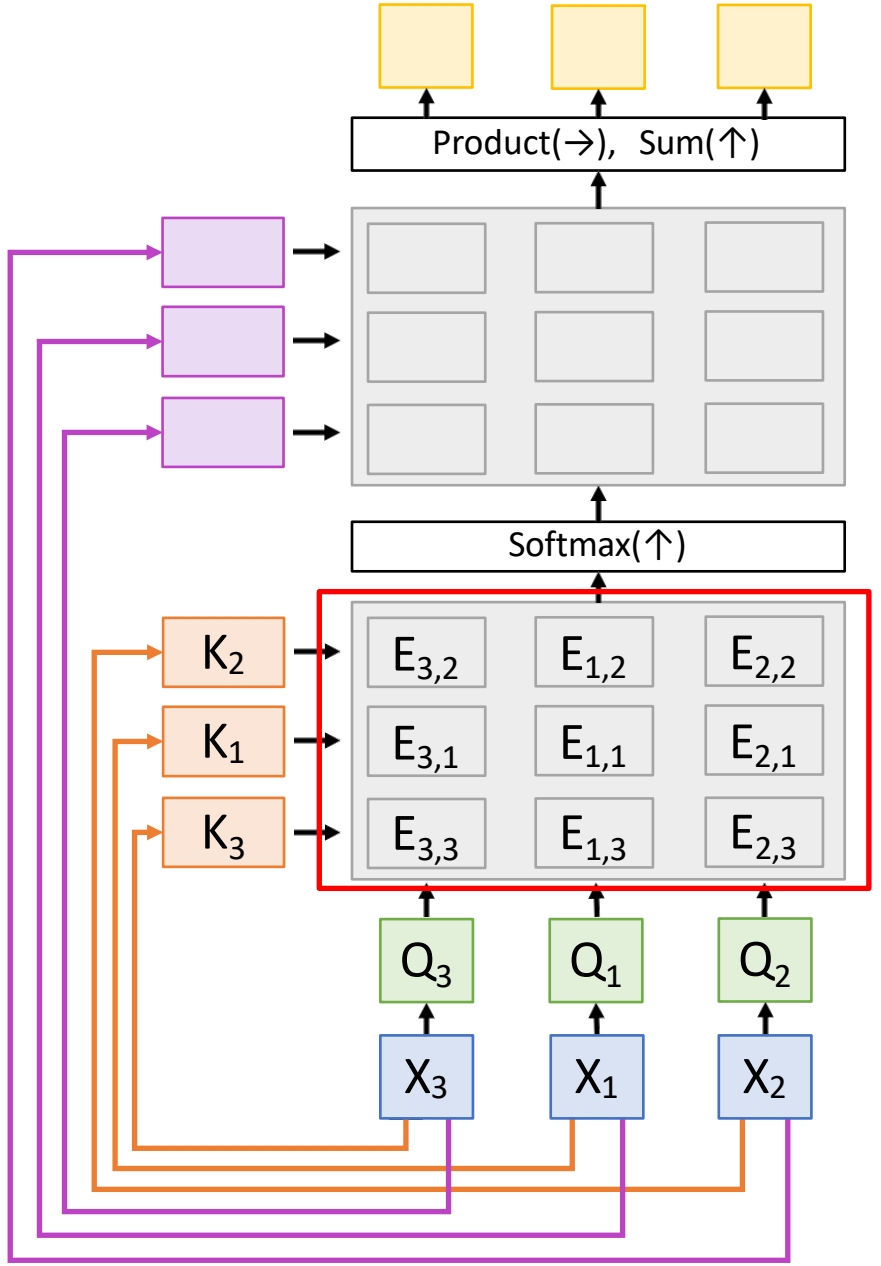
Similarities will be the same, but permuted

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)
Key matrix: W_K (Shape: $D_X \times D_Q$)
Value matrix: W_V (Shape: $D_X \times D_V$)
Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)
Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)
Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)
Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting** the input vectors:

Attention weights will be the same, but permuted

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$

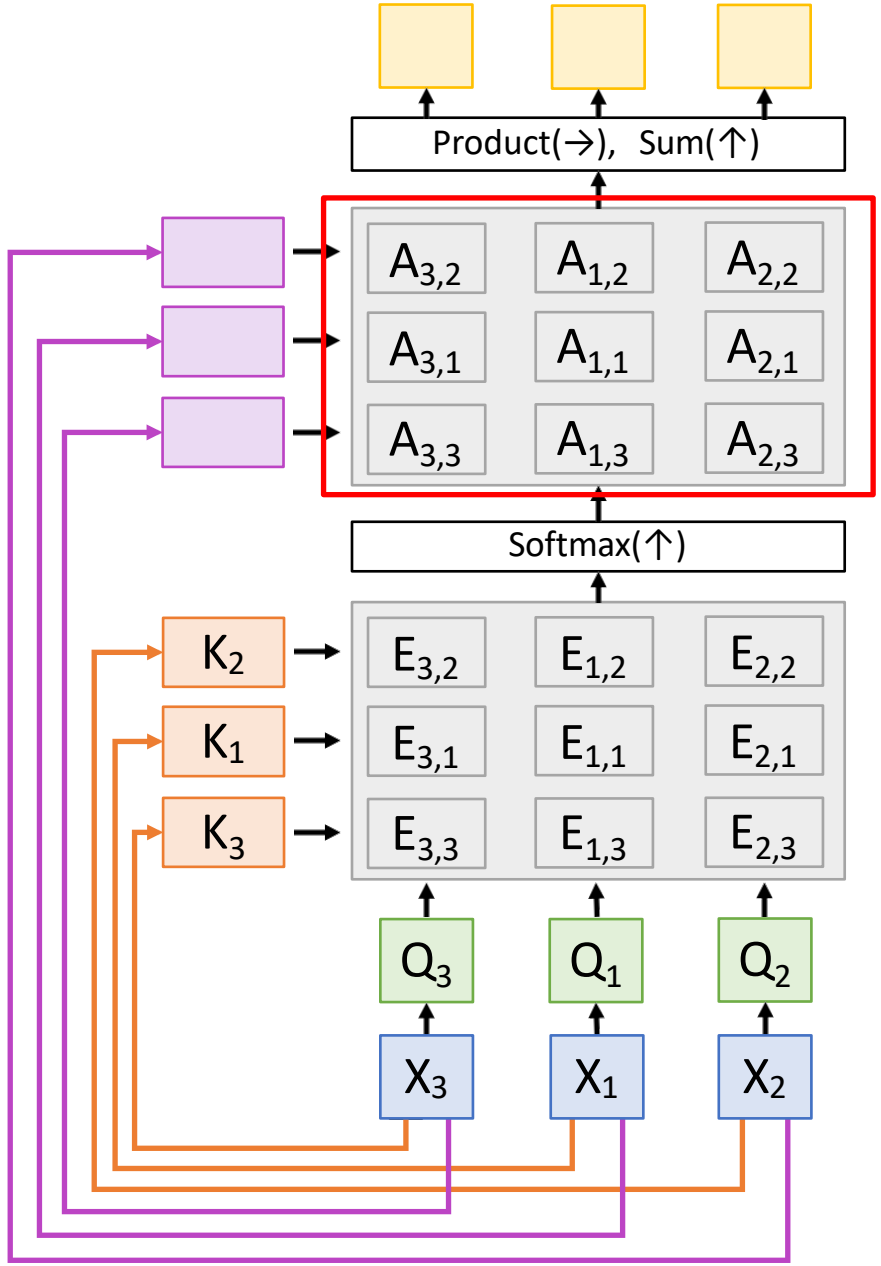
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

Consider **permuting** the input vectors:

Values will be the same, but permuted

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$

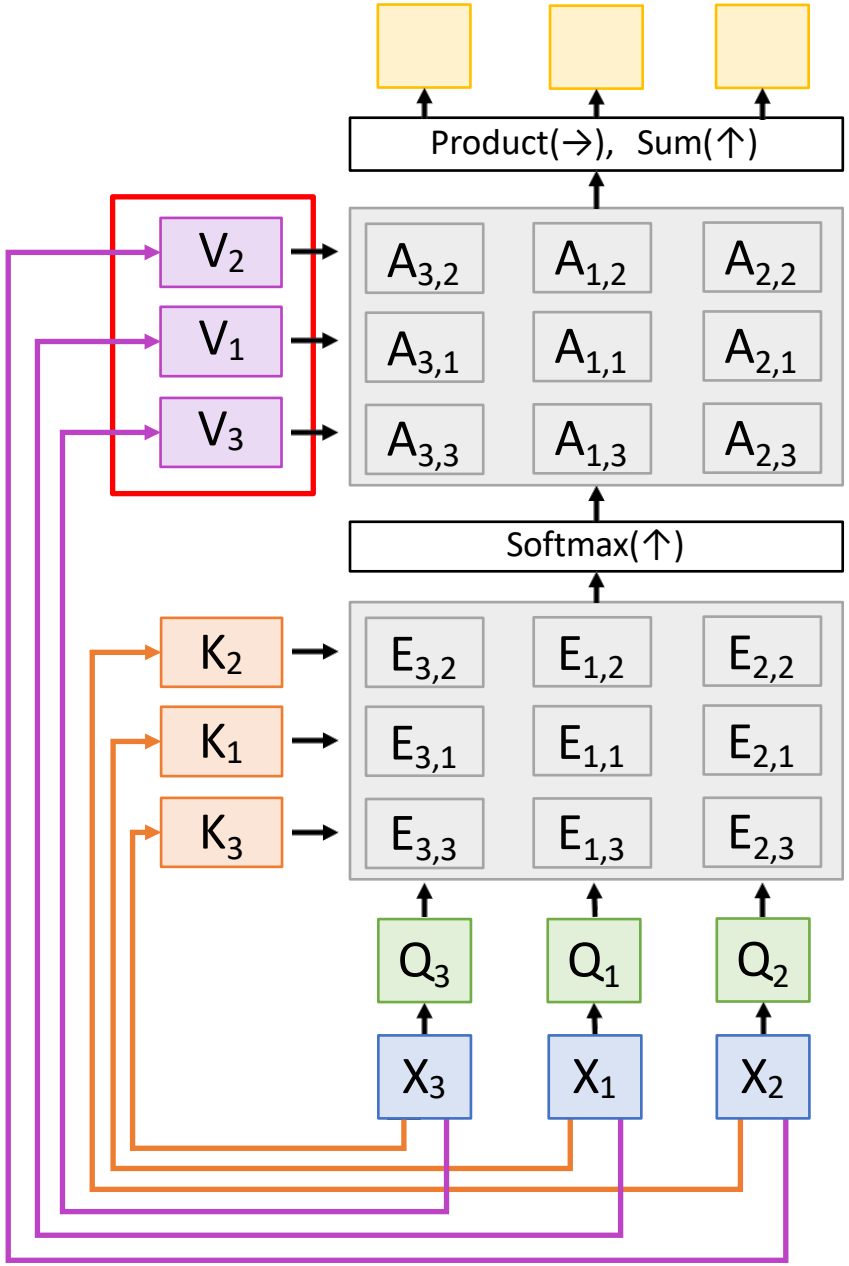
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Permutation Equivariant

$$f(s(x)) = s(f(x))$$

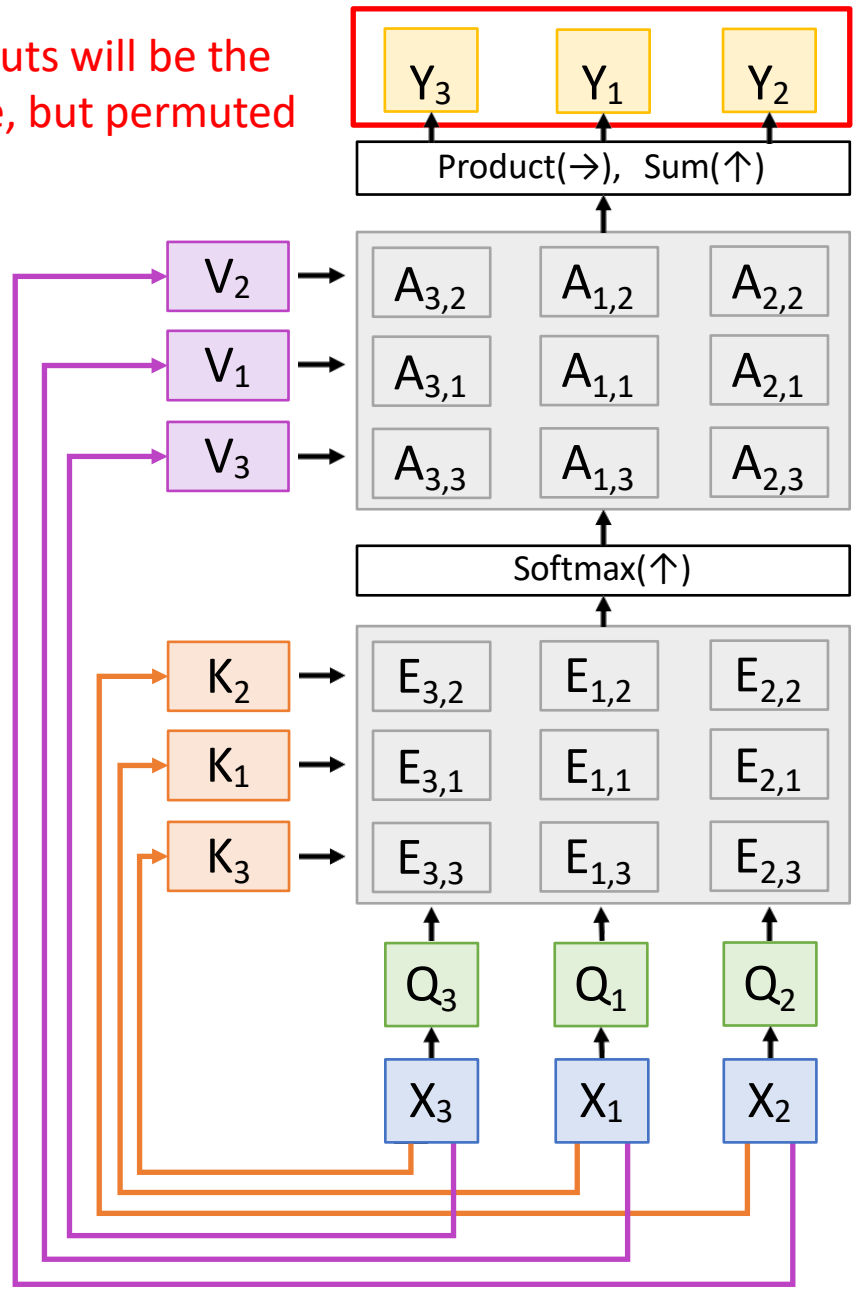
Inputs:

Input vectors: X (Shape: $N_X \times D_X$)
Key matrix: W_K (Shape: $D_X \times D_Q$)
Value matrix: W_V (Shape: $D_X \times D_V$)
Query matrix: W_Q (Shape: $D_X \times D_Q$)

Self-Attention layer works on **sets** of vectors

Computation:

Query vectors: $Q = XW_Q$
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)
Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)
Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)
Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Positional Encoding

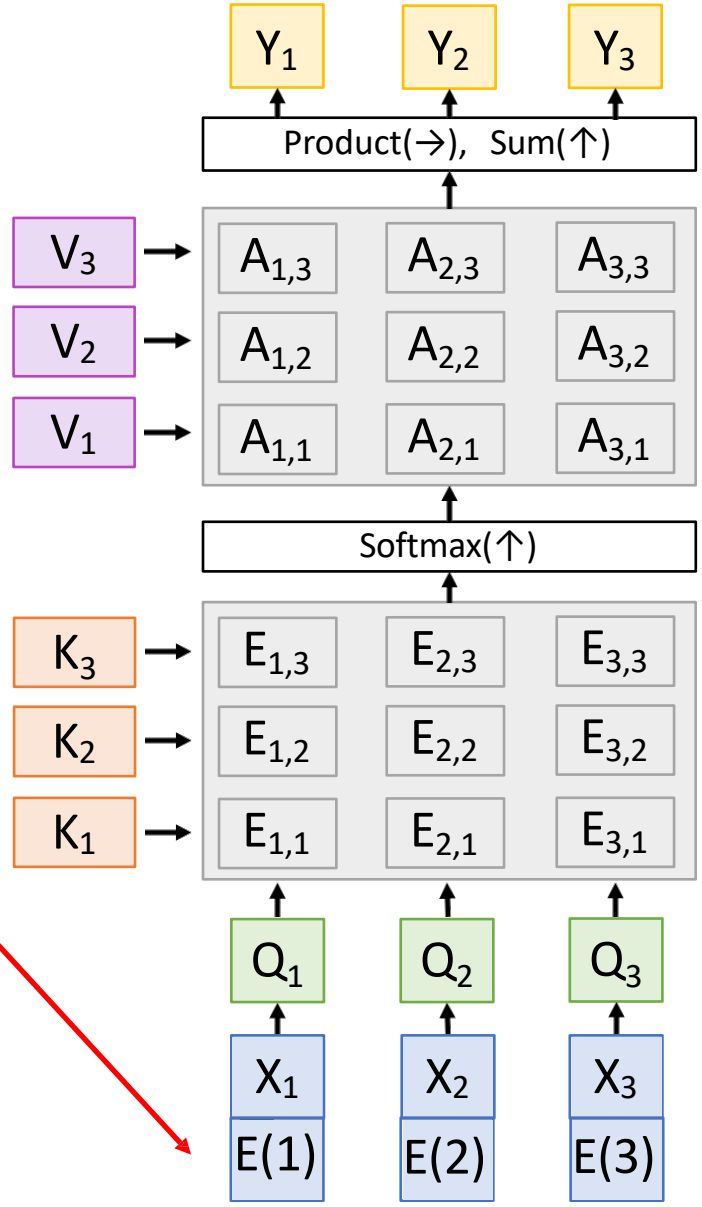
Inputs:
Input vectors: X (Shape: $N_X \times D_X$)
Key matrix: W_K (Shape: $D_X \times D_Q$)
Value matrix: W_V (Shape: $D_X \times D_V$)
Query matrix: W_Q (Shape: $D_X \times D_Q$)

Computation:
Query vectors: $Q = XW_Q$
Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)
Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)
Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: A = softmax(E, dim=1) (Shape: $N_X \times N_X$)
Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$

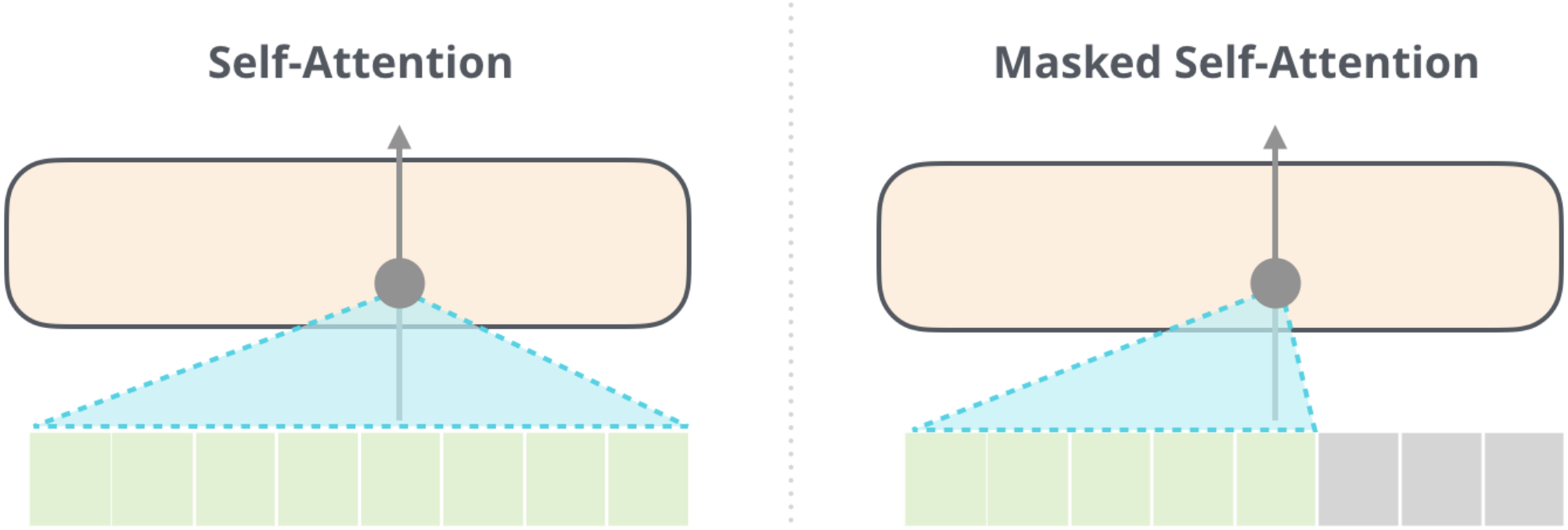
Self attention doesn't "know" the order of the vectors it is processing!

In order to make processing position-aware, concatenate input with **positional encoding**

E can be learned lookup table, or fixed function



Decoding: Masked Self-Attention



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Used for language modeling (predict next word)

Inputs:

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

Value matrix: W_V (Shape: D_X x D_V)

Query matrix: W_Q (Shape: D_X x D_Q)

Computation:

Query vectors: $Q = XW_Q$

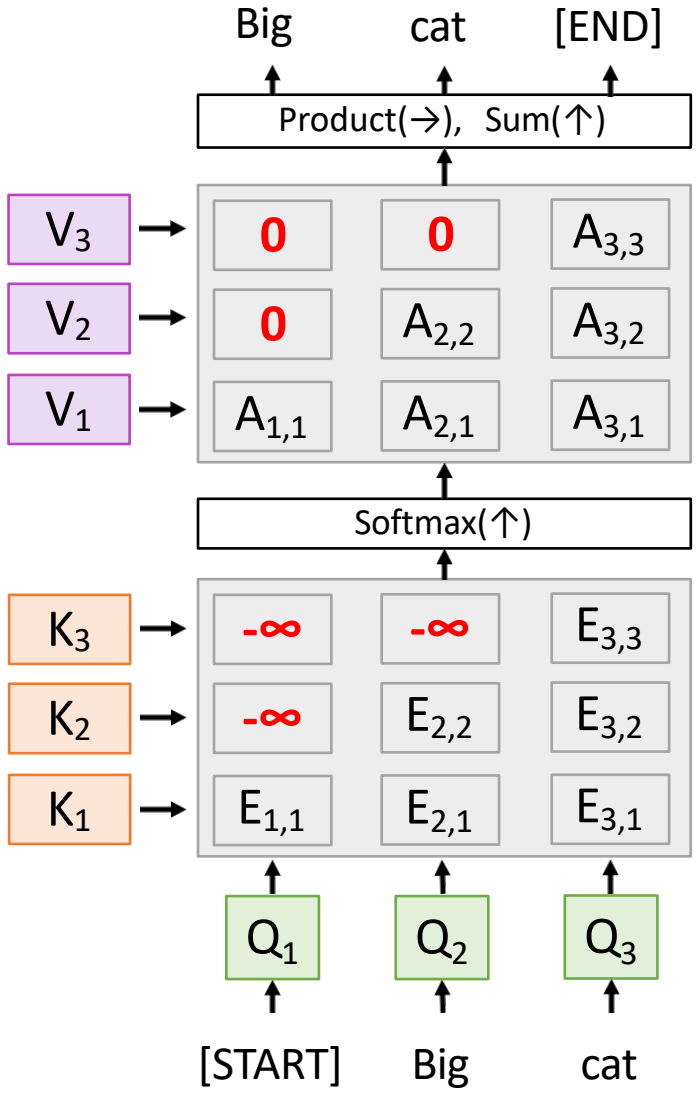
Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

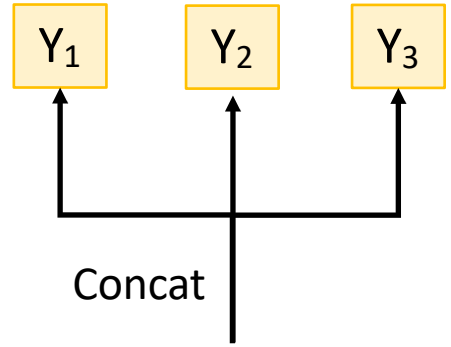
Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X)

Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$



Multihead Self-Attention Layer

Use H independent
"Attention Heads" in parallel



Inputs:

Input vectors: X (Shape: N_X x D_X)

Key matrix: W_K (Shape: D_X x D_Q)

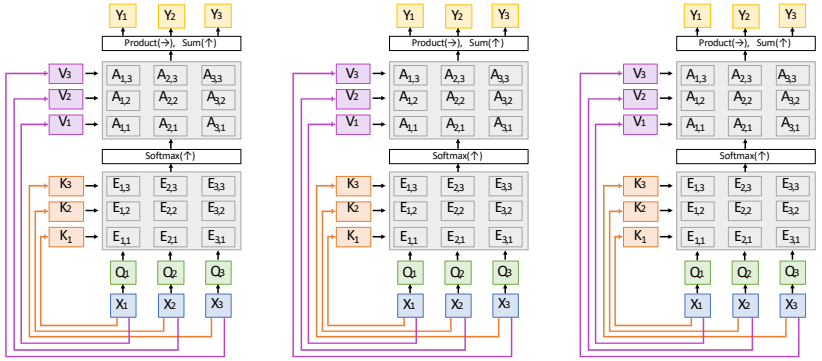
Value matrix: W_V (Shape: D_X x D_V)

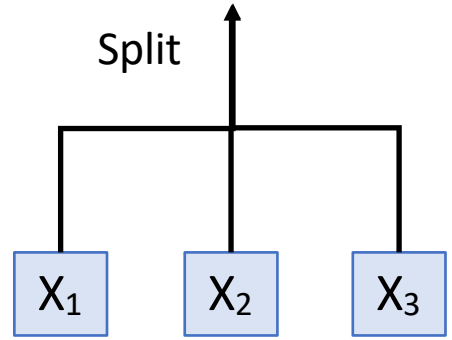
Query matrix: W_Q (Shape: D_X x D_Q)

Hyperparameters:

Query dimension D_Q

Number of heads H





Computation:

Query vectors: $Q = XW_Q$

Key vectors: $K = XW_K$ (Shape: N_X x D_Q)

Value Vectors: $V = XW_V$ (Shape: N_X x D_V)

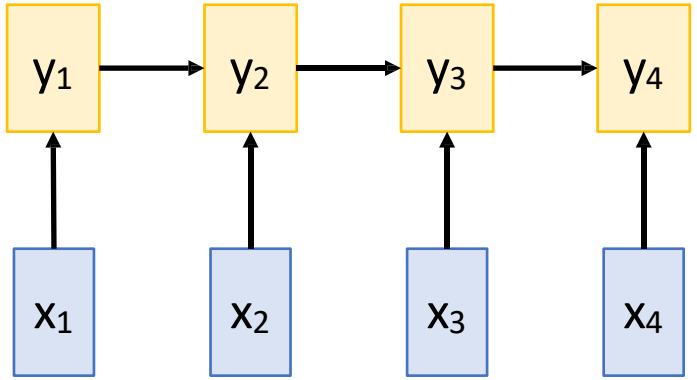
Similarities: $E = QK^T$ (Shape: N_X x N_X) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: A = softmax(E, dim=1) (Shape: N_X x N_X)

Output vectors: $Y = AV$ (Shape: N_X x D_V) $Y_i = \sum_j A_{i,j} V_j$

Summary: Three Ways of Processing Sequences

Recurrent Neural Network



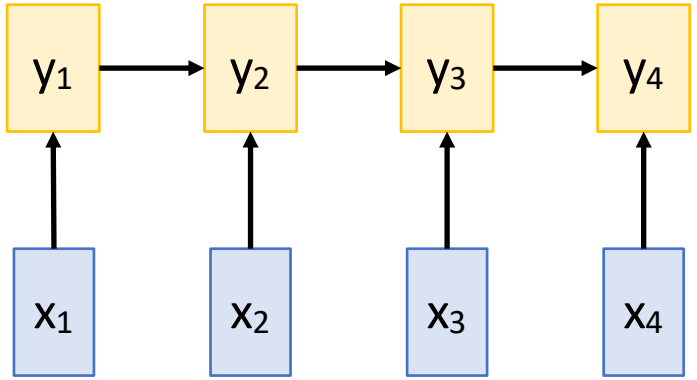
Works on **Ordered Sequences**

(+) Good at long sequences: After one RNN layer, h_T "sees" the whole sequence

(-) Not parallelizable: need to compute hidden states sequentially

Summary: Three Ways of Processing Sequences

Recurrent Neural Network

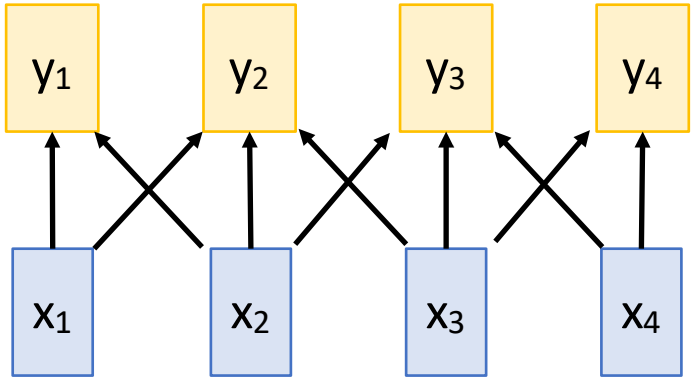


Works on **Ordered Sequences**

(+) Good at long sequences: After one RNN layer, h_T "sees" the whole sequence

(-) Not parallelizable: need to compute hidden states sequentially

1D Convolution



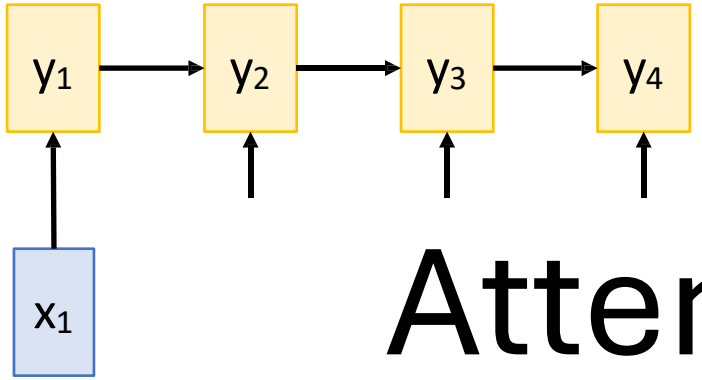
Works on **Multidimensional Grids**

(-) Bad at long sequences: Need to stack many conv layers for outputs to "see" the whole sequence

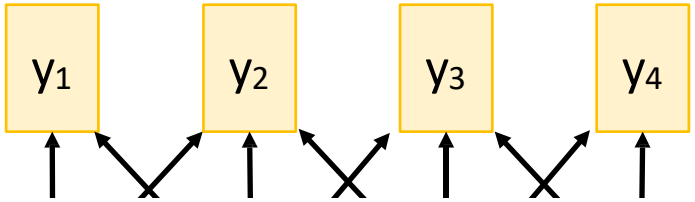
(+) Highly parallel: Each output can be computed in parallel

Summary: Three Ways of Processing Sequences

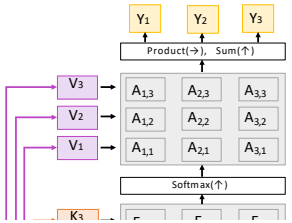
Recurrent Neural Network



1D Convolution



Self-Attention



Attention is all you need

Works on **Ordered Sequences**

(+) Good at long sequences: After one RNN layer, h_T "sees" the whole sequence

(-) Not parallelizable: need to compute hidden states sequentially

Works on **Multidimensional Grids**

(-) Bad at long sequences: Need to stack many conv layers for outputs to "see" the whole sequence

(+) Highly parallel: Each output can be computed in parallel

Works on **Sets of Vectors**

(-) Good at long sequences: after one self-attention layer, each output "sees" all inputs!

(+) Highly parallel: Each output can be computed in parallel

(-) Very memory intensive

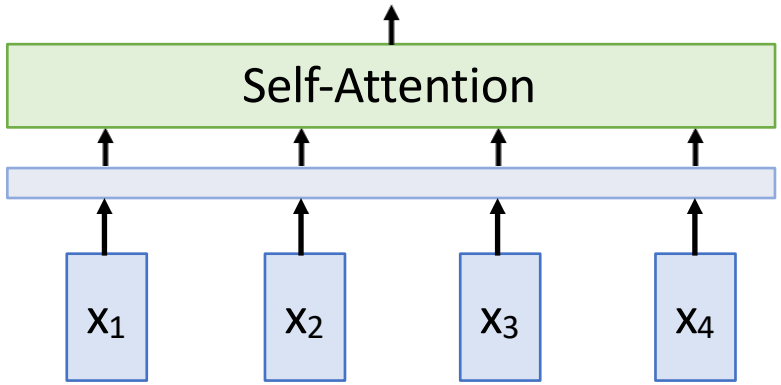
The Transformer



Vaswani et al, “Attention is all you need”, NeurIPS 2017

The Transformer

All vectors interact with each other

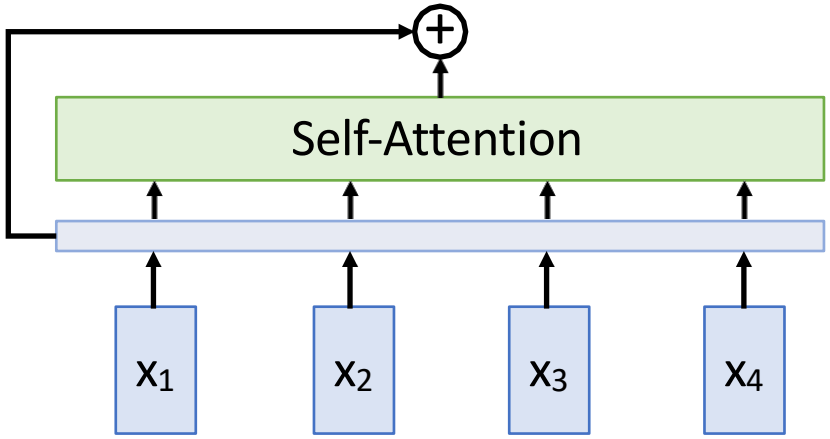


Vaswani et al, “Attention is all you need”, NeurIPS 2017

The Transformer

Residual connection

All vectors interact with each other



Vaswani et al, “Attention is all you need”, NeurIPS 2017

The Transformer

Recall **Layer Normalization**:

Given $h_1, \ldots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D)\sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

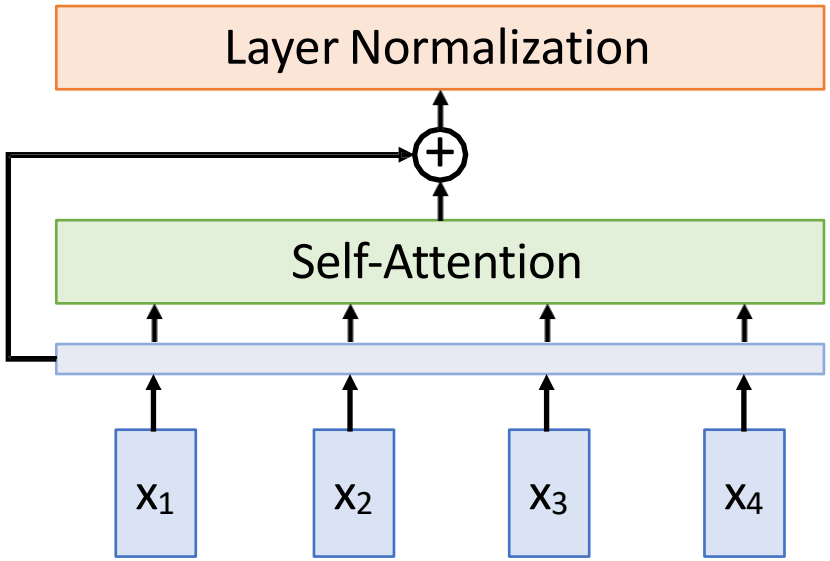
$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

Ba et al, 2016

Residual connection

All vectors interact with each other



Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer

Recall **Layer Normalization**:

Given $h_1, \ldots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D)\sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

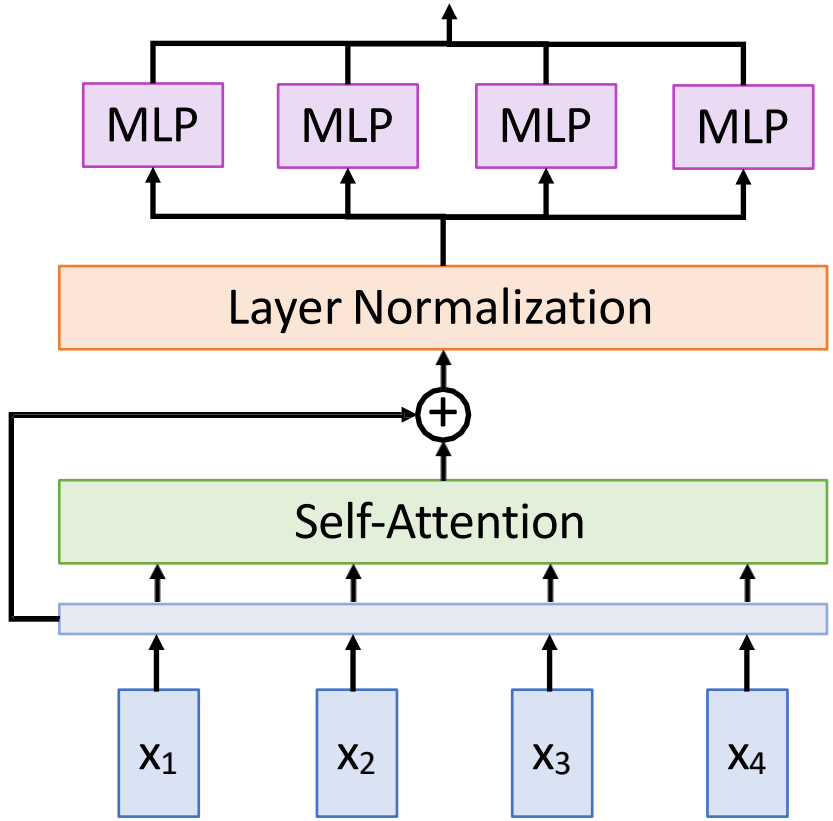
$y_i = \gamma * z_i + \beta$

Ba et al, 2016

MLP independently on each vector

Residual connection

All vectors interact with each other



Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer

Recall **Layer Normalization**:

Given $h_1, \ldots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D)\sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

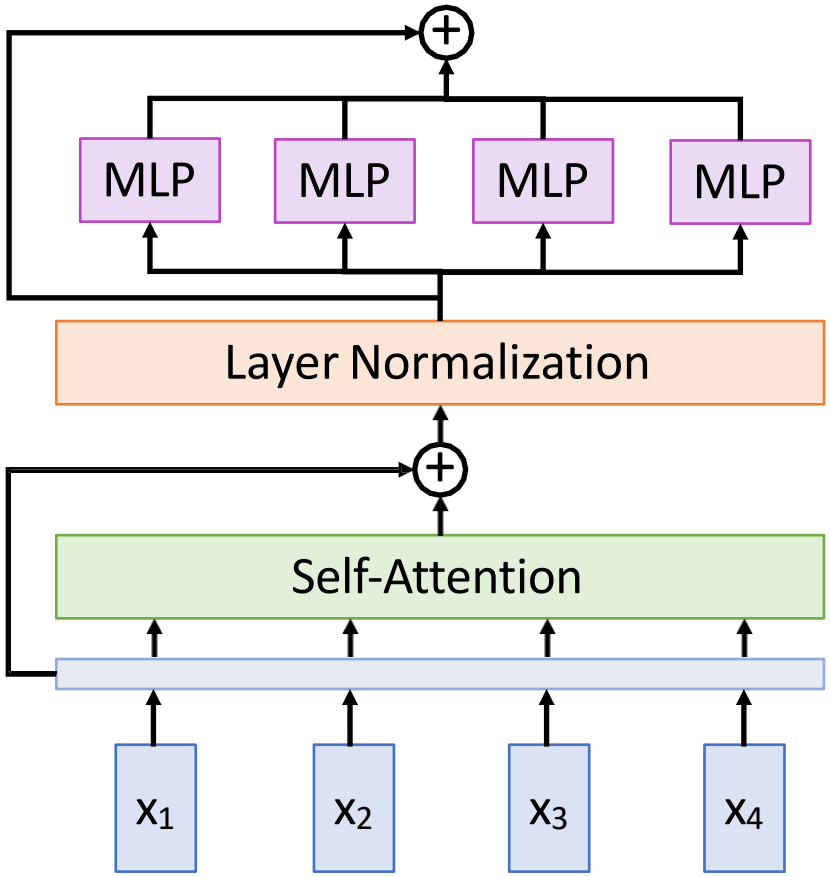
Ba et al, 2016

Residual connection

MLP independently on each vector

Residual connection

All vectors interact with each other



Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer

Recall **Layer Normalization**:

Given $h_1, \ldots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D)\sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

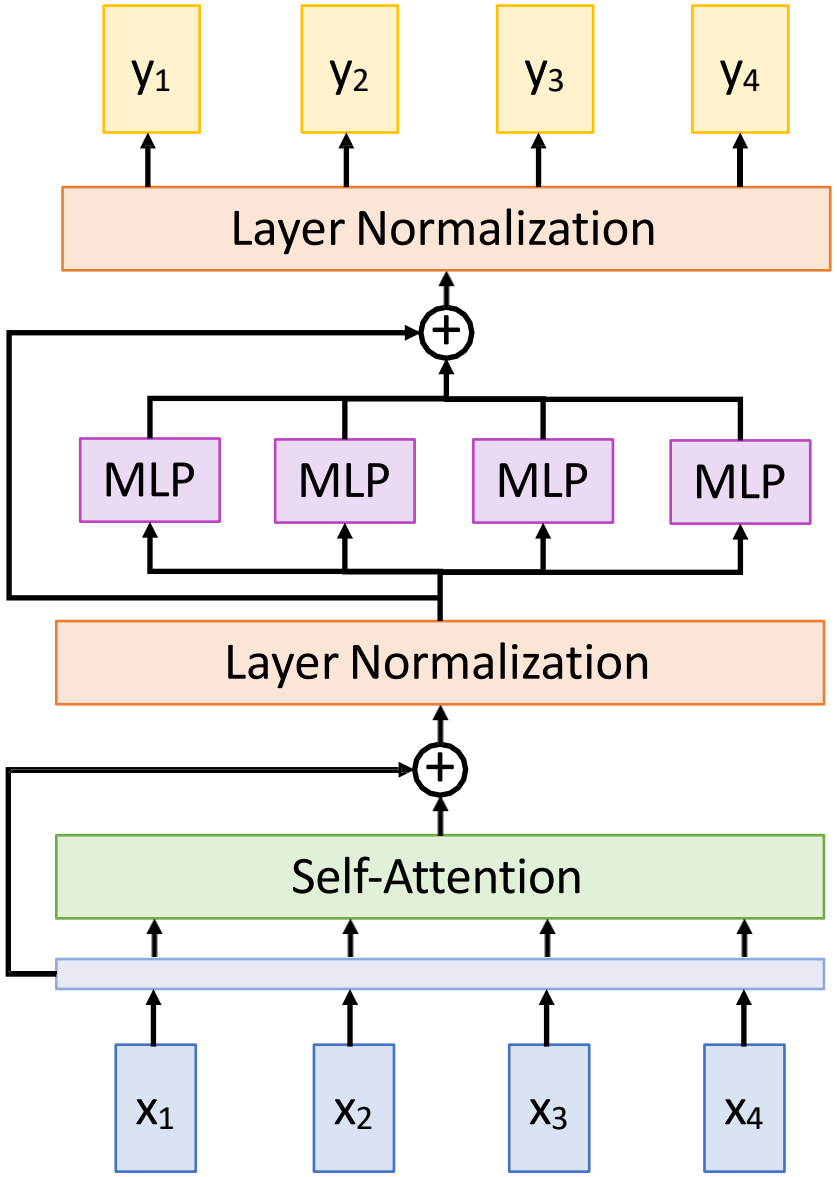
Ba et al, 2016

Residual connection

MLP independently on each vector

Residual connection

All vectors interact with each other



Vaswani et al, "Attention is all you need", NeurIPS 2017

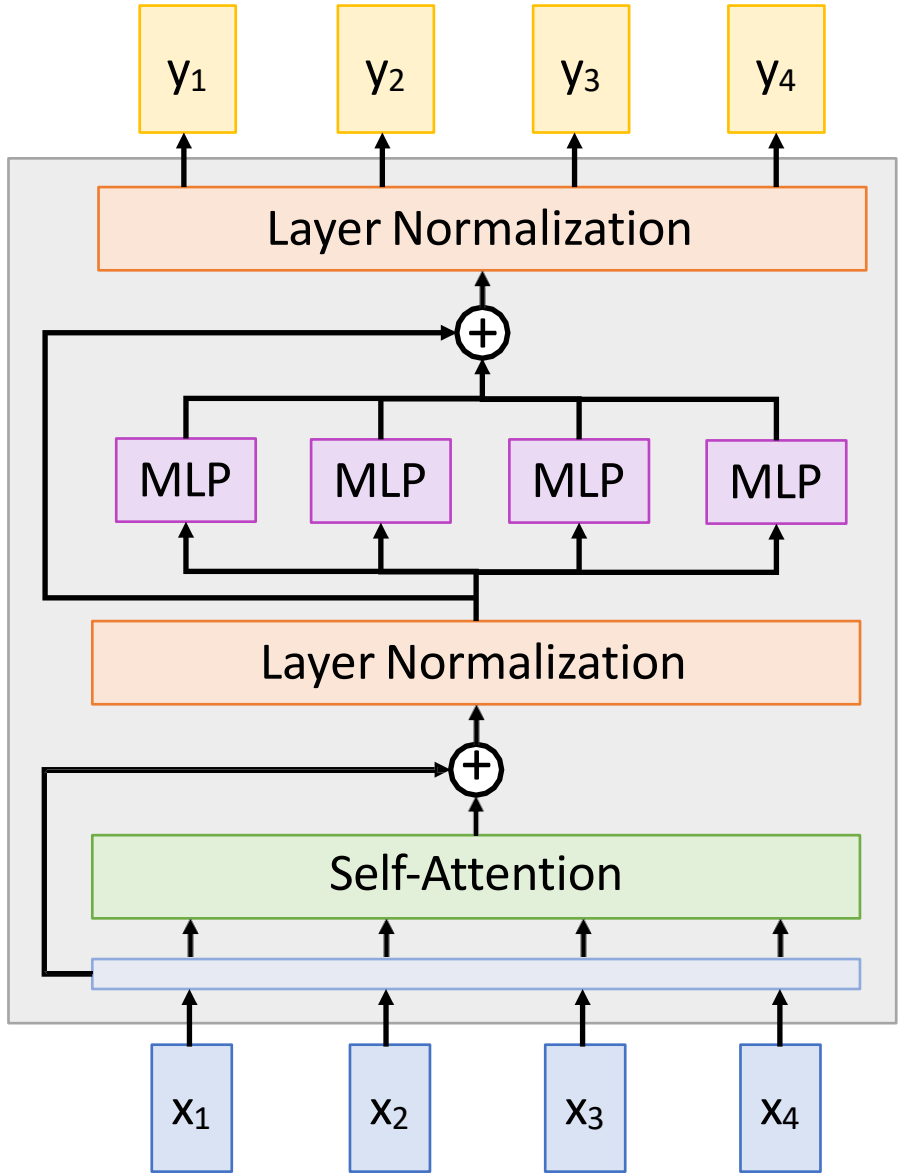
The Transformer

Transformer Block:
Input: Set of vectors x
Output: Set of vectors y

Self-attention is the only interaction between vectors!

Layer norm and MLP work independently per vector

Highly scalable, highly parallelizable



Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer

<u>Transformer Block</u>:
Input: Set of vectors x
Output: Set of vectors y

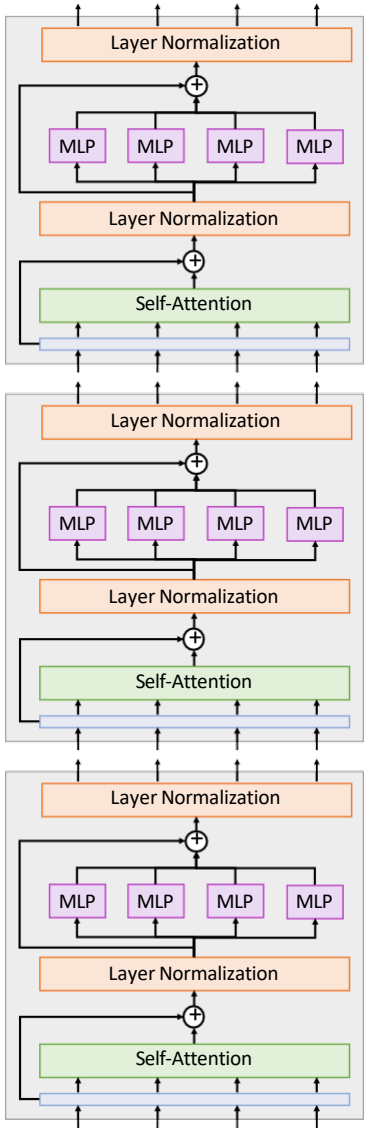
Self-attention is the only interaction between vectors!

Layer norm and MLP work independently per vector

Highly scalable, highly parallelizable

A **Transformer** is a sequence of transformer blocks

Vaswani et al:
12 blocks, D_Q=512, 6 heads



Vaswani et al, “Attention is all you need”, NeurIPS 2017

The Transformer: Transfer Learning

“ImageNet Moment for Natural Language Processing”

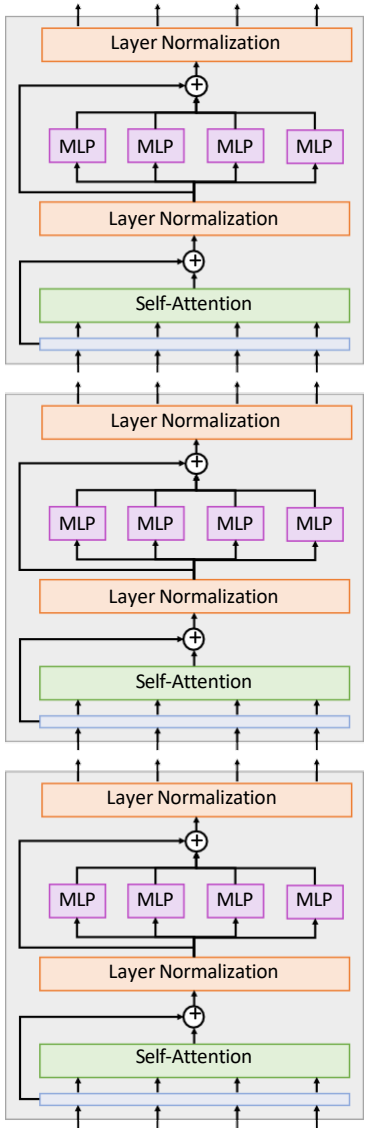
Pretraining:

Download a lot of text from the internet

Train a giant Transformer model for language modeling

Finetuning:

Fine-tune the Transformer on your own NLP task

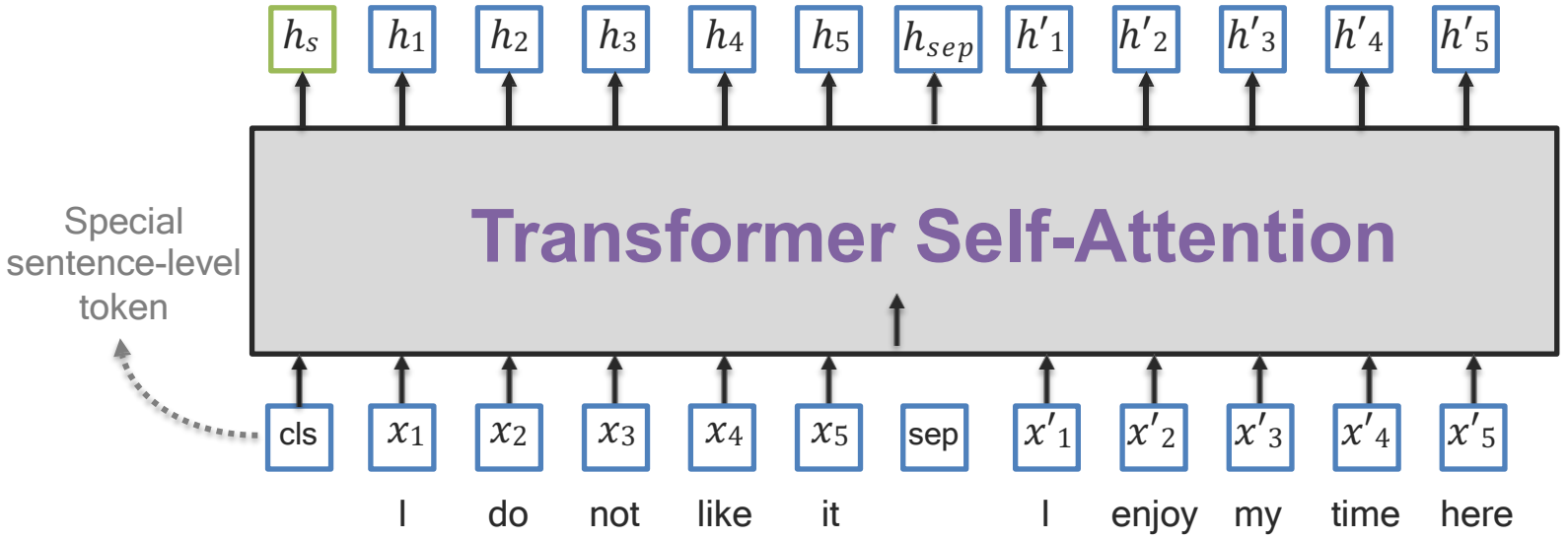


Devlin et al, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", EMNLP 2018

BERT: Bidirectional Encoder Representations

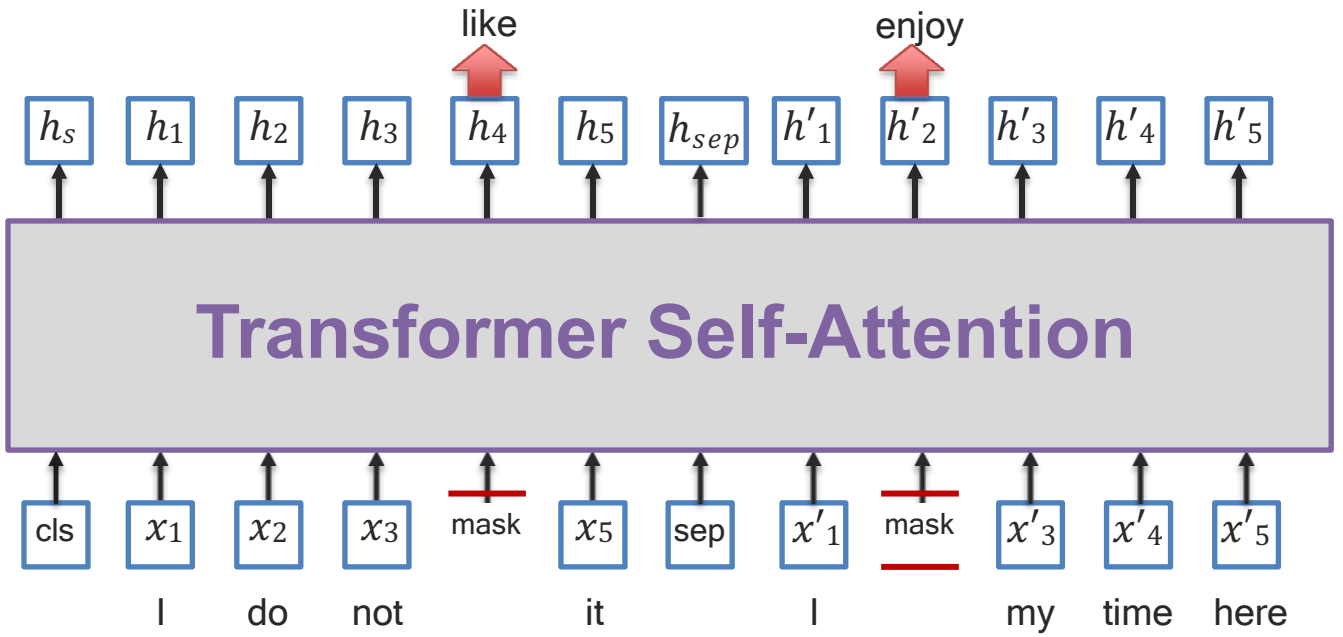
Advantages:

1. Jointly learn representation for token-level and sentence level
2. Same network architecture for pre-training and fine-tuning
3. Can be used learn relationship between sentences
4. Models bidirectional and long-range interactions between tokens



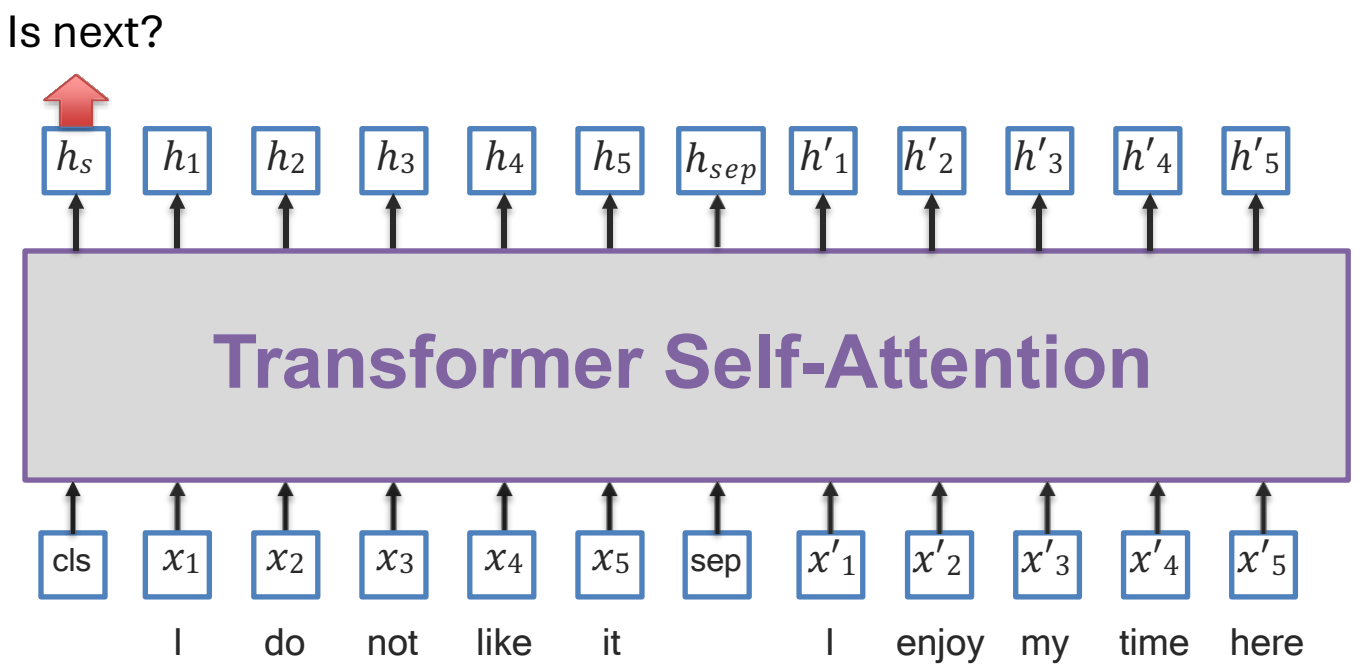
Self-supervised Pre-training

- **Masked Language Model:** Randomly mask input tokens and then try to predict them



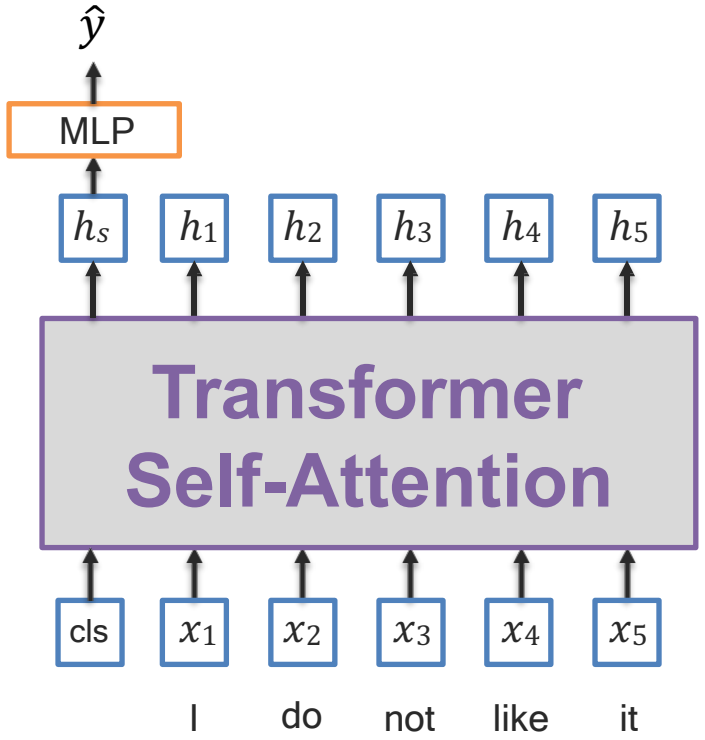
Self-supervised Pre-training

- **Next Sentence Prediction:** Given two sentences, predict if this is the next one or not



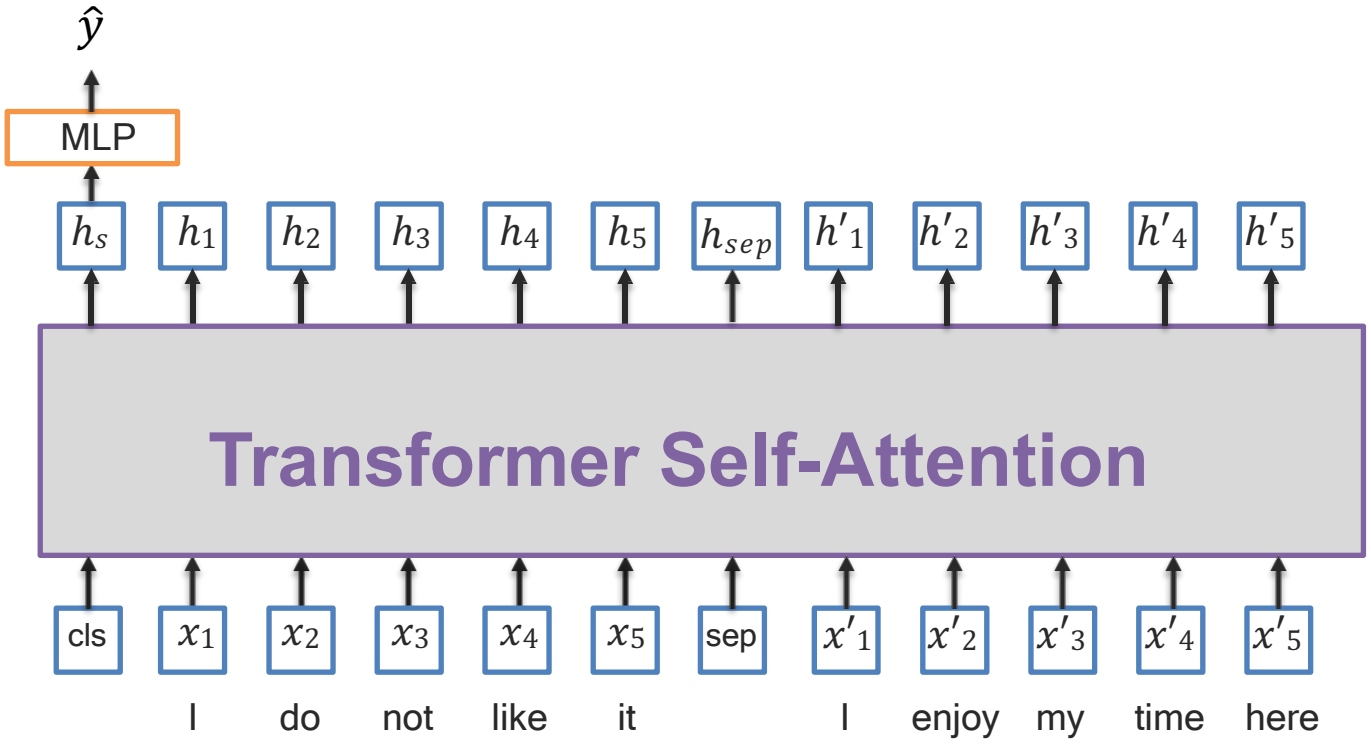
Fine-tunning

- Sentence-level classification



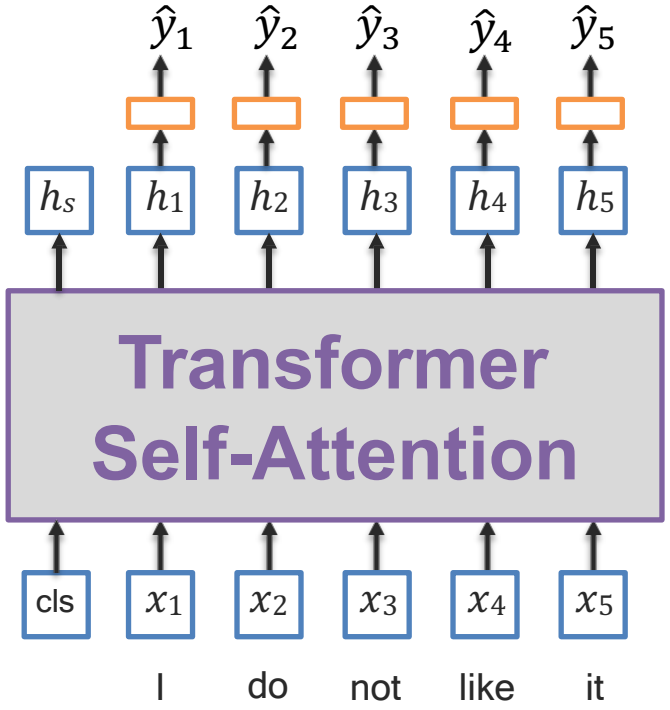
Fine-tunning

- More than one sentence

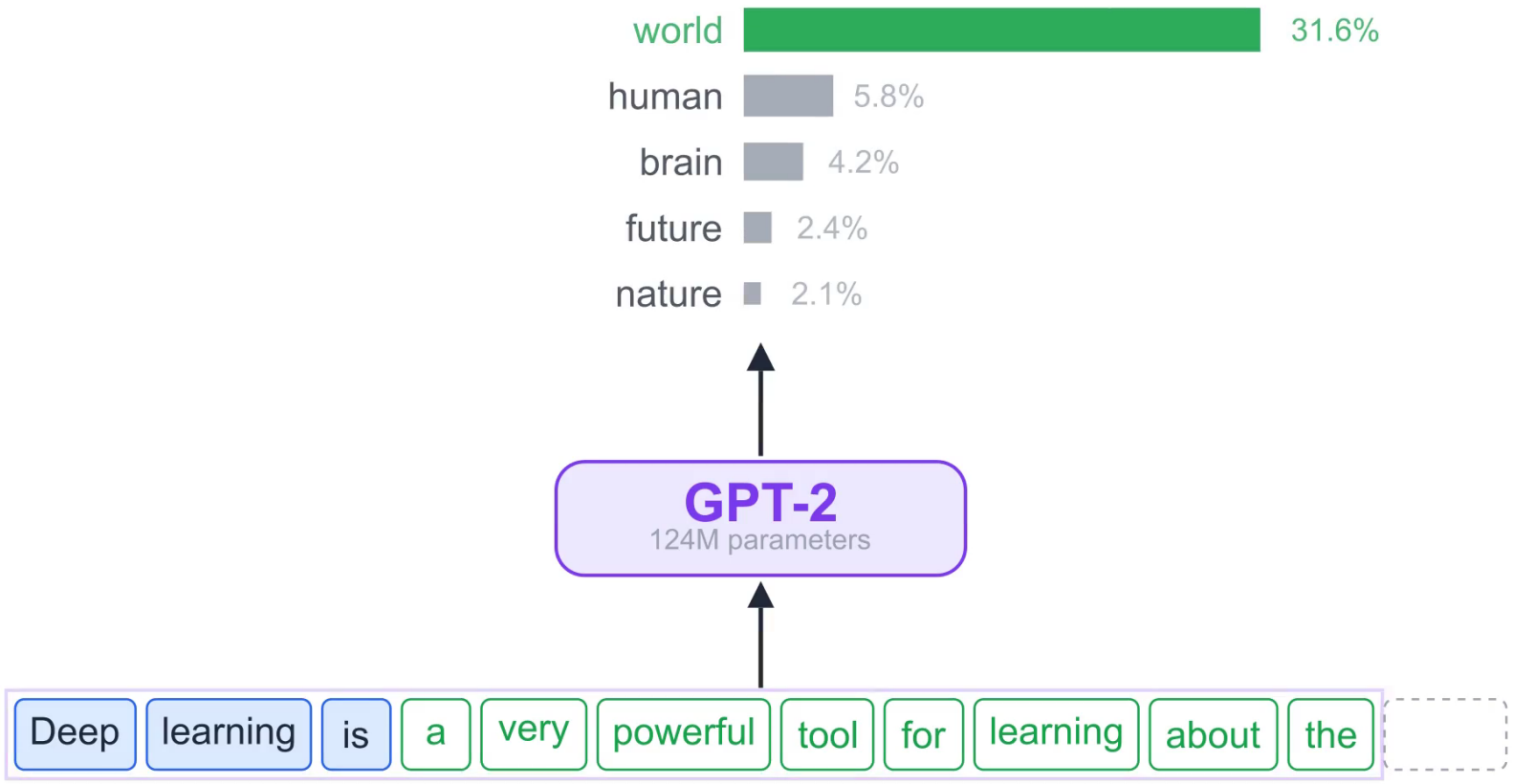


Fine-tunning

- Token-level classification



GPT-2 showed - autoregressive decoding is all you need



code: github.com/idansc/dl-course-2026/animations/code/gpt2_decoding.py

Scaling up Transformers

~$430,000 on Amazon AWS!

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	12	768	?	117M	40 GB	
GPT-2	24	1024	?	345M	40 GB	
GPT-2	36	1280	?	762M	40 GB	
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	40	1536	16	1.2B	174 GB	64x V100 GPU
Megatron-LM	54	1920	20	2.5B	174 GB	128x V100 GPU
Megatron-LM	64	2304	24	4.2B	174 GB	256x V100 GPU (10 days)
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)

Shoeybi et al, "Megatron-LM: Training Multi-Billion Parameter Language Models using Model Parallelism", 2019

GPT-3; Easy to scale

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128

GPT-2 (2019)	GPT-3 (2020)
1.5B parameters	175B parameters
40 GB of web text	300B tokens: web, books, Wikipedia
zero-shot (weak) or fine-tuned	few-shot: examples in the prompt
weights released	API only

Brown et al., "Language Models are Few-Shot Learners", NeurIPS 2020 (table: their Table 2.1); Radford et al., "Language Models are Unsupervised Multitask Learners", 2019.

Scaling up Transformers, 2020–2025

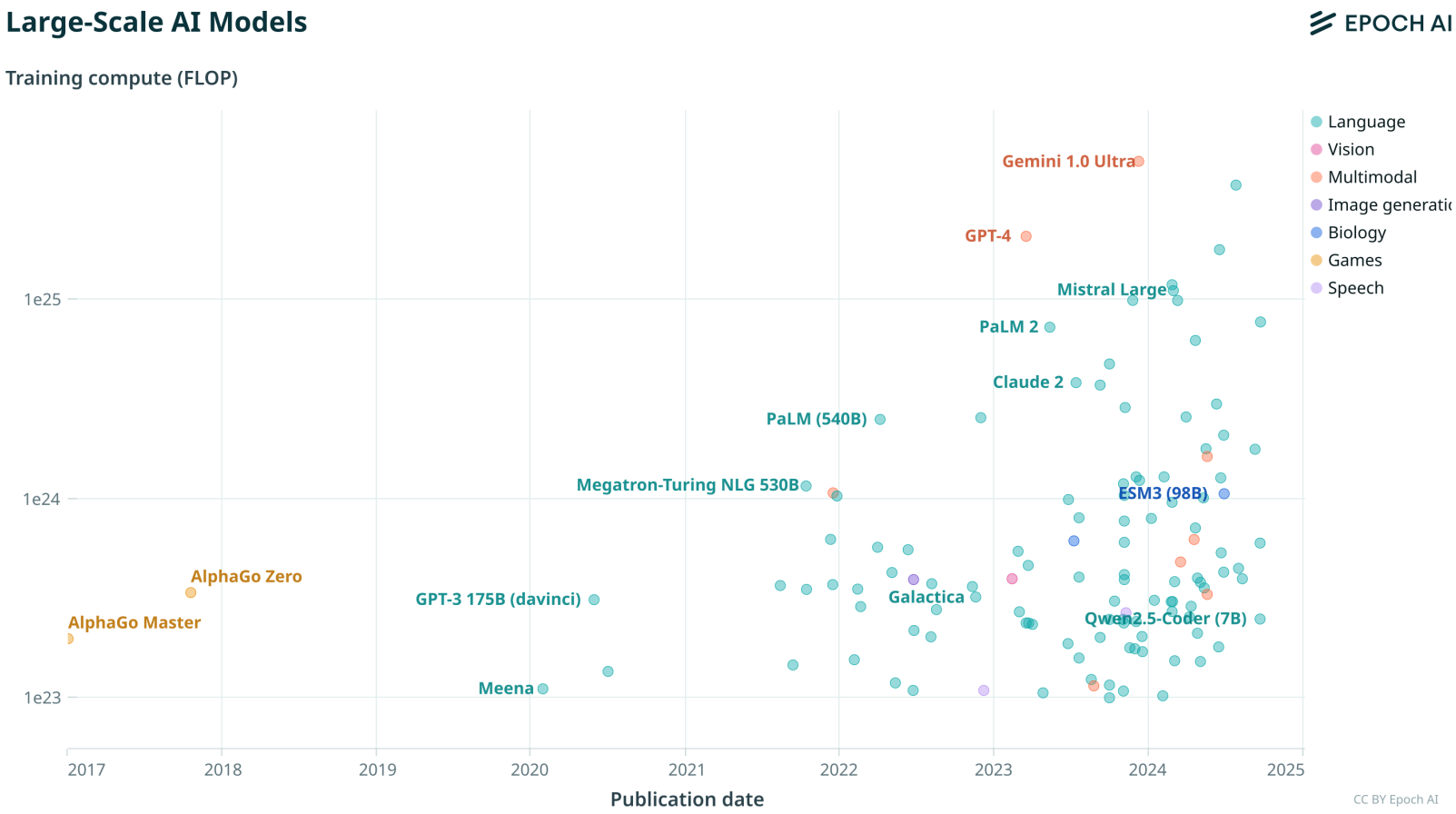
Model	Layers	Width	Heads	Params	Data	Training
GPT-3 (2020)	96	12288	96	175B	300B tokens	V100s; 3.1×10^{23} FLOPs
Llama 3.1 405B (2024)	126	16384	128	405B	15.6T tokens	16K H100; 3.8×10^{25} FLOPs
DeepSeek-V3 (2024)	61	7168	128	671B (37B active)	14.8T tokens	2,048 H800; 2.79M GPU-hours
Kimi K2 (2025)	61	7168	64	1.04T (32B active)	15.5T tokens	H800s; MuonClip optimizer

DeepSeek-V3, final run: ~$5.6M

Estimated compute for one frontier run: GPT-4 ~$78M, Gemini Ultra ~$191M (Stanford AI Index 2024)

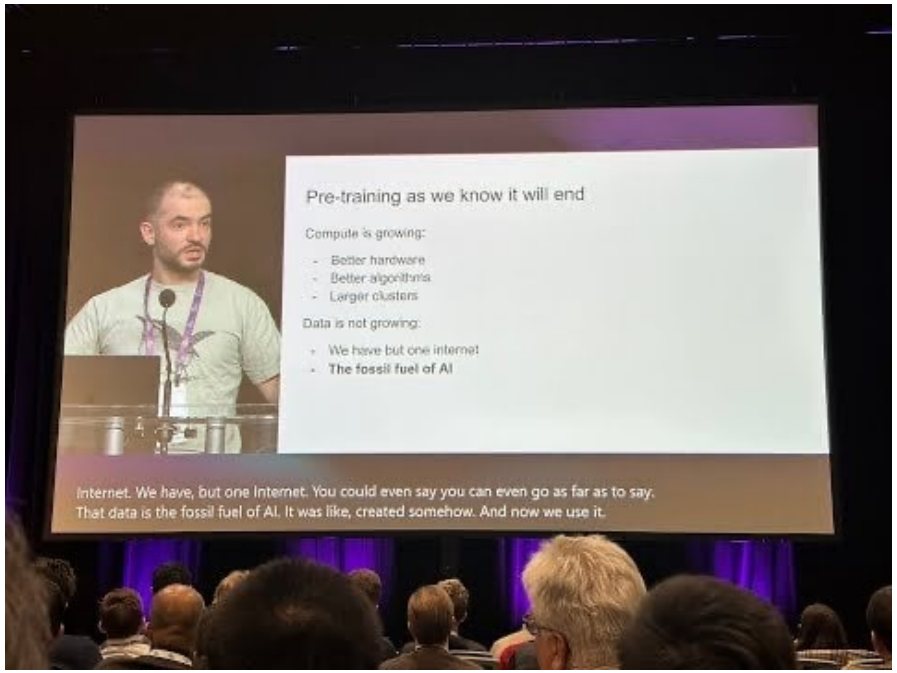
Brown et al. 2020; Grattafiori et al., The Llama 3 Herd of Models, 2024; DeepSeek-AI, DeepSeek-V3 Technical Report, 2024; Kimi Team, Kimi K2, 2025; Stanford AI Index 2024

LLMs Moore’s Law



Can pre-training continue?

- Compute is growing:
  - Better hardware
  - Better algorithms
  - Larger clusters
- Data is not growing:
  - We have but one internet
  - The fossil fuel of AI
- **RL Post-training?**
  - **Maybe; we'll discuses**



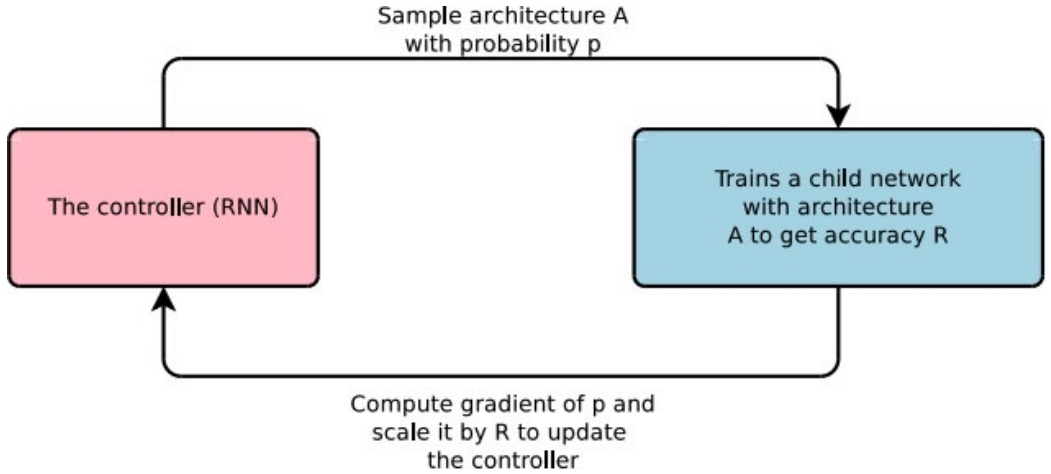
Ilya Sutskever at Neurips'24

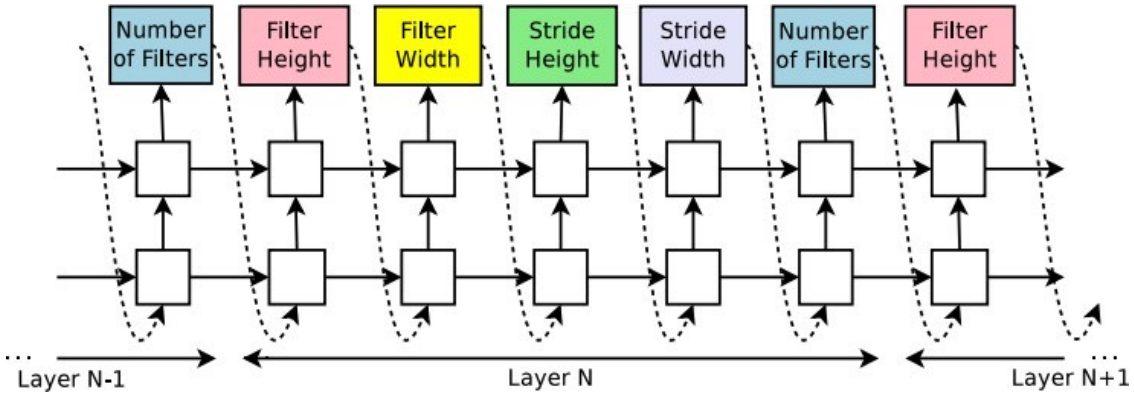
Neural Architecture Search

Designing neural network architectures is hard – let's automate it!

- One network (**controller**) outputs network architectures
- Sample **child networks** from controller and train them
- After training a batch of child networks, make a gradient step on controller network (Using **policy gradient**)
- Over time, controller learns to output good architectures!

- VERY EXPENSIVE!! Each gradient step on controller requires training a batch of child models!
- Original paper trained on 800 GPUs for 28 days!
- Followup work has focused on efficient search





Zoph and Le, "Neural Architecture Search with Reinforcement Learning", ICLR 2017

Architecture search since 2017

When	Idea	Examples
2018–20	Share weights: train one supernet, search = choose a path	ENAS: 1000× cheaper than the 2017 search · DARTS: gradient descent on the choice · Once-for-All: one net, a sub-net per device
2021	Search inside the Transformer	Primer: squared ReLU and a depthwise conv on Q, K, V; squared ReLU now ships in NVIDIA's Nemotron-4
2024	Cheap proxy tasks instead of full training runs	MAD: small synthetic tasks predict how an architecture scales · STAR: evolves hybrid language models
2024–25	Search inside an already trained LLM, for inference cost	Puzzle: Llama-3.1-70B → Nemotron-51B, 2.17× faster on one H100 · Jet-Nemotron: up to 53.6× faster decoding
2025–26	LLM agents propose, code and train the candidates	AIRA-Compose: 11 agents, 24 h → 1B models above Llama 3.2 · AgentNAS: an LLM's seed design, then classic NAS · ASI-Arch (claims contested)

Reality check (2019): random search with the same tricks is about as good, so every method has to beat it.

ENAS (Pham et al. 2018) · DARTS (Liu et al. 2019) · Once-for-All (Cai et al. 2020) · Primer (So et al. 2021) · MAD (Poli et al. 2024) · STAR (Liquid AI, 2025) · Puzzle (NVIDIA, 2025) · Jet-Nemotron (NVIDIA, 2025) · ASI-Arch (2025) · AIRA (2026) · AgentNAS (2026)

Count the parameters

Conv layer

AlexNet conv1: 64 filters, 11×11, 3 channels

$$64 \cdot (3 \cdot 11 \cdot 11) + 64$$

= 23,296

LSTM

$x \in \mathbb{R}^{32}$, $h \in \mathbb{R}^{64}$, four gates; PyTorch keeps two bias vectors

$$4 \cdot 64 \cdot (32 + 64) + 2 \cdot 4 \cdot 64$$

= 25,088

Transformer block

d = 64, 4 heads, d_ff = 128; +704 biases and LayerNorm

$$4 \cdot 64^2 + 2 \cdot 64 \cdot 128 + 704$$

= 33,472

Heads and sequence length do not change the count.

code: github.com/idansc/dl-course-2026/animations/code/count_params.py

Checked against nn.Conv2d, nn.LSTM and nn.TransformerEncoderLayer. Exam questions of this kind: exam bank, weeks 2–4.

Three architectures: what each one assumes

	Assumes	Shares weights across	Cost in length n
CNN	nearby pixels are related	positions	linear, local
RNN / LSTM	order; a state summarises the past	time steps	linear, sequential
Transformer	nothing about order (positions are added)	tokens	quadratic (attention)

Own summary of this lecture.

What makes all three trainable when they get deep?

ResNet block

$$h_{l+1} = h_l + F(h_l)$$

BatchNorm inside F

LSTM cell

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

(LayerNorm began on RNNs)

Transformer block

$$x_{l+1} = x_l + F(x_l)$$

LayerNorm after each sum

Gradients flow back through the sum, not through every weight matrix on the way.

Common thread: residual paths and normalization make depth trainable

He et al., CVPR 2016; Hochreiter and Schmidhuber 1997; Gers et al. 2000; Ba et al., Layer Normalization, 2016; Vaswani et al., NeurIPS 2017; Xiong et al., ICML 2020 (pre-norm).

Appendix

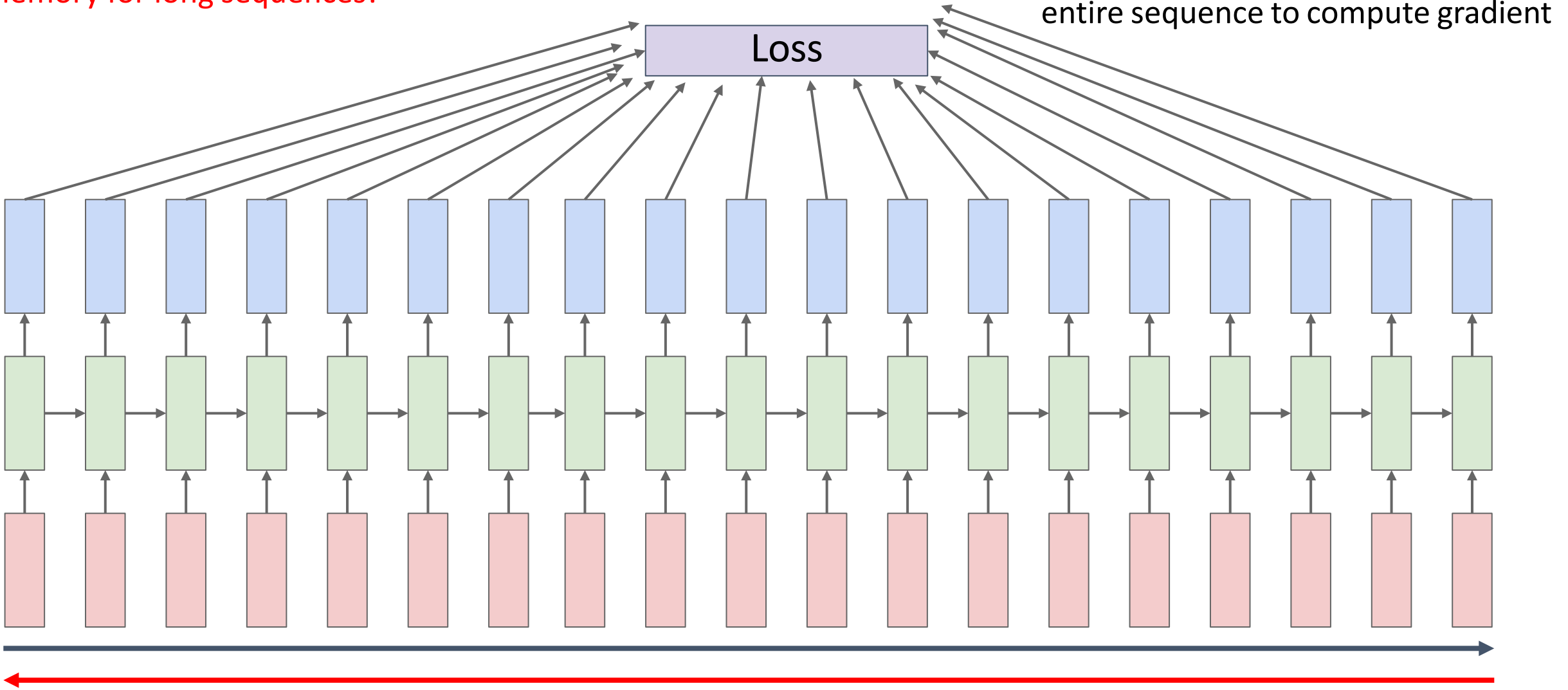
Backpropagation through time · min-char-rnn

CNN with self-attention

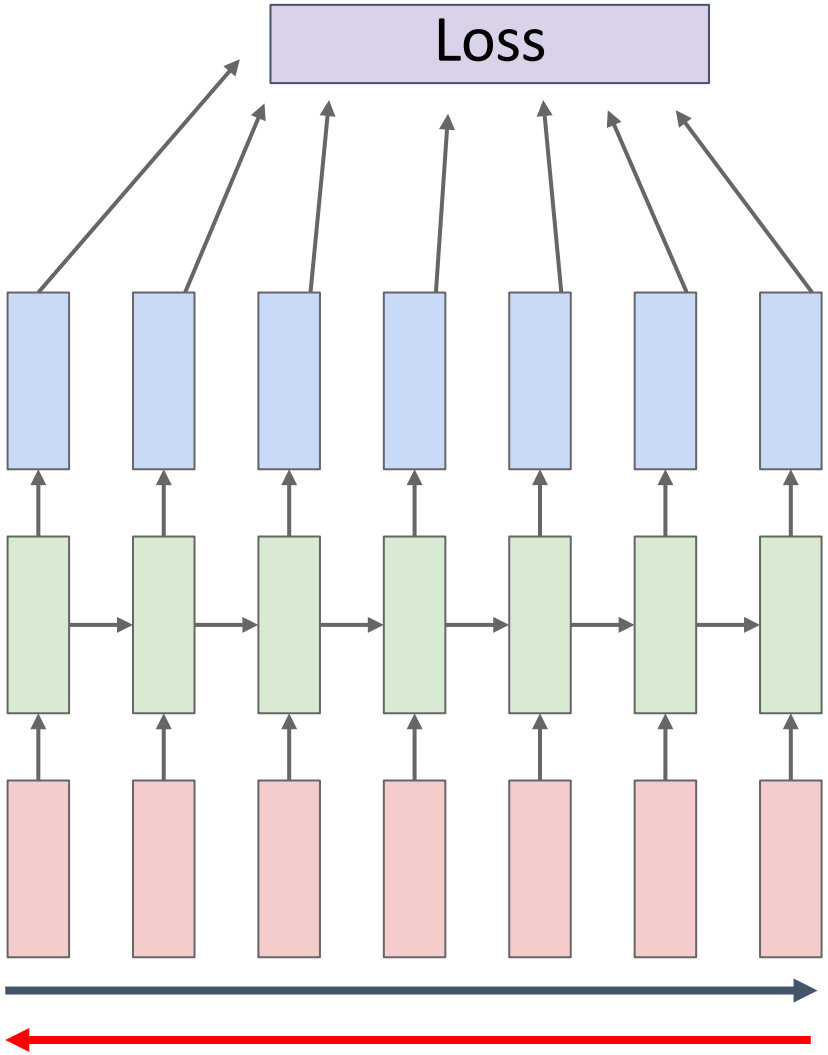
Appendix: Backpropagation Through Time

Problem: Takes a lot of memory for long sequences!

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

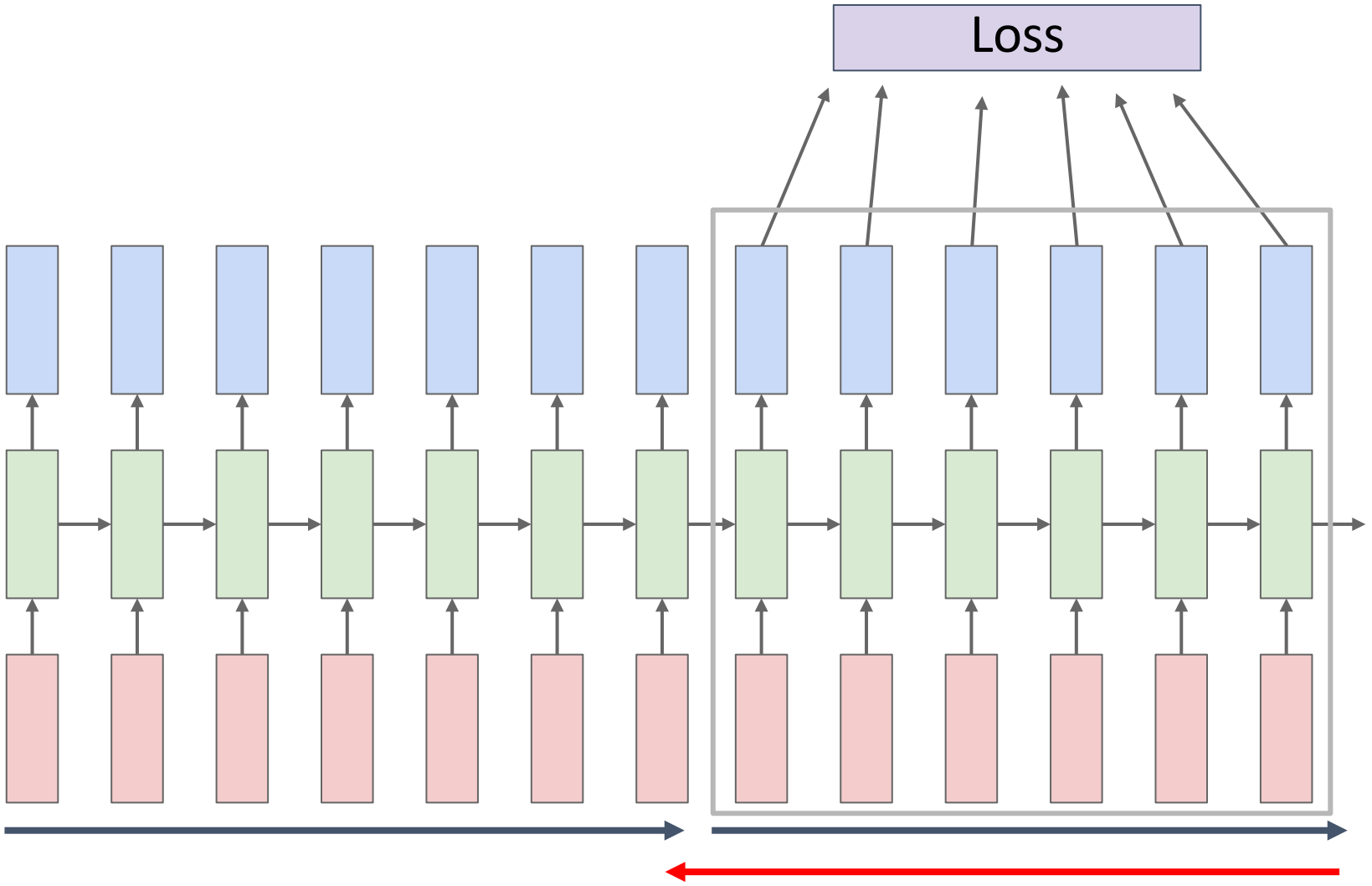


Appendix: Truncated Backpropagation Through Time



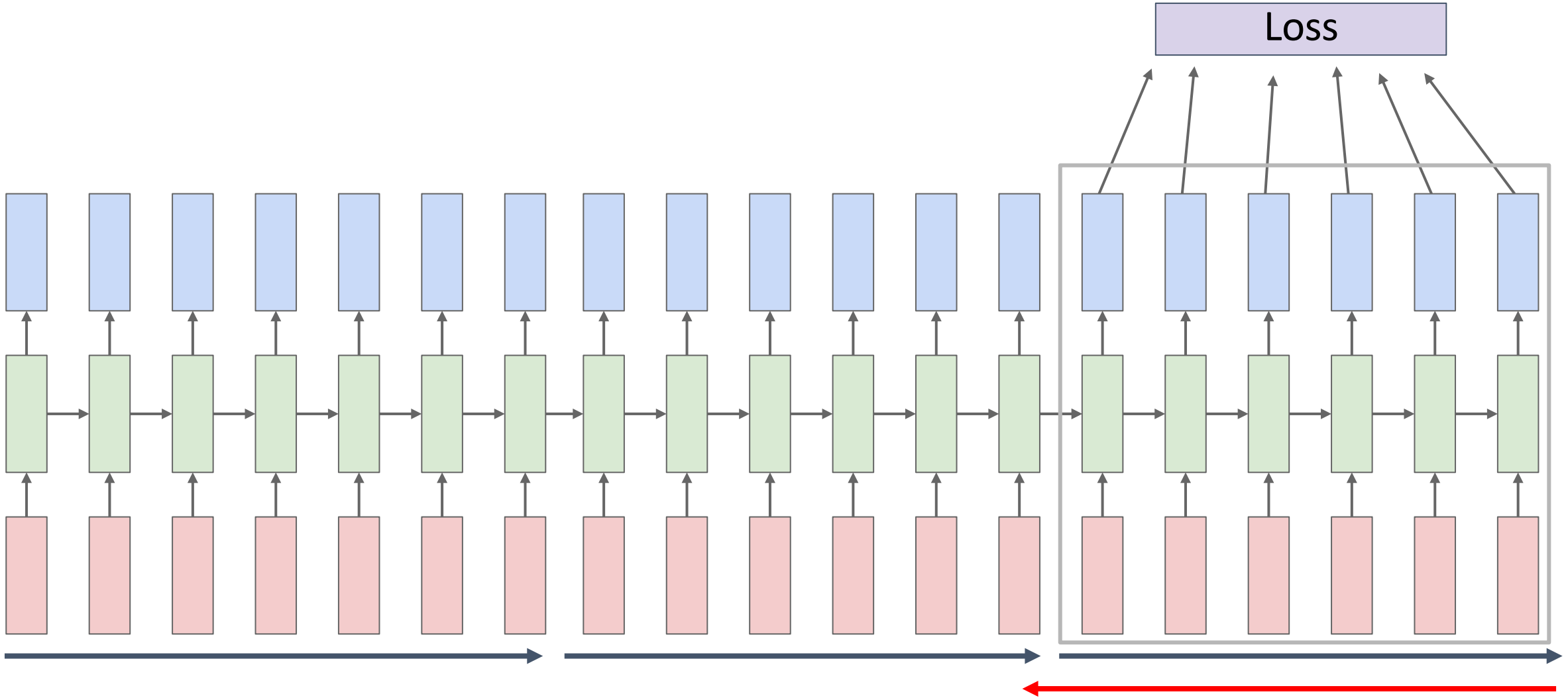
Run forward and backward through chunks of the sequence instead of whole sequence

Appendix: Truncated Backpropagation Through Time



Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Appendix: Truncated Backpropagation Through Time



min-char-rnn.py: 112 lines of Python

```python
"""
Minimal character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
BSD License
"""
import numpy as np

# data I/O
data = open('input.txt', 'r').read() # should be simple plain text file
chars = list(set(data))
data_size, vocab_size = len(data), len(chars)
print 'data has %d characters, %d unique.' % (data_size, vocab_size)
char_to_ix = { ch:i for i,ch in enumerate(chars) }
ix_to_char = { i:ch for i,ch in enumerate(chars) }

# hyperparameters
hidden_size = 100 # size of hidden layer of neurons
seq_length = 25 # number of steps to unroll the RNN for
learning_rate = 1e-1

# model parameters
Wxh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
Whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
Why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
bh = np.zeros((hidden_size, 1)) # hidden bias
by = np.zeros((vocab_size, 1)) # output bias

def lossFun(inputs, targets, hprev):
  """
  inputs,targets are both list of integers.
  hprev is Hx1 array of initial hidden state
  returns the loss, gradients on model parameters, and last hidden state
  """
  xs, hs, ys, ps = {}, {}, {}, {}
  hs[-1] = np.copy(hprev)
  loss = 0
  # forward pass
  for t in xrange(len(inputs)):
    xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
    xs[t][inputs[t]] = 1
    hs[t] = np.tanh(np.dot(Wxh, xs[t]) + np.dot(Whh, hs[t-1]) + bh) # hidden state
    ys[t] = np.dot(Why, hs[t]) + by # unnormalized log probabilities for next chars
    ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
    loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)
  # backward pass: compute gradients going backwards
  dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
  dbh, dby = np.zeros_like(bh), np.zeros_like(by)
  dhnext = np.zeros_like(hs[0])
  for t in reversed(xrange(len(inputs))):
    dy = np.copy(ps[t])
    dy[targets[t]] -= 1 # backprop into y
    dWhy += np.dot(dy, hs[t].T)
    dby += dy
    dh = np.dot(Why.T, dy) + dhnext # backprop into h
    dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
    dbh += dhraw
    dWxh += np.dot(dhraw, xs[t].T)
    dWhh += np.dot(dhraw, hs[t-1].T)
    dhnext = np.dot(Whh.T, dhraw)
  for dparam in [dWxh, dWhh, dWhy, dbh, dby]:
    np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
  return loss, dWxh, dWhh, dWhy, dbh, dby, hs[len(inputs)-1]
```

```python
def sample(h, seed_ix, n):
  """
  sample a sequence of integers from the model
  h is memory state, seed_ix is seed letter for first time step
  """
  x = np.zeros((vocab_size, 1))
  x[seed_ix] = 1
  ixes = []
  for t in xrange(n):
    h = np.tanh(np.dot(Wxh, x) + np.dot(Whh, h) + bh)
    y = np.dot(Why, h) + by
    p = np.exp(y) / np.sum(np.exp(y))
    ix = np.random.choice(range(vocab_size), p=p.ravel())
    x = np.zeros((vocab_size, 1))
    x[ix] = 1
    ixes.append(ix)
  return ixes

n, p = 0, 0
mWxh, mWhh, mWhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
while True:
  # prepare inputs (we're sweeping from left to right in steps seq_length long)
  if p+seq_length+1 >= len(data) or n == 0:
    hprev = np.zeros((hidden_size,1)) # reset RNN memory
    p = 0 # go from start of data
  inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
  targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]

  # sample from the model now and then
  if n % 100 == 0:
    sample_ix = sample(hprev, inputs[0], 200)
    txt = ''.join(ix_to_char[ix] for ix in sample_ix)
    print '----\n %s \n----' % (txt, )

  # forward seq_length characters through the net and fetch gradient
  loss, dWxh, dWhh, dWhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
  smooth_loss = smooth_loss * 0.999 + loss * 0.001
  if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress

  # perform parameter update with Adagrad
  for param, dparam, mem in zip([Wxh, Whh, Why, bh, by],
                                [dWxh, dWhh, dWhy, dbh, dby],
                                [mWxh, mWhh, mWhy, mbh, mby]):
    mem += dparam * dparam
    param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update

  p += seq_length # move data pointer
  n += 1 # iteration counter
```

(https://gist.github.com/karpathy/d4dee566867f8291f086)

Image Captioning: Failure Cases

Captions generated using neuraltalk2
All images are CC0 Public domain: fur coat, handstand, spider web, baseball



A woman is holding a cat in her hand



A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch

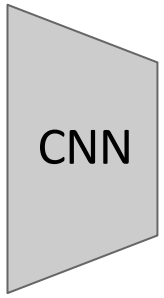


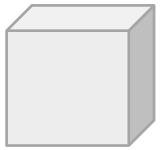
A man in a baseball uniform throwing a ball

Example: CNN with Self-Attention

Input Image





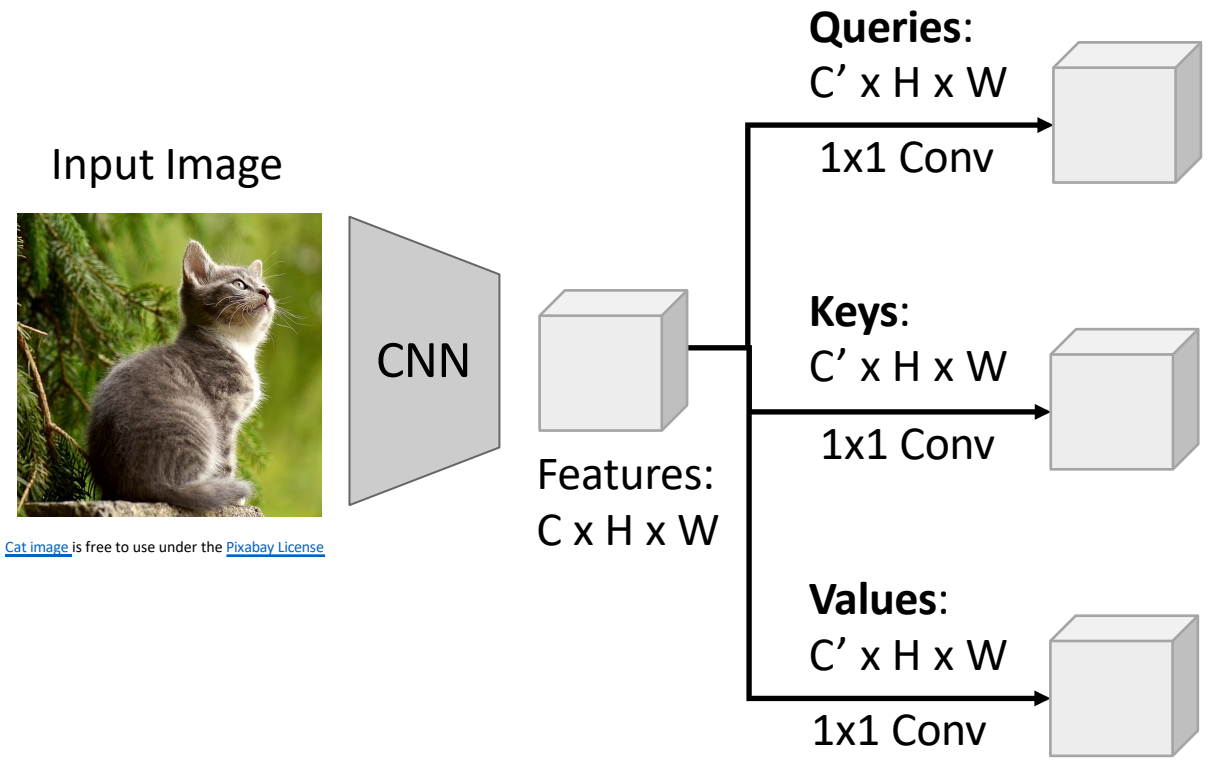


Features:
C x H x W

Cat image is free to use under the Pixabay License

Zhang et al, “Self-Attention Generative Adversarial Networks”, ICML 2018

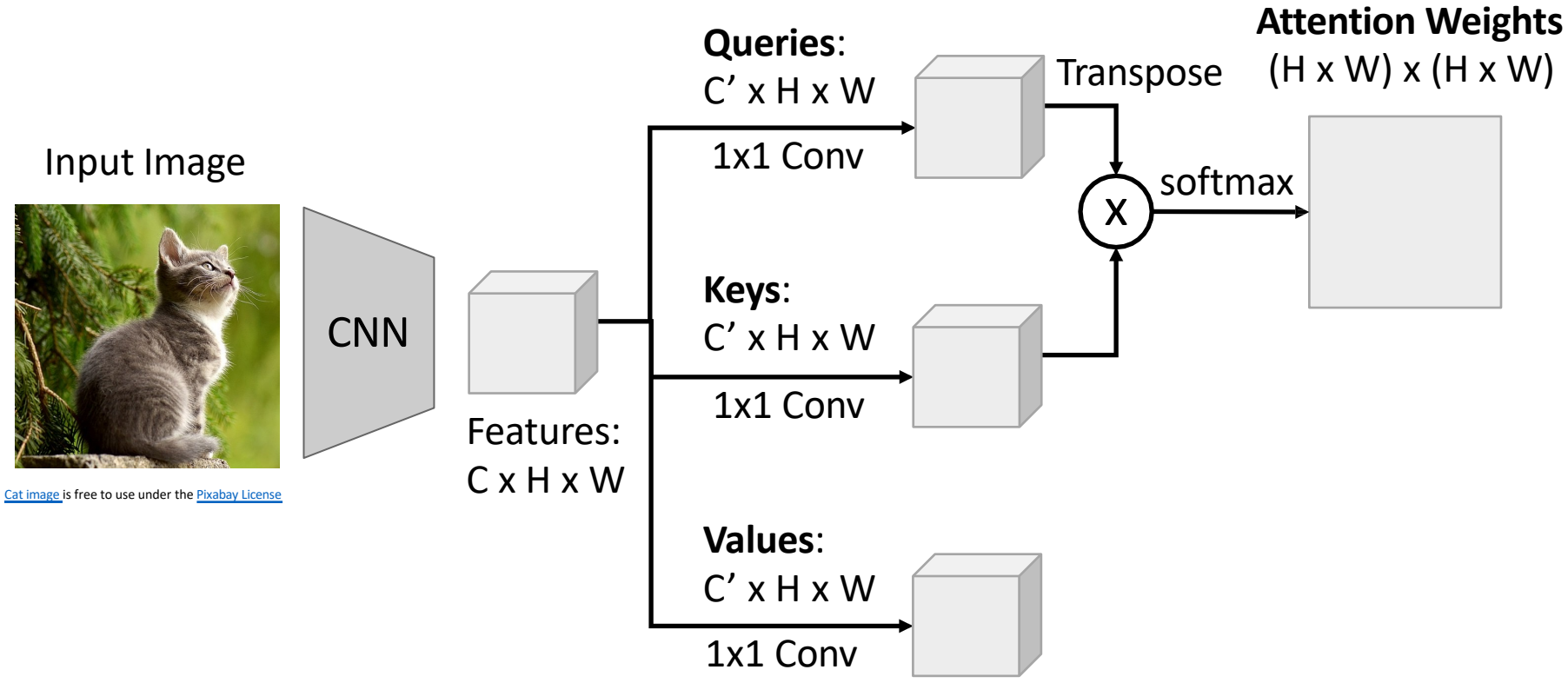
Example: CNN with Self-Attention



Cat image is free to use under the Pixabay License

Zhang et al, "Self-Attention Generative Adversarial Networks", ICML 2018

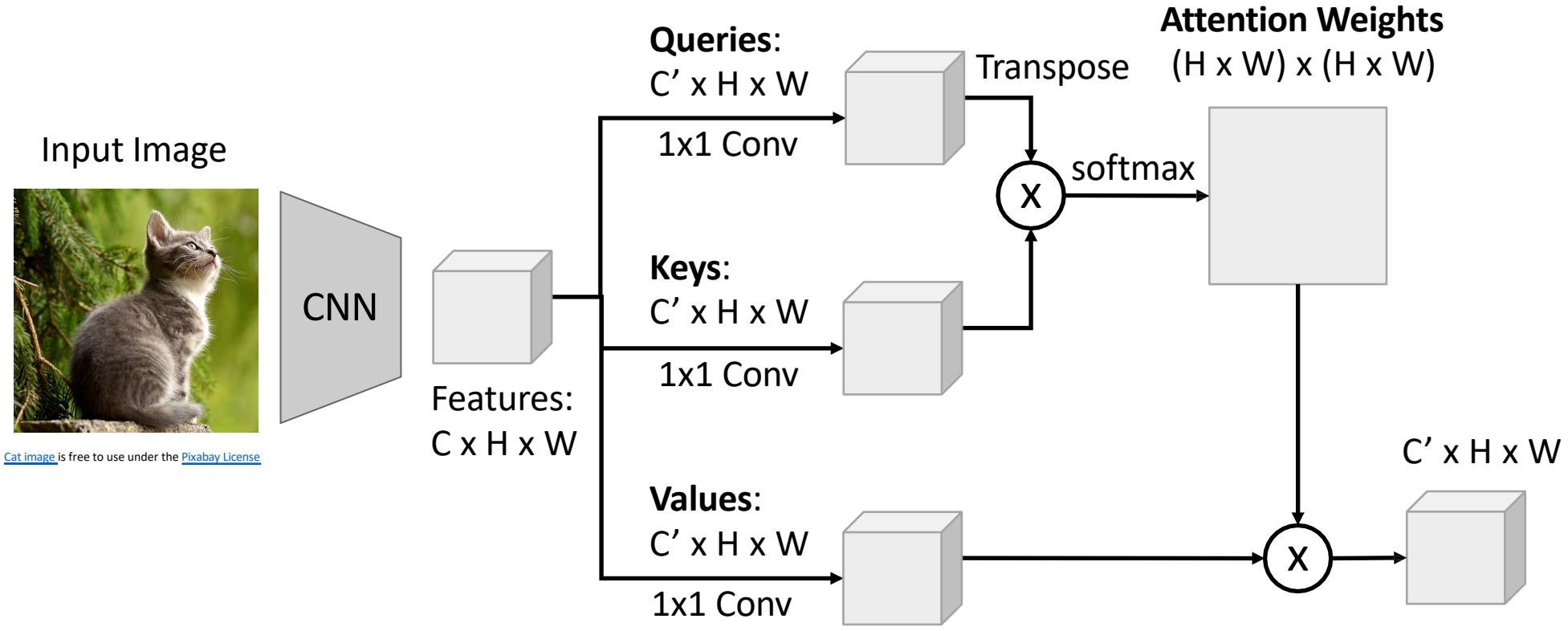
Example: CNN with Self-Attention



Cat image is free to use under the Pixabay License

Zhang et al, "Self-Attention Generative Adversarial Networks", ICML 2018

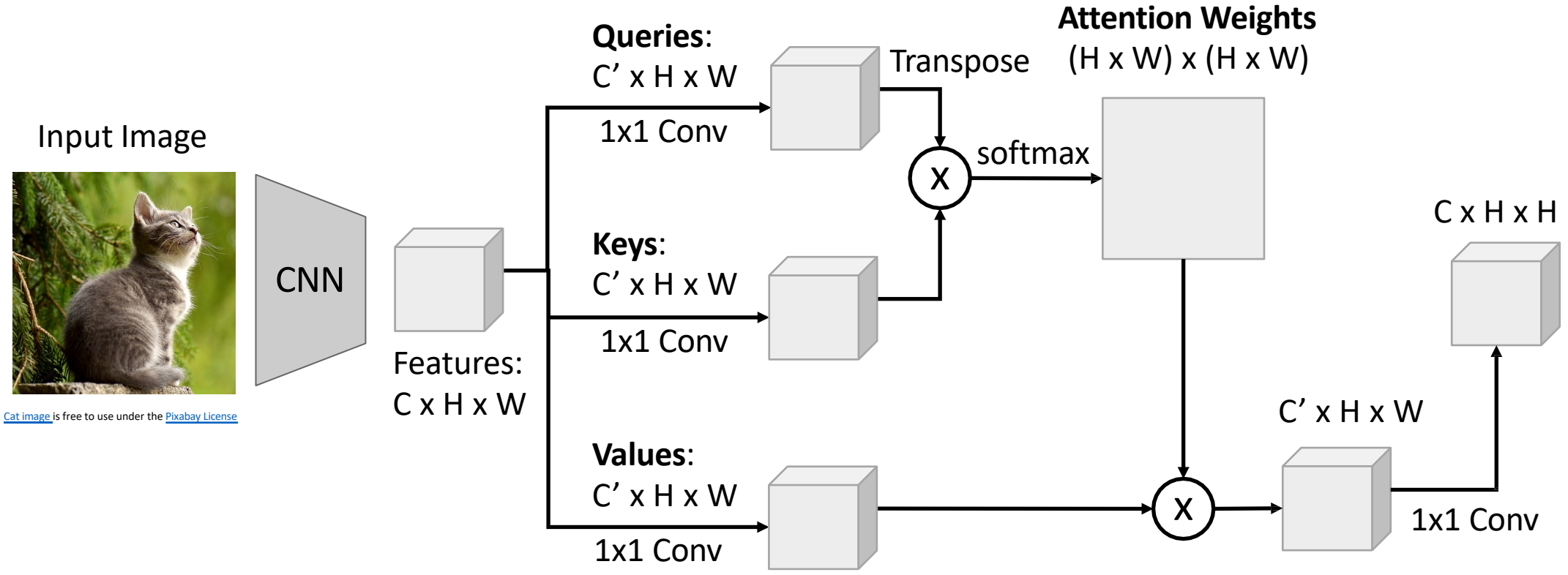
Example: CNN with Self-Attention



Cat image is free to use under the Pixabay License

Zhang et al, "Self-Attention Generative Adversarial Networks", ICML 2018

Example: CNN with Self-Attention



Cat image is free to use under the Pixabay License

Zhang et al, "Self-Attention Generative Adversarial Networks", ICML 2018

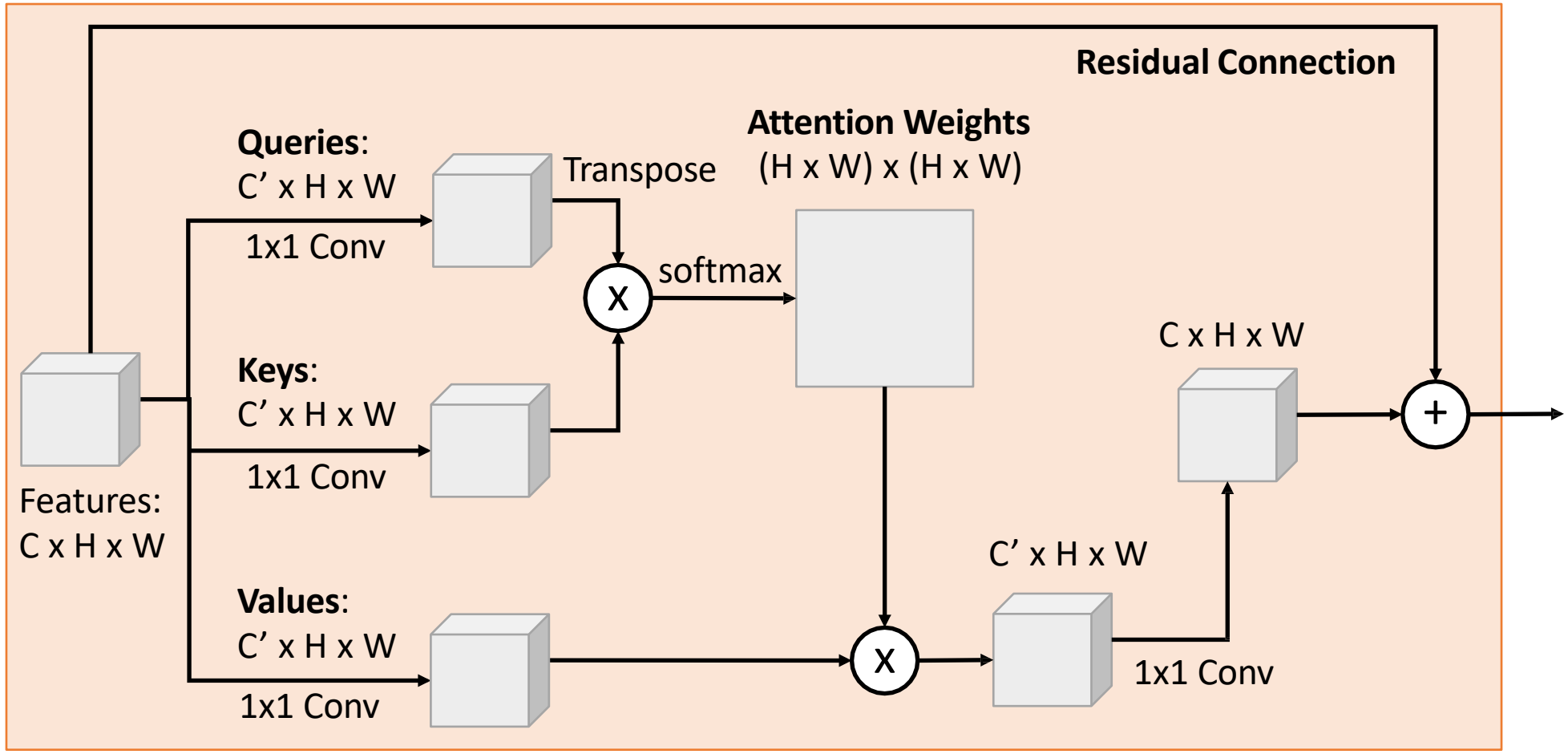
Example: CNN with Self-Attention

Input Image



Cat image is free to use under the Pixabay License

CNN



Self-Attention Module

Zhang et al, "Self-Attention Generative Adversarial Networks", ICML 2018